— Course Description —

Computer Architecture Mini-Course

Instructor: Prof. Milo Martin

Course Description

This three day mini-course is a broad overview of computer architecture, motivated by trends in semiconductor manufacturing, software evolution, and the emergence of parallelism at multiple levels of granularity. The first day discusses technology trends (including brief coverage of energy/power issues), instruction set architectures (for example, the differences between the x86 and ARM architectures), and memory hierarchy and caches. The second day focuses on core micro-architecture, including pipelining, instruction-level parallelism, superscalar execution, and dynamic (out-of-order) instruction scheduling. The third day touches upon data-level parallelism and overviews multicore chips.

The course is intended for software or hardware engineers with basic knowledge of computer organization (such as binary encoding of numbers, basic boolean logic, and familiarity with the concept of an assembly-level "instruction"). The material in this course is similar to what would be found in an advanced undergraduate or first-year graduate-level course on computer architecture. The course is well suited for: (1) software developers that desire more "under the hood" knowledge of how chips execute code and the performance implications thereof or (2) lower-level hardware/SoC or logic designers that seek understanding of state-of-the-art high-performance chip architectures.

The course will consist primarily of lectures, but it also includes three out-of-class reading assignments to be read before each day of class and discussed during the lectures.

Course Outline

Below is the the course outline for the three day course (starting 10am on the first day and ending at 5pm on the third day). The exact topics and order is tenative and subject to change.

Day 1: "Foundations & Memory Hierarchy"

- Introduction, motivation, & "What is Computer Architecture"
- Instruction set architectures
- Transistor technology trends and energy/power implications
- Memory hierarchy, caches, and virtual memory (two lectures)

Day 2: "Core Micro-Architecture"

- Pipelining
- Branch prediction
- Superscalar
- Hardware instruction schedulingk (two lectures)

Day 3: "Multicore & Parallelism"

- Multicore, coherence, and consistency (two lectures)
- Data-level parallelism
- Wrapup

Instructor and Bio

Prof. Milo Martin

Dr. Milo Martin is an Associate Professor at University of Pennsylvania, a private Ivy-league university in Philadelphia, PA. His research focuses on making computers more responsive and easier to design and program. Specific projects include computational sprinting, hardware transactional memory, adaptive cache coherence protocols, memory consistency models, hardware-aware verification of concurrent software, and hardware-assisted memory-safe implementations of unsafe programming language. Dr. Martin has published over 40 papers which collectively have received over 2500 citations. Dr. Martin is a recipient of the NSF CAREER award and received a PhD from the University of Wisconsin-Madison.

Computer Architecture
Mini-Course

March 2013

Prof. Milo Martin

Day 1 of 3

[spacer]

Computer Architecture

Unit 1: Introduction

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

What is Computer Architecture?

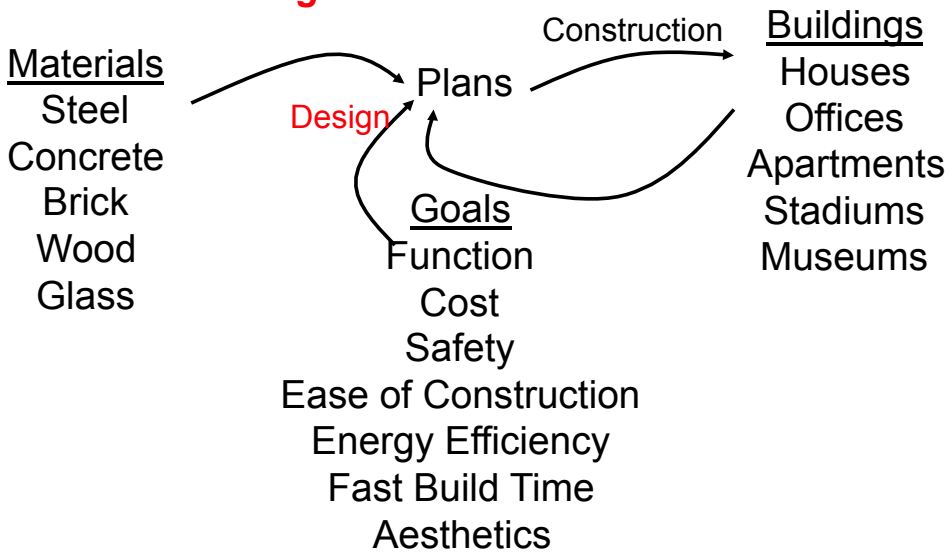
What is Computer Architecture?

- *"Computer Architecture* is the science and art of selecting and interconnecting hardware components to create computers that meet functional, performance and cost goals." - WWW Computer Architecture Page

- An analogy to architecture of buildings…

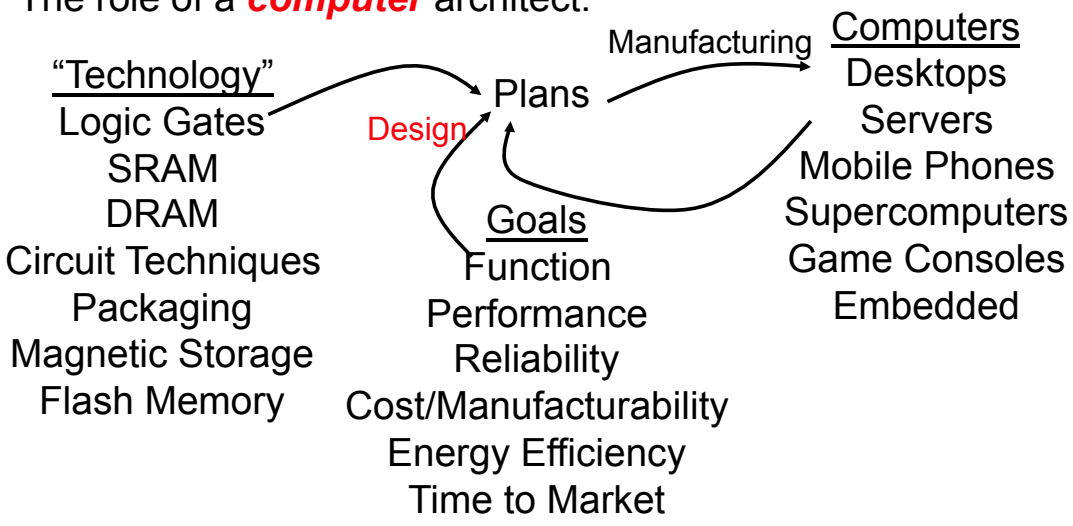
What is ~~Computer~~ Architecture?

The role of a ***building*** architect:



What is Computer Architecture?

The role of a ***computer*** architect:



Important differences: age (~60 years vs thousands), rate of change, automated mass production (magnifies design)

Computer Architecture Is Different…

- Age of discipline
  - 60 years (vs. five thousand years)
- Rate of change
  - All three factors (technology, applications, goals) are changing
  - Quickly
- Automated mass production
  - Design advances magnified over millions of chips
- Boot-strapping effect
  - Better computers help design next generation

Design Goals & Constraints

- **Functional**
  - Needs to be correct
    - And unlike software, difficult to update once deployed
  - What functions should it support (Turing completeness aside)

- **Reliable**
  - Does it ***continue*** to perform correctly?
  - Hard fault vs transient fault
  - Google story - memory errors and sun spots
  - Space satellites vs desktop vs server reliability

- **High performance**
  - "Fast" is only meaningful in the context of a set of important tasks
  - Not just "Gigahertz" – truck vs sports car analogy
  - Impossible goal: fastest possible design for all programs

Design Goals & Constraints

- **Low cost**
  - Per unit manufacturing cost (wafer cost)
  - Cost of making first chip after design (mask cost)
  - Design cost (huge design teams, why? Two reasons…)
  - (Dime/dollar joke)

- **Low power/energy**
  - Energy in (battery life, cost of electricity)
  - Energy out (cooling and related costs)
  - Cyclic problem, very much a problem today

- Challenge: balancing the relative importance of these goals
  - And the balance is constantly changing
    - No goal is absolutely important at expense of all others
  - Our focus: *performance,* only touch on cost, power, reliability

Shaping Force: Applications/Domains

- Another shaping force: **applications** (usage and context)
  - Applications and application domains have different requirements
    - Domain: group with similar character
  - Lead to different designs

- **Scientific**: weather prediction, genome sequencing
  - First computing application domain: naval ballistics firing tables
  - Need: large memory, heavy-duty floating point
  - Examples: CRAY T3E, IBM BlueGene

- **Commercial**: database/web serving, e-commerce, Google
  - Need: data movement, high memory + I/O bandwidth
  - Examples: Sun Enterprise Server, AMD Opteron, Intel Xeon

More Recent Applications/Domains

- **Desktop**: home office, multimedia, games
  - Need: integer, memory bandwidth, integrated graphics/network?
  - Examples: Intel Core 2, Core i7, AMD Athlon

- **Mobile**: laptops, mobile phones
  - Need: **low power**, integer performance, integrated wireless
  - Laptops: Intel Core 2 Mobile, Atom, AMD Turion
  - Smaller devices: ARM chips by Samsung, Qualcomm; Intel Atom

- **Embedded**: microcontrollers in automobiles, door knobs
  - Need: low power, **low cost**
  - Examples: ARM chips, dedicated digital signal processors (DSPs)
  - Over 6 billion ARM cores sold in 2010 (multiple per phone)

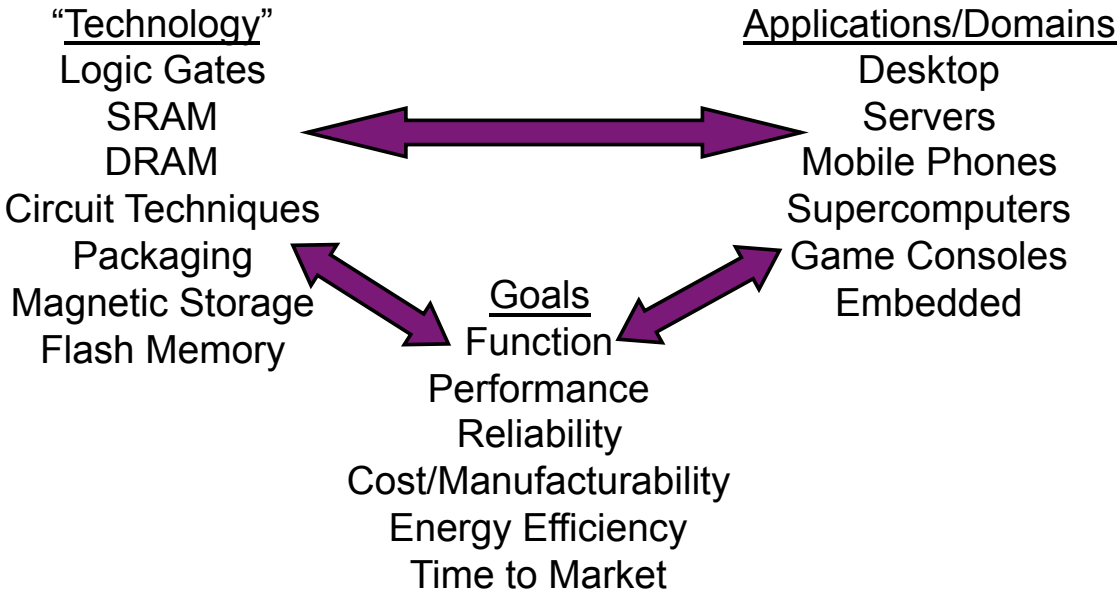
- **Deeply Embedded**: disposable "smart dust" sensors
  - Need: extremely low power, extremely low cost

Application Specific Designs

- This class is about **general-purpose CPUs**
  - Processor that can do anything, run a full OS, etc.
  - E.g., Intel Core i7, AMD Athlon, IBM Power, ARM, Intel Itanium
- In contrast to **application-specific chips**
  - Or **ASICs** (Application specific integrated circuits)
    - Also application-domain specific processors
  - Implement critical domain-specific functionality in hardware
    - Examples: video encoding, 3D graphics
  - General rules
    - \- Hardware is less flexible than software
    - \+ Hardware more effective (speed, power, cost) than software
    - \+ Domain specific more "parallel" than general purpose
      - But general mainstream processors becoming more parallel
- Trend: from specific to general (for a specific domain)

Technology Trends

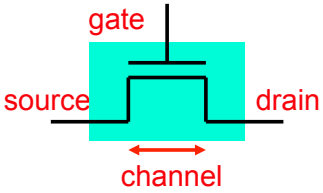
Constant Change: Technology



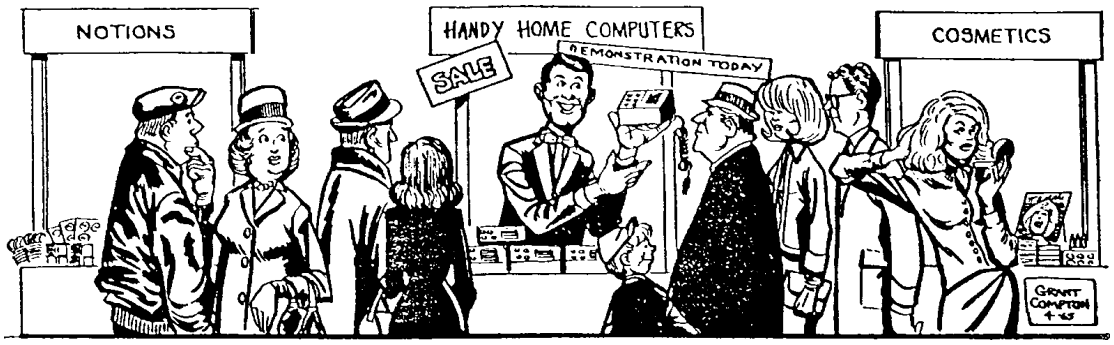
- Absolute improvement, **different rates of change**
- New application domains enabled by technology advances

"Technology"

- Basic element
  - Solid-state **transistor** (i.e., electrical switch)
  - Building block of **integrated circuits (ICs)**

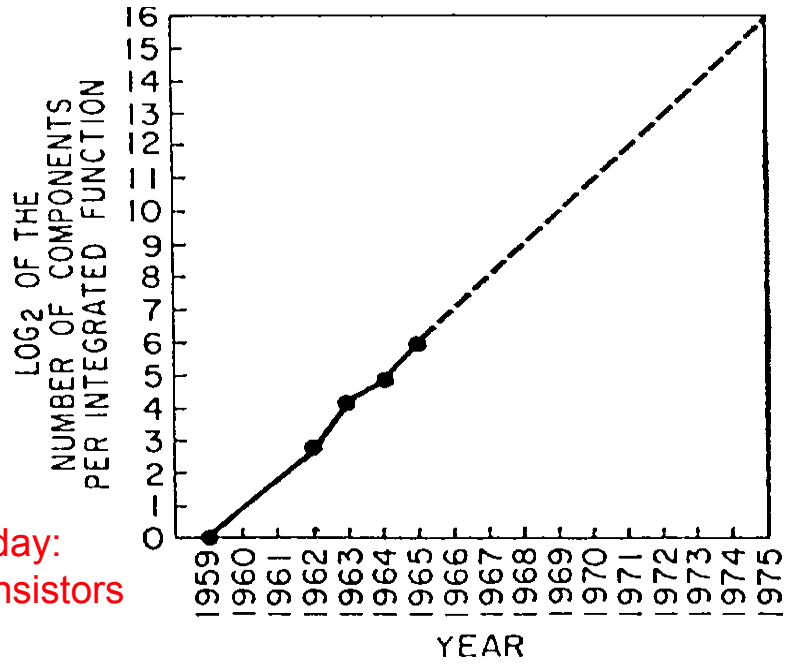


- What's so great about ICs? Everything
  + High performance, high reliability, low cost, low power
  + Lever of mass production
- Several kinds of integrated circuit families
  - **SRAM/logic**: optimized for speed (used for processors)
  - **DRAM**: optimized for density, cost, power (used for memory)
  - **Flash**: optimized for density, cost (used for storage)
  - Increasing opportunities for integrating multiple technologies
- Non-transistor storage and inter-connection technologies
  - Disk, optical storage, ethernet, fiber optics, wireless



Funny or Not Funny?

Moore's Law - 1965



Today:
2^{30} transistors

Technology Trends

- **Moore's Law**
  - Continued (up until now, at least) transistor miniaturization
- Some technology-based ramifications
  - Absolute improvements in density, speed, power, costs
  - SRAM/logic: density: ~30% (annual), speed: ~20%
  - DRAM: density: ~60%, speed: ~4%
  - Disk: density: ~60%, speed: ~10% (non-transistor)
  - Big improvements in flash memory and network bandwidth, too
- **Changing quickly and with respect to each other!!**
  - Example: density increases faster than speed
  - Trade-offs are constantly changing
  - **Re-evaluate/re-design for each technology generation**

Technology Change Drives Everything

- Computers get 10x faster, smaller, cheaper every 5-6 years!
  - A 10x quantitative change is qualitative change
  - Plane is 10x faster than car, and fundamentally different travel mode
- New applications become self-sustaining market segments
  - Recent examples: mobile phones, digital cameras, mp3 players, etc.
- Low-level improvements appear as discrete high-level jumps
  - Capabilities cross thresholds, enabling new applications and uses

Revolution I: The Microprocessor

- **Microprocessor revolution**
  - One significant technology threshold was crossed in 1970s
  - Enough transistors (~25K) to put a 16-bit processor on one chip
  - Huge performance advantages: fewer slow chip-crossings
  - Even bigger cost advantages: one "stamped-out" component

- Microprocessors have allowed new market segments
  - Desktops, CD/DVD players, laptops, game consoles, set-top boxes, mobile phones, digital camera, mp3 players, GPS, automotive

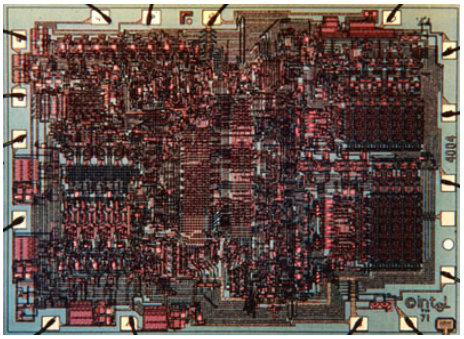
- And replaced incumbents in existing segments
  - Microprocessor-based system replaced supercomputers, "mainframes", "minicomputers", etc.

First Microprocessor

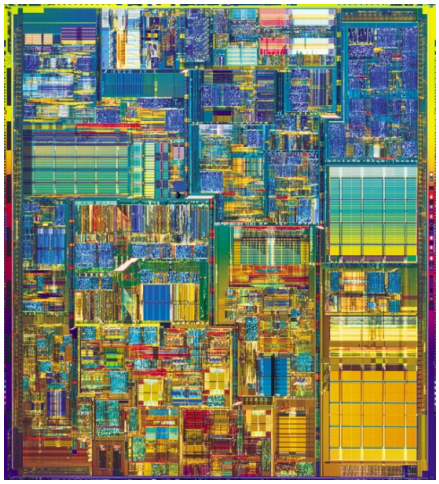
- Intel 4004 (1971)
  - Application: calculators
  - Technology: 10000 nm

  - 2300 transistors
  - 13 mm^2
  - 108 KHz
  - 12 Volts

  - 4-bit data
  - Single-cycle datapath



Pinnacle of Single-Core Microprocessors



- Intel Pentium4 (2003)
  - Application: desktop/server
  - Technology: 90nm (1/100x)
  - 55M transistors (20,000x)
  - 101 mm^2 (10x)
  - 3.4 GHz (10,000x)
  - 1.2 Volts (1/10x)
  - 32/64-bit data (16x)
  - 22-stage pipelined datapath
  - 3 instructions per cycle (superscalar)
  - Two levels of on-chip cache
  - data-parallel vector (SIMD) instructions, hyperthreading

Tracing the Microprocessor Revolution

- How were growing transistor counts used?
- Initially to widen the datapath
  - 4004: 4 bits → Pentium4: 64 bits
- … and also to add more powerful instructions
  - To amortize overhead of fetch and decode
  - To simplify programming (which was done by hand then)

Revolution II: Implicit Parallelism

- Then to **extract implicit instruction-level parallelism**
  - Hardware provides parallel resources, figures out how to use them
  - Software is oblivious

- Initially using pipelining …
  - Which also enabled increased clock frequency
- … caches …
  - Which became necessary as processor clock frequency increased
- … and integrated floating-point
- Then deeper pipelines and branch speculation
- Then multiple instructions per cycle (superscalar)
- Then dynamic scheduling (out-of-order execution)

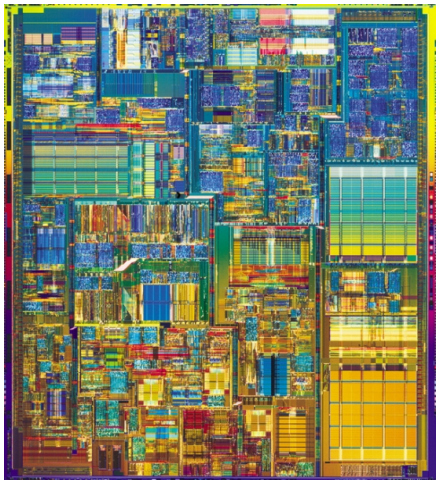
- We will talk about these things

Pinnacle of Single-Core Microprocessors

- Intel Pentium4 (2003)
  - Application: desktop/server
  - Technology: 90nm (1/100x)

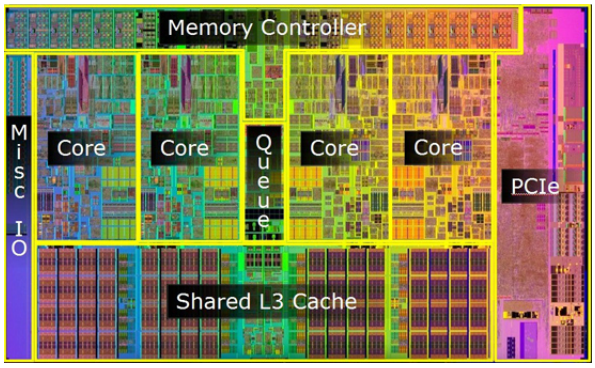
  - 55M transistors (20,000x)
  - 101 mm^2 (10x)
  - 3.4 GHz (10,000x)
  - 1.2 Volts (1/10x)

  - 32/64-bit data (16x)
  - 22-stage pipelined datapath
  - 3 instructions per cycle (superscalar)
  - Two levels of on-chip cache
  - data-parallel vector (SIMD) instructions, hyperthreading



Modern Multicore Processor

- Intel Core i7 (2009)
  - Application: desktop/server
  - Technology: 45nm (1/2x)



  - 774M transistors (12x)
  - 296 mm^2 (3x)
  - 3.2 GHz to 3.6 Ghz (~1x)
  - 0.7 to 1.4 Volts (~1x)

  - 128-bit data (2x)
  - 14-stage pipelined datapath (0.5x)
  - 4 instructions per cycle (~1x)
  - Three levels of on-chip cache
  - data-parallel vector (SIMD) instructions, hyperthreading
  - **Four-core multicore** (4x)

Revolution III: Explicit Parallelism

- Then to support **explicit data & thread level parallelism**
  - Hardware provides parallel resources, software specifies usage
  - Why? diminishing returns on instruction-level-parallelism
- First using (subword) vector instructions…, Intel's SSE
  - One instruction does four parallel multiplies
- … and general support for multi-threaded programs
  - Coherent caches, hardware synchronization primitives
- Then using support for multiple concurrent threads on chip
  - First with single-core multi-threading, now with multi-core
- Graphics processing units (GPUs) are highly parallel
  - Converging with general-purpose processors (CPUs)?

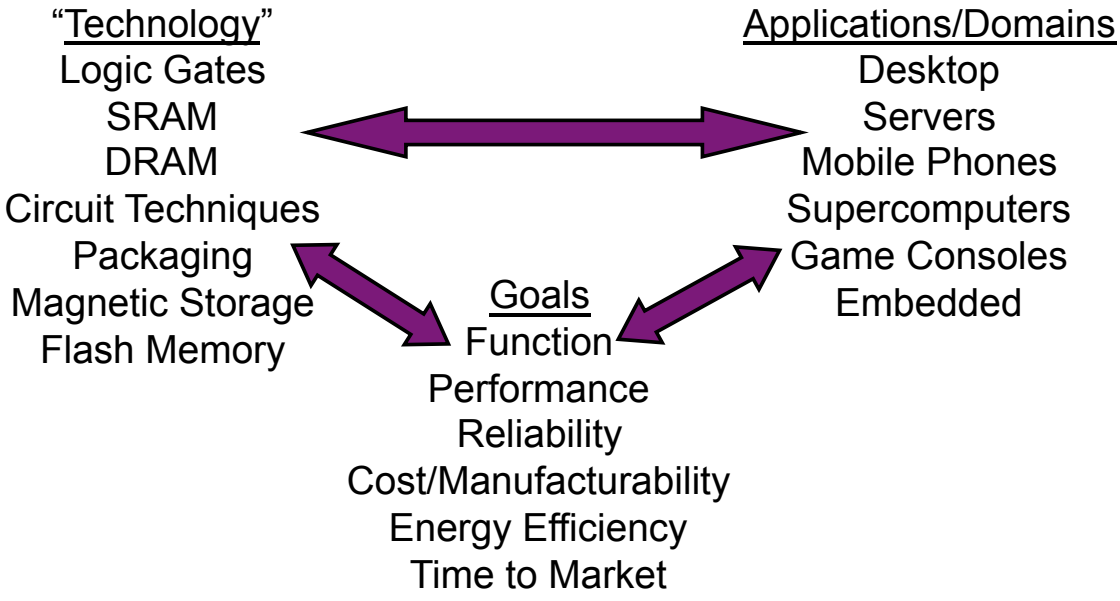
To ponder…

Is this decade's
"**multicore revolution**"
comparable to the original
"**microprocessor revolution**"?

Technology Disruptions

- Classic examples:
  - The transistor
  - Microprocessor
- More recent examples:
  - Multicore processors
  - Flash-based solid-state storage
- Near-term potentially disruptive technologies:
  - Phase-change memory (non-volatile memory)
  - Chip stacking (also called 3D die stacking)
- Disruptive "end-of-scaling"
  - "If something can't go on forever, it must stop eventually"
  - Can we continue to shrink transistors for ever?
  - Even if more transistors, not getting as energy efficient as fast

Recap: Constant Change



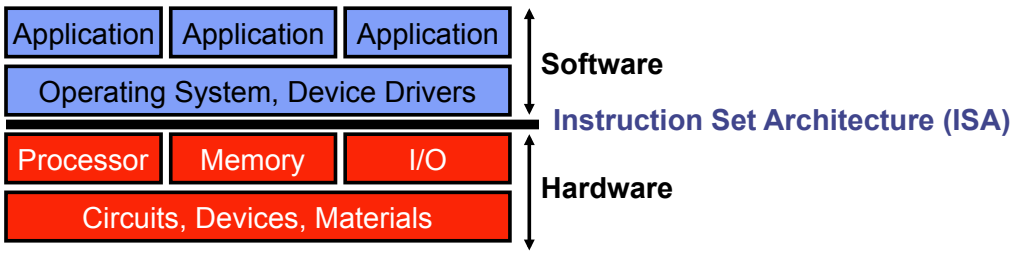
Managing This Mess

- Architect must consider all factors
  - Goals/constraints, applications, implementation technology
- Questions
  - How to deal with all of these inputs?
  - How to manage changes?
- Answers
  - Accrued institutional knowledge (stand on each other's shoulders)
  - Experience, rules of thumb
  - Discipline: clearly defined end state, keep your eyes on the ball
  - **Abstraction and layering**

Pervasive Idea: Abstraction and Layering

- **Abstraction**: only way of dealing with complex systems
  - Divide world into objects, each with an…
    - **Interface**: knobs, behaviors, knobs → behaviors
    - **Implementation**: "black box" (ignorance+apathy)
  - Only specialists deal with implementation, rest of us with interface
  - Example: car, only mechanics know how implementation works
- **Layering**: abstraction discipline makes life even simpler
  - Divide objects in system into layers, layer *n* objects…
    - Implemented using interfaces of layer *n* – 1
    - Don't need to know interfaces of layer *n* – 2 (sometimes helps)
- **Inertia**: a dark side of layering
  - Layer interfaces become entrenched over time ("standards")
  - – Very difficult to change even if benefit is clear (example: Digital TV)
- **Opacity**: hard to reason about performance across layers

Abstraction, Layering, and Computers



- **Computer architecture**
  - Definition of **ISA** to facilitate implementation of software layers
- This course mostly on **computer micro-architecture**
  - Design of chip to implement **ISA**
- Touch on compilers & OS (*n* +1), circuits (*n* -1) as well

Course Overview

Why Study Computer Architecture?

- **Understand where computers are going**
  - Future capabilities drive the (computing) world
  - Real world-impact: no computer architecture → no computers!
- **Understand high-level design concepts**
  - The best architects understand all the levels
    - Devices, circuits, architecture, compiler, applications
- **Understand computer performance**
  - Writing well-tuned (fast) software requires knowledge of hardware
- **Write better software**
  - The best software designers also understand hardware
  - Need to understand hardware to write fast software
- **Design hardware**

Course Information

- Course goals
  - **See the "big ideas" in computer architecture**
    - Pipelining, parallelism, caching, locality, abstraction, etc.
  - Exposure to examples of good (and some bad) engineering
  - Understand how architectural ideas are evaluated (readings)

- Prerequisites
  - **General computer science/engineering background**
  - Basic computer organization concepts
    - digital logic, binary arithmetic
  - Familiarity with concept of "instruction" and assembly language

- Course level
  - Advanced undergraduate or first-year graduate-level course

Course Topics & Schedule (1 of 2)

- Day 1: Foundations & memory hierarchy
  - Introduction
  - Technology trends
  - ISAs (instruction set architectures)
  - Caches & memory hierarchy, virtual memory
- Day 2: Core micro-architecture & implicit parallelism
  - Pipelining
  - Branch prediction
  - Superscalar
  - Instruction scheduling – in hardware and software
- Day 3: Explicit parallelism
  - Multicore, synchronization, cache coherence, memory consistency
  - Data-level parallelism, vectors, and just a bit on GPUs
  - Xbox 360 & Wrapup

Course Topics & Schedule (2 of 2)

- A key course theme: **Parallelism**
  - Instruction-level: superscalar, dynamic scheduling, speculation
  - Data-level: vectors and GPUs
  - Thread-level: multicore, cache coherence, memory consistency

- Covered briefly
  - Virtual memory
  - GPUs

- Not covered
  - Off-chip memory technologies (DRAM)
  - Reliability
  - Input/output devices
  - Datacenter architectures & deeply embedded

Instructor Bio: Prof. Milo M. K. Martin

- Associate Professor
  - University of Pennsylvania (or "Penn"); private Ivy-league university in Philadelphia
  - PhD from the University of Wisconsin-Madison

- My research focuses on…
  making computers more responsive and easier to design and program

- Specific projects include:
  - Computational sprinting
  - Hardware transactional memory & adaptive cache coherence
  - Memory consistency models
  - Hardware-aware verification of concurrent software
  - Hardware-assisted memory safety of unsafe programming language

- Published over 40 papers; have received over 2500 citations

- Recipient of the NSF CAREER award

Full Disclosure

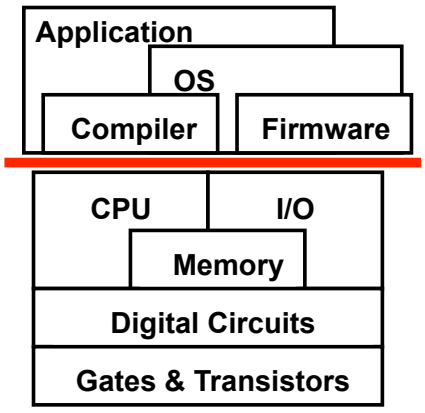
- Potential sources of bias or conflict of interest
- Most of my funding is governmental (your tax $$$ at work)
  - National Science Foundation (NSF)
  - DARPA & ONR
- My non-governmental sources of research funding
  - NVIDIA (sub-contract of large DARPA project, now completed)
  - Intel
  - Sun/Oracle (hardware donation)
- Collaborators and colleagues
  - Intel, IBM, AMD, Oracle, Microsoft, Google, VMWare, ARM, etc.
  - (Just about every major computer company)
- Sabbatical at Google for 2013-2014

Computer Architecture

Unit 2: Instruction Set Architectures

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

Instruction Set Architecture (ISA)



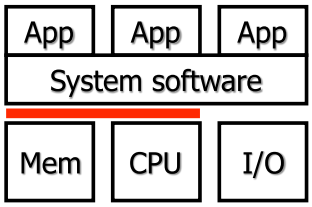
- What is an ISA?
  - A functional contract
- All ISAs similar in high-level ways
  - But many design choices in details
  - Two "philosophies": CISC/RISC
    - Difference is blurring
- Good ISA…
  - Enables high-performance
  - At least doesn't get in the way
- Compatibility is a powerful force
  - Tricks: binary translation, μISAs

Readings

- Suggested reading:
  - "The Evolution of RISC Technology at IBM" by John Cocke and V. Markstein

Execution Model

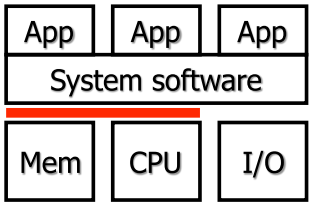
Program Compilation




```
int array[100], sum;
void array_sum() {
   for (int i=0; i<100;i++) {
      sum += array[i];
   }
}
```

- **Program** written in a "high-level" programming language
  - C, C++, Java, C#
  - Hierarchical, structured control: loops, functions, conditionals
  - Hierarchical, structured data: scalars, arrays, pointers, structures
- **Compiler**: translates program to **assembly**
  - Parsing and straight-forward translation
  - Compiler also optimizes
  - Compiler itself another application … who compiled compiler?

Assembly & Machine Language

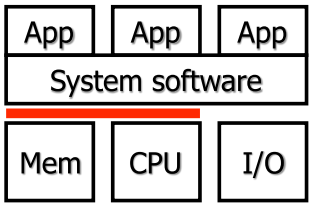


- **Assembly language**
  - Human-readable representation
- **Machine language**
  - Machine-readable representation
  - 1s and 0s (often displayed in "hex")
- **Assembler**
  - Translates assembly to machine

Machine code	Assembly code
x9A00	CONST R5, #0
x9200	CONST R1, array
xD320	HICONST R1, array
x9464	CONST R2, sum
xD520	HICONST R2, sum
x6640	LDR R3, R1, #0
x6880	LDR R4, R2, #0
x18C4	ADD R4, R3, R4
x7880	STR R4, R2, #0
x1261	ADD R1, R1, #1
x1BA1	ADD R5, R5, #1
x2B64	CMPI R5, #100
x03F8	BRn array_sum_loop

Example is in "LC4" a toy
instruction set architecture, or **ISA**

Example Assembly Language & ISA

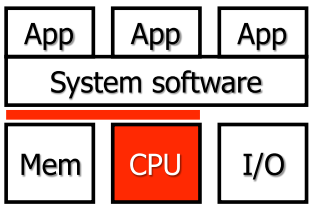


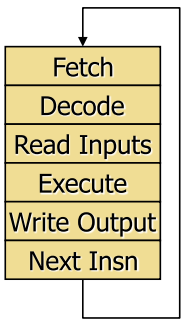
- **MIPS**: example of real ISA
  - 32/64-bit operations
  - 32-bit insns
  - 64 registers
    - 32 integer, 32 floating point
  - ~100 different insns

```
    .data
array: .space 100
sum:   .word 0
    .text
array_sum:
    li $5, 0
    la $1, array
    la $2, sum
array_sum_loop:
    lw $3, 0($1)
    lw $4, 0($2)
    add $4, $3, $4
    sw $4, 0($2)
    addi $1, $1, 1
    addi $5, $5, 1
    li $6, 100
    blt $5, $6, array_sum_loop
```

Example code is MIPS, but all ISAs are similar at some level

Instruction Execution Model





Instruction → Insn

- The computer is just finite state machine
  - **Registers** (few of them, but fast)
  - **Memory** (lots of memory, but slower)
  - **Program counter** (next insn to execute)
    - Called "instruction pointer" in x86
- A computer executes **instructions**
  - **Fetches** next instruction from memory
  - **Decodes** it (figure out what it does)
  - **Reads** its **inputs** (registers & memory)
  - **Executes** it (adds, multiply, etc.)
  - **Write** its **outputs** (registers & memory)
  - **Next insn** (adjust the program counter)
- **Program is just "data in memory"**
  - Makes computers programmable ("universal")

What is an ISA?

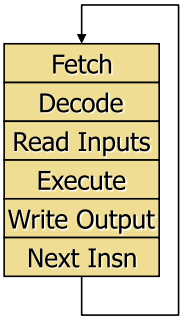
What Is An ISA?

- **ISA (instruction set architecture)**
  - A well-defined hardware/software interface
  - The **"contract"** between software and hardware
    - **Functional definition** of storage locations & operations
      - Storage locations: registers, memory
      - Operations: add, multiply, branch, load, store, etc
    - **Precise description** of how to invoke & access them
- Not in the "contract": non-functional aspects
  - How operations are implemented
  - Which operations are fast and which are slow and when
  - Which operations take more power and which take less
- Instructions
  - Bit-patterns hardware interprets as commands
  - Instruction → Insn (instruction is too long to write in slides)

A Language Analogy for ISAs

- Communication
  - Person-to-person → software-to-hardware
- Similar structure
  - Narrative → program
  - Sentence → insn
  - Verb → operation (add, multiply, load, branch)
  - Noun → data item (immediate, register value, memory value)
  - Adjective → addressing mode
- Many different languages, many different ISAs
  - Similar basic structure, details differ (sometimes greatly)
- Key differences between languages and ISAs
  - Languages evolve organically, many ambiguities, inconsistencies
  - ISAs are explicitly engineered and extended, unambiguous

The Sequential Model



- **Basic structure of all modern ISAs**
  - Often called VonNeuman, but in ENIAC before
- **Program order**: total order on dynamic insns
  - Order and **named storage** define computation
- Convenient feature: **program counter (PC)**
  - Insn itself stored in memory at location pointed to by PC
  - Next PC is next insn unless insn says otherwise
- Processor logically executes loop at left
- **Atomic**: insn finishes before next insn starts
  - Implementations can break this constraint physically
  - But must maintain illusion to preserve correctness

ISA Design Goals

What Makes a Good ISA?

- **Programmability**
  - Easy to express programs efficiently?
- **Performance/Implementability**
  - Easy to design high-performance implementations?
  - More recently
    - Easy to design low-power implementations?
    - Easy to design low-cost implementations?
- **Compatibility**
  - Easy to maintain as languages, programs, and technology evolve?
  - x86 (IA32) generations: 8086, 286, 386, 486, Pentium, PentiumII, PentiumIII, Pentium4, Core2, Core i7, …

Programmability

- Easy to express programs efficiently?
  - For whom?
- Before 1980s: **human**
  - Compilers were terrible, most code was hand-assembled
  - Want high-level coarse-grain instructions
    - As similar to high-level language as possible
- After 1980s: **compiler**
  - Optimizing compilers generate much better code that you or I
  - Want low-level fine-grain instructions
    - Compiler can't tell if two high-level idioms match exactly or not
- This shift changed what is considered a "good" ISA…

Implementability

- Every ISA can be implemented
  - Not every ISA can be implemented efficiently
- Classic high-performance implementation techniques
  - Pipelining, parallel execution, out-of-order execution (more later)
- Certain ISA features make these difficult
  - Variable instruction lengths/formats: complicate decoding
  - Special-purpose registers: complicate compiler optimizations
  - Difficult to interrupt instructions: complicate many things
    - Example: memory copy instruction

Performance, Performance, Performance

- How long does it take for a program to execute?
  - Three factors

1. How many insn must execute to complete program?
   - **Instructions per program** during execution
   - "Dynamic insn count" (not number of "static" insns in program)
2. How quickly does the processor "cycle"?
   - **Clock frequency** (cycles per second) 1 gigahertz (Ghz)
   - or expressed as reciprocal, **Clock period** nanosecond (ns)
   - Worst-case delay through circuit for a particular design
3. How many *cycles* does each instruction take to execute?
   - **Cycles per Instruction** (CPI) or reciprocal, **Insn per Cycle** (IPC)

Execution time =
(instructions/program) * (seconds/cycle) * (cycles/instruction)

Maximizing Performance

Execution time =
(instructions/program) * (seconds/cycle) * (cycles/instruction)

(1 billion instructions) * (1ns per cycle) * (1 cycle per insn)
= 1 second

- Instructions per program:
  - Determined by program, compiler, instruction set architecture (ISA)
- Cycles per instruction: "CPI"
  - Typical range today: 2 to 0.5
  - Determined by program, compiler, ISA, micro-architecture
- Seconds per cycle: "clock period"
  - Typical range today: 2ns to 0.25ns
  - Reciprocal is frequency: 0.5 Ghz to 4 Ghz (1 Htz = 1 cycle per sec)
  - Determined by micro-architecture, technology parameters
- For minimum execution time, minimize each term
  - Difficult: ***often pull against one another***

Example: Instruction Granularity

Execution time =
(instructions/program) * (seconds/cycle) * (cycles/instruction)

- **CISC** (Complex Instruction Set Computing) **ISAs**
  - Big heavyweight instructions (lots of work per instruction)
  + Low "insns/program"
  – Higher "cycles/insn" and "seconds/cycle"
    - We have the technology to get around this problem

- **RISC** (Reduced Instruction Set Computer) **ISAs**
  - Minimalist approach to an ISA: simple insns only
  + Low "cycles/insn" and "seconds/cycle"
  – Higher "insn/program", but hopefully not as much
    - Rely on compiler optimizations

Compiler Optimizations

- Primarily goal: reduce instruction count
  - Eliminate redundant computation, keep more things in registers
    + Registers are faster, fewer loads/stores
    – An ISA can make this difficult by having too few registers

- But also…
  - Reduce branches and jumps (later)
  - Reduce cache misses (later)
  - Reduce dependences between nearby insns (later)
    – An ISA can make this difficult by having implicit dependences

- How effective are these?
  + Can give 4X performance over unoptimized code
  – Collective wisdom of 40 years ("Proebsting's Law"): 4% per year
  - Funny but … shouldn't leave 4X performance on the table

Compatibility

- In many domains, ISA must remain compatible
  - IBM's 360/370 (the *first* "ISA family")
  - Another example: Intel's x86 and Microsoft Windows
    - x86 one of the worst designed ISAs EVER, but survives
- **Backward compatibility**
  - New processors supporting old programs
    - Can't drop features (**caution in adding new ISA features**)
    - Or, update software/OS to emulate dropped features (slow)
- **Forward (upward) compatibility**
  - Old processors supporting new programs
    - Include a "CPU ID" so the software can test of features
    - Add ISA hints by overloading no-ops (example: x86's PAUSE)
    - New firmware/software on old processors to emulate new insn

Translation and Virtual ISAs

- New compatibility interface: ISA + translation software
  - **Binary-translation**: transform static image, run native
  - **Emulation**: unmodified image, interpret each dynamic insn
    - Typically optimized with just-in-time (JIT) compilation
  - Examples: FX!32 (x86 on Alpha), Rosetta (PowerPC on x86)
  - Performance overheads reasonable (many advances over the years)

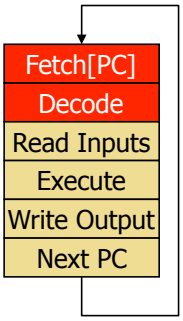
- **Virtual ISAs**: designed for translation, not direct execution
  - Target for high-level compiler (one per language)
  - Source for low-level translator (one per ISA)
  - Goals: Portability (abstract hardware nastiness), flexibility over time
  - Examples: Java Bytecodes, C# CLR (Common Language Runtime) NVIDIA's "PTX"

Ultimate Compatibility Trick

- Support old ISA by…
  - …having a simple processor for that ISA somewhere in the system
  - How did PlayStation2 support PlayStation1 games?
    - Used PlayStation processor for I/O chip **& emulation**

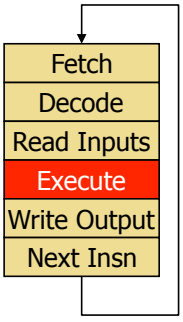
Aspects of ISAs

Length and Format



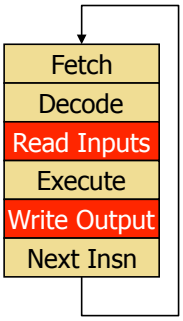
- **Length**
  - Fixed length
    - Most common is 32 bits
    - + Simple implementation (next PC often just PC+4)
    - – Code density: 32 bits to increment a register by 1
  - Variable length
    - + Code density
      - x86 averages 3 bytes (ranges from 1 to 16)
    - – Complex fetch (where does next instruction begin?)
  - Compromise: two lengths
    - E.g., MIPS16 or ARM's Thumb
- **Encoding**
  - A few simple encodings simplify decoder
    - x86 decoder one nasty piece of logic

Operations and Datatypes



- Datatypes
  - Software: attribute of data
  - Hardware: attribute of operation, data is just 0/1's
- All processors support
  - Integer arithmetic/logic (8/16/32/64-bit)
  - IEEE754 floating-point arithmetic (32/64-bit)
- More recently, most processors support
  - "Packed-integer" insns, e.g., MMX
  - "Packed-floating point" insns, e.g., SSE/SSE2/AVX
  - For "data parallelism", more about this later
- Other, infrequently supported, data types
  - Decimal, other fixed-point arithmetic

Where Does Data Live?



- **Registers**
  - "short term memory"
  - Faster than memory, quite handy
  - Named directly in instructions
- **Memory**
  - "longer term memory"
  - Accessed via "addressing modes"
    - Address to read or write calculated by instruction
- "Immediates"
  - Values spelled out as bits in instructions
  - Input only

How Many Registers?

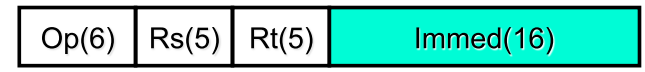
- Registers faster than memory, have as many as possible?
  - **No**
- One reason registers are faster: there are **fewer of them**
  - Small is fast (hardware truism)
- Another: they are **directly addressed** (no address calc)
  - More registers, means more bits per register in instruction
  - Thus, fewer registers per instruction or larger instructions
- **Not everything can be put in registers**
  - Structures, arrays, anything pointed-to
  - Although compilers are getting better at putting more things in
- More registers means **more saving/restoring**
  - Across function calls, traps, and context switches
- Trend toward more registers:
  - 8 (x86) → 16 (x86-64), 16 (ARM v7) → 32 (ARM v8)

Memory Addressing

- **Addressing mode:** way of specifying address
  - Used in memory-memory or load/store instructions in register ISA
- Examples
  - **Displacement:** R1=mem[R2+immed]
  - **Index-base:** R1=mem[R2+R3]
  - **Memory-indirect:** R1=mem[mem[R2]]
  - **Auto-increment:** R1=mem[R2], R2= R2+1
  - **Auto-indexing:** R1=mem[R2+immed], R2=R2+immed
  - **Scaled:** R1=mem[R2+R3*immed1+immed2]
  - **PC-relative:** R1=mem[PC+imm]
- What high-level program idioms are these used for?
- What implementation impact? What impact on insn count?

Addressing Modes Examples

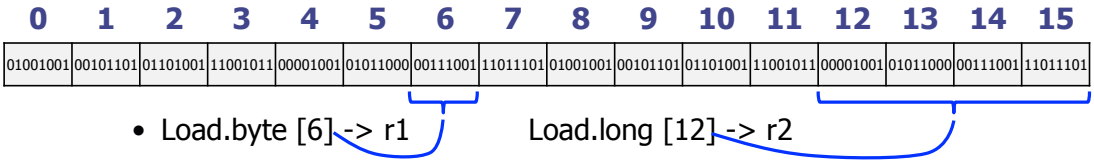
- MIPS



  - **Displacement**: R1+offset (16-bit)
  - Why? Experiments on VAX (ISA with every mode) found:
    - 80% use small displacement (or displacement of zero)
    - Only 1% accesses use displacement of more than 16bits
- Other ISAs (SPARC, x86) have reg+reg mode, too
  - Impacts both implementation and insn count? (How?)
- x86 (MOV instructions)
  - **Absolute**: zero + offset (8/16/32-bit)
  - **Register indirect**: R1
  - **Displacement**: R1+offset (8/16/32-bit)
  - **Indexed**: R1+R2
  - **Scaled:** R1 + (R2*Scale) + offset(8/16/32-bit) Scale = 1, 2, 4, 8

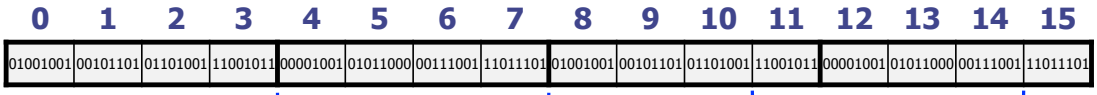
Access Granularity & Alignment

- **Byte addressability**
  - An address points to a byte (8 bits) of data
  - The ISA's minimum granularity to read or write memory
  - ISAs also support wider load/stores
    - "Half" (2 bytes), "Longs" (4 bytes), "Quads" (8 bytes)



- Load.byte [6] -> r1
- Load.long [12] -> r2

However, physical memory systems operate on **even larger chunks**



- Load.long [4] -> r1
- Load.long [11] -> r2
- **"unaligned"**

- **Access alignment**: if address % size is not 0, then it is "unaligned"
  - A single unaligned access may require multiple physical memory accesses

Handling Unaligned Accesses

- **Access alignment**: if address % size is not 0, then it is "unaligned"
  - A single unaligned access may require multiple physical memory accesses

- How do handle such unaligned accesses?
  1. Disallow (unaligned operations are considered illegal)
     - MIPS takes this route
  2. Support in hardware? (allow such operations)
     - x86 allows regular loads/stores to be unaligned
       - Unaligned access still slower, adds significant hardware complexity
  3. Trap to software routine? (allow, but hardware traps to software)
     - Simpler hardware, but high penalty when unaligned
  4. In software (compiler can use regular instructions when possibly unaligned
     - Load, shift, load, shift, and (slow, needs help from compiler)
  5. MIPS? ISA support: unaligned access by compiler using two instructions
     - Faster than above, but still needs help from compiler

```
lwl @XXXX10; lwr @XXXX10
```

Operand Model: Register or Memory?

- "Load/store" architectures
  - Memory access instructions (loads and stores) are distinct
  - Separate addition, subtraction, divide, etc. operations
  - Examples: MIPS, ARM, SPARC, PowerPC

- Alternative: mixed operand model (x86, VAX)
  - Operand can be from register **or** memory
  - x86 example: `addl 100, 4(%eax)`
    - 1. Loads from memory location [4 + %eax]
    - 2. Adds "100" to that value
    - 3. Stores to memory location [4 + %eax]
    - Would requires three instructions in MIPS, for example.

How Much Memory? Address Size

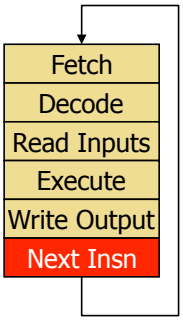
- What does "64-bit" in a 64-bit ISA mean?
  - **Each program can address (i.e., use) 2^{64} bytes**
  - 64 is the **address size**
  - Alternative (wrong) definition: width of arithmetic operations

- Most critical, inescapable ISA design decision
  - Too small? Will limit the lifetime of ISA
  - May require nasty hacks to overcome (E.g., x86 segments)

- x86 evolution:
  - 4-bit (4004), 8-bit (8008), 16-bit (8086), 24-bit (80286),
  - 32-bit + protected memory (80386)
  - 64-bit (AMD's Opteron & Intel's Pentium4)

- All ISAs moving to 64 bits (if not already there)

Control Transfers



- Default next-PC is PC + sizeof(current insn)
  - Branches and jumps can change that
- **Computing targets**: where to jump to
  - For all branches and jumps
  - PC-relative: for branches and jumps with function
  - Absolute: for function calls
  - Register indirect: for returns, switches & dynamic calls
- **Testing conditions**: whether to jump at all
  - Implicit condition codes or "flags" (x86)

```
cmp R1,10   // sets "negative" flag
branch-neg target
```

  - Use registers & separate branch insns (MIPS)

```
set-less-than R2,R1,10
branch-not-equal-zero R2,target
```


ISAs Also Include Support For…

- Function calling conventions
  - Which registers are saved across calls, how parameters are passed
- Operating systems & memory protection
  - Privileged mode
  - System call (TRAP)
  - Exceptions & interrupts
  - Interacting with I/O devices
- Multiprocessor support
  - "Atomic" operations for synchronization
- Data-level parallelism
  - Pack many values into a wide register
    - Intel's SSE2: four 32-bit float-point values into 128-bit register
  - Define parallel operations (four "adds" in one cycle)

The RISC vs. CISC Debate

RISC and CISC

- **RISC**: reduced-instruction set computer
  - Coined by Patterson in early 80's
  - RISC-I (Patterson), MIPS (Hennessy), IBM 801 (Cocke)
  - Examples: PowerPC, ARM, SPARC, Alpha, PA-RISC
- **CISC**: complex-instruction set computer
  - Term didn't exist before "RISC"
  - Examples: x86, VAX, Motorola 68000, etc.
- Philosophical war started in mid 1980's
  - RISC "won" the technology battles
  - CISC won the high-end commercial space (1990s to today)
    - Compatibility was a strong force
  - RISC winning the embedded computing space

CISCs and RISCs

- The CISCs: x86, VAX (**V**irtual **A**ddress e**X**tension to PDP-11)
  - Variable length instructions: 1-321 bytes!!!
  - 14 registers + PC + stack-pointer + condition codes
  - Data sizes: 8, 16, 32, 64, 128 bit, decimal, string
  - Memory-memory instructions for all data sizes
  - Special insns: `crc`, `insque`, `polyf`, and a cast of hundreds
  - x86: "Difficult to explain and impossible to love"
- The RISCs: MIPS, PA-RISC, SPARC, PowerPC, Alpha, ARM
  - 32-bit instructions
  - 32 integer registers, 32 floating point registers
    - ARM has 16 registers
  - Load/store architectures with few addressing modes
  - Why so many basically similar ISAs? Everyone wanted their own

Historical Development

- Pre 1980
  - Bad compilers (so assembly written by hand)
  - Complex, high-level ISAs (**easier to write assembly**)
  - Slow multi-chip micro-programmed implementations
    - Vicious feedback loop
- Around 1982
  - Moore's Law makes single-chip microprocessor possible…
    - **…but only for small, simple ISAs**
  - Performance advantage of this "integration" was compelling
- **RISC manifesto**: create ISAs that…
  - **Simplify single-chip implementation**
  - **Facilitate optimizing compilation**

The RISC Design Tenets

- **Single-cycle execution**
  - CISC: many multicycle operations
- **Hardwired (simple) control**
  - CISC: "microcode" for multi-cycle operations
- **Load/store architecture**
  - CISC: register-memory and memory-memory
- **Few memory addressing modes**
  - CISC: many modes
- **Fixed-length instruction format**
  - CISC: many formats and lengths
- **Reliance on compiler optimizations**
  - CISC: hand assemble to get good performance
- **Many registers** (compilers can use them effectively)
  - CISC: few registers

RISC vs CISC Performance Argument

- Performance equation:
  - (instructions/program) * (cycles/instruction) * (seconds/cycle)

- **CISC** (Complex Instruction Set Computing)
  - Reduce "instructions/program" with "complex" instructions
    - But tends to increase "cycles/instruction" or clock period
  - Easy for assembly-level programmers, good code density
- **RISC** (Reduced Instruction Set Computing)
  - Improve "cycles/instruction" with many single-cycle instructions
  - Increases "instruction/program", but hopefully not as much
    - **Help from smart compiler**
  - Perhaps improve clock cycle time (seconds/cycle)
    - **via aggressive implementation allowed by simpler insn**

The Debate

- RISC argument
  - CISC is fundamentally handicapped
  - For a given technology, RISC implementation will be better (faster)
    - Current technology enables single-chip RISC
    - When it enables single-chip CISC, RISC will be pipelined
    - When it enables pipelined CISC, RISC will have caches
    - When it enables CISC with caches, RISC will have next thing...

- CISC rebuttal
  - CISC flaws not fundamental, can be fixed with more transistors
  - Moore's Law will narrow the RISC/CISC gap (true)
    - Good pipeline: RISC = 100K transistors, CISC = 300K
    - By 1995: 2M+ transistors had evened playing field
  - Software costs dominate, **compatibility** is paramount

Intel's x86 Trick: RISC Inside

- 1993: Intel wanted "out-of-order execution" in Pentium Pro
  - Hard to do with a coarse grain ISA like x86
- Solution? Translate x86 to RISC micro-ops **(μops)** in hardware

  ```
  push $eax
  ```
  becomes (we think, uops are proprietary)
  ```
  store $eax, -4($esp)
  addi $esp,$esp,-4
  ```
  + Processor maintains **x86 ISA externally for compatibility**
  + But executes **RISC μISA internally for implementability**
  - Given translator, x86 almost as easy to implement as RISC
    - Intel implemented "out-of-order" before any RISC company
    - "out-of-order" also helps x86 more (because ISA limits compiler)
  - Also used by other x86 implementations (AMD)
- Different **μops** for different designs
  - **Not part of the ISA specification**, not publically disclosed

Potential Micro-op Scheme

- Most instructions are a **single** micro-op
  - Add, xor, compare, branch, etc.
  - Loads example: mov -4(%rax), %ebx
  - Stores example: mov %ebx, -4(%rax)
- Each memory access adds a micro-op
  - "addl -4(%rax), %ebx" is two micro-ops (load, add)
  - "addl %ebx, -4(%rax)" is three micro-ops (load, add, store)
- Function call (CALL) – 4 uops
  - Get program counter, store program counter to stack, adjust stack pointer, unconditional jump to function start
- Return from function (RET) – 3 uops
  - Adjust stack pointer, load return address from stack, jump register
- Again, just a basic idea, micro-ops are specific to each chip

Winner for Desktop PCs: CISC

- x86 was first mainstream 16-bit microprocessor by ~2 years
  - IBM put it into its PCs…
  - Rest is historical inertia, Moore's law, and "financial feedback"
    - x86 is most difficult ISA to implement and do it fast but…
    - Because Intel sells the most **non-embedded** processors…
    - It hires more and better engineers…
    - Which help it maintain competitive performance …
    - **And given competitive performance, compatibility wins…**
    - So Intel sells the most **non-embedded** processors…
  - AMD as a competitor keeps pressure on x86 performance
- Moore's Law has helped Intel in a big way
  - Most engineering problems can be solved with more transistors

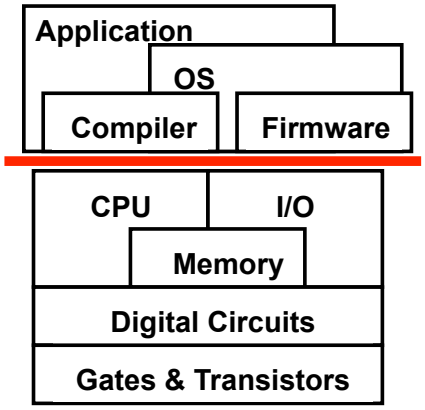
Winner for Embedded: RISC

- ARM (Acorn RISC Machine → Advanced RISC Machine)
  - First ARM chip in mid-1980s (from Acorn Computer Ltd).
  - 3 billion units sold in 2009 (>60% of all 32/64-bit CPUs)
  - Low-power and **embedded** devices (phones, for example)
    - Significance of embedded? ISA Compatibility less powerful force
- 32-bit RISC ISA
  - 16 registers, PC is one of them
  - Rich addressing modes, e.g., auto increment
  - Condition codes, each instruction can be conditional
- ARM does not sell chips; it licenses its ISA & core designs
- ARM chips from many vendors
  - Qualcomm, Freescale (was Motorola), Texas Instruments, STMicroelectronics, Samsung, Sharp, Philips, etc.

Redux: Are ISAs Important?

- Does "quality" of ISA actually matter?
  - Not for performance (mostly)
    - Mostly comes as a design complexity issue
    - Insn/program: everything is compiled, compilers are good
    - Cycles/insn and seconds/cycle: μISA, many other tricks
  - What about power efficiency? ***Maybe***
    - ARMs are most power efficient today…
      - …but Intel is moving x86 that way (e.g, Intel's Atom)
    - **Open question: can x86 be as power efficient as ARM?**
- Does "nastiness" of ISA matter?
  - Mostly no, only compiler writers and hardware designers see it
- Even compatibility is not what it used to be
  - Software emulation
  - **Open question: will "ARM compatibility" be the next x86?**

Instruction Set Architecture (ISA)



- What is an ISA?
  - A functional contract
- All ISAs similar in high-level ways
  - But many design choices in details
  - Two "philosophies": CISC/RISC
    - Difference is blurring
- Good ISA…
  - Enables high-performance
  - At least doesn't get in the way
- Compatibility is a powerful force
  - Tricks: binary translation, μISAs

[spacer]

Computer Architecture

Unit 3: Technology & Energy

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

This Unit: Technology & Energy

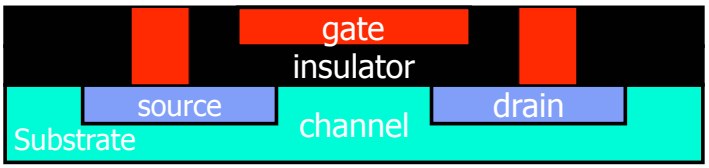
- Technology basis
  - Fabrication (manufacturing) & cost
  - Transistors & wires
  - Implications of transistor scaling (Moore's Law)
- Energy & power

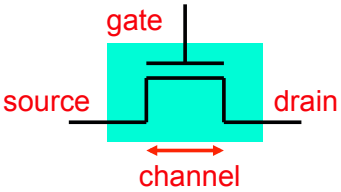
Readings

- Assigned reading:
  - G. Moore, "Cramming More Components onto Integrated Circuits"

Technology & Fabrication

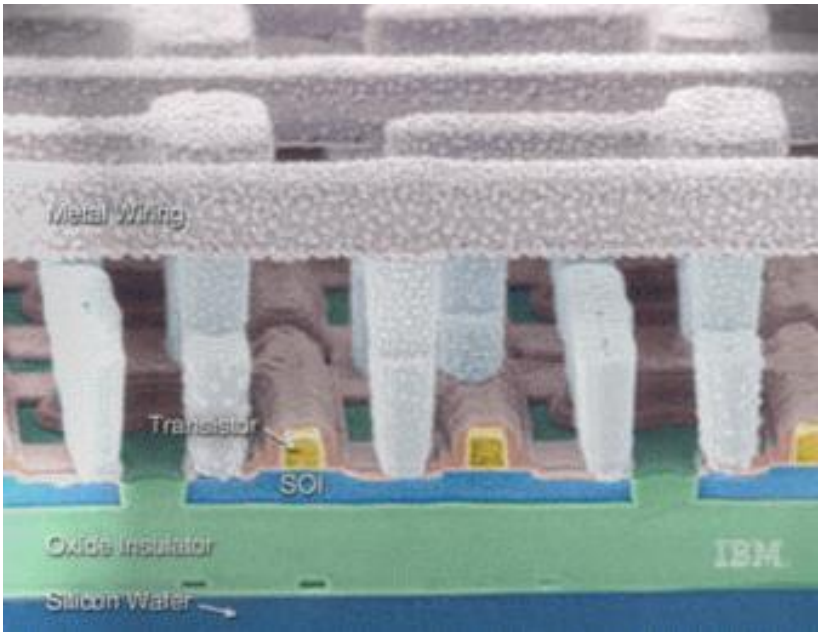
Semiconductor Technology





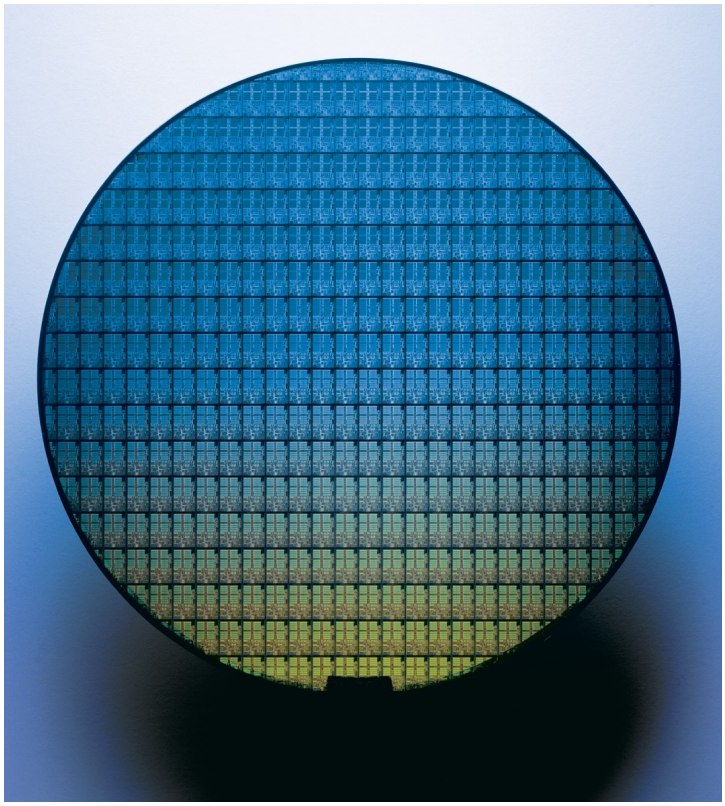
- Basic technology element: **MOSFET**
  - Solid-state component acts like electrical switch
  - **MOS**: metal-oxide-semiconductor
    - Conductor, insulator, semi-conductor
- **FET**: field-effect transistor
  - Channel conducts source→drain only when voltage applied to gate
- **Channel length**: characteristic parameter (short → fast)
  - Aka "feature size" or "technology"
  - Currently: 0.022 micron (μm), 22 nanometers (nm)
  - Continued miniaturization (scaling) known as "**Moore's Law**"
    - Won't last forever, physical limits approaching (or are they?)

Transistors and Wires



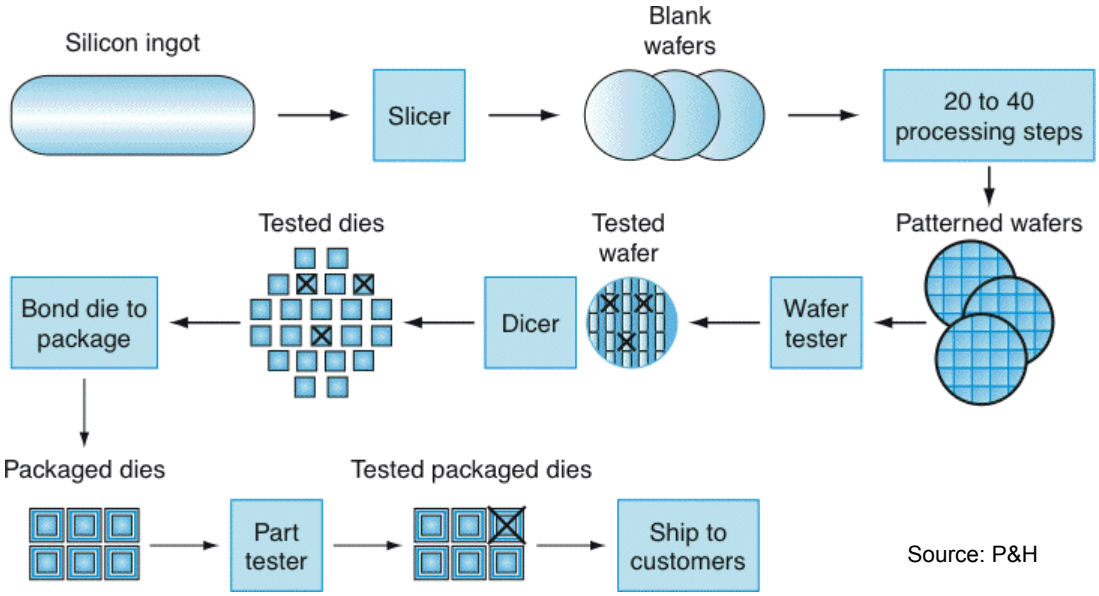
©IBM

From slides © Krste Asanović, MIT



Intel
Pentium M
Wafer

Manufacturing Steps

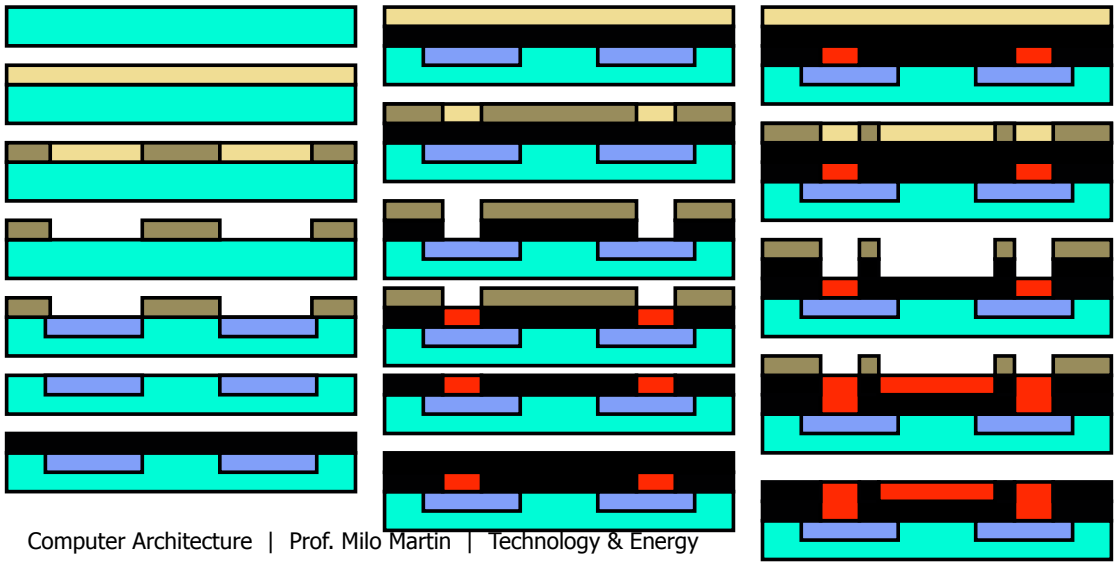


Source: P&H

Manufacturing Steps

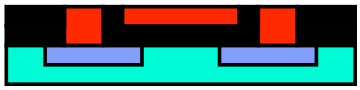
- Multi-step photo-/electro-chemical process
  - More steps, higher unit cost

+ Fixed cost mass production ($1 million+ for "mask set")

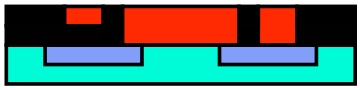


Manufacturing Defects

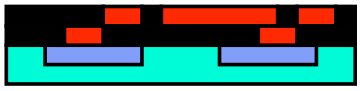
Correct:



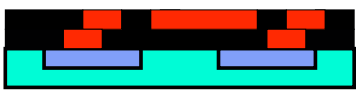
Defective:



Defective:



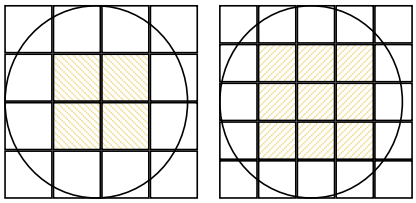
Slow:

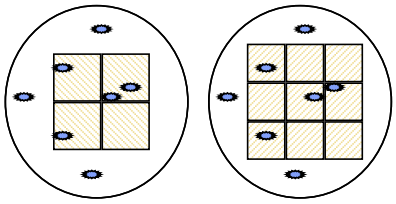


- Defects can arise
  - Under-/over-doping
  - Over-/under-dissolved insulator
  - Mask mis-alignment
  - Particle contaminants
- Try to minimize defects
  - Process margins
  - Design rules
    - Minimal transistor size, separation
- Or, tolerate defects
  - Redundant or "spare" memory cells
  - Can substantially improve yield

Cost Implications of Defects

- Chips built in multi-step chemical processes on **wafers**
  - Cost / wafer is constant, f(wafer size, number of steps)
- Chip (die) cost is related to **area**
  - Larger chips means fewer of them
- Cost is **more than** linear in area
  - Why? random defects
  - Larger chip, more chance of defect
    - Result: lower "yield" (fewer working chips)





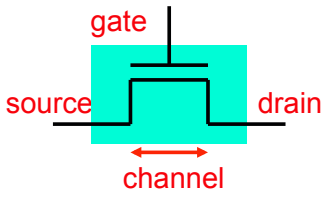
- **Wafer yield**: % wafer that is chips
- **Die yield**: % chips that work
- Yield is increasingly non-binary - fast vs slow chips

Manufacturing Cost

- **Chip cost vs system cost**
  - Cost of memory, storage, display, battery, etc.
- **Cost vs price**
  - Microprocessors not commodities; relationship complicated
  - Specialization, compatibility, different cost/performance/power
  - Economies of scale
- **Unit costs**: die manufacturing, testing, packaging, burn-in
  - Die cost based on area & defect rate (yield)
  - Package cost related to heat dissipation & number of pins
- **Fixed costs**: design & verification, fab cost
  - Amortized over "proliferations", e.g., Core i3, i5, i7 variants
  - Building new "fab" costs billions of dollars today
  - Both getting worse; trend toward "foundry" & "fabless" models

Technology Scaling Trends

Moore's Law: Technology Scaling



- **Moore's Law**: aka "technology scaling"
  - Continued miniaturization (esp. reduction in channel length)
  - \+ Improves switching speed, power/transistor, area(cost)/transistor
  - – Reduces transistor reliability
  - Literally: DRAM density (transistors/area) doubles every 18 months
  - Public interpretation: performance doubles every 18 months
    - Not quite right, but helps performance in several ways…

Moore's Effect #1: Transistor Count

- Linear shrink in each dimension
  - 180nm, 130nm, 90nm, 65nm, 45nm, 32nm, 22nm, …
  - Each generation is a 1.414 linear shrink
    - Shrink each dimension (2D)
  - Results in 2x more transistors (1.414*1.414) per area
- Generally reduces cost per transistor
- More transistors can increase performance
  - Job of a computer architect: use the ever-increasing number of transistors
  - Today, desktop/laptop processor chips have ~1 billion transistors

Moore's Effect #2: Delay

- **First-order: speed scales proportional to gate length**
  - Has provided much of the performance gains in the past
- Delay is proportional to: capacitance * resistance
- Scaling helps wire and gate delays in some ways…
  - \+ Transistors become shorter (Resistance↓), narrower (Capacitance↓)
  - \+ Wires become shorter (Length↓ → Resistance↓)
- Hurts in others…
  - – Transistors become narrower (Resistance↑)
  - – Gate insulator thickness becomes smaller (Capacitance↑)
  - – Wires becomes thinner (Resistance↑)
- What to do?
  - Take the good, use wire/transistor sizing to counter the bad
  - Exploit new materials: Aluminum → Copper, metal gate, high-K

Moore's Effect #3: Cost

- Mixed impact on unit integrated circuit cost
  - + Either lower cost for same functionality…
  - + Or same cost for more functionality
  - – Difficult to achieve high yields

- – Increases startup cost
  - More expensive fabrication equipment
  - Takes longer to design, verify, and test chips

- – Process variation across chip increasing
  - Some transistors slow, some fast
  - Increasingly active research area: dealing with this problem

Moore's Effect #4: Psychological

- **Moore's Curve**: common interpretation of Moore's Law
  - "CPU performance doubles every 18 months"
  - Self fulfilling prophecy: 2X every 18 months is ~1% per week
    - Q: Would you add a feature that improved performance 20% if it would delay the chip 8 months?
  - Processors under Moore's Curve (arrive too late) fail spectacularly
    - E.g., Intel's Itanium, Sun's Millennium

Moore's Law in the Future

- Won't last forever, approaching physical limits
  - "If something must eventually stop, it can't go on forever"
  - But betting against it has proved foolish in the past
  - Perhaps will "slow" rather than stop abruptly

- Transistor count will likely continue to scale
  - "Die stacking" is on the cusp of becoming main stream
  - Uses the third dimension to increase transistor count

- But transistor performance scaling?
  - Running into physical limits
  - Example: gate oxide is less than 10 silicon atoms thick!
    - Can't decrease it much further
  - Power is becoming the limiting factor

Power & Energy

Power/Energy Are Increasingly Important

- **Battery life** for mobile devices
  - Laptops, phones, cameras
- **Tolerable temperature** for devices without active cooling
  - Power means temperature, active cooling means **cost**
  - No room for a fan in a cell phone, no market for a hot cell phone
- **Electric bill** for compute/data centers
  - Pay for power twice: once in, once out (to cool)
- **Environmental concerns**
  - "Computers" account for growing fraction of energy consumption

Energy & Power

- **Energy**: measured in Joules or Watt-seconds
  - Total amount of energy stored/used
  - Battery life, electric bill, environmental impact
  - Instructions per Joule (car analogy: miles per gallon)
- **Power**: energy per unit time (measured in Watts)
  - Related to "performance" (which is also a "per unit time" metric)
  - Power impacts power supply and cooling requirements (cost)
    - Power-density (Watt/mm^2): important related metric
  - Peak power vs average power
    - E.g., camera, power "spikes" when you actually take a picture
  - Joules per second (car analogy: gallons per hour)
- Two sources:
  - **Dynamic power**: active switching of transistors
  - **Static power**: leakage of transistors even while inactive

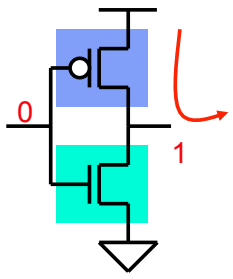
Technology Basis of Transistor Speed

- Physics 101: delay through an electrical component ~ **RC**
  - **Resistance (R)** ~ length / cross-section area
    - Slows rate of charge flow
  - **Capacitance (C)** ~ length * area / distance-to-other-plate
    - Stores charge
  - **Voltage (V)**
    - Electrical pressure
  - **Threshold Voltage (V_t)**
    - Voltage at which a transistor turns "on"
    - Property of transistor based on fabrication technology
  - **Switching time ~ to (R * C) / (V – V_t)**

- Components contribute to capacitance & resistance
  - Transistors
  - Wires (longer the wire, the more the capacitance & resistance)

Dynamic Power

- **Dynamic power ($P_{dynamic}$)**: aka switching or active power
  - Energy to switch a gate (0 to 1, 1 to 0)
  - Each gate has capacitance (C)
    - Charge stored is ∝ C * V
    - Energy to charge/discharge a capacitor is ∝ to C * V^2
    - Time to charge/discharge a capacitor is ∝ to V
      - Result: frequency ~ to V
  - **$P_{dynamic}$ ≈ N * C * V^2 * f * A**
    - N: number of transistors
    - C: capacitance per transistor (size of transistors)
    - V: voltage (supply voltage for gate)
    - f: frequency (transistor switching freq. is ∝ to clock freq.)
    - A: activity factor (not all transistors may switch this cycle)

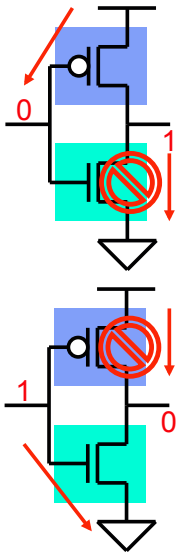


Reducing Dynamic Power

- Target each component: $\mathbf{P_{dynamic} \approx N * C * V^2 * f * A}$
- **Reduce number of transistors** (N)
  - Use fewer transistors/gates
- **Reduce capacitance** (C)
  - Smaller transistors (Moore's law)
- **Reduce voltage** (V)
  - Quadratic reduction in energy consumption!
  - But also slows transistors (transistor speed is ~ to V)
- **Reduce frequency** (f)
  - Slower clock frequency (reduces power but not energy) Why?
- **Reduce activity** (A)
  - "Clock gating" disable clocks to unused parts of chip
  - Don't switch gates unnecessarily

Static Power

- **Static power (P_{static})**: aka idle or leakage power
  - Transistors don't turn off all the way
  - Transistors "leak"
    - Analogy: leaky valve
  - $\mathbf{P_{static} \approx N * V * e^{-V_t}}$
  - N: number of transistors
  - V: voltage
  - **V_t (threshold voltage)**: voltage at which transistor conducts (begins to switch)
- Switching speed vs leakage trade-off
- The lower the **V_t**:
  - Faster transistors (linear)
    - Transistor speed $\propto$ to $V - V_t$
  - Leakier transistors (exponential)



Reducing Static Power

- Target each component: $P_{static} \approx N * V * e^{-Vt}$
- **Reduce number of transistors** (N)
  - Use fewer transistors/gates
- **Disable transistors** (also targets N)
  - "Power gating" disable power to unused parts (long latency to power up)
  - Power down units (or entire cores) not being used
- **Reduce voltage** (V)
  - Linear reduction in static energy consumption
  - But also slows transistors (transistor speed is ~ to V)
- **Dual V_t** – use a mixture of high and low V_t transistors
  - Use slow, low-leak transistors in SRAM arrays
  - Requires extra fabrication steps (cost)
- **Low-leakage transistors**
  - High-K/Metal-Gates in Intel's 45nm process, "tri-gate" in Intel's 22nm
- Reducing frequency can hurt energy efficiency due to leakage power

Continuation of Moore's Law

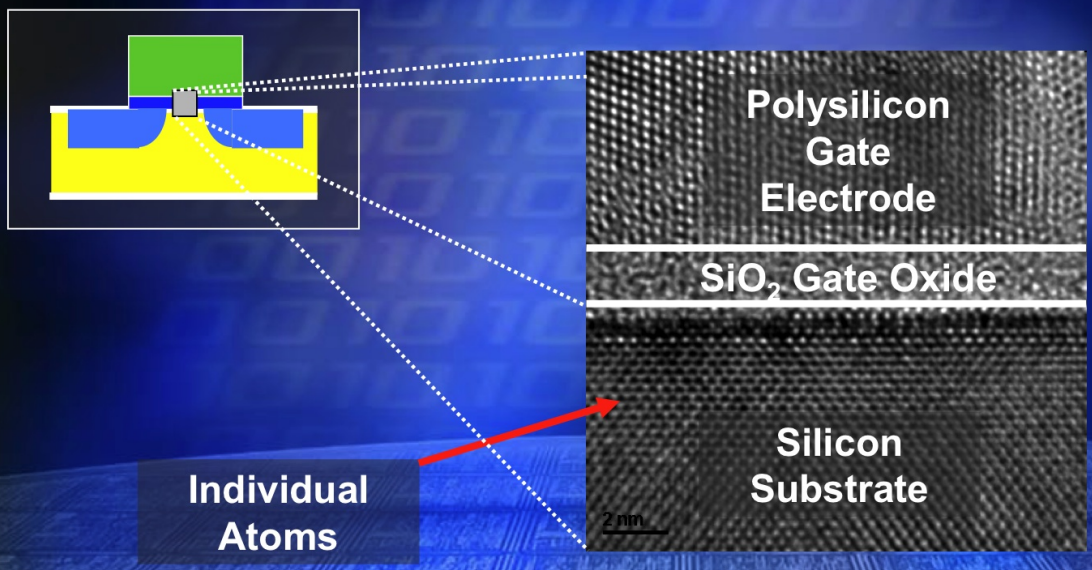
Process Name	P856	P858	Px60	P1262	P1264	P1266	P1268	P1270
1st Production	1997	1999	2001	2003	2005	2007	2009	2011
Process Generation	0.25μm	0.18μm	0.13μm	90 nm	65 nm	45 nm	32 nm	22 nm
Wafer Size (mm)	200	200	200/300	300	300	300	300	300
Inter-connect	Al	Al	Cu	Cu	Cu	Cu	Cu	?
Channel	Si	Si	Si	Strained Si	Strained Si	Strained Si	Strained Si	Strained Si
Gate dielectric	SiO_2	SiO_2	SiO_2	SiO_2	SiO_2	High-k	High-k	High-k
Gate electrode	Poly-silicon	Poly-silicon	Poly-silicon	Poly-silicon	Poly-silicon	Metal	Metal	Metal

Introduction targeted at this time

Subject to change

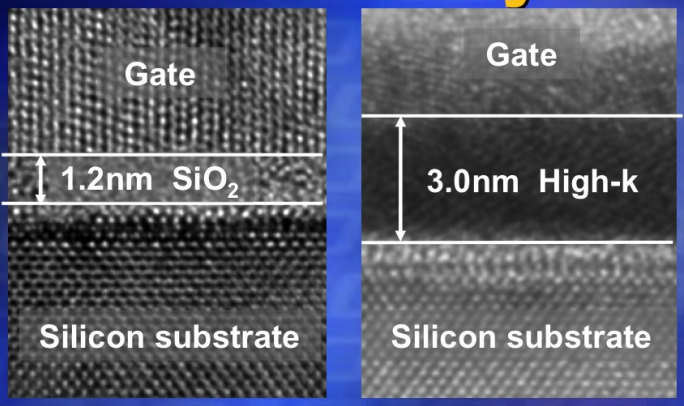
Intel found a solution for High-k and metal gate

Gate dielectric today is only a few molecular layers thick



intel

High-k Dielectric reduces leakage substantially



Benefits compared to current process technologies

	High-k vs. SiO_2	Benefit
Capacitance	60% greater	*Much faster transistors*
Gate dielectric leakage	> 100x reduction	*Far cooler*

intel

Dynamic Voltage/Frequency Scaling

- **Dynamically trade-off power for performance**
  - Change the voltage and frequency at runtime
  - Under control of operating system
- Recall: $P_{dynamic} \approx N * C * \mathbf{V^2} * \mathbf{f} * A$
  - Because frequency $\propto$ to $V - V_t$...
  - $P_{dynamic} \propto$ to $\mathbf{V^2(V - V_t)} \approx \mathbf{V^3}$
- Reduce both voltage and frequency linearly
  - **Cubic decrease in dynamic power**
  - Linear decrease in performance (actually sub-linear)
    - Thus, only about quadratic in energy
  - Linear decrease in static power
    - Thus, static energy can become dominant
- Newer chips can adjust frequency on a per-core basis

Dynamic Voltage/Frequency Scaling

	Mobile PentiumIII "**SpeedStep**"	Transmeta 5400 "LongRun"	Intel X-Scale (StrongARM2)
f (MHz)	300–1000 (step=50)	200–700 (step=33)	50–800 (step=50)
V (V)	0.9–1.7 (step=0.1)	1.1–1.6V (cont)	0.7–1.65 (cont)
High-speed	3400MIPS @ 34W	1600MIPS @ 2W	800MIPS @ 0.9W
Low-power	1100MIPS @ 4.5W	300MIPS @ 0.25W	62MIPS @ 0.01W

- Dynamic voltage/frequency scaling
  - **Favors parallelism**
- Example: Intel Xscale
  - 1 GHz → 200 MHz reduces energy used by 30x
    - But around 5x slower
  - 5 x 200 MHz in parallel, use **1/6th the energy**
  - Power is driving the trend toward multi-core

Moore's Effect on Power

+ Moore's Law reduces power/transistor…
  - Reduced sizes and surface areas reduce capacitance (C)
– …but increases power density and total power
  - By increasing transistors/area and total transistors
  - Faster transistors → higher frequency → more power
  - Hotter transistors leak more (thermal runaway)
- What to do? Reduce voltage (V)
  + Reduces dynamic power quadratically, static power linearly
    - Already happening: Intel 486 (5V) → Core2 (1.3V)
  - Trade-off: reducing V means either…
    – Keeping V_t the same and reducing frequency (f)
    – Lowering V_t and increasing leakage exponentially
  - Use techniques like high-K and dual-V_T
- The end of voltage scaling & "dark silicon"

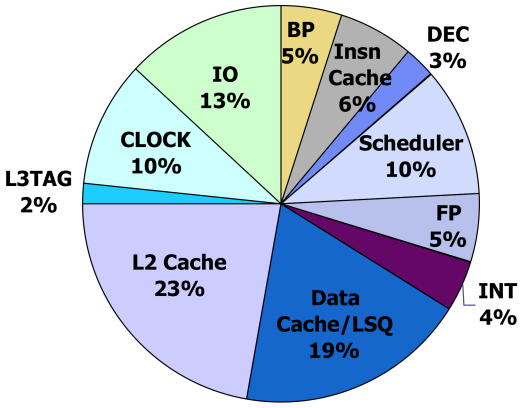
Trends in Power

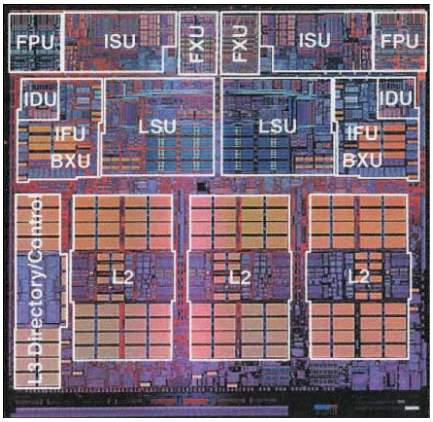
	386	486	Pentium	Pentium II	Pentium4	Core2	Core i7
Year	1985	1989	1993	1998	2001	2006	2009
Technode (nm)	1500	800	350	180	130	65	45
Transistors (M)	0.3	1.2	3.1	5.5	42	291	731
Voltage (V)	**5**	**5**	**3.3**	**2.9**	**1.7**	**1.3**	**1.2**
Clock (MHz)	16	25	66	200	1500	3000	3300
Power (W)	1	5	16	35	**80**	**75**	**130**
Peak MIPS	6	25	132	600	4500	24000	52800
MIPS/W	6	5	8	17	56	320	406

- Supply voltage decreasing over time
  - But "voltage scaling" is perhaps reaching its limits
- Emphasis on power starting around 2000
  - Resulting in slower frequency increases
  - Also note number of cores increasing (2 in Core 2, 4 in Core i7)

Processor Power Breakdown

- Power breakdown for IBM POWER4
  - Two 4-way superscalar, 2-way multi-threaded cores, 1.5MB L2
  - Big power components are L2, data cache, scheduler, clock, I/O
  - Implications on "complicated" versus "simple" cores



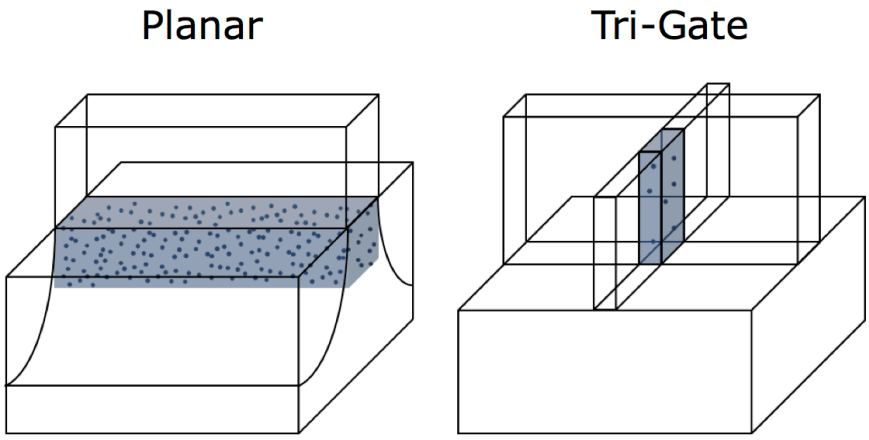


Implications on Software

- Software-controlled dynamic voltage/frequency scaling
  - Example: video decoding
    - Too high a clock frequency – wasted energy (battery life)
    - Too low a clock frequency – quality of video suffers
  - "Race to sleep" versus "slow and steady" approaches
- Managing low-power modes
  - Don't want to "wake up" the processor every millisecond
- Tuning software
  - Faster algorithms can be converted to lower-power algorithms
  - Via dynamic voltage/frequency scaling
- Exploiting parallelism & heterogeneous cores
  - NVIDIA Tegra 3: 5 cores (4 "normal" cores & 1 "low power" core)
- Specialized hardware accelerators

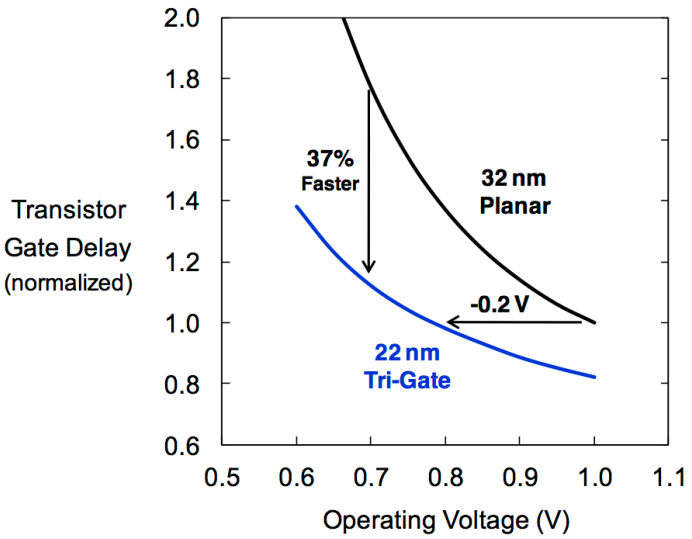
Recent Technology Update From Intel

Reduced Channel Doping



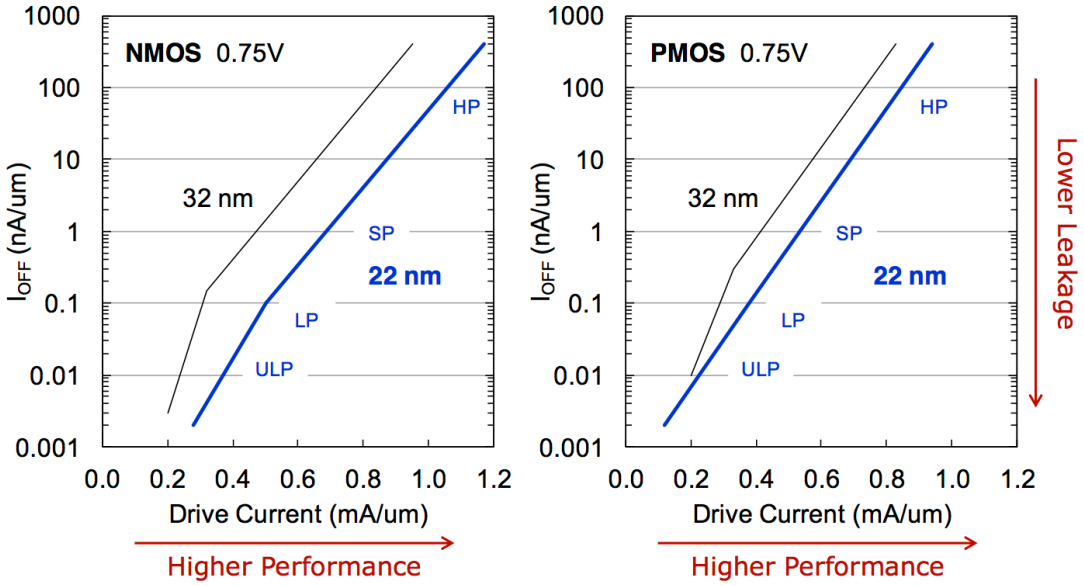
Fully depleted Tri-Gate structure has reduced channel doping, providing improved performance and reduced variability

Performance/Power Benefits



Tri-Gate provides 37% speed up at low voltage or 50% active power reduction at same performance

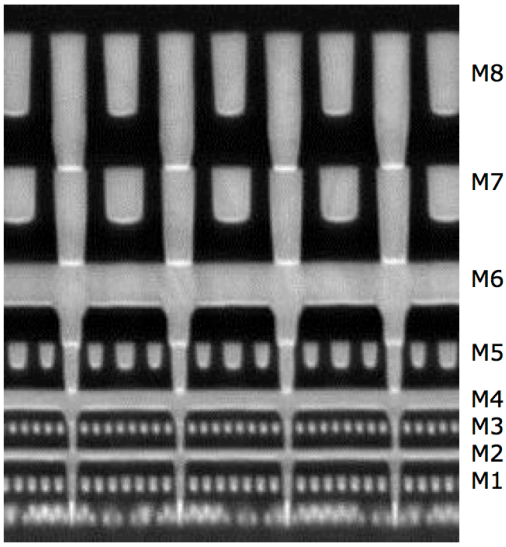
Transistor Performance vs. Leakage



22 nm SoC technology offers a wide range of transistors

22 nm Interconnects

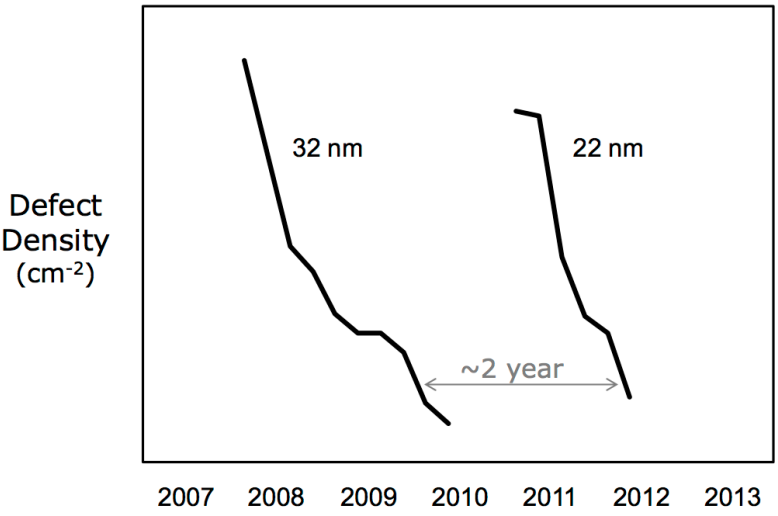
Layer	Pitch
TM	14 um
M8	360 nm
M7	320 nm
M6	240 nm
M5	160 nm
M4	112 nm
M3	80 nm
M2	80 nm
M1	90 nm



Minimum pitch scaled ~0.7x from 32 nm for ~2x transistor density improvement

IDF2012 INTEL DEVELOPER FORUM

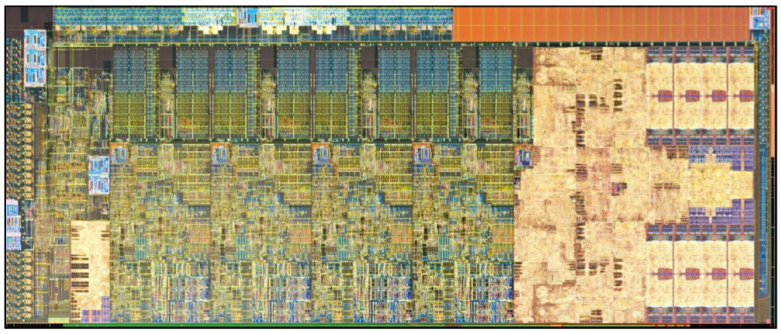
22 nm Defect Density Trend



22 nm defect density now at low levels needed for volume manufacturing

IDF2012 INTEL DEVELOPER FORUM

3RD Generation Intel® Core™ Processor



22 nm Tri-Gate Technology

4 Cores + Integrated Graphics

1.4 Billion Transistors, 160 mm^2

IDF2012
INTEL DEVELOPER FORUM

27

Summary

Technology Summary

- Has a first-order impact on computer architecture
  - Performance (transistor delay, wire delay)
  - Cost (die area & defects)
  - **Changing rapidly**
- Most significant trends for architects
  - More and more transistors
    - What to do with them? → integration → **parallelism**
  - Logic is improving faster than memory & cross-chip wires
    - "Memory wall" → caches, more integration

  } Rest of course
- Power and energy
  - Voltage vs frequency, parallelism, special-purpose hardware
- This unit: a quick overview, just scratching the surface

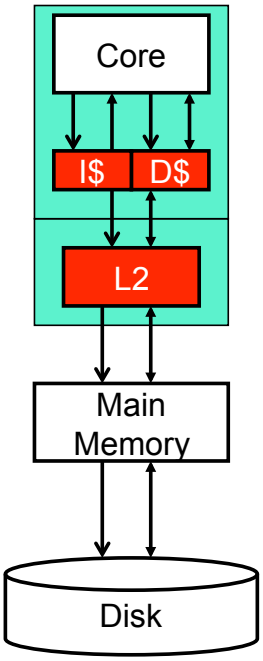
[spacer]

Computer Architecture

Unit 4: Caches

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

This Unit: Caches



- "Cache": hardware managed
  - Hardware automatically retrieves missing data
  - Built from fast on-chip SRAM
  - In contrast to off-chip, DRAM "main memory"
- **Average access time** of a memory component
  - $latency_{avg} = latency_{hit} + (\%_{miss} * latency_{miss})$
  - Hard to get low $latency_{hit}$ and $\%_{miss}$ in one structure → memory hierarchy
- Cache ABCs (**associativity, block size, capacity**)
- **Performance optimizations**
  - Prefetching & data restructuring
- **Handling writes**
  - Write-back vs. write-through
- **Memory hierarchy**
  - Smaller, faster, expensive → bigger, slower, cheaper

Reading

- Assigned Reading
  - "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers", by Norm Jouppi, ISCA 1990

Motivation

- Processor can compute only as fast as memory
  - A 3Ghz processor can execute an "add" operation in 0.33ns
  - Today's "main memory" latency is more than 33ns
  - Naïve implementation:
    - loads/stores can be 100x slower than other operations
- Unobtainable goal:
  - Memory that operates at processor speeds
  - Memory as large as needed for all running programs
  - Memory that is cost effective
- Can't achieve all of these goals at once
  - Example: latency of an SRAM is at least: sqrt(number of bits)

Memories (SRAM & DRAM)

Types of Memory

- **Static RAM (SRAM)**
  - 6 or 8 transistors per bit
    - Two inverters (4 transistors) + transistors for reading/writing
  - Optimized for speed (first) and density (second)
  - Fast (sub-nanosecond latencies for small SRAM)
    - Speed roughly proportional to its area (~ sqrt(number of bits))
  - Mixes well with standard processor logic

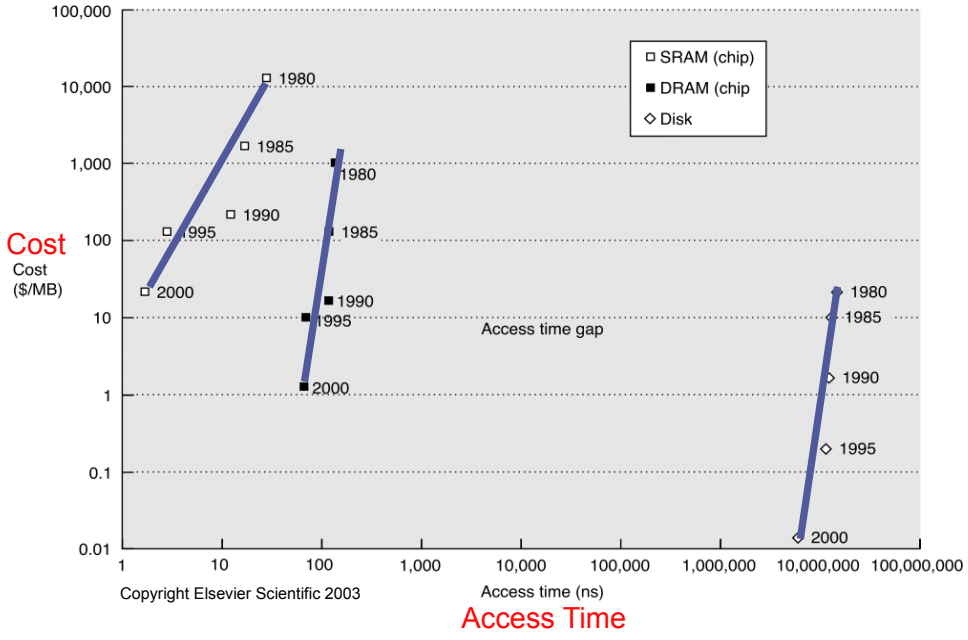
- **Dynamic RAM (DRAM)**
  - 1 transistor + 1 capacitor per bit
  - Optimized for density (in terms of cost per bit)
  - Slow (>30ns internal access, ~50ns pin-to-pin)
  - Different fabrication steps (does not mix well with logic)

- Nonvolatile storage: Magnetic disk, Flash RAM

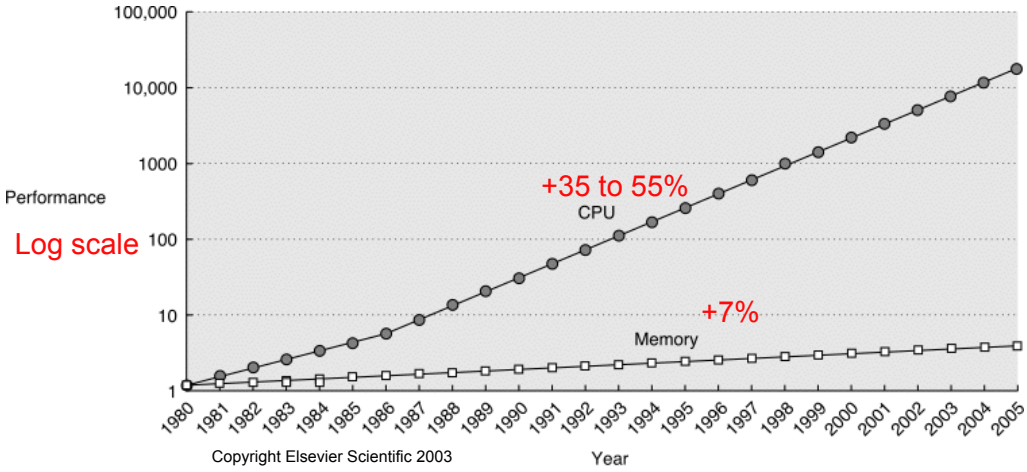
Memory & Storage Technologies

- **Cost** - what can $200 buy (2009)?
  - SRAM: 16MB
  - DRAM: 4,000MB (4GB) – 250x cheaper than SRAM
  - Flash: 64,000MB (64GB) – 16x cheaper than DRAM
  - Disk: 2,000,000MB (2TB) – 32x vs. Flash (512x vs. DRAM)
- **Latency**
  - SRAM: <1 to 2ns (on chip)
  - DRAM: ~50ns – 100x or more slower than SRAM
  - Flash: 75,000ns (75 microseconds) – 1500x vs. DRAM
  - Disk: 10,000,000ns (10ms) – 133x vs Flash (200,000x vs DRAM)
- **Bandwidth**
  - SRAM: 300GB/sec (e.g., 12-port 8-byte register file @ 3Ghz)
  - DRAM: ~25GB/s
  - Flash: 0.25GB/s (250MB/s), 100x less than DRAM
  - Disk: 0.1 GB/s (100MB/s), 250x vs DRAM, **sequential** access only

Memory Technology Trends



The "Memory Wall"



- Processors get faster more quickly than memory (note log scale)
  - Processor speed improvement: 35% to 55%
  - Memory latency improvement: 7%

The Memory Hierarchy

Known From the Beginning

"Ideally, one would desire an infinitely large memory capacity such that any particular word would be immediately available … We are forced to recognize the possibility of constructing a hierarchy of memories, each of which has a greater capacity than the preceding but which is less quickly accessible."

Burks, Goldstine, VonNeumann

"Preliminary discussion of the logical design of an electronic computing instrument"

IAS memo 1946

Big Observation: Locality & Caching

- **Locality of memory references**
  - Empirical property of real-world programs, few exceptions
- **Temporal locality**
  - Recently referenced data is likely to be referenced again soon
  - **Reactive**: "cache" recently used data in small, fast memory
- **Spatial locality**
  - More likely to reference data near recently referenced data
  - **Proactive**: "cache" large chunks of data to include nearby data
- Both properties hold for data and instructions
- Cache: "Hashtable" of recently used blocks of data
  - In hardware, finite-sized, transparent to software

Spatial and Temporal Locality Example

- Which memory accesses demonstrate spatial locality?
- Which memory accesses demonstrate temporal locality?

```
int sum = 0;
int x[1000];

for(int c = 0; c < 1000; c++){
  sum += c;

  x[c] = 0;
}
```


Library Analogy

- Consider books in a library

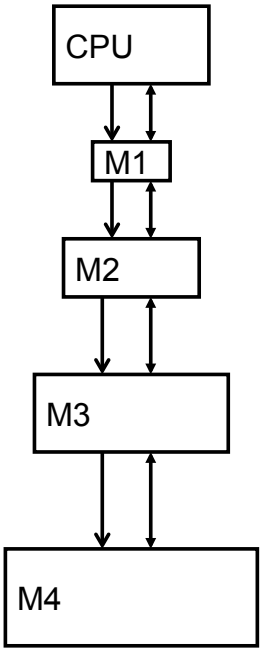
- Library has lots of books, but it is slow to access
  - Far away (time to walk to the library)
  - Big (time to walk within the library)

- How can you avoid these latencies?
  - Check out books, take them home with you
    - Put them on desk, on bookshelf, etc.
  - But desks & bookshelves have limited capacity
    - Keep recently used books around (**temporal locality**)
    - Grab books on related topic at the same time (**spatial locality**)
    - Guess what books you'll need in the future (prefetching)

Library Analogy Explained

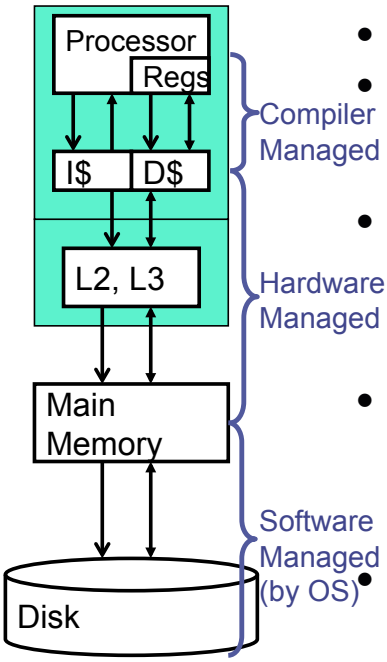
- Registers ↔ books on your desk
  - Actively being used, small capacity
- Caches ↔ bookshelves
  - Moderate capacity, pretty fast to access
- Main memory ↔ library
  - Big, holds almost all data, but slow
- Disk (virtual memory) ↔ inter-library loan
  - Very slow, but hopefully really uncommon

Exploiting Locality: Memory Hierarchy



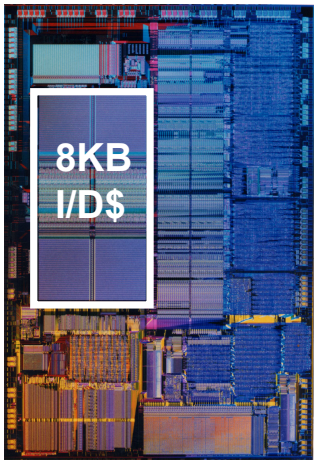
- Hierarchy of memory components
  - Upper components
    - Fast ↔ Small ↔ Expensive
  - Lower components
    - Slow ↔ Big ↔ Cheap
- Connected by "buses"
  - Which also have latency and bandwidth issues
- Most frequently accessed data in M1
  - M1 + next most frequently accessed in M2, etc.
  - Move data up-down hierarchy
- Optimize average access time
  - $latency_{avg} = latency_{hit} + (\%_{miss} * latency_{miss})$
  - Attack each component

Concrete Memory Hierarchy

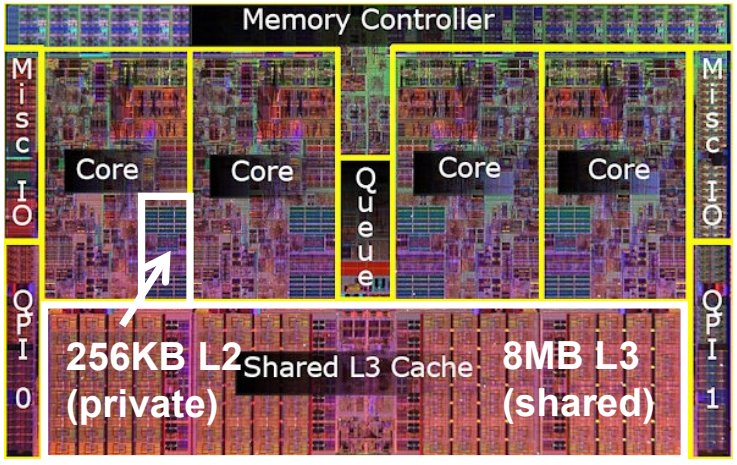


- 0th level: **Registers**
- 1st level: **Primary caches**
  - Split instruction (I$) and data (D$)
  - Typically 8KB to 64KB each
- 2nd level: **2nd and 3rd cache** (L2, L3)
  - On-chip, typically made of SRAM
  - 2nd level typically ~256KB to 512KB
  - "Last level cache" typically 4MB to 16MB
- 3rd level: **main memory**
  - Made of DRAM ("Dynamic" RAM)
  - Typically 1GB to 4GB for desktops/laptops
    - Servers can have 100s of GB
- 4th level: **disk (swap and files)**
  - Uses magnetic disks or flash drives

Evolution of Cache Hierarchies



Intel 486



Intel Core i7 (quad core)

- Chips today are 30–70% cache by area

Caches

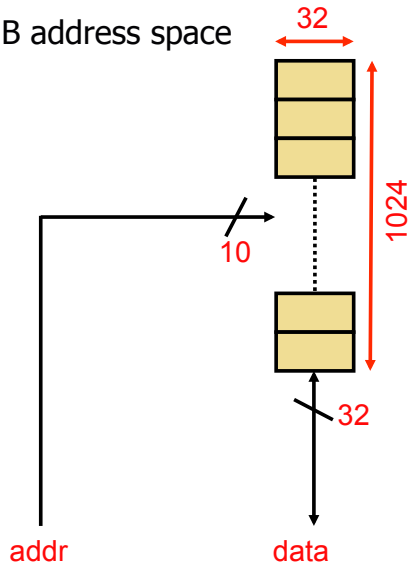
Analogy to a Software Hashtable

- What is a "hash table"?
  - What is it used for?
  - How does it work?

- Short answer:
  - Maps a "key" to a "value"
    - Constant time lookup/insert
  - Have a table of some size, say N, of "buckets"
  - Take a "key" value, apply a hash function to it
  - Insert and lookup a "key" at "hash(key) modulo N"
    - Need to store the "key" and "value" in each bucket
    - Need to check to make sure the "key" matches
  - Need to handle conflicts/overflows somehow (chaining, re-hashing)

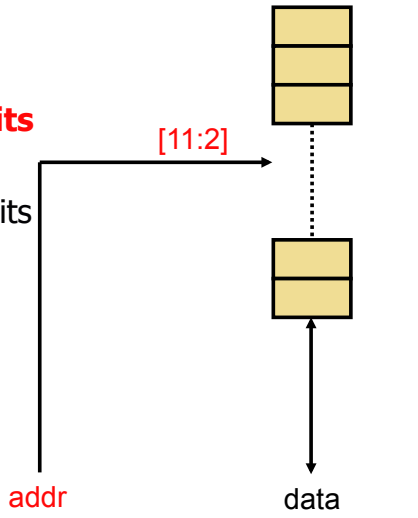
Hardware Cache Organization

- **Cache is a hardware hashtable**
- The setup
  - 32-bit ISA → 4B words/addresses, 2^{32} B address space
- Logical cache organization
  - 4KB, organized as 1K 4B **blocks**
  - Each block can hold a 4-byte word
- Physical cache implementation
  - 1K (1024 bit) by 4B **SRAM**
  - Called **data array**
  - 10-bit address input
  - 32-bit data input/output



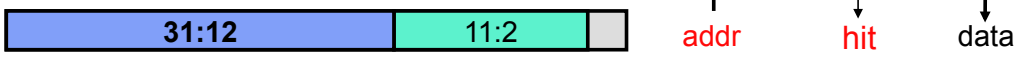
Looking Up A Block

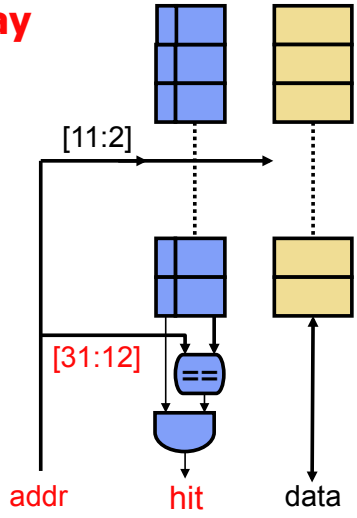
- Q: which 10 of the 32 address bits to use?
- A: bits [11:2]
  - 2 least significant (LS) bits [1:0] are the **offset bits**
    - Locate byte within word
    - Don't need these to locate word
  - Next 10 LS bits [11:2] are the **index bits**
    - These locate the word
    - Nothing says index must be these bits
    - But these work best in practice
      - Why? (think about it)



Knowing that You Found It

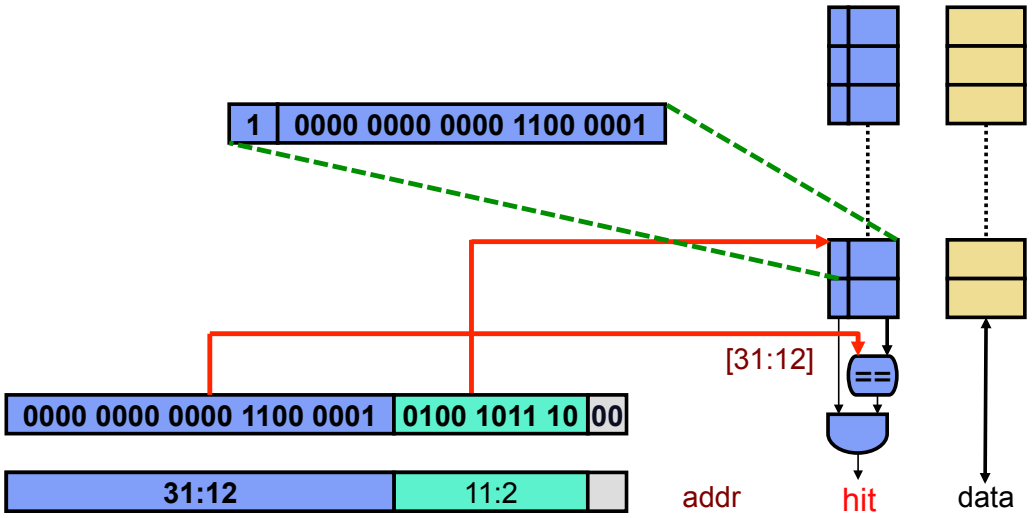
- Each cache row corresponds to 2^{20} blocks
  - How to know which if any is currently there?
  - Tag each cache word with remaining address bits [31:12]
- Build separate and parallel **tag array**
  - 1K by 21-bit SRAM
  - 20-bit (next slide) **tag** + 1 **valid bit**
- Lookup algorithm
  - Read tag indicated by index bits
  - If tag matches & valid bit set:
    then: Hit → data is good
    else: Miss → data is garbage, wait…





A Concrete Example

- Lookup address `x000C14B8`
  - Index = addr [11:2] = (addr >> 2) & x7FF = `x12E`
  - Tag = addr [31:12] = (addr >> 12) = `x000C1`



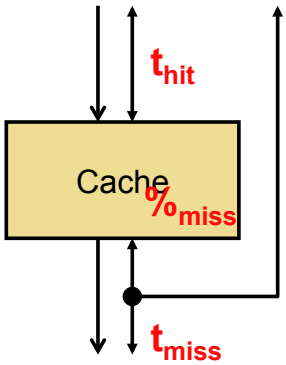
Handling a Cache Miss

- What if requested data isn't in the cache?
  - How does it get in there?

- **Cache controller**: finite state machine
  - Remembers miss address
  - Accesses next level of memory
  - Waits for response
  - Writes data/tag into proper locations

  - All of this happens on the **fill path**
  - Sometimes called **backside**

Cache Performance Equation



- For a cache
  - **Access**: read or write to cache
  - **Hit**: desired data found in cache
  - **Miss**: desired data not found in cache
    - Must get from another component
    - No notion of "miss" in register file
  - **Fill**: action of placing data into cache

  - $\%_{miss}$ (miss-rate): #misses / #accesses
  - t_{hit}: time to read data from (write data to) cache
  - t_{miss}: time to read data into cache

- Performance metric: average access time

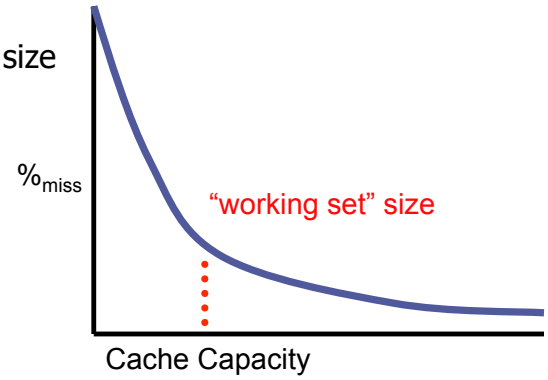
$$t_{avg} = t_{hit} + (\%_{miss} * t_{miss})$$

Measuring Cache Performance

- Ultimate metric is t_{avg}
  - Cache capacity and circuits roughly determines t_{hit}
  - Lower-level memory structures determine t_{miss}
  - Measure $\%_{miss}$
    - Hardware performance counters
    - Simulation

Capacity and Performance

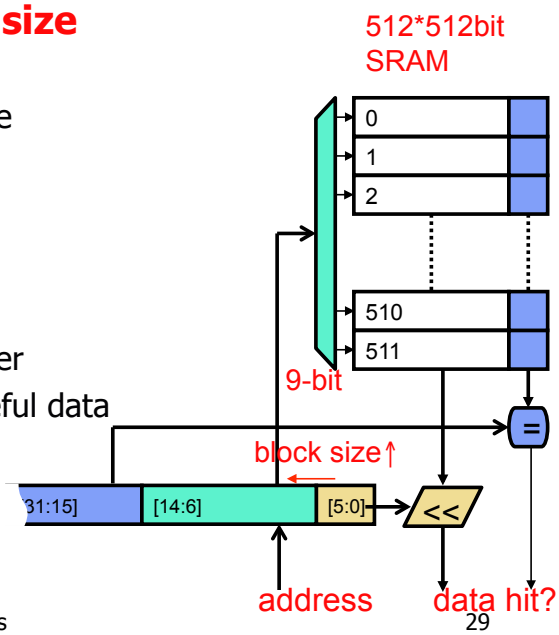
- Simplest way to reduce $\%_{miss}$: increase capacity
  - \+ Miss rate decreases monotonically
    - **"Working set"**: insns/data program is actively using
    - Diminishing returns
  - – However t_{hit} increases
    - Latency grows with cache size
  - t_{avg} ?



- Given capacity, manipulate $\%_{miss}$ by changing **organization**

Block Size

- Given capacity, manipulate $\%_{miss}$ by changing organization
- One option: increase **block size**
  - Exploit **spatial locality**
  - Notice index/offset bits change
  - Tag remain the same
- Ramifications
  - \+ Reduce $\%_{miss}$ (up to a point)
  - \+ Reduce tag overhead (why?)
  - – Potentially useless data transfer
  - – Premature replacement of useful data
  - – Fragmentation



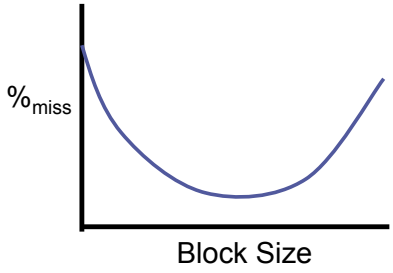
Larger Blocks to Lower Tag Overhead

- Tag overhead of 32KB cache with 1024 32B frames
  - 32B frames → 5-bit offset
  - 1024 frames → 10-bit index
  - 32-bit address – 5-bit offset – 10-bit index = 17-bit tag
  - (17-bit tag + 1-bit valid) * 1024 frames = 18Kb tags = 2.2KB tags
  - ~6% overhead

- Tag overhead of 32KB cache with 512 64B frames
  - 64B frames → 6-bit offset
  - 512 frames → 9-bit index
  - 32-bit address – 6-bit offset – 9-bit index = 17-bit tag
  - (17-bit tag + 1-bit valid) * 512 frames = 9Kb tags = 1.1KB tags
  - \+ ~3% overhead

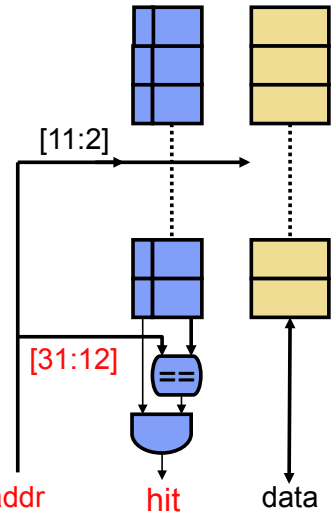
Effect of Block Size on Miss Rate

- Two effects on miss rate
  - + **Spatial prefetching (good)**
    - For blocks with adjacent addresses
    - Turns miss/miss into miss/hit pairs
  - – **Interference (bad)**
    - For blocks with non-adjacent addresses (but in adjacent frames)
    - Turns hits into misses by disallowing simultaneous residence
    - Consider entire cache as one big block
- Both effects always present
  - Spatial "prefetching" dominates initially
    - Depends on size of the cache
  - Reasonable block sizes are 32B–128B
- But also increases traffic
  - More data moved, not all used



Cache Conflicts

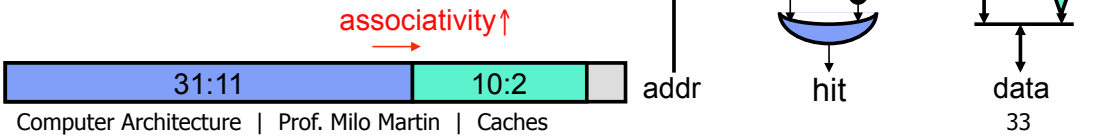
- Consider two frequently-accessed variables…
- What if their addresses have the same "index" bits?
  - Such addresses "conflict" in the cache
  - Can't hold both in the cache at once…
  - Can results in lots of misses (bad!)
- Conflicts increase cache miss rate
  - Worse, result in non-robust performance
  - Small program change -> changes memory layout -> changes cache mapping of variables -> dramatically increase/decrease conflicts
- How can we mitigate conflicts?

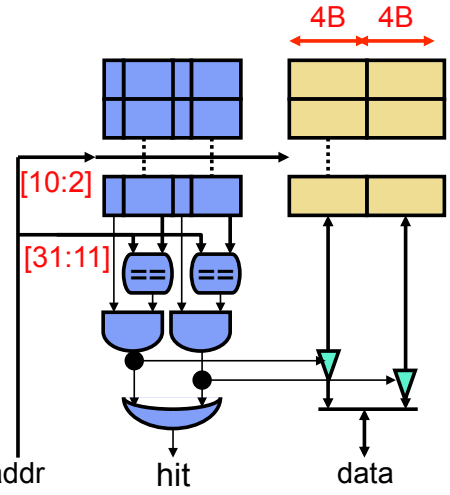


Associativity

- **Set-associativity**
  - Block can reside in one of few frames
  - Frame groups called ***sets***
  - Each frame in set called a ***way***
  - This is **2-way set-associative (SA)**
  - 1-way → **direct-mapped (DM)**
  - 1-set → **fully-associative (FA)**

  + Reduces conflicts
  – Increases latency$_{hit}$:
    - additional tag match & muxing

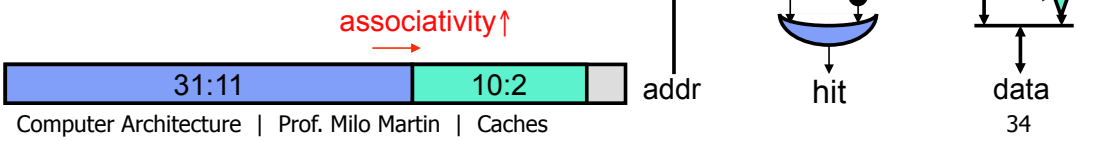


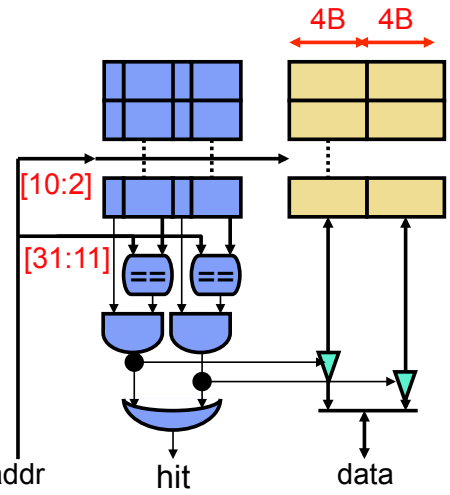


Associativity

- Lookup algorithm
  - Use index bits to find set
  - Read data/tags in all frames in parallel
  - **Any** (match and valid bit), Hit

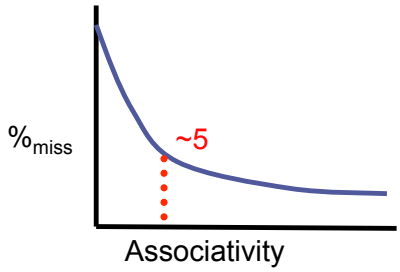
  - Notice tag/index/offset bits
    - Only 9-bit index (versus 10-bit for direct mapped)





Associativity and Performance

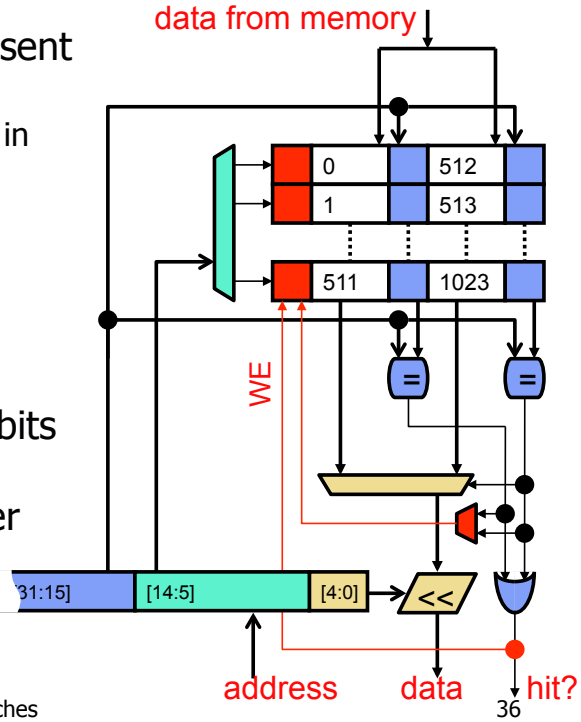
- Higher associative caches
  - + Have better (lower) $\%_{miss}$
    - Diminishing returns
  - – However t_{hit} increases
    - The more associative, the slower
  - What about t_{avg}?



- Block-size and number of sets should be powers of two
  - Makes indexing easier (just rip bits out of the address)
- 3-way set-associativity? No problem

Miss Handling & Replacement Policies

- Set-associative caches present a new design choice
  - On cache miss, which block in set to replace (kick out)?
- Add **LRU** field to each set
  - "Least recently used"
  - LRU data is encoded "way"
- Each access updates LRU bits
- Psudeo-LRU used for larger associativity caches



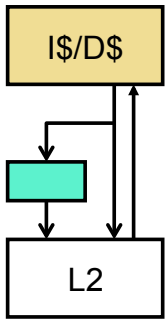
Replacement Policies

- Set-associative caches present a new design choice
  - On cache miss, which block in set to replace (kick out)?
- Some options
  - **Random**
  - **FIFO (first-in first-out)**
  - **LRU (least recently used)**
    - Fits with temporal locality, LRU = least likely to be used in future
  - **NMRU (not most recently used)**
    - An easier to implement approximation of LRU
    - Is LRU for 2-way set-associative caches
  - **Belady's**: replace block that will be used furthest in future
    - Unachievable optimum

Cache Optimizations

Prefetching

- Bring data into cache proactively/**speculatively**
  - If successful, reduces number of caches misses
- Key: anticipate upcoming miss addresses accurately
  - Can do in software or hardware
- Simple hardware prefetching: **next block prefetching**
  - Miss on address **X** → anticipate miss on **X+block-size**
  - \+ Works for insns: sequential execution
  - \+ Works for data: arrays
- Table-driven hardware prefetching
  - Use **predictor** to detect strides, common patterns
- Effectiveness determined by:
  - **Timeliness**: initiate prefetches sufficiently in advance
  - **Coverage**: prefetch for as many misses as possible
  - **Accuracy**: don't pollute with unnecessary data



Software Prefetching

- Use a special "prefetch" instruction
  - Tells the hardware to bring in data, doesn't actually read it
  - Just a hint
- Inserted by programmer or compiler
- Example

```
int tree_add(tree_t* t) {
  if (t == NULL) return 0;
  __builtin_prefetch(t->left);
  return t->val + tree_add(t->right) + tree_add(t->left);
}
```

  - 20% performance improvement for large trees (>1M nodes)
    - But ~15% slowdown for small trees (<1K nodes)
- Multiple prefetches bring multiple blocks in parallel
  - More "memory-level" parallelism (MLP)

Software Restructuring: Data

- Capacity misses: poor spatial or temporal locality
  - Several code restructuring techniques to improve both
    - Loop blocking (break into cache-sized chunks), loop fusion
  - Compiler must know that restructuring preserves semantics
- **Loop interchange**: spatial locality
  - Example: row-major matrix: `X[i][j]` followed by `X[i][j+1]`
  - Poor code: `X[I][j]` followed by `X[i+1][j]`

```
for (j = 0; j<NCOLS; j++)
   for (i = 0; i<NROWS; i++)
      sum += X[i][j];
```

  - Better code

```
for (i = 0; i<NROWS; i++)
   for (j = 0; j<NCOLS; j++)
      sum += X[i][j];
```


Software Restructuring: Data

- **Loop "blocking" or "tiling"**: temporal locality
  - Consider following "image blur" code

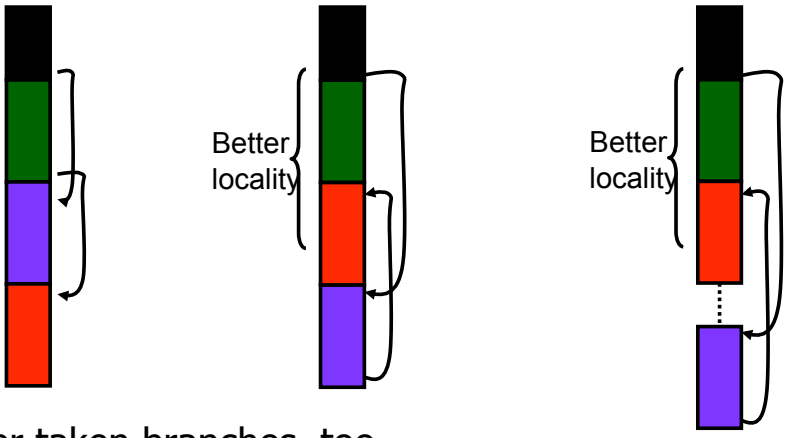
```
for (i = 1; i<NROWS-1; i++)
   for (j = 1; j<NCOLS-1; j++)
      Y[i][j] = avg(X[i][j], X[i+1][j], X[i-1][j],
                    X[i][j+1], X[i][j-1]);
```

  - How many times is each block brought into the cache?
    - Answer: For a large image matrix, ~3 times
    - Why? exploits only spatial locality (not temporal locality)
- Can be reduced to ~1 time by exploiting temporal locality
  - Assume NROWS-2 and NCOLS-2 evenly divisible by TILE_SIZE

```
for (ii = 1; ii<NROWS-1; ii+=TILE_SIZE)
   for (jj = 1; jj<NCOLS-1; jj+=TILE_SIZE)
      for (i = ii; i<ii+TILE_SIZE; i++)
         for (j = jj; j<jj+TILE_SIZE; j++)
            Y[i][j] = avg(X[i][j], X[i+1][j], …)
```

Software Restructuring: Code

- Compiler an layout code for temporal and spatial locality
  - If (a) { **code1;** } else { **code2;** } **code3;**
  - But, code2 case never happens (say, error condition)

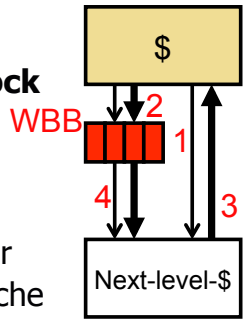


- Fewer taken branches, too

What About Stores? Handling Cache Writes

Handling Cache Writes

- When to propagate new value to (lower level) memory?
- **Option #1: Write-through**: immediately
  - On hit, update cache
  - Immediately send the write to the next level
- **Option #2: Write-back**: when block is replaced
  - Requires additional "**dirty**" bit per block
    - Replace **clean** block: **no extra traffic**
    - Replace **dirty** block: **extra "writeback" of block**
  - \+ **Writeback-buffer (WBB)**:
    - Hide latency of writeback (keep off critical path)
    - Step#1: Send "fill" request to next-level
    - Step#2: While waiting, write dirty block to buffer
    - Step#3: When new blocks arrives, put it into cache
    - Step#4: Write buffer contents to next-level



Write Propagation Comparison

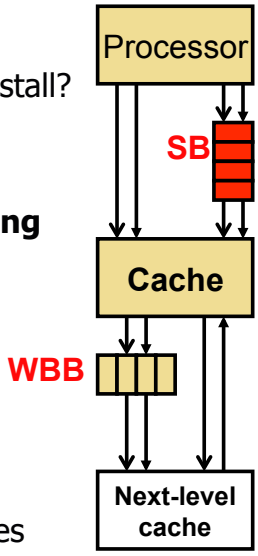
- **Write-through**
  - – Creates additional traffic
    - Consider repeated write hits
  - – Next level must handle small writes (1, 2, 4, 8-bytes)
  - \+ No need for dirty bits in cache
  - \+ No need to handle "writeback" operations
    - Simplifies miss handling (no write-back buffer)
  - Sometimes used for L1 caches (for example, by IBM)
  - Usually **write-non-allocate**: on write miss, just write to next level
- **Write-back**
  - \+ Key advantage: uses less bandwidth
  - Reverse of other pros/cons above
  - Used by Intel, (AMD), and many ARM cores
  - Second-level and beyond are generally write-back caches
  - Usually **write-allocate**: on write miss, fill block from next level

Write Misses and Store Buffers

- Read miss?
  - Load can't go on without the data, it must stall
- Write miss?
  - Technically, no instruction is waiting for data, why stall?

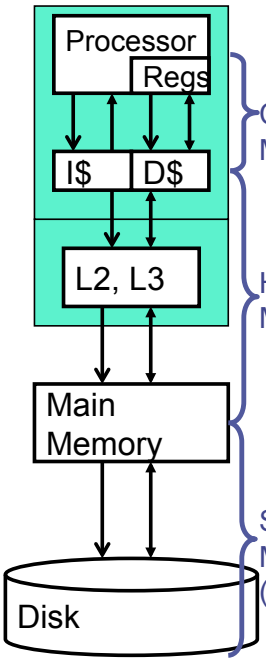
- **Store buffer**: a small buffer
  - Stores put address/value to store buffer, **keep going**
  - Store buffer writes stores to D$ in the background
  - Loads must search store buffer (in addition to D$)
  - \+ Eliminates stalls on write misses (mostly)
  - – Creates some problems (later)

- Store buffer vs. writeback-buffer
  - Store buffer: "in front" of D$, for hiding store misses
  - Writeback buffer: "behind" D$, for hiding writebacks



Cache Hierarchies

Concrete Memory Hierarchy

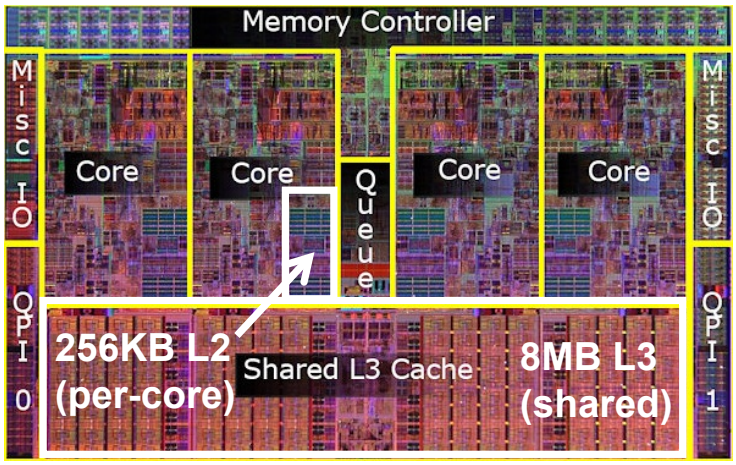


- 0th level: **Registers**
- 1st level: **Primary caches**
  - Split instruction (I$) and data (D$)
  - Typically 8KB to 64KB each
- 2nd level: **2nd and 3rd cache** (L2, L3)
  - On-chip, typically made of SRAM
  - 2nd level typically ~256KB to 512KB
  - "Last level cache" typically 4MB to 16MB
- 3rd level: **main memory**
  - Made of DRAM ("Dynamic" RAM)
  - Typically 1GB to 4GB for desktops/laptops
    - Servers can have 100s of GB
- 4th level: **disk (swap and files)**
  - Uses magnetic disks or flash drives

Designing a Cache Hierarchy

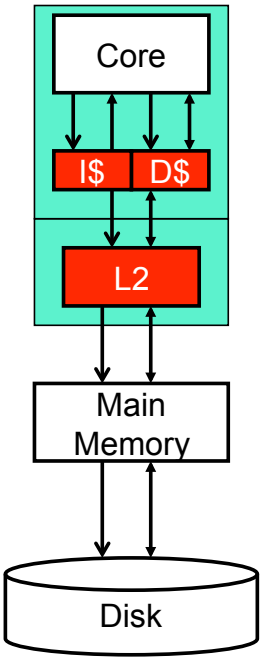
- For any memory component: t_{hit} vs. $\%_{miss}$ tradeoff
- Upper components (I$, D$) emphasize low t_{hit}
  - **Frequent access → t_{hit} important**
  - t_{miss} is not bad → $\%_{miss}$ less important
  - Lower capacity and lower associativity (to reduce t_{hit})
  - Small-medium block-size (to reduce conflicts)
  - **Split instruction & data cache to allow simultaneous access**
- Moving down (L2, L3) emphasis turns to $\%_{miss}$
  - **Infrequent access → t_{hit} less important**
  - t_{miss} is bad → $\%_{miss}$ important
  - High capacity, associativity, and block size (to reduce $\%_{miss}$)
  - **Unified insn & data caching to dynamic allocate capacity**

Example Cache Hierarchy: Core i7



- Each core:
  - 32KB insn & 32KB data, 8-way set-associative, 64-byte blocks
  - 256KB second-level cache, 8-way set-associative, 64-byte blocks
- 8MB shared cache, 16-way set-associative

Summary



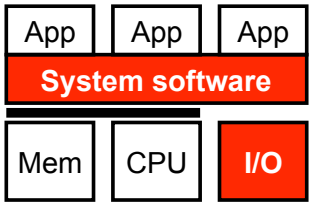
- "Cache": hardware managed
  - Hardware automatically retrieves missing data
  - Built from fast on-chip SRAM
  - In contrast to off-chip, DRAM "main memory"
- **Average access time** of a memory component
  - $latency_{avg} = latency_{hit} + (\%_{miss} * latency_{miss})$
  - Hard to get low $latency_{hit}$ and $\%_{miss}$ in one structure → memory hierarchy
- Cache ABCs (**associativity, block size, capacity**)
- **Performance optimizations**
  - Prefetching & data restructuring
- **Handling writes**
  - Write-back vs. write-through
- **Memory hierarchy**
  - Smaller, faster, expensive → bigger, slower, cheaper

Computer Architecture

Unit 5: Virtual Memory

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

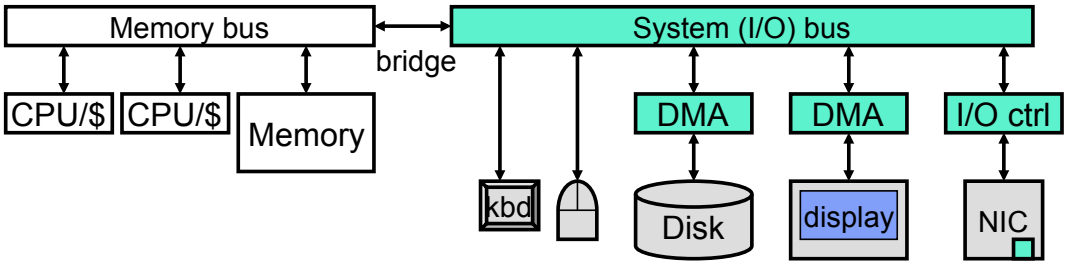
This Unit: Virtual Memory



- The operating system (OS)
  - A super-application
  - Hardware support for an OS
- Virtual memory
  - Page tables and address translation
  - TLBs and memory hierarchy issues

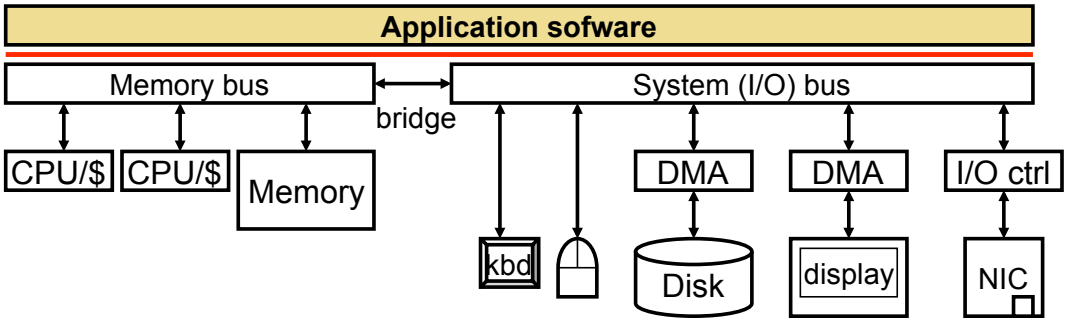
A Computer System: Hardware

- CPUs and memories
  - Connected by memory bus
- **I/O peripherals**: storage, input, display, network, …
  - With separate or built-in DMA
  - Connected by **system bus** (which is connected to memory bus)



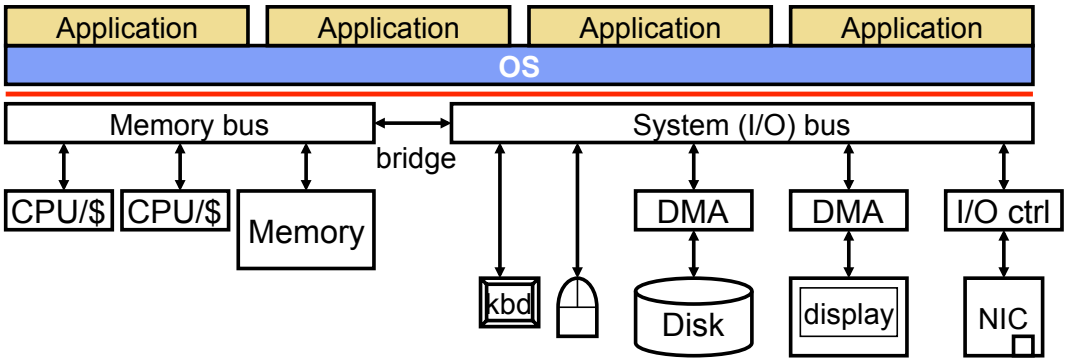
A Computer System: + App Software

- **Application software**: computer must do something



A Computer System: + OS

- **Operating System (OS):** virtualizes hardware for apps
  - **Abstraction**: provides **services** (e.g., threads, files, etc.)
    + Simplifies app programming model, raw hardware is nasty
  - **Isolation**: gives each app illusion of private CPU, memory, I/O
    + Simplifies app programming model
    + Increases hardware resource utilization



Operating System (OS) and User Apps

- Sane system development requires a split
  - Hardware itself facilitates/enforces this split

- **Operating System (OS)**: a super-privileged process
  - Manages hardware resource allocation/revocation for all processes
  - Has direct access to resource allocation features
  - Aware of many nasty hardware details
  - Aware of other processes
  - Talks directly to input/output devices (device driver software)

- **User-level apps**: ignorance is bliss
  - Unaware of most nasty hardware details
  - Unaware of other apps (and OS)
  - Explicitly denied access to resource allocation features

System Calls

- Controlled transfers to/from OS

- **System Call**: a user-level app "function call" to OS
  - Leave description of what you want done in registers
  - SYSCALL instruction (also called TRAP or INT)
    - Can't allow user-level apps to invoke arbitrary OS code
    - Restricted set of legal OS addresses to jump to (**trap vector**)
  - Processor jumps to OS using trap vector
    - Sets privileged mode
  - OS performs operation
  - OS does a "return from system call"
    - Unsets privileged mode

- Used for I/O and other operating system services

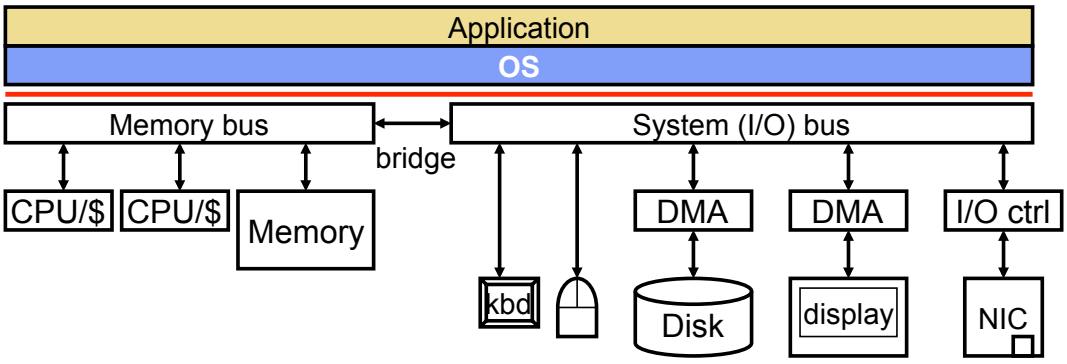
Input/Output (I/O)

- Applications use "system calls" to initiate I/O
- Only operating system (OS) talks directly to the I/O device
  - Send commands, query status, etc.
  - OS software uses special uncached load/store operations
  - Hardware sends these reads/writes across I/O bus to device
- Hardware also provides "Direct Memory Access (DMA)"
  - For big transfers, the I/O device accesses the memory directly
  - Example: DMA used to transfer an entire block to/from disk
- Interrupt-driven I/O
  - The I/O device tells the software its transfer is complete
  - Tells the hardware to raise an "interrupt" (door bell)
  - Processor jumps into the OS
  - Inefficient alternative: polling

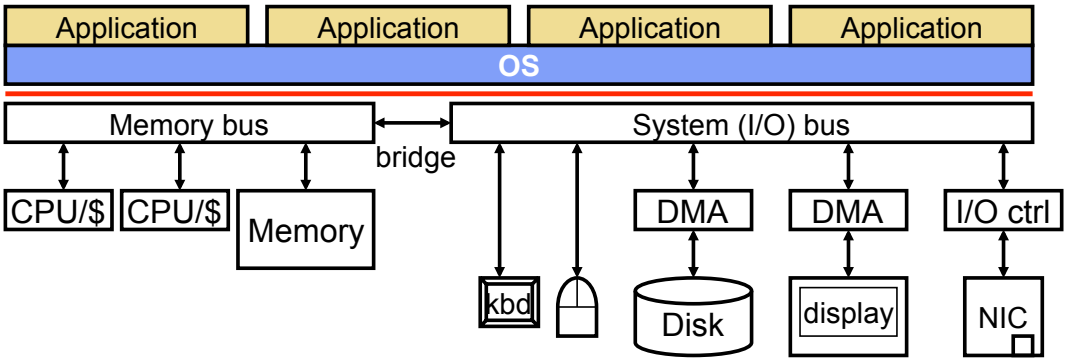
Interrupts

- **Exceptions**: synchronous, generated by running app
  - E.g., illegal insn, divide by zero, etc.
- **Interrupts**: asynchronous events generated externally
  - E.g., timer, I/O request/reply, etc.
- **"Interrupt" handling**: same mechanism for both
  - "Interrupts" are on-chip signals/bits
    - Either internal (e.g., timer, exceptions) or from I/O devices
  - Processor continuously monitors interrupt status, when one is high…
  - Hardware jumps to some preset address in OS code (interrupt vector)
  - Like an asynchronous, non-programmatic SYSCALL
- **Timer**: programmable on-chip interrupt
  - Initialize with some number of micro-seconds
  - Timer counts down and interrupts when reaches zero

A Computer System: + OS



A Computer System: + OS



Virtualizing Processors

- How do multiple apps (and OS) share the processors?
  - **Goal: applications think there are an infinite # of processors**

- Solution: time-share the resource
  - Trigger a **context switch** at a regular interval (~1ms)
    - **Pre-emptive**: app doesn't yield CPU, OS forcibly takes it
      + Stops greedy apps from starving others
  - **Architected state**: PC, registers
    - Save and restore them on context switches
    - Memory state?
  - **Non-architected state**: caches, predictor tables, etc.
    - Ignore or flush
- Operating system responsible to handle context switching
  - Hardware support is just a timer interrupt

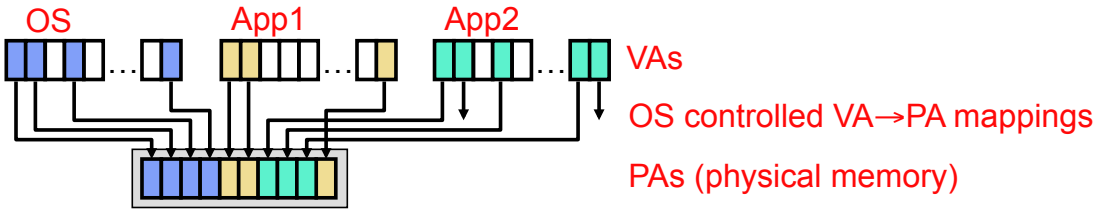
Virtualizing Main Memory

- How do multiple apps (and the OS) share main memory?
  - **Goal: each application thinks it has infinite memory**

- One app may want more memory than is in the system
  - App's insn/data footprint may be larger than main memory
  - **Requires main memory to act like a cache**
    - With disk as next level in memory hierarchy (slow)
    - Write-back, write-allocate, large blocks or "pages"
  - No notion of "program not fitting" in registers or caches (why?)
- Solution:
  - Part #1: treat memory as a "cache"
    - Store the overflowed blocks in "swap" space on disk
  - Part #2: add a level of indirection (address translation)

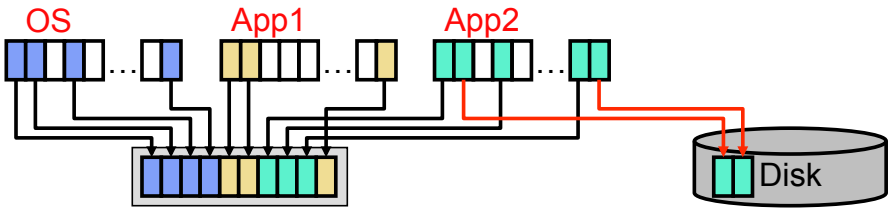
Virtual Memory (VM)

- **Virtual Memory (VM)**:
  - Level of indirection
  - Application generated addresses are **virtual addresses (VAs)**
    - Each process ***thinks*** it has its own 2^N bytes of address space
  - Memory accessed using **physical addresses (PAs)**
  - VAs translated to PAs at some coarse granularity (page)
  - OS controls VA to PA mapping for itself and all other processes
  - Logically: translation performed before every insn fetch, load, store
  - Physically: hardware acceleration removes translation overhead



Virtual Memory (VM)

- Programs use **virtual addresses (VA)**
  - VA size (N) aka machine size (e.g., Core 2 Duo: 48-bit)
- Memory uses **physical addresses (PA)**
  - PA size (M) typically M<N, especially if N=64
  - 2^M is most physical memory machine supports
- VA→PA at **page** granularity (VP→PP)
  - Mapping need not preserve contiguity
  - VP need not be mapped to any PP
  - Unmapped VPs live on disk (swap) or nowhere (if not yet touched)



VM is an Old Idea: Older than Caches

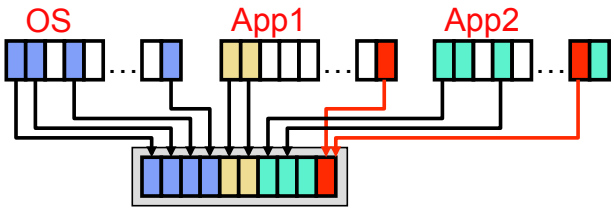
- Original motivation: **single-program compatibility**
  - IBM System 370: a family of computers with one software suite
  - \+ Same program could run on machines with different memory sizes
  - – Prior, programmers explicitly accounted for memory size

- But also: **full-associativity + software replacement**
  - Memory t_{miss} is high: extremely important to reduce $\%_{miss}$

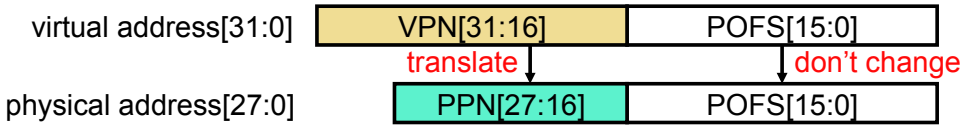
Parameter	I$/D$	L2	Main Memory
t_{hit}	2ns	10ns	**30ns**
t_{miss}	**10ns**	**30ns**	**10ms (10M ns)**
Capacity	8–64KB	128KB–2MB	**64MB–64GB**
Block size	16–32B	32–256B	**4+KB**
Assoc./Repl.	1–4, LRU	4–16, LRU	**Full, "working set"**

Uses of Virtual Memory

- More recently: **isolation** and **multi-programming**
  - Each app thinks it has 2^N B of memory, its stack starts 0xFFFFFFFF,…
  - Apps prevented from reading/writing each other's memory
    - Can't even address the other program's memory!
- **Protection**
  - Each page with a read/write/execute permission set by OS
  - Enforced by hardware
- **Inter-process communication**.
  - Map same physical pages into multiple virtual address spaces
  - Or share files via the UNIX `mmap()` call



Address Translation

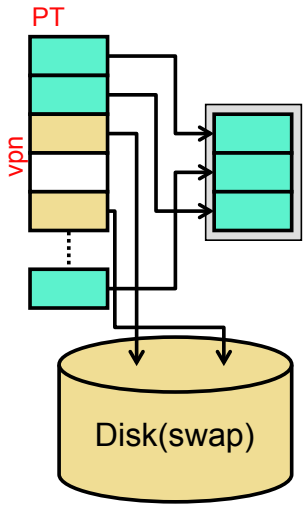


- VA→PA mapping called **address translation**
  - Split VA into **virtual page number (VPN)** & **page offset (POFS)**
  - Translate VPN into **physical page number (PPN)**
  - POFS is not translated
  - VA→PA = [VPN, POFS] → [PPN, POFS]

- Example above
  - 64KB pages → 16-bit POFS
  - 32-bit machine → 32-bit VA → 16-bit VPN
  - Maximum 256MB memory → 28-bit PA → 12-bit PPN

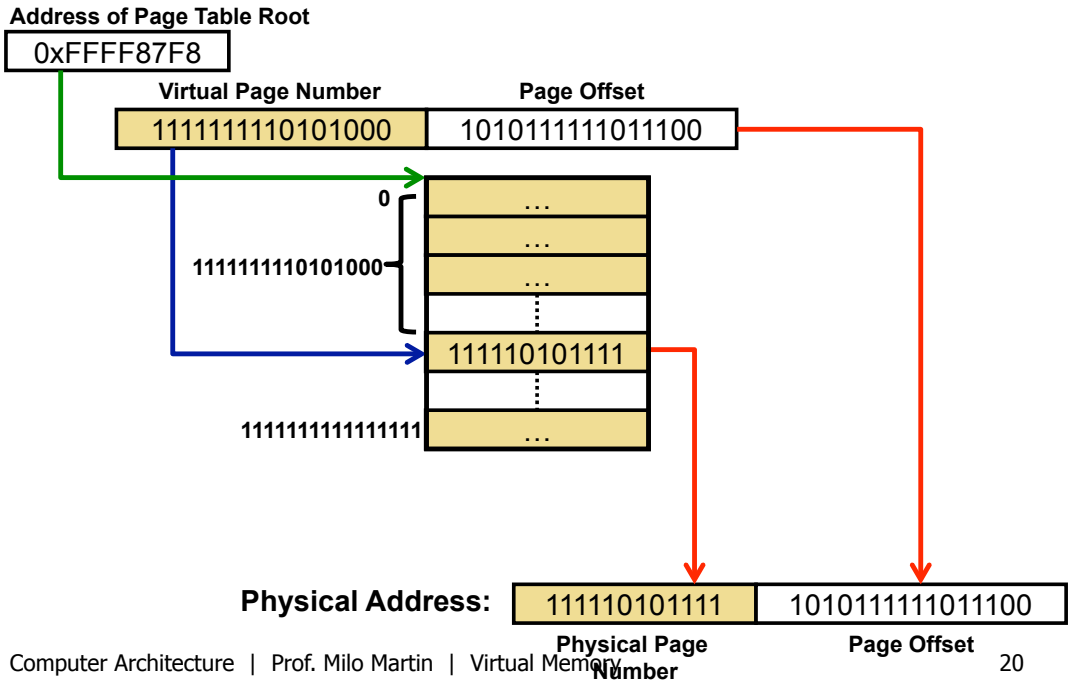
Address Translation Mechanics I

- How are addresses translated?
  - In software (for now) but with hardware acceleration (a little later)
- Each process allocated a **page table (PT)**
  - **Software data structure constructed by OS**
  - Maps VPs to PPs or to disk (swap) addresses
    - VP entries empty if page never referenced
  - Translation is table lookup



Page Table Example

Example: Memory access at address 0xFFA8AFBA



Page Table Size

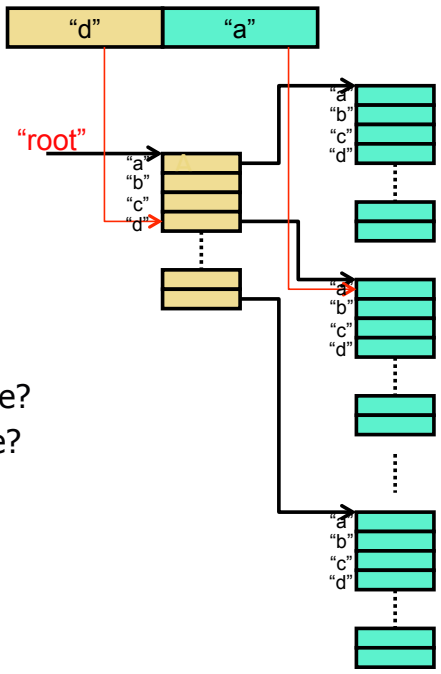
- How big is a page table on the following machine?
  - 32-bit machine
  - 4B page table entries (PTEs)
  - 4KB pages

  - 32-bit machine → 32-bit VA → 2^32 = 4GB virtual memory
  - 4GB virtual memory / 4KB page size → 1M VPs
  - 1M VPs * 4 Bytes per PTE → 4MB

- How big would the page table be with 64KB pages?
- How big would it be for a 64-bit machine?
- Page tables can get big
  - There are ways of making them smaller

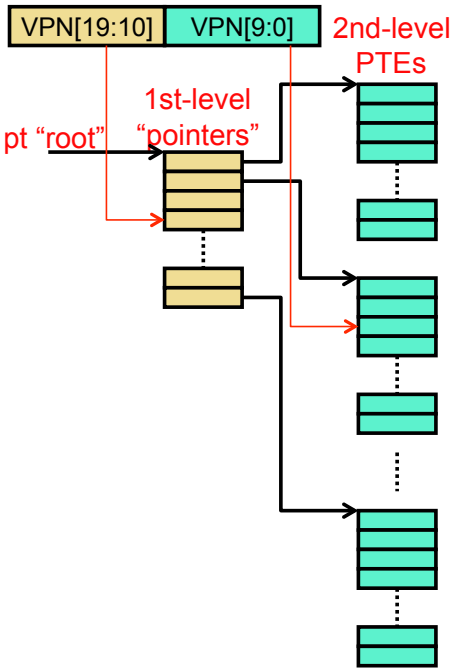
"Trie"

- What is a "**trie**" data structure
  - Also called a "prefix tree"
- What is it used for?
- What properties does it have?
  - How is it different from a binary tree?
  - How is it different than a hash table?



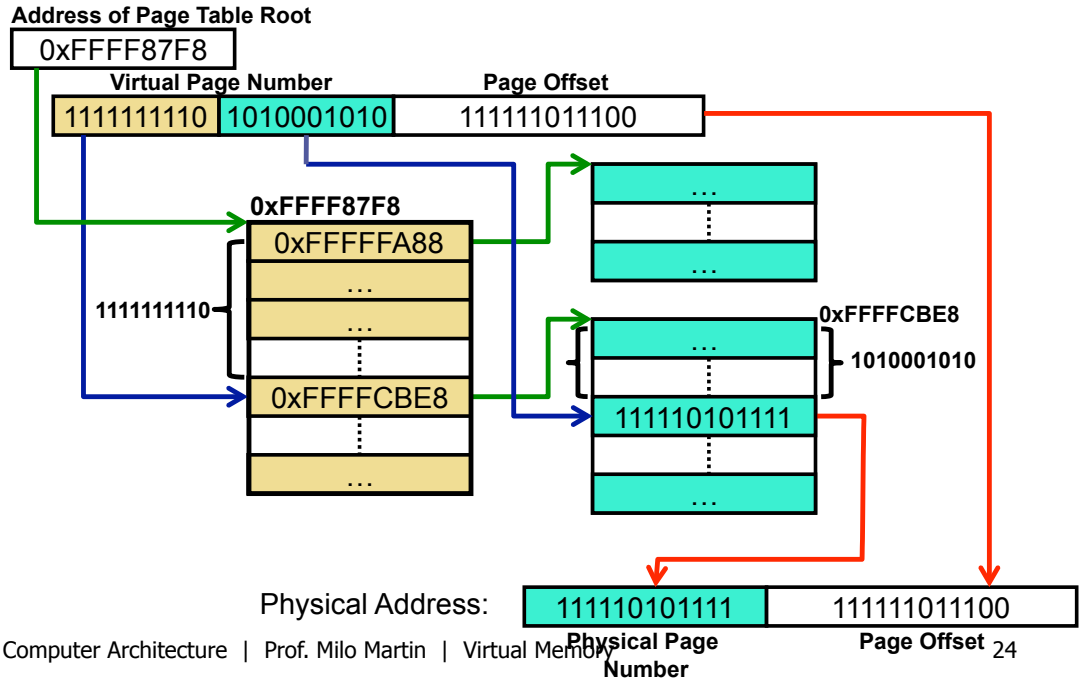
Multi-Level Page Table (PT)

- One way: **multi-level page tables**
  - Tree of page tables ("trie")
  - Lowest-level tables hold PTEs
  - Upper-level tables hold pointers to lower-level tables
  - Different parts of VPN used to index different levels
- 20-bit VPN
  - Upper 10 bits index 1st-level table
  - Lower 10 bits index 2nd-level table
  - In reality, often more than 2 levels



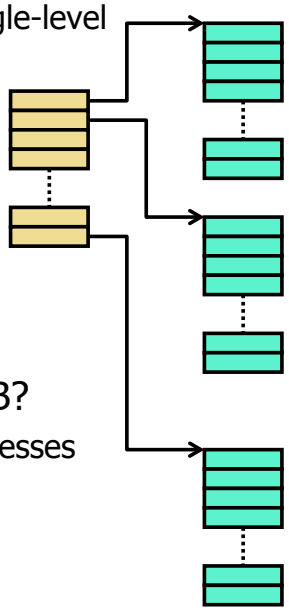
Multi-Level Address Translation

Example: Memory access at address 0xFFA8AFBA



Multi-Level Page Table (PT)

- Have we saved any space?
  - Isn't total size of 2nd level tables same as single-level table (i.e., 4MB)?
  - Yes, but…
- Large virtual address regions **unused**
  - Corresponding 2nd-level tables need not exist
  - Corresponding 1st-level pointers are *null*
- How large for contiguous layout of 256MB?
  - Each 2nd-level table maps 4MB of virtual addresses
  - One 1st-level + 64 2nd-level pages
  - 65 total pages = 260KB (much less than 4MB)



Page-Level Protection

- **Page-level protection**
  - Piggy-back page-table mechanism
  - Map VPN to PPN + Read/Write/Execute permission bits
  - Attempt to execute data, to write read-only data?
    - Exception → OS terminates program
  - Useful (for OS itself actually)

VA Bits <47:39>	VA Bits <38:30>	VA Bits <29:21>	VA Bits <20:12>	VA Bits <11:0>
Level 1 table index	Level 2 table index	Level 3 table index	Level 4 table (page) index	Page offset address

- 4-level lookup, 4KB translation granule, 48-bit address
  - 9 address bits per level

VA Bits <41:29>	VA Bits <28:16>	VA Bits <15:0>
Level 1 table index	Level 2 table (page) index	Page offset address

- 2-level lookup, 64KB page/page table size, 42-bit address
  - 13 address bits per level
  - 3 levels for 48 bits of VA – top level table is a partial table

63	52	48	12	2	1	0
Upper attributes	SBZ	Address out	SBZ	Lower attributes and validity		

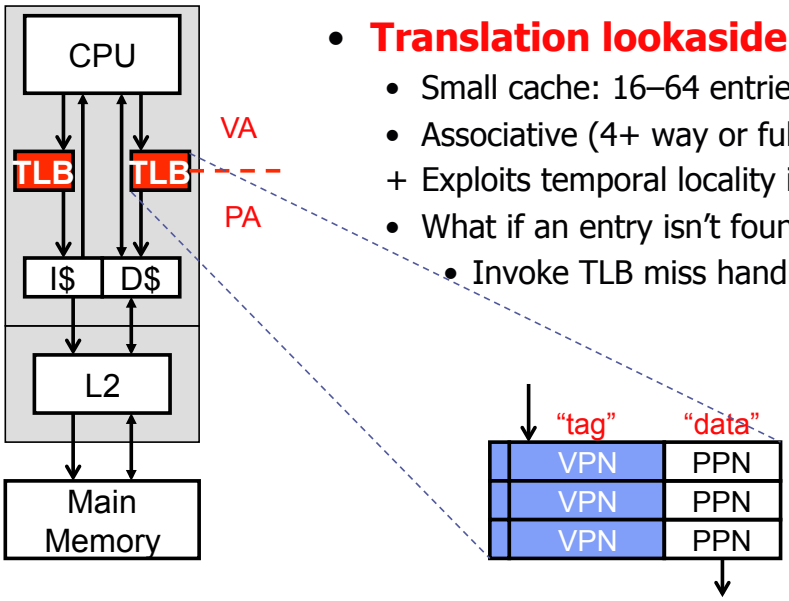
- 64-bit Translation table entry format

ARMv8 Technology Preview *By Richard Grisenthwaite Lead Architect and Fellow. ARM*

Address Translation Mechanics II

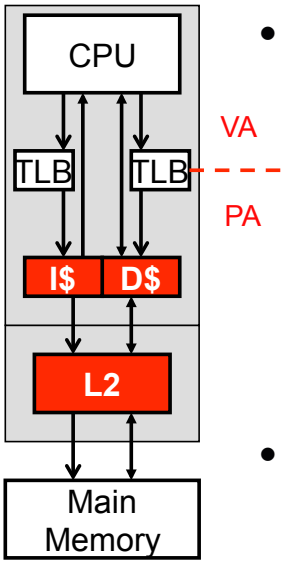
- Conceptually
  - Translate VA to PA before every cache access
  - Walk the page table before every load/store/insn-fetch
  - Would be terribly inefficient (even in hardware)
- In reality
  - **Translation Lookaside Buffer (TLB)**: cache translations
  - Only walk page table on TLB miss
- Hardware truisms
  - Functionality problem? Add indirection (e.g., VM)
  - Performance problem? Add cache (e.g., TLB)

Translation Lookaside Buffer



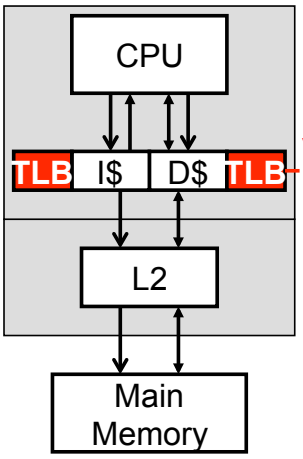
- **Translation lookaside buffer (TLB)**
  - Small cache: 16–64 entries
  - Associative (4+ way or fully associative)
  + Exploits temporal locality in page table
  - What if an entry isn't found in the TLB?
    - Invoke TLB miss handler

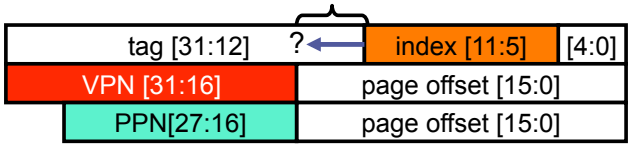
Serial TLB & Cache Access



- **"Physical" caches**
  - Indexed and tagged by **physical addresses**
  + Natural, "lazy" sharing of caches between apps/OS
    - VM ensures isolation (via **physical addresses**)
    - No need to do anything on context switches
    - Multi-threading works too
  + Cached inter-process communication works
    - Single copy indexed by physical address
  – Slow: adds at least one cycle to t_{hit}
- Note: **TLBs are by definition "virtual"**
  - Indexed and tagged by **virtual addresses**
  - Flush across context switches
  - Or extend with process identifier tags (x86)

Parallel TLB & Cache Access





VA
PA

- What about parallel access?
  - Only if…
    (cache size) / (associativity) ≤ page size
  - Index bits same in virt. and physical addresses!
- Access TLB in parallel with cache
  - Cache access needs tag only at very end
  - \+ Fast: no additional t_{hit} cycles
  - \+ No context-switching/aliasing problems
  - Dominant organization used today
- Example: Core 2, 4KB pages, 32KB, 8-way SA L1 data cache
  - Implication: associativity allows bigger caches

TLB Organization

- **Like caches**: TLBs also have ABCs
  - Capacity
  - Associativity (At least 4-way associative, fully-associative common)
  - What does it mean for a TLB to have a block size of two?
    - Two consecutive VPs share a single tag
  - **Like caches**: there can be second-level TLBs
- Example: AMD Opteron
  - 32-entry fully-assoc. TLBs, 512-entry 4-way L2 TLB (insn & data)
  - 4KB pages, 48-bit virtual addresses, four-level page table
- **Rule of thumb**: TLB should "cover" size of on-chip caches
  - In other words: (#PTEs in TLB) * page size ≥ cache size
  - Why? Consider relative miss latency in each…

TLB Misses

- **TLB miss:** translation not in TLB, but in page table
  - Two ways to "fill" it, both relatively fast
- **Hardware-managed TLB**: e.g., x86, recent SPARC, ARM
  - Page table root in hardware register, hardware "walks" table
  - \+ Latency: saves cost of OS call (avoids pipeline flush)
  - – Page table format is hard-coded
- **Software-managed TLB**: e.g., Alpha, MIPS
  - Short (~10 insn) OS routine walks page table, updates TLB
  - \+ Keeps page table format flexible
  - – Latency: one or two memory accesses + OS call (pipeline flush)
- Trend is towards hardware TLB miss handler

Page Faults

- **Page fault**: PTE not in TLB or page table
  - → page not in memory
  - Or no valid mapping → segmentation fault
  - Starts out as a TLB miss, detected by OS/hardware handler
- **OS software routine**:
  - Choose a physical page to replace
    - **"Working set"**: refined LRU, tracks active page usage
  - If dirty, write to disk
  - Read missing page from disk
    - Takes so long (~10ms), OS schedules another task
  - Requires yet another data structure: **frame map**
    - Maps physical pages to <process, virtual page> pairs
  - Treat like a normal TLB miss from here

Summary

- OS virtualizes memory and I/O devices

- Virtual memory
  - "infinite" memory, isolation, protection, inter-process communication
  - Page tables
  - Translation buffers
    - Parallel vs serial access, interaction with caching
  - Page faults

Computer Architecture
Mini-Course

March 2013

Prof. Milo Martin

Day 2 of 3

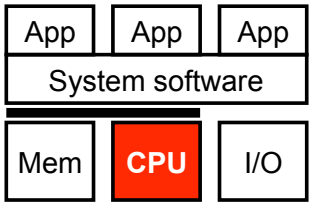
[spacer]

Computer Architecture

Unit 6: Pipelining

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

This Unit: Pipelining



- Single-cycle & multi-cycle datapaths
- Latency vs throughput & performance
- Basic pipelining
- Data hazards
  - Bypassing
  - Load-use stalling
- Pipelined multi-cycle operations
- Control hazards
  - Branch prediction

In-Class Exercise

- You have a washer, dryer, and "folder"
  - Each takes 30 minutes per load
  - How long for one load in total?
  - How long for two loads of laundry?
  - How long for 100 loads of laundry?

- Now assume:
  - Washing takes 30 minutes, drying 60 minutes, and folding 15 min
  - How long for one load in total?
  - How long for two loads of laundry?
  - How long for 100 loads of laundry?

[spacer]

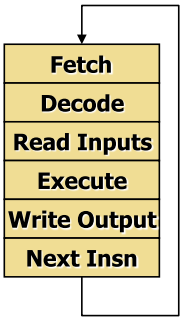
In-Class Exercise Answers

- You have a washer, dryer, and "folder"
  - Each takes 30 minutes per load
  - How long for one load in total? **90 minutes**
  - How long for two loads of laundry? 90 + 30 = **120 minutes**
  - How long for 100 loads of laundry? 90 + 30*99 = **3060 min**

- Now assume:
  - Washing takes 30 minutes, drying 60 minutes, and folding 15 min
  - How long for one load in total? **105 minutes**
  - How long for two loads of laundry? 105 + 60 = **165 minutes**
  - How long for 100 loads of laundry? 105 + 60*99 = **6045 min**

Datapath Background

Recall: The Sequential Model



- **Basic structure of all modern ISAs**
  - Often called VonNeuman, but in ENIAC before
- **Program order**: total order on dynamic insns
  - Order and **named storage** define computation
- Convenient feature: **program counter (PC)**
  - Insn itself stored in memory at location pointed to by PC
  - Next PC is next insn unless insn says otherwise
- Processor logically executes loop at left
- **Atomic**: insn finishes before next insn starts
  - Implementations can break this constraint physically
  - **But must maintain illusion to preserve correctness**

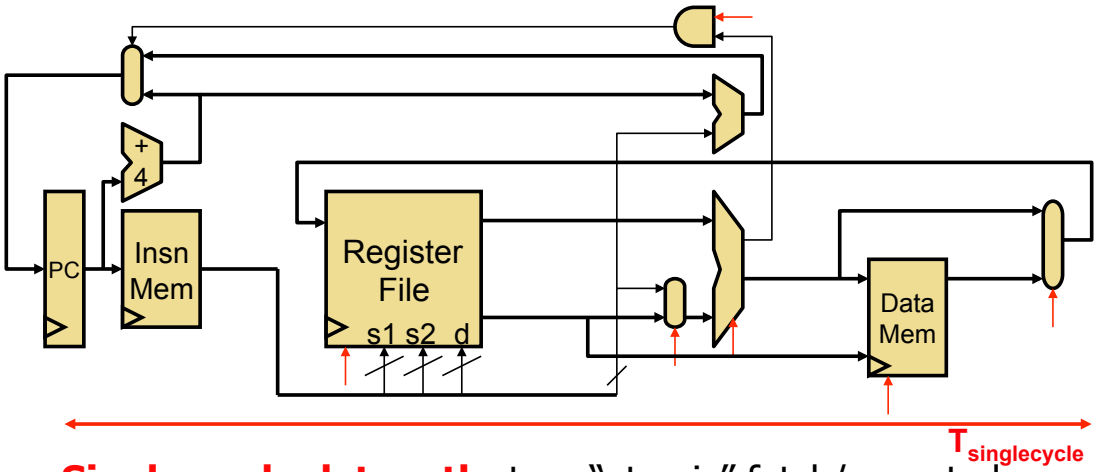
Recall: Maximizing Performance

Execution time =
(instructions/program) * (seconds/cycle) * (cycles/instruction)

(1 billion instructions) * (1ns per cycle) * (1 cycle per insn)
= 1 second

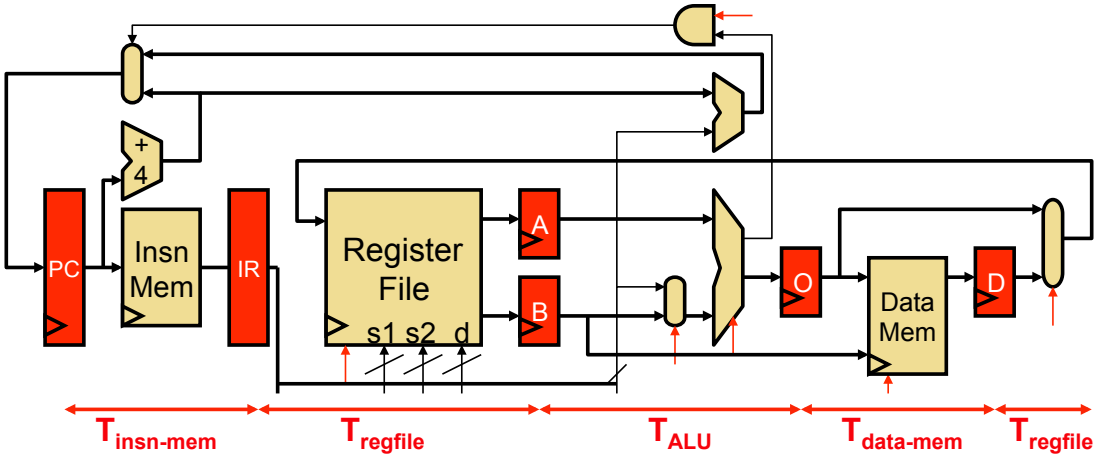
- Instructions per program:
  - Determined by program, compiler, instruction set architecture (ISA)
- **Cycles per instruction: "CPI"**
  - Typical range today: 2 to 0.5
  - Determined by program, compiler, ISA, micro-architecture
- **Seconds per cycle: "clock period" - same each cycle**
  - Typical range today: 2ns to 0.25ns
  - Reciprocal is frequency: 0.5 Ghz to 4 Ghz (1 Htz = 1 cycle per sec)
  - Determined by micro-architecture, technology parameters
- For minimum execution time, minimize each term
  - Difficult: ***often pull against one another***

Single-Cycle Datapath



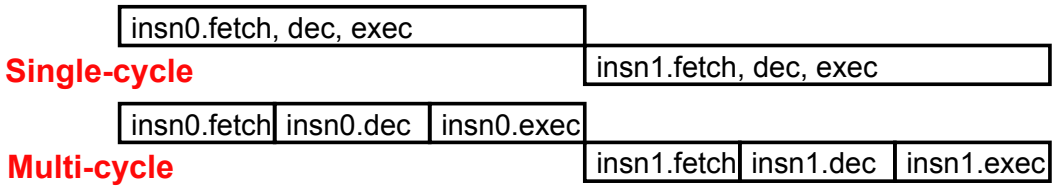
- **Single-cycle datapath**: true "atomic" fetch/execute loop
  - Fetch, decode, execute one complete instruction every cycle
  + Takes 1 cycle to execution any instruction by definition ("CPI" is 1)
  – Long clock period: to accommodate slowest instruction (worst-case delay through circuit, must wait this long *every* time)

Multi-Cycle Datapath



- **Multi-cycle datapath**: attacks slow clock
  - Fetch, decode, execute one complete insn over multiple cycles
  - **Allows insns to take different number of cycles**
  + Opposite of single-cycle: short clock period (less "work" per cycle)
  – Multiple cycles per instruction (higher "CPI")

Recap: Single-cycle vs. Multi-cycle



- **Single-cycle datapath**:
  - Fetch, decode, execute one complete instruction every cycle
  - \+ Low CPI: 1 by definition
  - – Long clock period: to accommodate slowest instruction

- **Multi-cycle datapath**: attacks slow clock
  - Fetch, decode, execute one complete insn over multiple cycles
  - **Allows insns to take different number of cycles**
  - ± Opposite of single-cycle: short clock period, high CPI (think: CISC)

Single-cycle vs. Multi-cycle Performance

- Single-cycle
  - Clock period = 50ns, CPI = 1
  - Performance = **50ns/insn**

- Multi-cycle has opposite performance split of single-cycle
  - \+ Shorter clock period
  - – Higher CPI

- Multi-cycle
  - Branch: 20% (**3** cycles), load: 20% (**5** cycles), ALU: 60% (**4** cycles)
  - Clock period = **11ns**, CPI = (20%*3)+(20%*5)+(60%*4) = 4
    - Why is clock period 11ns and not 10ns? overheads
  - Performance = **44ns/insn**

- Aside: CISC makes perfect sense in multi-cycle datapath

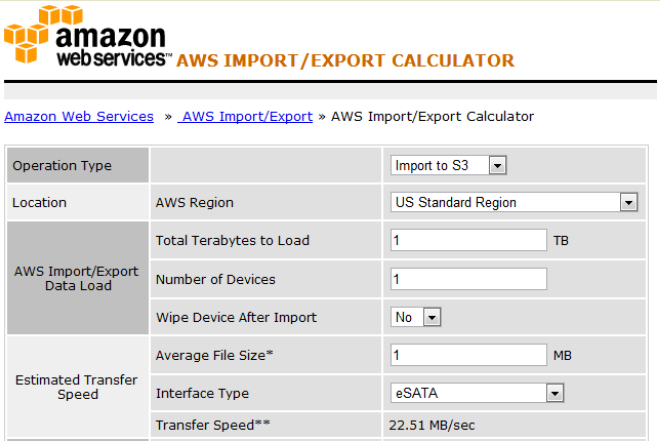
Pipelined Datapath

Performance: Latency vs. Throughput

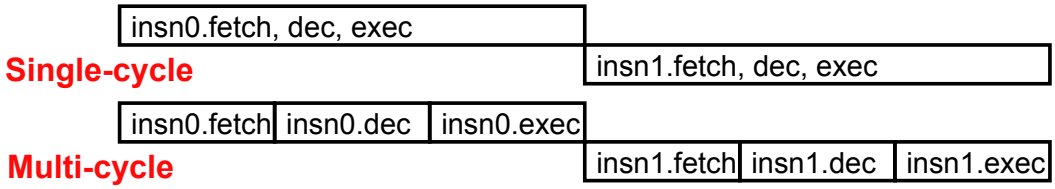
- **Latency (execution time)**: time to finish a fixed task
- **Throughput (bandwidth)**: number of tasks in fixed time
  - Different: exploit parallelism for throughput, not latency (e.g., bread)
  - Often contradictory (latency **vs.** throughput)
    - Will see many examples of this
  - Choose definition of performance that matches your goals
    - Scientific program? Latency, web server: throughput?
- Example: move people 10 miles
  - Car: capacity = 5, speed = 60 miles/hour
  - Bus: capacity = 60, speed = 20 miles/hour
  - Latency: **car = 10 min**, bus = 30 min
  - Throughput: car = 15 PPH (count return trip), **bus = 60 PPH**
- Fastest way to send 10TB of data? (at 1+ gbits/second)

Amazon Does This...

Available Internet Connection	Theoretical Min. Number of Days to Transfer 1TB at 80% Network Utilization	When to Consider AWS Import/Export?
T1 (1.544Mbps)	82 days	100GB or more
10Mbps	13 days	600GB or more
T3 (44.736Mbps)	3 days	2TB or more
100Mbps	1 to 2 days	5TB or more
1000Mbps	Less than 1 day	60TB or more

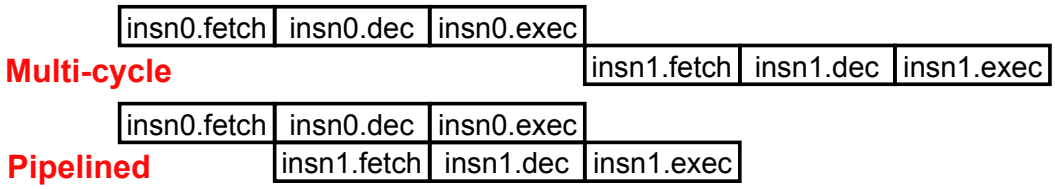


Latency versus Throughput



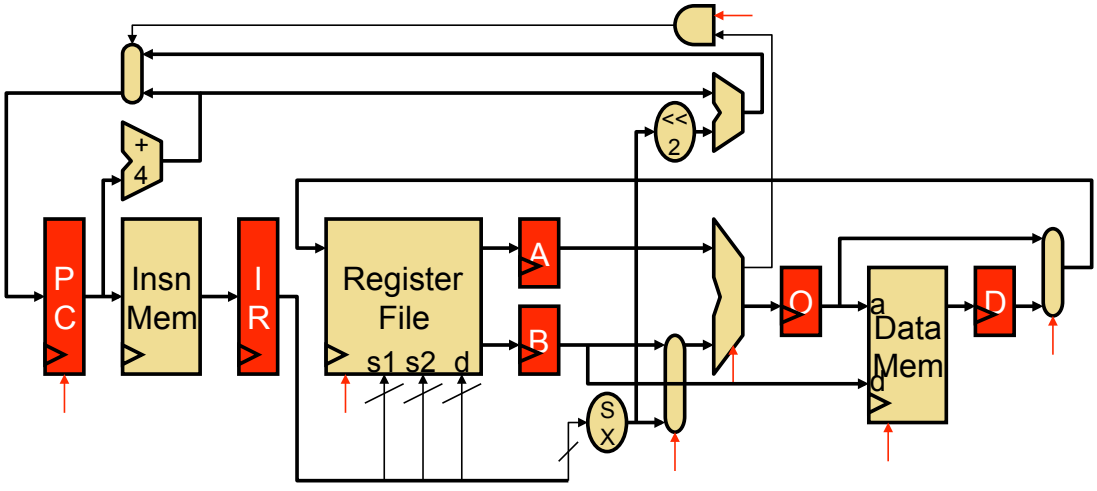
- Can we have both low CPI and short clock period?
  - Not if datapath executes only one insn at a time
- Latency and throughput: two views of performance …
  - (1) at the program level and (2) at the instructions level
- Single instruction latency
  - Doesn't matter: programs comprised of billions of instructions
  - Difficult to reduce anyway
- Goal is to make programs, not individual insns, go faster
  - Instruction throughput → program latency
  - Key: **exploit inter-insn parallelism**

Pipelining

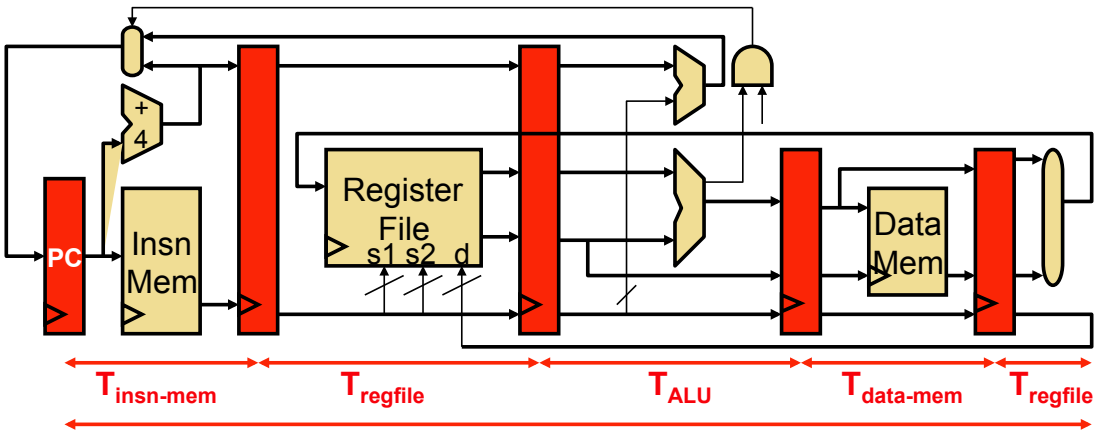


- Important performance technique
  - **Improves instruction throughput rather instruction latency**
- Begin with multi-cycle design
  - When insn advances from stage 1 to 2, next insn enters at stage 1
  - Form of parallelism: "insn-stage parallelism"
  - Maintains illusion of sequential fetch/execute loop
  - Individual instruction takes the same number of stages
  - \+ **But instructions enter and leave at a much faster rate**
- Laundry analogy

5 Stage Multi-Cycle Datapath

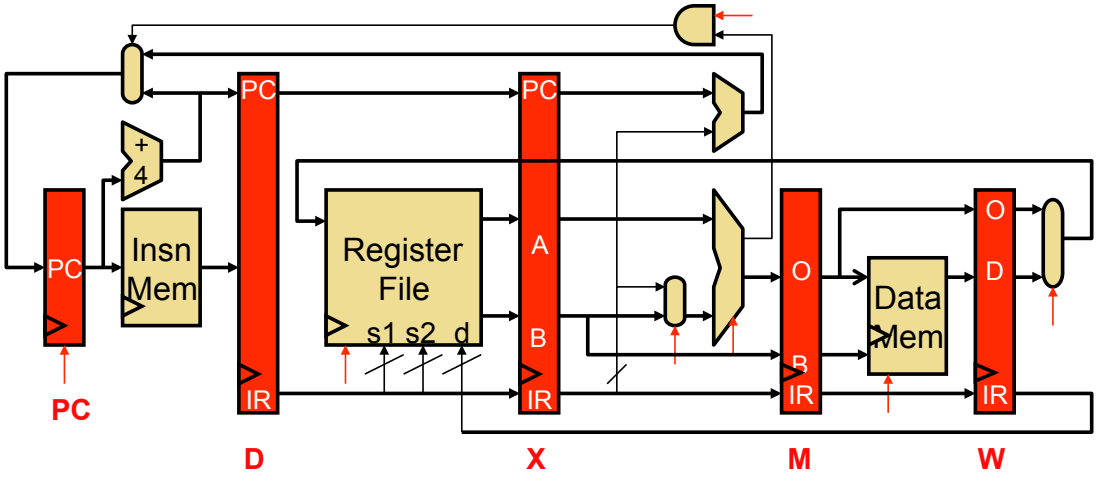


5 Stage Pipeline: Inter-Insn Parallelism



- **Pipelining**: cut datapath into N stages (here 5)
  - One insn in each stage in each cycle
  - \+ Clock period = MAX($T_{insn-mem}$, $T_{regfile}$, T_{ALU}, $T_{data-mem}$)
  - \+ Base CPI = 1: insn enters and leaves every cycle
  - – Actual CPI > 1: pipeline must often "stall"
  - Individual insn latency increases (pipeline overhead), not the point

5 Stage Pipelined Datapath



- Five stage: **F**etch, **D**ecode, e**X**ecute, **M**emory, **W**riteback
  - Nothing magical about 5 stages (Pentium 4 had 22 stages!)
- Latches (pipeline registers) named by stages they begin
  - **PC**, **D**, **X**, **M**, **W**

More Terminology & Foreshadowing

- **Scalar pipeline**: one insn per stage per cycle
  - Alternative: "superscalar" (later)
- **In-order pipeline**: insns enter execute stage in order
  - Alternative: "out-of-order" (later)
- **Pipeline depth**: number of pipeline stages
  - Nothing magical about five
  - Contemporary high-performance cores have ~15 stage pipelines

Instruction Convention

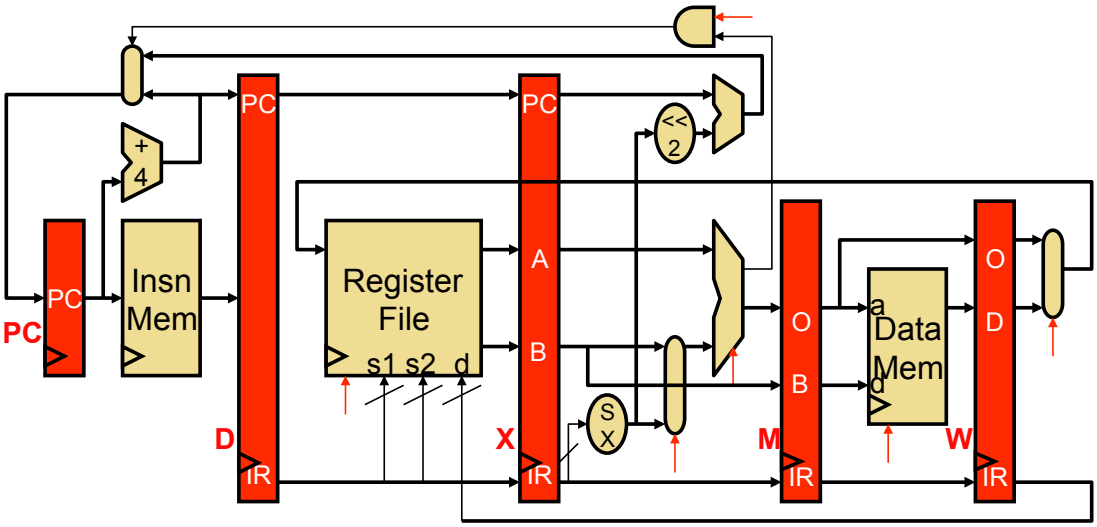
- Different ISAs use inconsistent register orders
- Some ISAs (for example MIPS)
  - Instruction destination (i.e., output) **on the left**
  - add $1, $2, $3 means $1←$2+$3
- Other ISAs
  - Instruction destination (i.e., output) **on the right**

  `add r1,r2,r3` means `r1+r2→r3`

  `ld 8(r5),r4` means `mem[r5+8]→r4`

  `st r4,8(r5)` means `r4→mem[r5+8]`
- Will try to specify to avoid confusion, next slides MIPS style

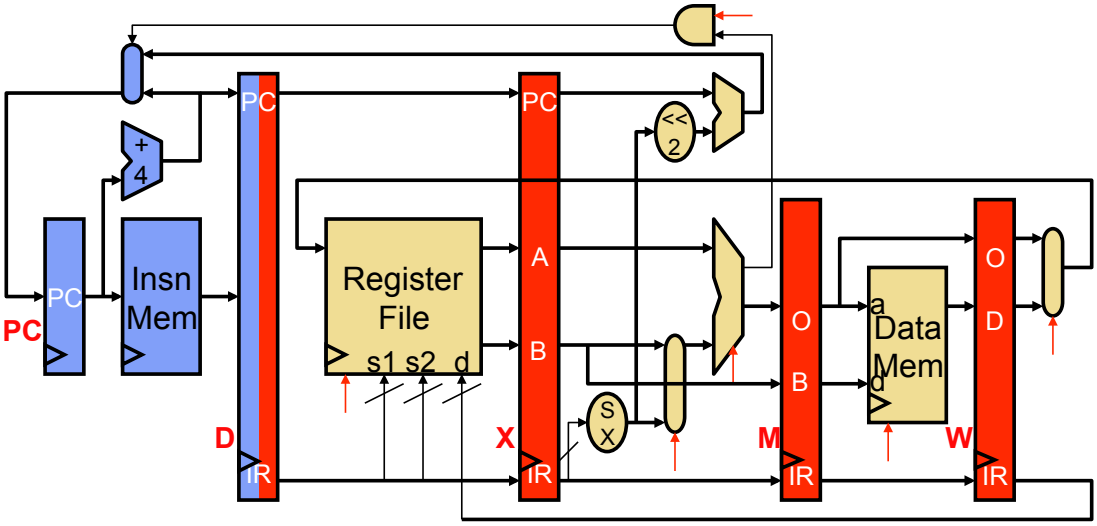
Pipeline Example: Cycle 1




```
add $3,$2,$1
```

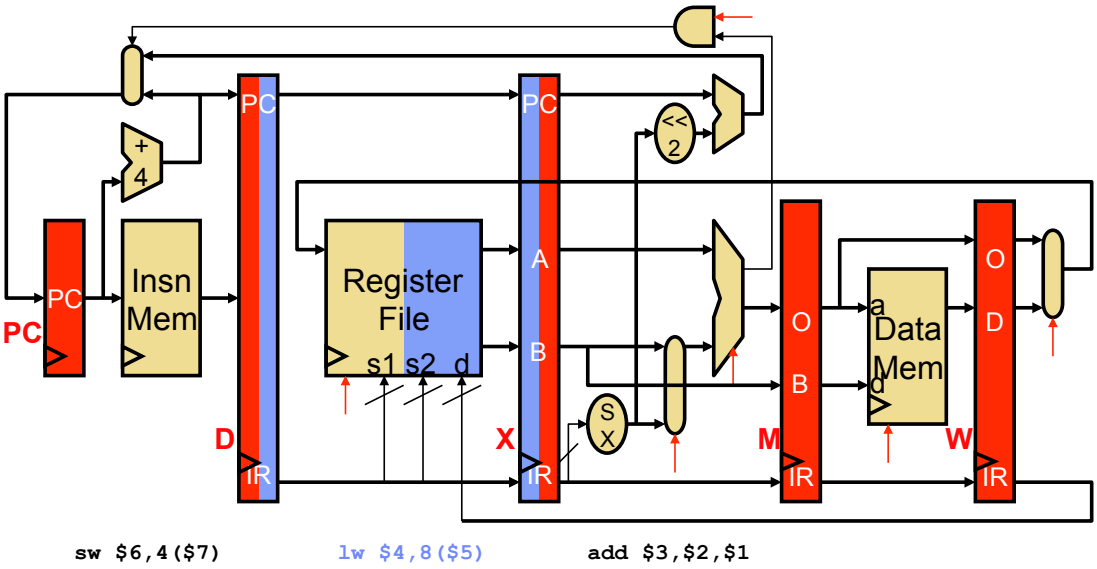
- 3 instructions

Pipeline Example: Cycle 2

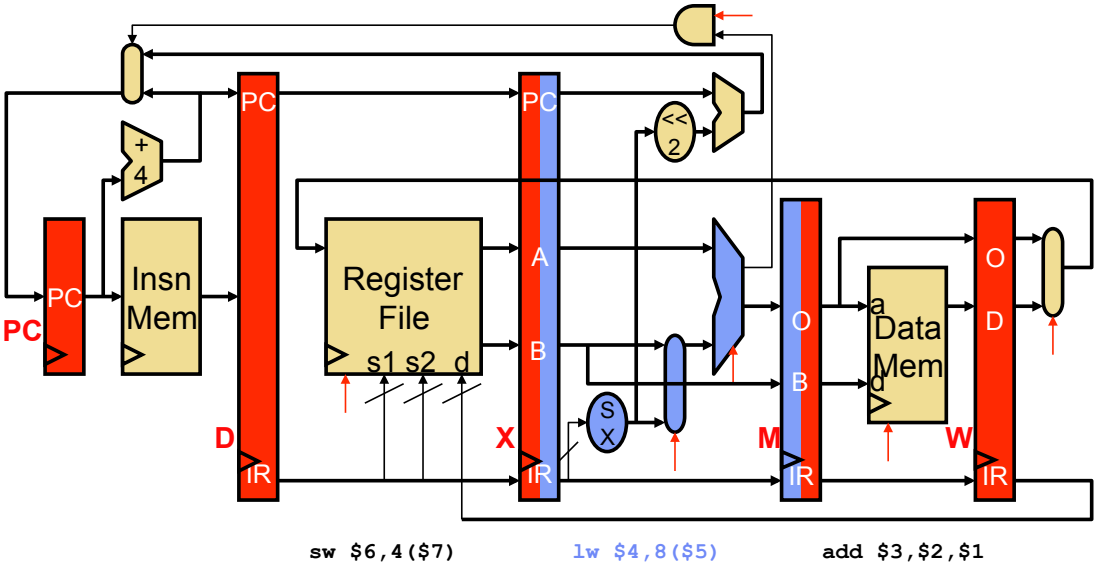



```
lw $4,8($5)          add $3,$2,$1
```

Pipeline Example: Cycle 3

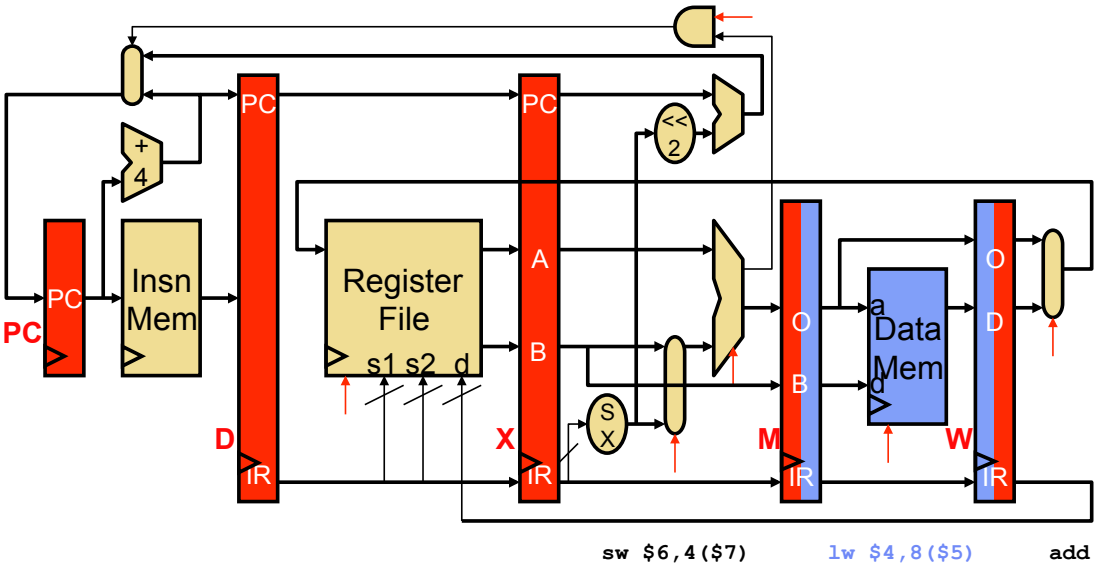


Pipeline Example: Cycle 4

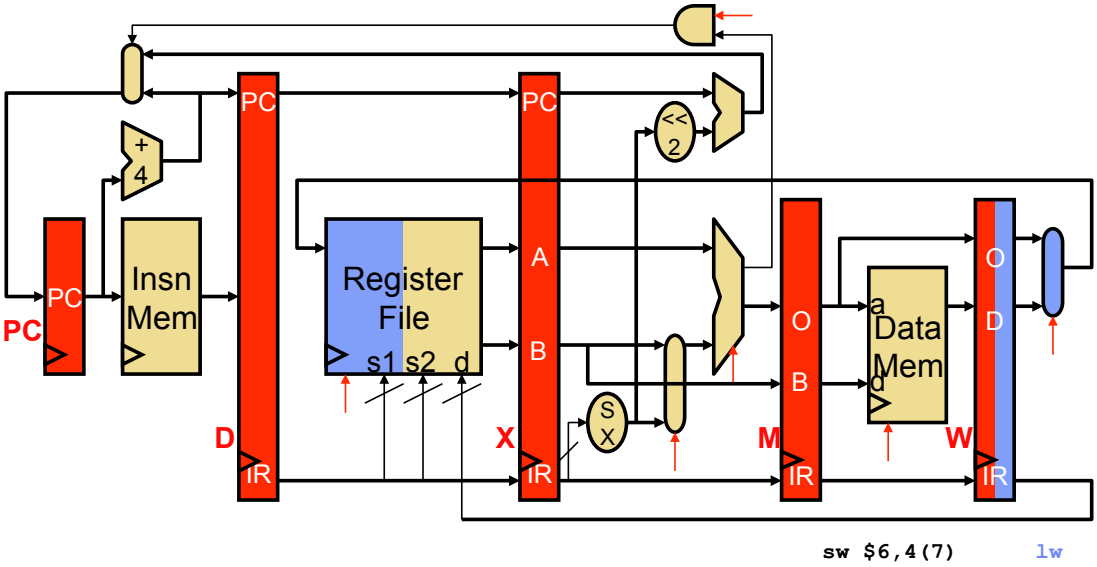


- 3 instructions

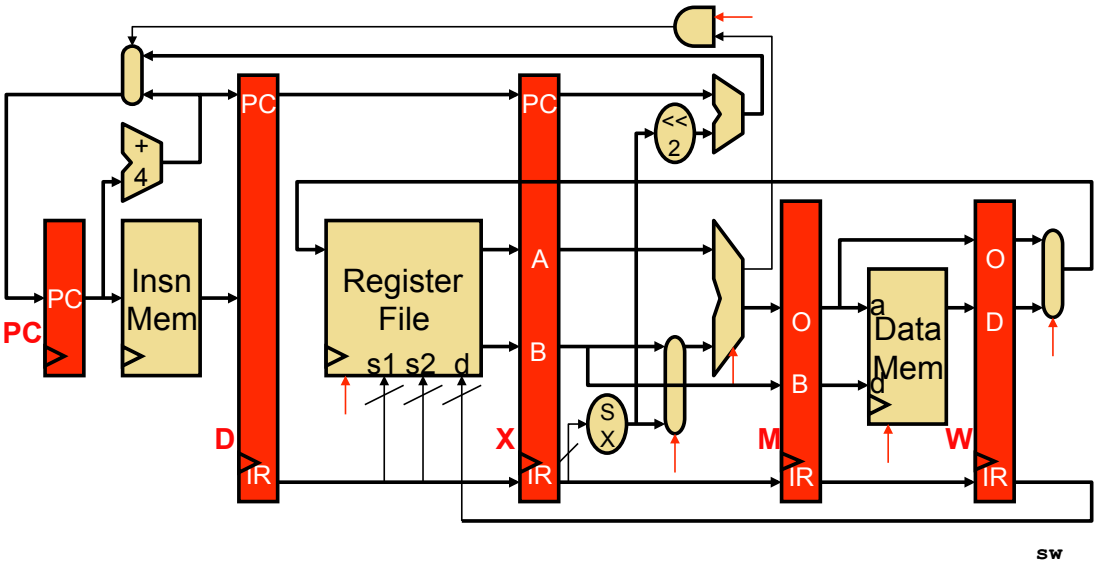
Pipeline Example: Cycle 5



Pipeline Example: Cycle 6



Pipeline Example: Cycle 7



Pipeline Diagram

- **Pipeline diagram**: shorthand for what we just saw
  - Across: cycles
  - Down: insns
  - Convention: **X** means `lw $4,8($5)` finishes execute stage and writes into M latch at end of cycle 4

	1	2	3	4	5	6	7	8	9
`add $3,$2,$1`	F	D	X	M	W				
`lw $4,8($5)`		F	D	**X**	M	W			
`sw $6,4($7)`			F	D	X	M	W		

Example Pipeline Perf. Calculation

- Single-cycle
  - Clock period = 50ns, CPI = 1
  - Performance = 50ns/insn
- Multi-cycle
  - Branch: 20% (3 cycles), load: 20% (5 cycles), ALU: 60% (4 cycles)
  - Clock period = 11ns, CPI = (20%*3)+(20%*5)+(60%*4) = 4
  - Performance = 44ns/insn
- 5-stage pipelined
  - Clock period = **12ns** approx. (50ns / 5 stages) + overheads

  \+ CPI = **1** (each insn takes 5 cycles, but 1 completes each cycle)

  \+ Performance = **12ns/insn**

  – Well actually … CPI = 1 + some penalty for pipelining (next)
    - CPI = **1.5** (on average insn completes every 1.5 cycles)
    - Performance = **18ns/insn**
    - Much higher performance than single-cycle or multi-cycle

Q1: Why Is Pipeline Clock Period …

- … > (delay thru datapath) / (number of pipeline stages)?
  - Three reasons:
    - Latches add delay
    - Pipeline stages have different delays, clock period is max delay
    - Extra datapaths for pipelining (bypassing paths)
  - These factors have implications for ideal number pipeline stages
    - Diminishing clock frequency gains for longer (deeper) pipelines

Q2: Why Is Pipeline CPI…

- … > 1?
  - CPI for scalar in-order pipeline is 1 **+ stall penalties**
  - Stalls used to resolve hazards
    - **Hazard**: condition that jeopardizes sequential illusion
    - **Stall**: pipeline delay introduced to restore sequential illusion

- Calculating pipeline CPI
  - **Frequency of stall** * **stall cycles**
  - Penalties add (stalls generally don't overlap in in-order pipelines)
  - 1 + (stall-freq$_1$*stall-cyc$_1$) + (stall-freq$_2$*stall-cyc$_2$) + …

- Correctness/performance/make common case fast
  - Long penalties OK if they are rare, e.g., 1 + (0.01 * 10) = 1.1
  - Stalls also have implications for ideal number of pipeline stages

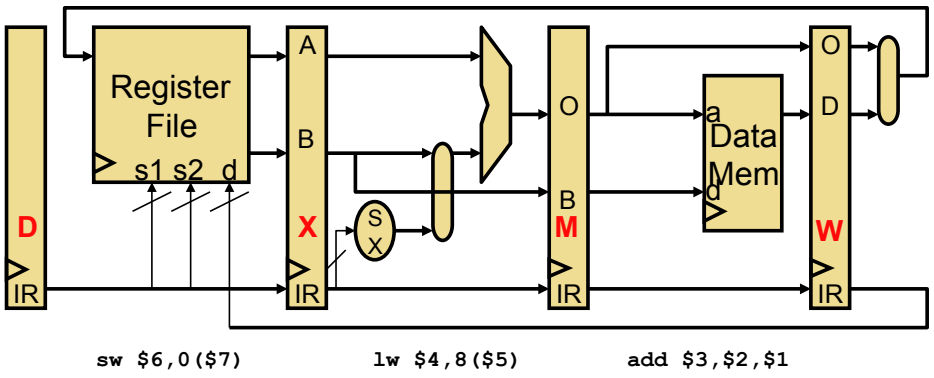
Data Dependences, Pipeline Hazards, and Bypassing

Dependences and Hazards

- **Dependence**: relationship between two insns
  - **Data**: two insns use same storage location
  - **Control**: one insn affects whether another executes at all
  - Not a bad thing, programs would be boring without them
  - Enforced by making older insn go before younger one
    - Happens naturally in single-/multi-cycle designs
    - But not in a pipeline

- **Hazard**: dependence & possibility of wrong insn order
  - Effects of wrong insn order cannot be externally visible
    - **Stall**: for order by keeping younger insn in same stage
  - Hazards are a bad thing: stalls reduce performance

Data Hazards



- Let's forget about branches and the control for a while
- The three insn sequence we saw earlier executed fine…
  - But it wasn't a real program
  - Real programs have **data dependences**
    - They pass values via registers and memory

Dependent Operations

- Independent operations

```
add $3,$2,$1
add $6,$5,$4
```

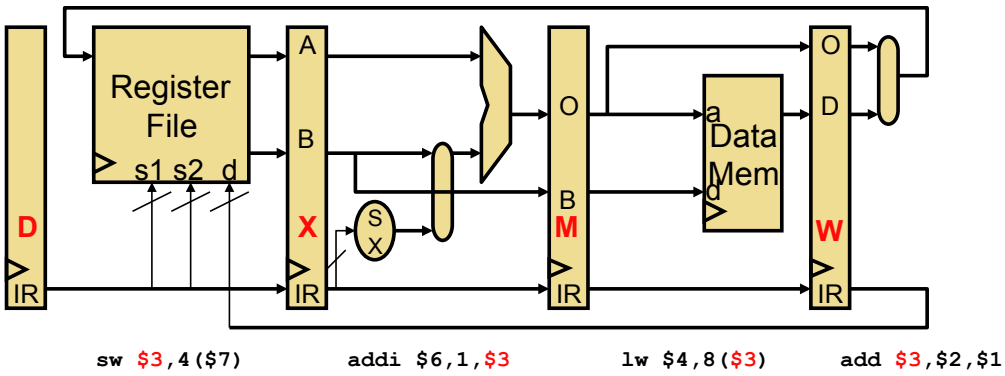
- Would this program execute correctly on a pipeline?

```
add $3,$2,$1
add $6,$5,$3
```

- What about this program?

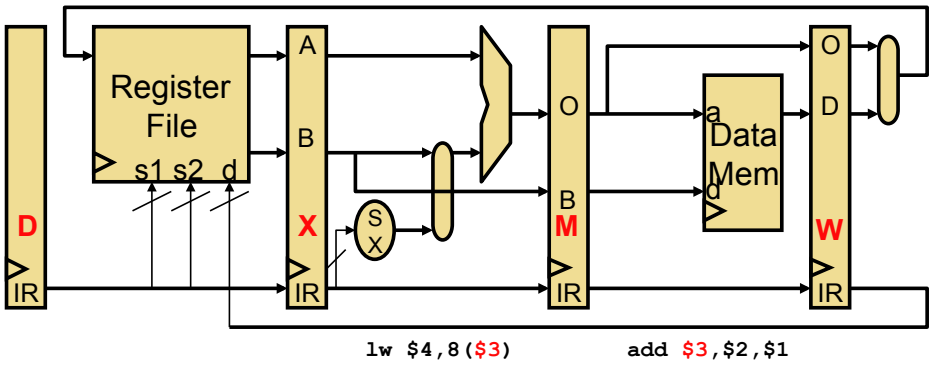
```
add $3,$2,$1
lw $4,8($3)
addi $6,1,$3
sw $3,8($7)
```


Data Hazards



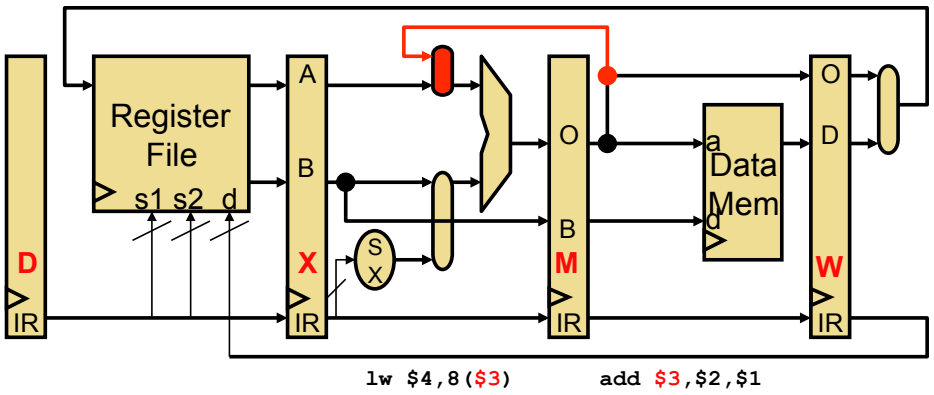
- Would this "program" execute correctly on this pipeline?
  - Which insns would execute with correct inputs?
  - `add` is writing its result into `$3` in current cycle
  - `lw` read `$3` two cycles ago → got wrong value
  - `addi` read `$3` one cycle ago → got wrong value
  - `sw` is reading `$3` this cycle → maybe (depending on regfile design)

Observation!



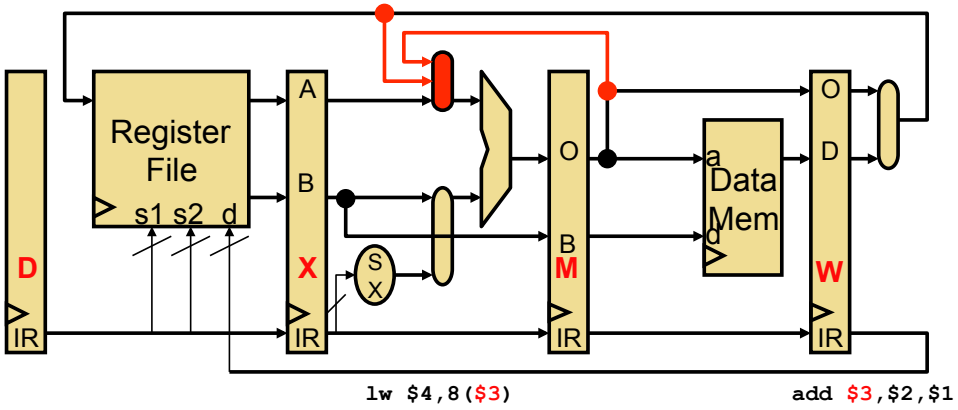
- Technically, this situation is broken
  - `lw $4,8($3)` has already read `$3` from regfile
  - `add $3,$2,$1` hasn't yet written `$3` to regfile
- But fundamentally, everything is OK
  - `lw $4,8($3)` hasn't actually used `$3` yet
  - `add $3,$2,$1` has already computed `$3`

Bypassing



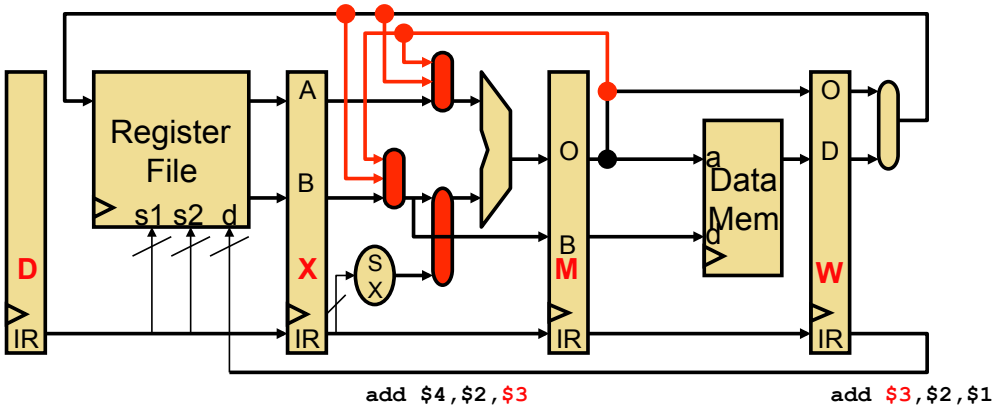
- **Bypassing**
  - Reading a value from an intermediate (μarchitectural) source
  - Not waiting until it is available from primary source
  - Here, we are bypassing the register file
  - Also called **forwarding**

WX Bypassing



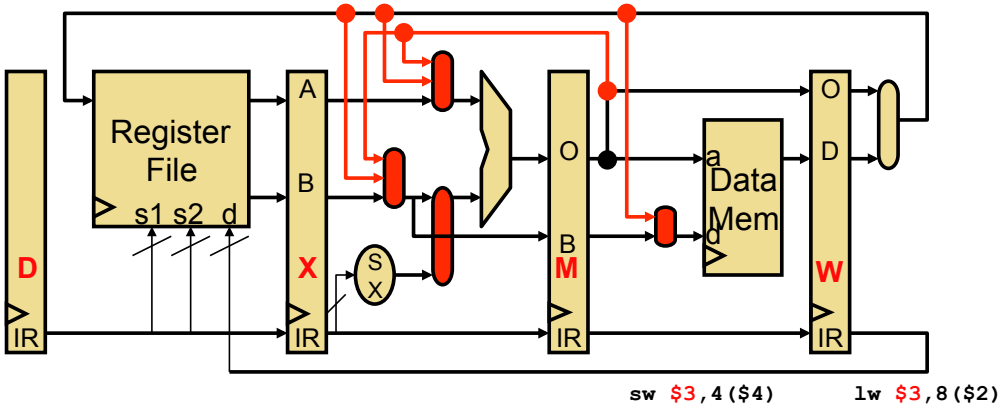
- What about this combination?
  - Add another bypass path and MUX (multiplexor) input
  - First one was an **MX** bypass
  - This one is a **WX** bypass

ALUinB Bypassing



- Can also bypass to ALU input B

WM Bypassing?



- Does WM bypassing make sense?
  - Not to the address input (why not?)

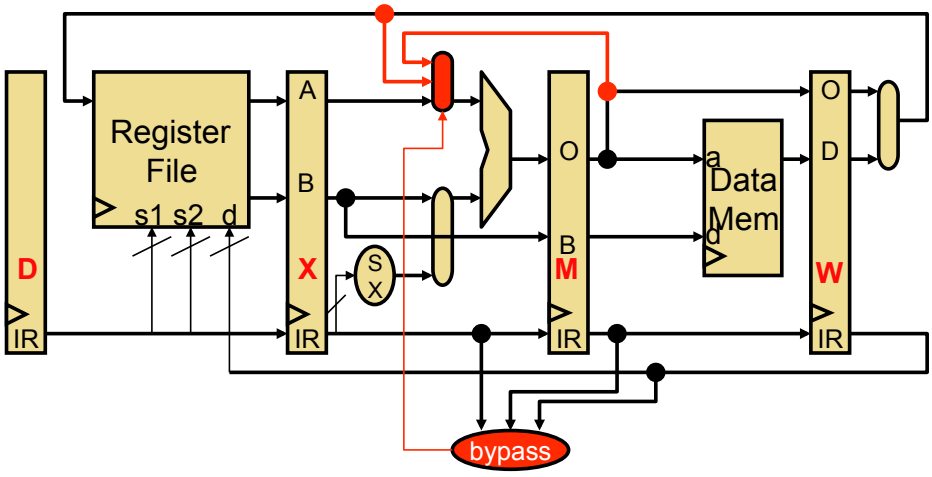
```
sw $4,4($3)      lw $3,8($2)
```

X

  - But to the store data input, yes

```
sw $3,4($4)      lw $3,8($2)
```


Bypass Logic



- Each multiplexor has its own, here it is for "ALUinA"

  (X.IR.RegSrc1 == M.IR.RegDest) => 0
  (X.IR.RegSrc1 == W.IR.RegDest) => 1
  Else => 2

Pipeline Diagrams with Bypassing

- If bypass exists, "from"/"to" stages execute in same cycle
  - Example: MX bypass

```
                    1  2  3  4  5  6  7  8  9  10
add r2,r3➔r1        F  D  X  M  W
sub r1,r4➔r2           F  D  X  M  W
```

  - Example: WX bypass

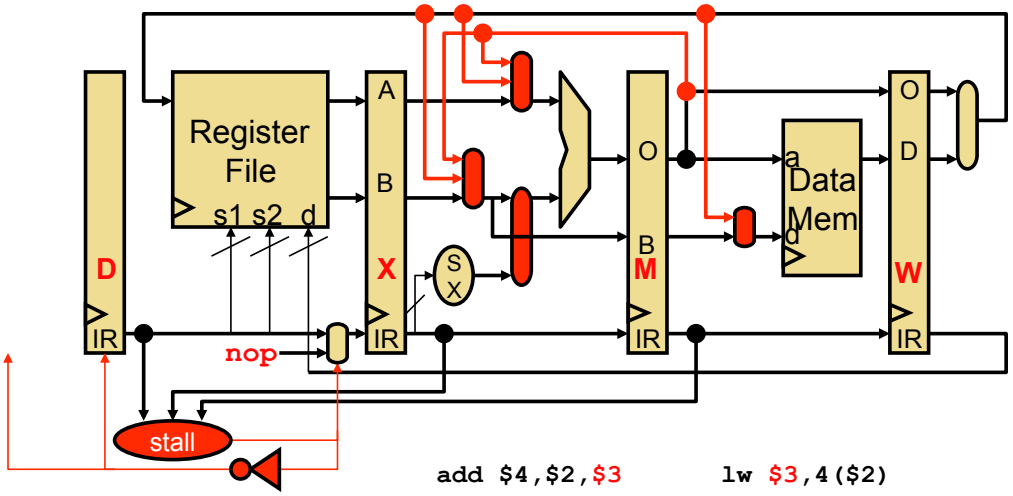
```
                    1  2  3  4  5  6  7  8  9  10
add r2,r3➔r1        F  D  X  M  W
ld [r7+4]➔r5           F  D  X  M  W
sub r1,r4➔r2              F  D  X  M  W
```

  - Example: WM bypass

```
                    1  2  3  4  5  6  7  8  9  10
add r2,r3➔r1        F  D  X  M  W
?                      F  D  X  M  W
```

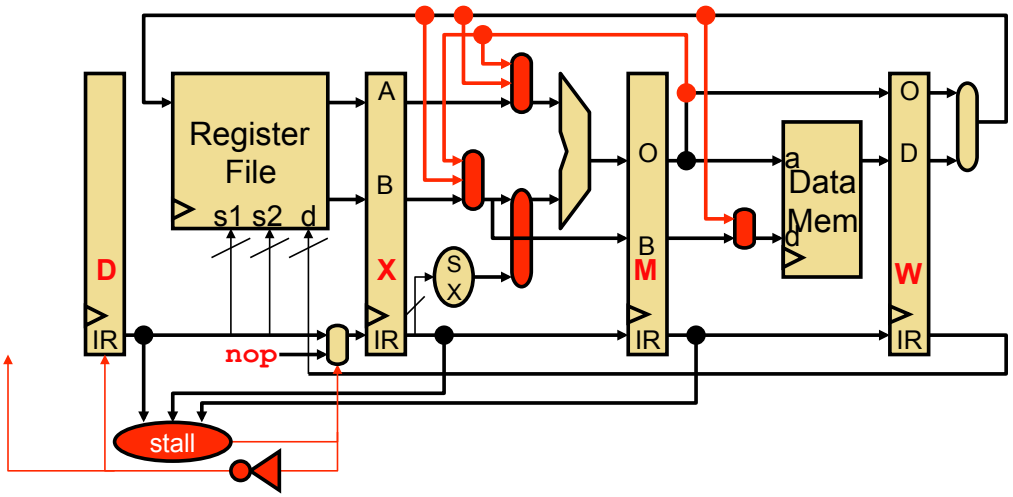
    - Can you think of a code example that uses the WM bypass?

Have We Prevented All Data Hazards?



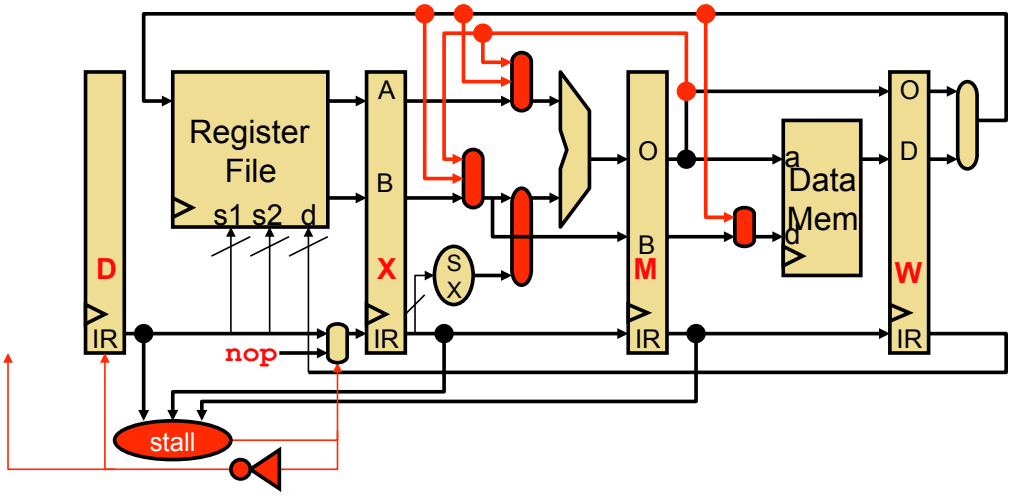
- No. Consider a "load" followed by a dependent "add" insn
- Bypassing alone isn't sufficient!
- Hardware solution: detect this situation and inject a stall cycle
- Software solution: ensure compiler doesn't generate such code

Stalling on Load-To-Use Dependences



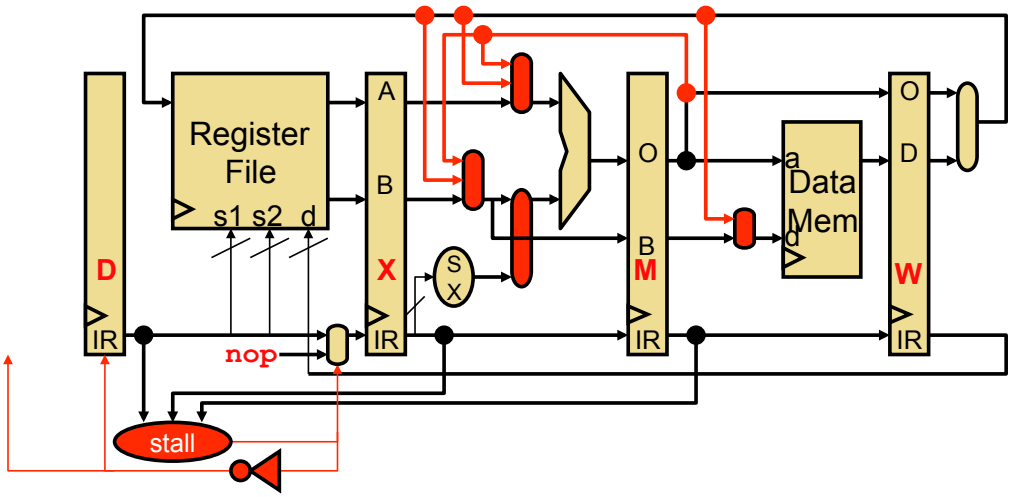
- Prevent "D insn" from advancing this cycle
  - Write `nop` into X.IR (effectively, insert `nop` in hardware)
  - Keep same "D insn", same PC next cycle
- Re-evaluate situation next cycle

Stalling on Load-To-Use Dependences




```
Stall = (X.IR.Operation == LOAD) &&
        (   (D.IR.RegSrc1 == X.IR.RegDest) ||
            ((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
        )
```

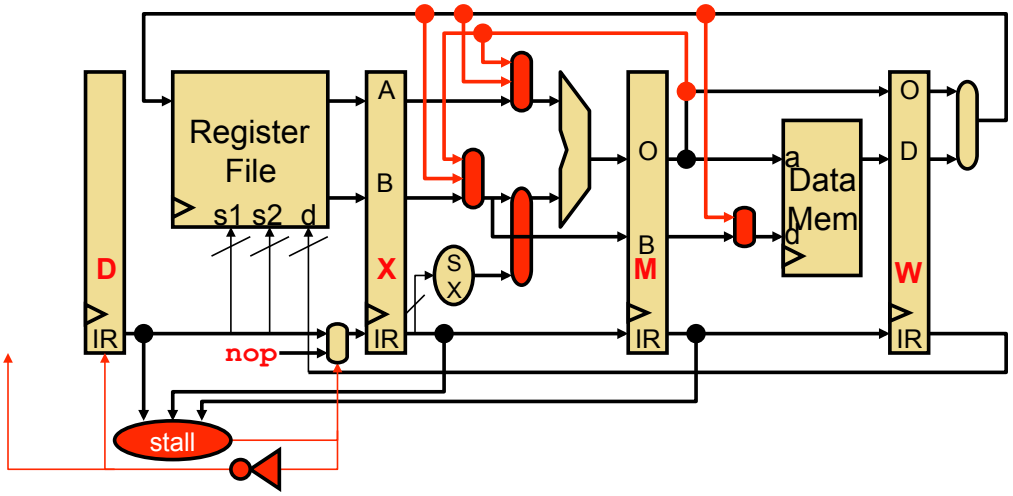
Stalling on Load-To-Use Dependences




```
add $4,$2,$3      (stall bubble)      lw $3,4($2)
```

Stall = (X.IR.Operation == LOAD) &&
((D.IR.RegSrc1 == X.IR.RegDest) ||
((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
)

Stalling on Load-To-Use Dependences




```
add $4,$2,$3      (stall bubble)      lw $3,…
```

Stall = (X.IR.Operation == LOAD) &&
((D.IR.RegSrc1 == X.IR.RegDest) ||
((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
)

Performance Impact of Load/Use Penalty

- Assume
  - Branch: 20%, load: 20%, store: 10%, other: 50%
  - 50% of loads are followed by dependent instruction
    - require 1 cycle stall (I.e., insertion of 1 `nop`)

- Calculate CPI
  - CPI = 1 + (1 * 20% * 50%) = **1.1**

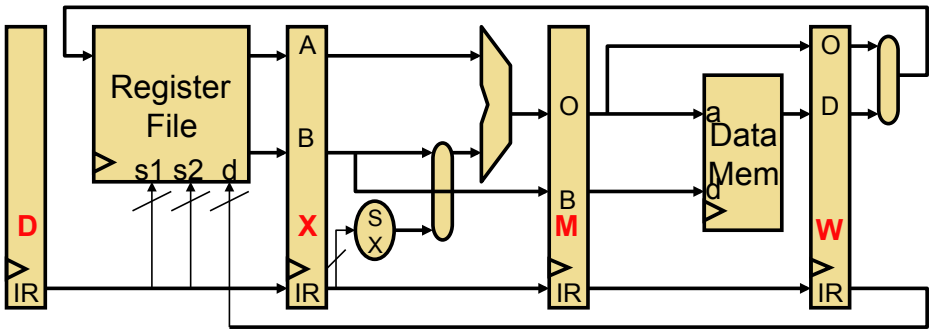
Reducing Load-Use Stall Frequency

	1	2	3	4	5	6	7	8	9
`add $3,$2,$1`	F	D	X	M	W				
`lw $4,4($3)`		F	D	X	M	W			
`addi $6,$4,1`			F	D	**d***	X	M	W	
`sub $8,$3,$1`				F	**d***	D	X	M	W

- Use compiler scheduling to reduce load-use stall frequency
  - More on compiler scheduling later

	1	2	3	4	5	6	7	8	9
`add $3,$2,$1`	F	D	X	M	W				
`lw $4,4($3)`		F	D	X	M	W			
`sub $8,$3,$1`			F	D	X	M	W		
`addi $6,$4,1`				F	D	X	M	W	

Dependencies Through Memory



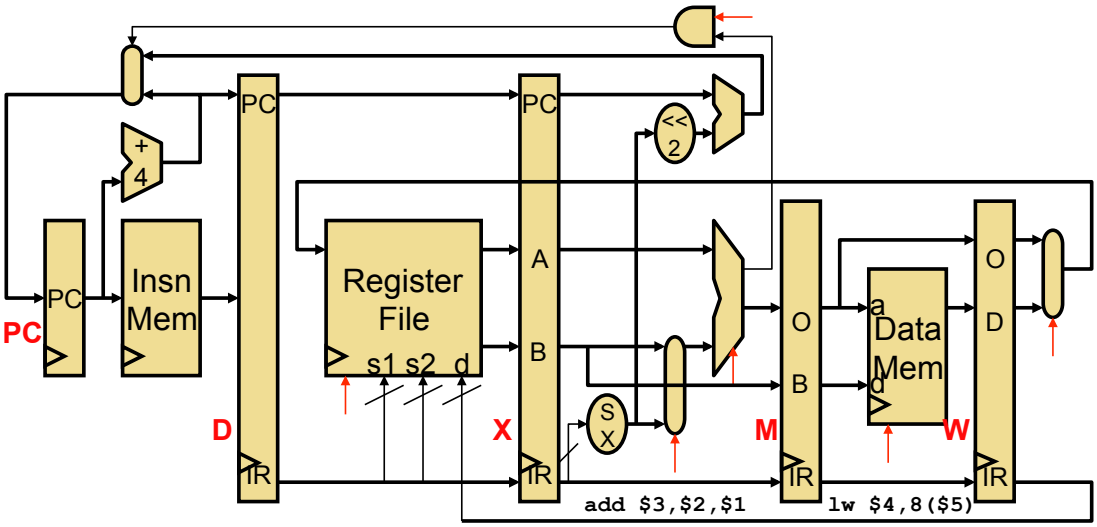
`lw $4,8($1)` `sw $5,8($1)`

- Are "load to store" memory dependencies a problem? No
  - **`lw`** following **`sw`** to same address in next cycle, gets right value
  - Why? Data mem read/write always take place in same stage
- Are there any other sort of hazards to worry about?

Structural Hazards

- **Structural hazards**
  - Two insns trying to use same circuit at same time
    - E.g., structural hazard on register file write port
- **To avoid structural hazards**
  - Avoided if:
    - Each insn uses every structure exactly once
    - For at most one cycle
    - All instructions travel through all stages
  - Add more resources:
    - Example: two memory accesses per cycle (Fetch & Memory)
    - Split instruction & data memories allows simultaneous access
- **Tolerate structure hazards**
  - Add stall logic to stall pipeline when hazards occur

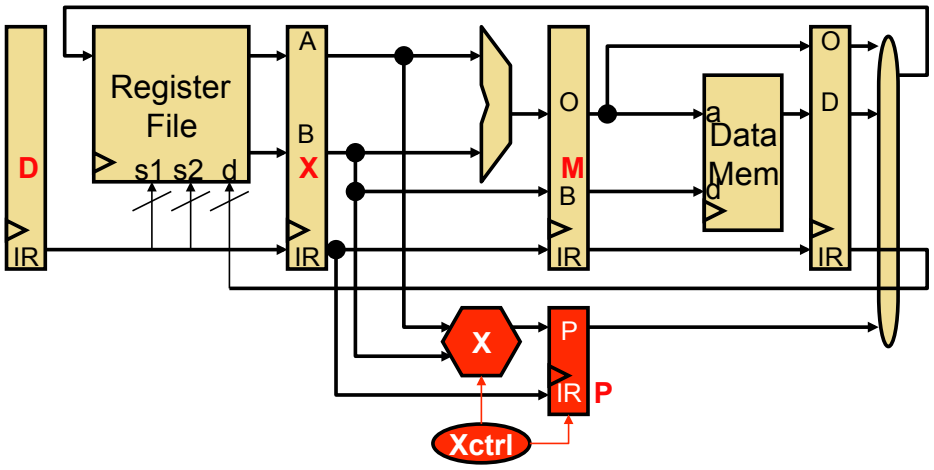
Why Does Every Insn Take 5 Cycles?



- Could/should we allow **`add`** to skip M and go to W? No
  - It wouldn't help: peak fetch still only 1 insn per cycle
  - **Structural hazards**: imagine **`add`** after **`lw`** (only 1 reg. write port)

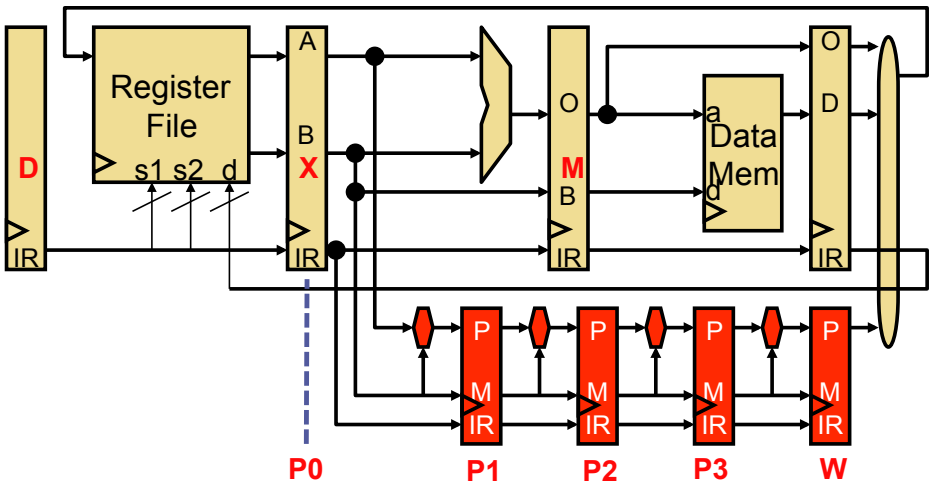
Multi-Cycle Operations
(if time permits)

Pipelining and Multi-Cycle Operations



- What if you wanted to add a multi-cycle operation?
  - E.g., 4-cycle multiply
  - **P**: separate output latch connects to W stage
  - Controlled by pipeline control finite state machine (FSM)

A Pipelined Multiplier



- Multiplier itself is often pipelined, what does this mean?
  - Product/multiplicand register/ALUs/latches replicated
  - Can start different multiply operations in consecutive cycles
  - **But still takes 4 cycles to generate output value**

Pipeline Diagram with Multiplier

- Allow independent instructions

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $6,$7,1`		F	D	X	M	W			

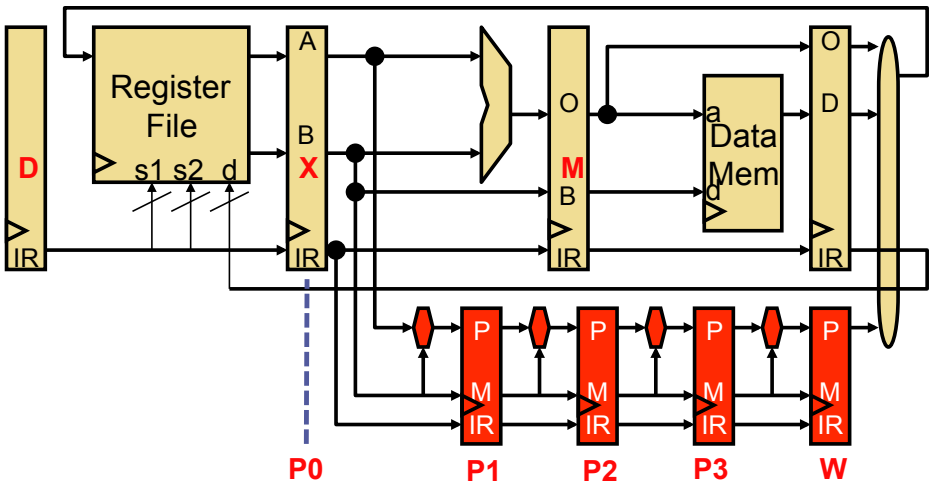
- Even allow independent multiplies

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`mul $6,$7,$8`		F	D	P0	P1	P2	P3	W	

- But must stall subsequent dependent instructions:

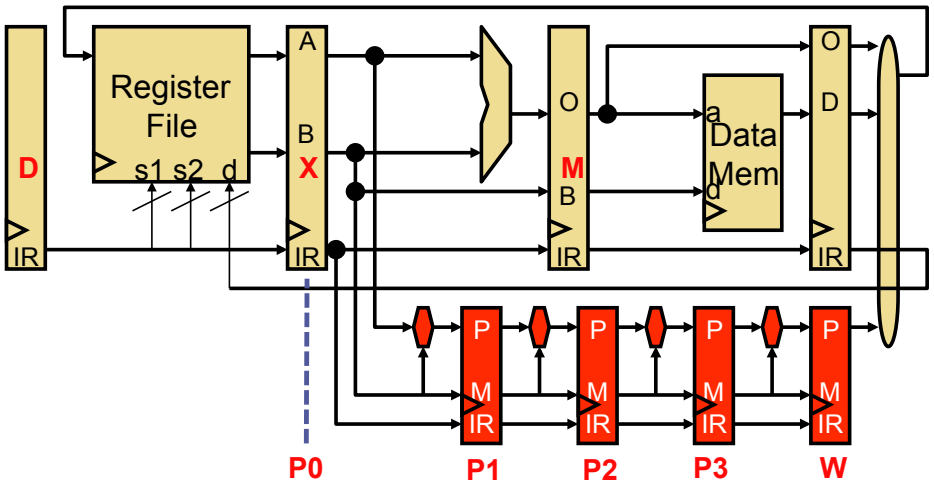
	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $6,$4,1`		F	D	**d***	**d***	**d***	X	M	W

What about Stall Logic?



	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $6,$4,1`		F	D	**d***	**d***	**d***	X	M	W

What about Stall Logic?



Stall = (OldStallLogic) ||
(D.IR.RegSrc1 == P0.IR.RegDest) || (D.IR.RegSrc2 == P0.IR.RegDest) ||
(D.IR.RegSrc1 == P1.IR.RegDest) || (D.IR.RegSrc2 == P1.IR.RegDest) ||
(D.IR.RegSrc1 == P2.IR.RegDest) || (D.IR.RegSrc2 == P2.IR.RegDest)

Multiplier Write Port Structural Hazard

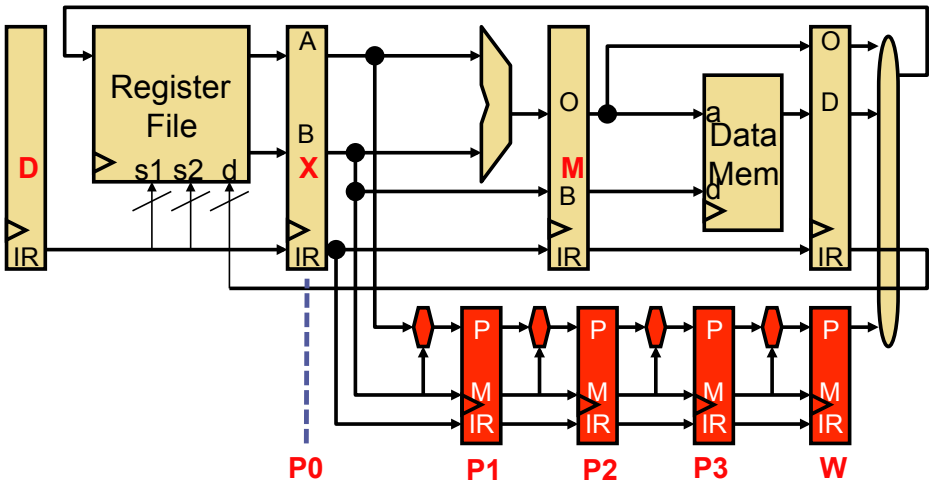
- What about…
  - Two instructions trying to write register file in same cycle?
  - Structural hazard!
- Must prevent:

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $6,$1,1`		F	D	X	M	W			
`add $5,$6,$10`			F	D	X	M	**W**		

- Solution? stall the subsequent instruction

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $6,$1,1`		F	D	X	M	W			
`add $5,$6,$10`			F	**d***	D	X	M	**W**	

Preventing Structural Hazard



- Fix to problem on previous slide:

  Stall = (OldStallLogic) ||

  (**D.IR.RegDest "is valid" &&**

  D.IR.Operation != MULT && P0.IR.RegDest "is valid")

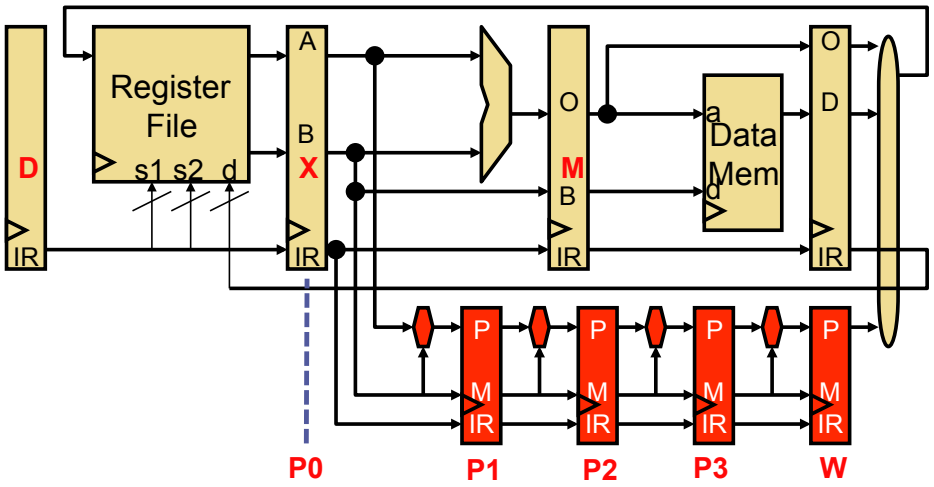
More Multiplier Nasties

- What about…
  - Mis-ordered writes to the same register
  - Software thinks **`add`** gets **`$4`** from **`addi`**, actually gets it from **`mul`**

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $4,$1,1`		F	D	X	M	**W**			
…									
…									
`add $10,$4,$6`					F	D	X	M	W

- Common? Not for a 4-cycle multiply with 5-stage pipeline
  - More common with deeper pipelines
  - In any case, must be correct

Preventing Mis-Ordered Reg. Write



- Fix to problem on previous slide:
  Stall = (OldStallLogic) ||
  ((**D.IR.RegDest == X.IR.RegDest) && (X.IR.Operation == MULT)**)

Corrected Pipeline Diagram

- With the correct stall logic
  - Prevent mis-ordered writes to the same register
  - Why two cycles of delay?

	1	2	3	4	5	6	7	8	9
`mul $4,$3,$5`	F	D	P0	P1	P2	P3	W		
`addi $4,$1,1`		F	**d***	**d***	D	X	M	**W**	
…									
…									
`add $10,$4,$6`					F	D	X	M	W

- **Multi-cycle operations complicate pipeline logic**

Pipelined Functional Units

- Almost all multi-cycle functional units are pipelined
  - Each operation takes N cycles
  - But can start initiate a new (independent) operation every cycle
  - Requires internal latching and some hardware replication
  - + A cheaper way to add bandwidth than multiple non-pipelined units

	1	2	3	4	5	6	7	8	9	10	11
`mulf f0,f1,f2`	F	D	E*	E*	E*	E*	W				
`mulf f3,f4,f5`		F	D	E*	E*	E*	E*	W			

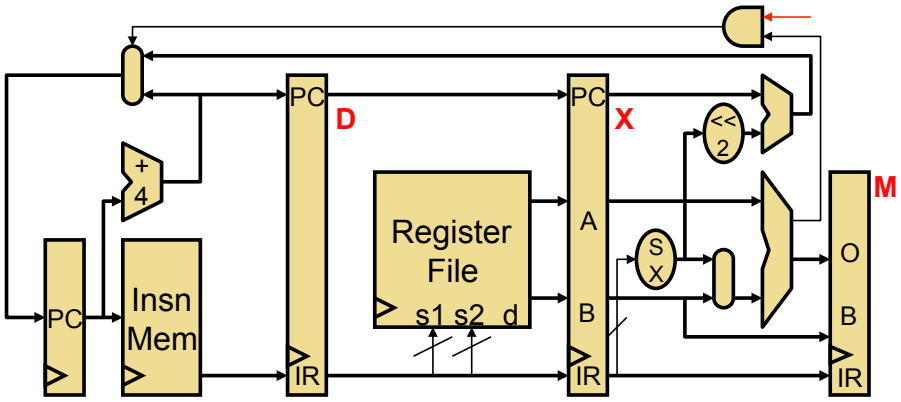
  - One exception: int/FP divide: difficult to pipeline and not worth it

	1	2	3	4	5	6	7	8	9	10	11
`divf f0,f1,f2`	F	D	E/	E/	E/	E/	W				
`divf f3,f4,f5`		F	D	**s***	**s***	**s***	E/	E/	E/	E/	W

  - **s* = structural hazard, two insns need same structure**
    - ISAs and pipelines designed to have few of these
    - Canonical example: all insns forced to go through M stage

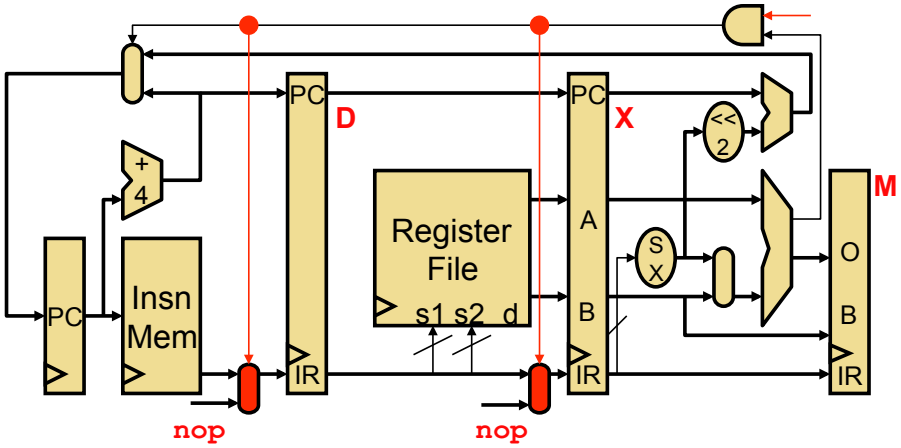
Control Dependences and Branch Prediction

What About Branches?



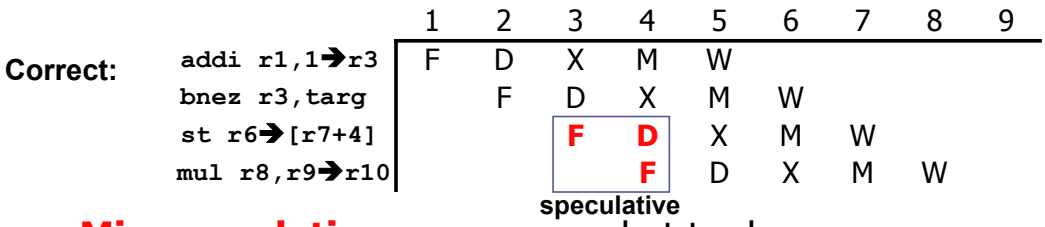
- **Branch speculation**
  - Could just stall to wait for branch outcome (two-cycle penalty)
  - **Fetch past branch insns before branch outcome is known**
    - Default: assume "**not-taken**" (at fetch, can't tell it's a branch)

Branch Recovery

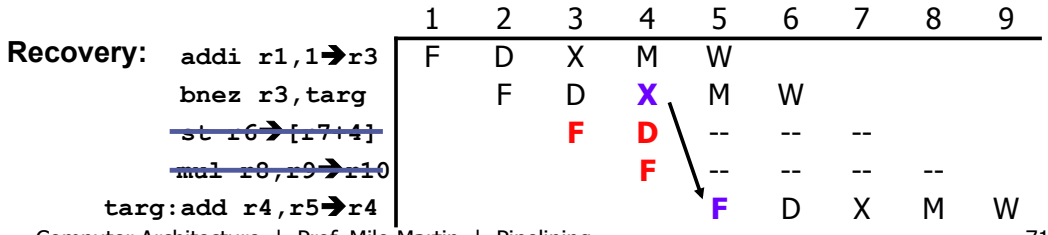


- **Branch recovery**: what to do when branch is actually taken
  - Insns that will be written into D and X are wrong
  - **Flush them**, i.e., replace them with `nops`
  - \+ They haven't had written permanent state yet (regfile, DMem)
  - – Two cycle penalty for taken branches

Branch Speculation and Recovery



- **Mis-speculation recovery**: what to do on wrong guess
  - Not too painful in an short, in-order pipeline
  - Branch resolves in X
  - \+ Younger insns (in F, D) haven't changed permanent state
  - **Flush** insns currently in D and X (i.e., replace with `nops`)



Branch Performance

- Back of the envelope calculation
  - **Branch: 20%**, load: 20%, store: 10%, other: 50%
  - Say, **75% of branches are taken**

- CPI = 1 + 20% * 75% * 2 =
  1 + **0.20 * 0.75 * 2** = 1.3
  - **Branches cause 30% slowdown**
    - Worse with deeper pipelines (higher mis-prediction penalty)

- Can we do better than assuming branch is not taken?

Big Idea: Speculative Execution

- Speculation: "risky transactions on chance of profit"

- **Speculative execution**
  - Execute before all parameters known with certainty
  - **Correct speculation**
    + Avoid stall, improve performance
  - **Incorrect speculation (mis-speculation)**
    – Must abort/flush/squash incorrect insns
    – Must undo incorrect changes (recover pre-speculation state)

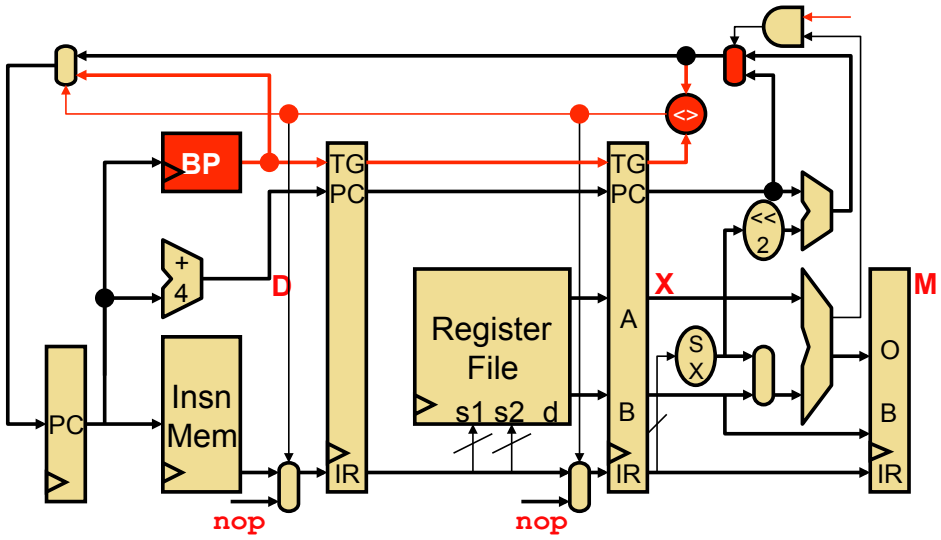
- **Control speculation**: speculation aimed at control hazards
  - Unknown parameter: are these the correct insns to execute next?

Control Speculation Mechanics

- Guess branch target, start fetching at guessed position
  - Doing nothing is implicitly guessing target is PC+4
  - Can actively guess other targets: **dynamic branch prediction**

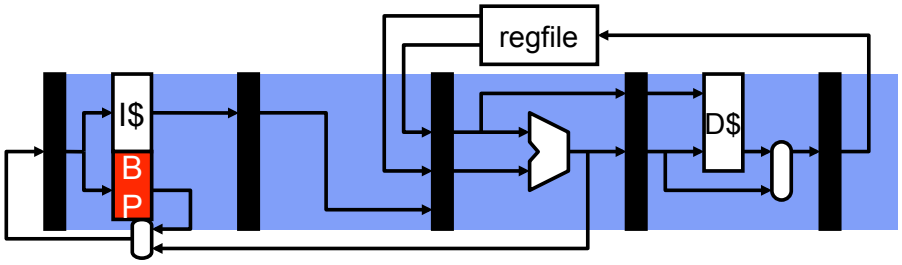
- Execute branch to verify (check) guess
  - Correct speculation? keep going
  - Mis-speculation? Flush mis-speculated insns
    - Hopefully haven't modified permanent state (Regfile, DMem)
    + Happens naturally in in-order 5-stage pipeline

Dynamic Branch Prediction



- **Dynamic branch prediction**: hardware guesses outcome
  - Start fetching from guessed address
  - Flush on **mis-prediction**

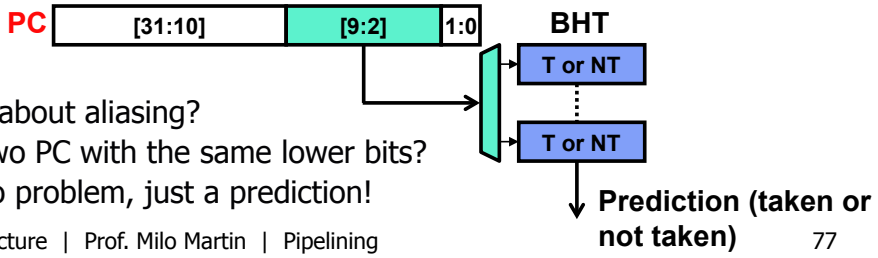
Dynamic Branch Prediction Components



- Step #1: is it a branch?
  - Easy after decode...
- Step #2: is the branch taken or not taken?
  - **Direction predictor** (applies to conditional branches only)
  - Predicts taken/not-taken
- Step #3: if the branch is taken, where does it go?
  - Easy after decode…

Branch Direction Prediction

- **Learn from past, predict the future**
  - Record the past in a hardware structure
- **Direction predictor (DIRP)**
  - Map conditional-branch PC to taken/not-taken (T/N) decision
  - Individual conditional branches often biased or weakly biased
    - 90%+ one way or the other considered **"biased"**
    - Why? Loop back edges, checking for uncommon conditions
- **Branch history table (BHT)**: simplest predictor
  - PC indexes table of bits (0 = N, 1 = T), no tags
  - Essentially: branch will go same way it went last time



  - What about aliasing?
    - Two PC with the same lower bits?
    - No problem, just a prediction!

Branch History Table (BHT)

- **Branch history table (BHT)**: simplest direction predictor
  - PC indexes table of bits (0 = N, 1 = T), no tags
  - Essentially: branch will go same way it went last time
  - Problem: **inner loop branch** below

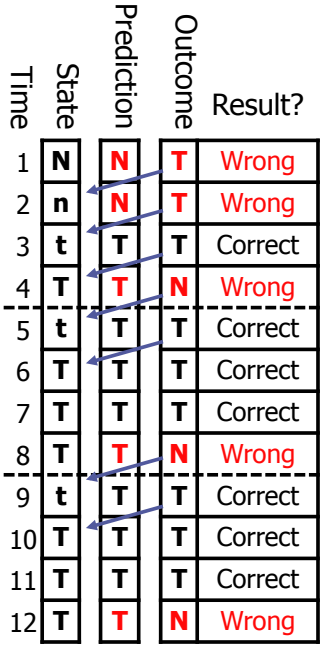
```
for (i=0;i<100;i++)
    for (j=0;j<3;j++)
        // whatever
```

    - Two "built-in" mis-predictions per inner loop iteration
    - Branch predictor "changes its mind too quickly"

Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	T	T	T	Correct
3	T	T	T	Correct
4	T	T	N	Wrong
5	N	N	T	Wrong
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	N	N	T	Wrong
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

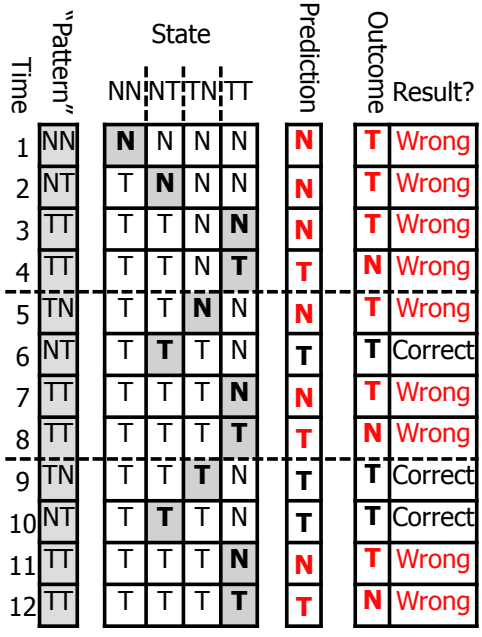
Two-Bit Saturating Counters (2bc)

- **Two-bit saturating counters (2bc)** [Smith 1981]
  - Replace each single-bit prediction
    - (0,1,2,3) = (N,n,t,T)
  - Adds "hysteresis"
    - Force predictor to mis-predict twice before "changing its mind"
  - One mispredict each loop execution (rather than two)
    - \+ Fixes this pathology (which is not contrived, by the way)
    - Can we do even better?



Correlated Predictor

- **Correlated (two-level) predictor** [Patt 1991]
  - Exploits observation that branch outcomes are correlated
  - Maintains separate prediction per (PC, BHR) pairs
    - **Branch history register (BHR)**: recent branch outcomes
  - Simple working example: assume program has one branch
    - BHT: one 1-bit DIRP entry
    - BHT+**2BHR**: 2^2 = **4** 1-bit DIRP entries
  - – Why didn't we do better?
    - BHT not long enough to capture pattern



Correlated Predictor – 3 Bit Pattern

- **Try 3 bits of history**
- 2^3 DIRP entries per pattern

Time	"Pattern"	State NNN	NNT	NTN	NTT	TNN	TNT	TTN	TTT	Prediction	Outcome	Result?
1	NNN	**N**	N	N	N	N	N	N	N	**N**	**T**	Wrong
2	NNT	T	**N**	N	N	N	N	N	N	**N**	**T**	Wrong
3	NTT	T	T	N	**N**	N	N	N	N	**N**	**T**	Wrong
4	TTT	T	T	N	T	N	N	N	**N**	**N**	**N**	Correct
5	TTN	T	T	N	T	N	N	**N**	N	**N**	**T**	Wrong
6	TNT	T	T	N	T	N	**N**	T	N	**N**	**T**	Wrong
7	NTT	T	T	N	**T**	N	T	T	N	**T**	**T**	Correct
8	TTT	T	T	N	T	N	T	T	**N**	**N**	**N**	Correct
9	TTN	T	T	N	T	N	T	**T**	N	**T**	**T**	Correct
10	TNT	T	T	N	T	N	**T**	T	N	**T**	**T**	Correct
11	NTT	T	T	N	**T**	N	T	T	N	**T**	**T**	Correct
12	TTT	T	T	N	T	N	T	T	**N**	**N**	**N**	Correct

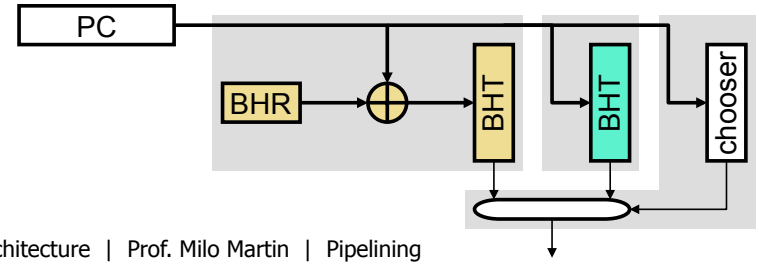
+ No mis-predictions after predictor learns all the relevant patterns!

Correlated Predictor Design

- Design choice: how many history bits (BHR size)?
  - Tricky one
  - \+ Given unlimited resources, longer BHRs are better, but…
  - – BHT utilization decreases
    - – Many history patterns are never seen
    - – Many branches are history independent (don't care)
    - • PC xor BHR allows multiple PCs to dynamically share BHT
    - • BHR length < $\log_2$(BHT size)
  - – Predictor takes longer to train
  - • Typical length: 8–12

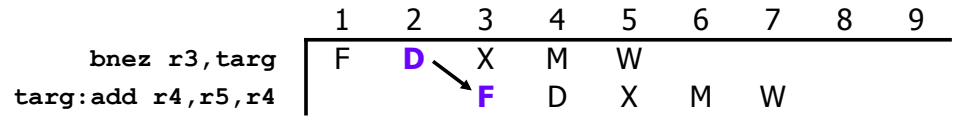
Hybrid Predictor

- **Hybrid (tournament) predictor** [McFarling 1993]
  - Attacks correlated predictor BHT capacity problem
  - Idea: combine two predictors
    - **Simple BHT** predicts history independent branches
    - **Correlated predictor** predicts only branches that need history
    - **Chooser** assigns branches to one predictor or the other
    - Branches start in simple BHT, move mis-prediction threshold
  - \+ Correlated predictor can be made **smaller**, handles fewer branches
  - \+ 90–95% accuracy



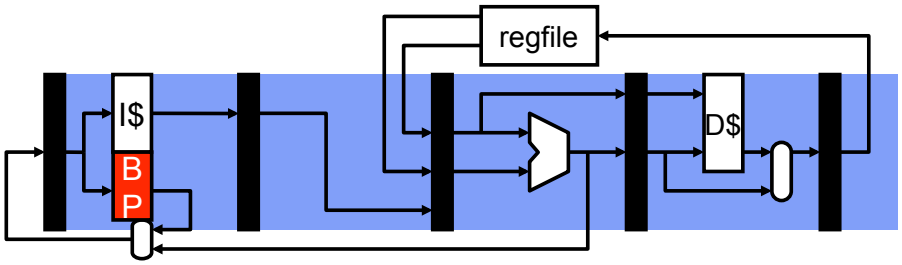
When to Perform Branch Prediction?

- Option #1: During Decode
  - Look at instruction opcode to determine branch instructions
  - Can calculate next PC from instruction (for PC-relative branches)
  - – One cycle "mis-fetch" penalty **even if branch predictor is correct**



- Option #2: During Fetch?
  - How do we do that?

Revisiting Branch Prediction Components



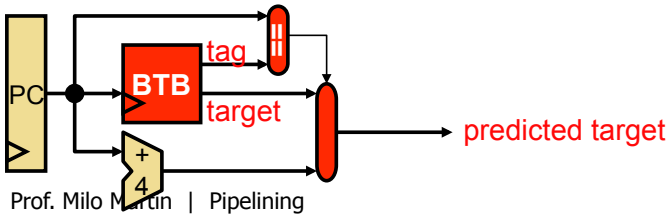
- Step #1: is it a branch?
  - Easy after decode... during fetch: **predictor**
- Step #2: is the branch taken or not taken?
  - **Direction predictor** (as before)
- Step #3: if the branch is taken, where does it go?
  - **Branch target predictor (BTB)**
  - Supplies target PC if branch is taken

Branch Target Buffer (BTB)

- As before: learn from past, predict the future
  - Record the past branch targets in a hardware structure
- **Branch target buffer (BTB)**:
  - "guess" the future PC based on past behavior
  - "Last time the branch X was taken, it went to address Y"
    - "So, in the future, if address X is fetched, fetch address Y next"
- Operation
  - A small RAM: address = PC, data = target-PC
  - Access at Fetch *in parallel* with instruction memory
    - predicted-target = BTB[hash(PC)]
  - Updated at X whenever target != predicted-target
    - BTB[hash(PC)] = target
  - Hash function is just typically just extracting lower bits (as before)
  - Aliasing? No problem, this is only a prediction

Branch Target Buffer (continued)

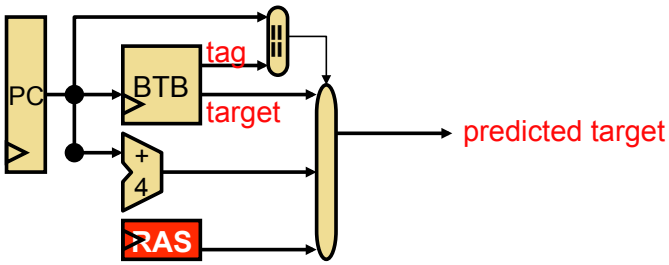
- At Fetch, how does insn know it's a branch & should read BTB? It doesn't have to…
  - **…all insns access BTB in parallel with Imem Fetch**
- Key idea: **use BTB to predict which insn are branches**
  - Implement by "tagging" each entry with its corresponding PC
  - Update BTB on every taken branch insn, record target PC:
    - BTB[PC].tag = PC, BTB[PC].target = target of branch
  - All insns access at Fetch *in parallel* with Imem
    - Check for tag match, signifies insn at that PC is a branch
    - Predicted PC = (BTB[PC].tag == PC) ? BTB[PC].target : PC+4



Why Does a BTB Work?

- Because most control insns use **direct targets**
  - Target encoded in insn itself → same "taken" target every time
- What about **indirect targets**?
  - Target held in a register → can be different each time
  - Two indirect call idioms
    - \+ Dynamically linked functions (DLLs): target always the same
    - Dynamically dispatched (virtual) functions: hard but uncommon
  - Also two indirect unconditional jump idioms
    - Switches: hard but uncommon
    - – Function returns: hard and common but…

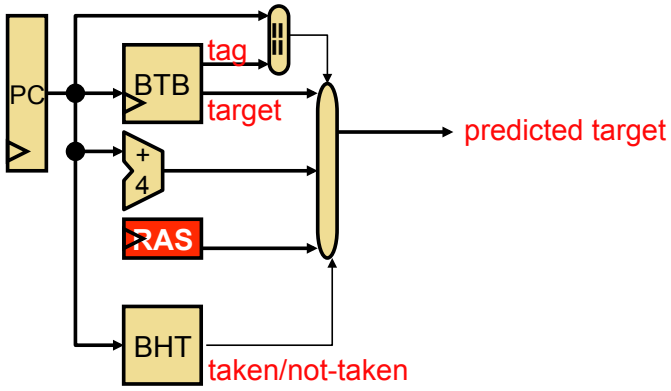
Return Address Stack (RAS)



- **Return address stack (RAS)**
  - Call instruction? RAS[TopOfStack++] = PC+4
  - Return instruction? Predicted-target = RAS[--TopOfStack]

Putting It All Together

- BTB & branch direction predictor during fetch



- If branch prediction correct, no taken branch penalty

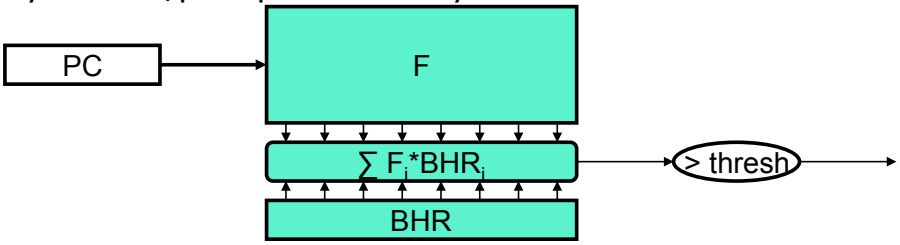
Branch Prediction Performance

- Dynamic branch prediction
  - 20% of instruction branches
  - Simple predictor: branches predicted with 75% accuracy
    - CPI = 1 + (20% * **25%** * 2) = **1.1**
  - More advanced predictor: 95% accuracy
    - CPI = 1 + (20% * **5%** * 2) = **1.02**

- Branch mis-predictions still a big problem though
  - Pipelines are long: typical mis-prediction penalty is 10+ cycles
  - For cores that do more per cycle, predictions more costly (later)

Research: Perceptron Predictor

- **Perceptron predictor** [Jimenez]
  - Attacks predictor size problem using machine learning approach
  - History table replaced by table of function coefficients F_i (signed)
  - Predict taken if $\sum(BHR_i * F_i) >$ threshold
  + Table size #PC*|BHR|*|F| (can use long BHR: ~60 bits)
    – Equivalent correlated predictor would be $\#PC*2^{|BHR|}$
  - How does it learn? Update F_i when branch is taken
    - $BHR_i == 1 ? F_i{+}{+} : F_i{-}{-}$;
    - "don't care" F_i bits stay near 0, important F_i bits saturate
  + Hybrid BHT/perceptron accuracy: 95–98%



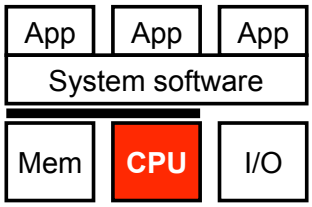
More Research: GEHL Predictor

- Problem with both correlated predictor and perceptron
  - Same predictor area dedicated to 1st history bit (1 column) …
  - … as to 2nd, 3rd, 10th, 60th…
  - Not a good use of space: 1st bit much more important than 60th
- **GEometric History-Length predictor** [Seznec, ISCA'05]
  - Multiple predictors, indexed with geometrically longer history (0, 4, 16, 32)
    - Predictors are (partially) tagged, no separate "chooser"
    - Predict: use *matching* entry from predictor with longest history
    - Mis-predict: create entry in predictor with next-longest history
    - Only 25% of predictor area used for bits 16-32 (not 50%)
    - Helps amortize cost of tagging
  - \+ Trains quickly
  - 95-97% accurate

Pipeline Depth

- Trend had been to deeper pipelines
  - 486: 5 stages (50+ gate delays / clock)
  - Pentium: 7 stages
  - Pentium II/III: 12 stages
  - Pentium 4: 22 stages (~10 gate delays / clock) **"super-pipelining"**
  - Core1/2: 14 stages
- Increasing **pipeline depth**
  - \+ Increases clock frequency (reduces period)
    - But double the stages reduce the clock period by less than 2x
  - – Decreases IPC (increases CPI)
    - Branch mis-prediction penalty becomes longer
    - Non-bypassed data hazard stalls become longer
  - At some point, actually causes performance to decrease, but when?
    - 1GHz Pentium 4 was slower than 800 MHz PentiumIII
  - "Optimal" pipeline depth is program and technology specific

Summary



- Single-cycle & multi-cycle datapaths
- Latency vs throughput & performance
- Basic pipelining
- Data hazards
  - Bypassing
  - Load-use stalling
- Pipelined multi-cycle operations
- Control hazards
  - Branch prediction

Computer Architecture

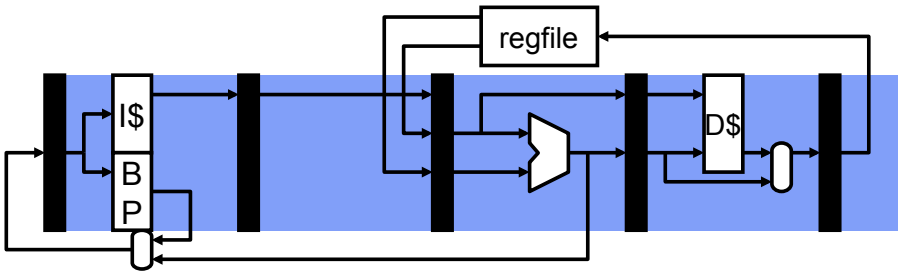
Unit 7: Superscalar Pipelines

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

A Key Theme: Parallelism

- Previously: pipeline-level parallelism
  - Work on execute of one instruction in parallel with decode of next
- Next: instruction-level parallelism (ILP)
  - Execute multiple independent instructions fully in parallel
- Then:
  - Static & dynamic scheduling
    - Extract much more ILP
  - Data-level parallelism (DLP)
    - Single-instruction, multiple data (one insn., four 64-bit adds)
  - Thread-level parallelism (TLP)
    - Multiple software threads running on multiple cores

"Scalar" Pipeline & the Flynn Bottleneck



- So far we have looked at **scalar pipelines**
  - One instruction per stage
    - With control speculation, bypassing, etc.
  - Performance limit (aka "Flynn Bottleneck") is CPI = IPC = 1
  - Limit is never even achieved (hazards)
  - Diminishing returns from "super-pipelining" (hazards + overhead)

An Opportunity...

- But consider:

```
ADD r1, r2 -> r3
ADD r4, r5 -> r6
```

  - Why not execute them ***at the same time***? (We can!)

- What about:

```
ADD r1, r2 -> r3
ADD r4, r3 -> r6
```

  - In this case, ***dependences*** prevent parallel execution

- What about three instructions at a time?
  - Or four instructions at a time?

What Checking Is Required?

- For two instructions: 2 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
```

- For three instructions: 6 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
ADD src1_3, src2_3 -> dest_3   (4 checks)
```

- For four instructions: 12 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
ADD src1_3, src2_3 -> dest_3   (4 checks)
ADD src1_4, src2_4 -> dest_4   (6 checks)
```

- Plus checking for load-to-use stalls from prior *n* loads

What Checking Is Required?

- For two instructions: 2 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
```

- For three instructions: 6 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
ADD src1_3, src2_3 -> dest_3   (4 checks)
```

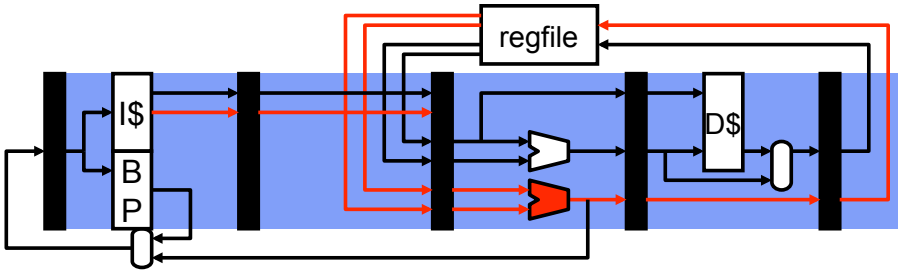
- For four instructions: 12 checks

```
ADD src1_1, src2_1 -> dest_1
ADD src1_2, src2_2 -> dest_2   (2 checks)
ADD src1_3, src2_3 -> dest_3   (4 checks)
ADD src1_4, src2_4 -> dest_4   (6 checks)
```

- Plus checking for load-to-use stalls from prior *n* loads

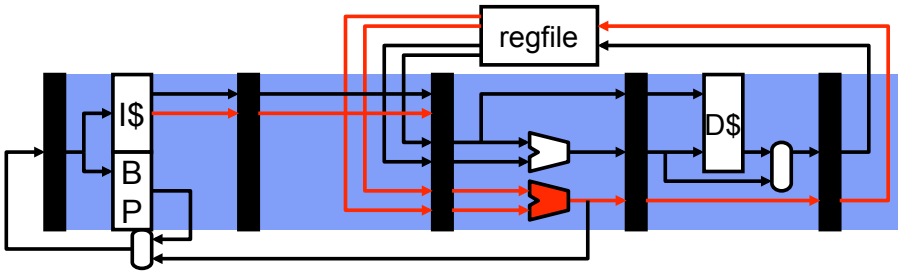
How do we build such "superscalar" hardware?

Multiple-Issue or "Superscalar" Pipeline



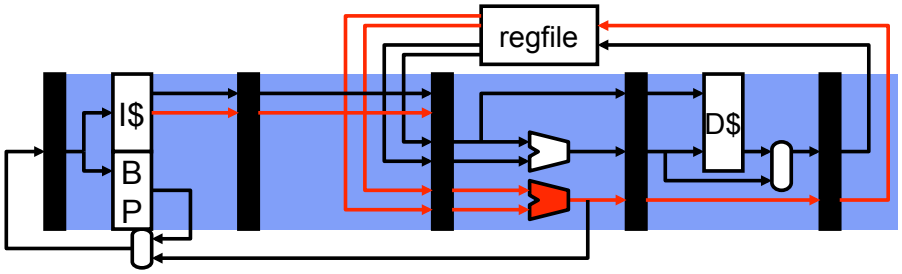
- Overcome this limit using **multiple issue**
  - Also called **superscalar**
  - Two instructions per stage at once, or three, or four, or eight…
  - **"Instruction-Level Parallelism (ILP)"** [Fisher, IEEE TC'81]
- Today, typically "4-wide" (Intel Core i7, AMD Opteron)
  - Some more (Power5 is 5-issue; Itanium is 6-issue)
  - Some less (dual-issue is common for simple cores)

A Typical Dual-Issue Pipeline (1 of 2)



- Fetch an entire 16B or 32B cache block
  - 4 to 8 instructions (assuming 4-byte average instruction length)
  - Predict a single branch per cycle
- Parallel decode
  - Need to check for conflicting instructions
    - **Is output register of I_1 is an input register to I_2?**
  - Other stalls, too (for example, load-use delay)

A Typical Dual-Issue Pipeline (2 of 2)



- Multi-ported register file
  - Larger area, latency, power, cost, complexity
- Multiple execution units
  - Simple adders are easy, but bypass paths are expensive
- Memory unit
  - Single load per cycle (stall at decode) probably okay for dual issue
  - Alternative: add a read port to data cache
    - Larger area, latency, power, cost, complexity

How Much ILP is There?

- The compiler tries to "schedule" code to avoid stalls
  - Even for scalar machines (to fill load-use delay slot)
  - Even harder to schedule multiple-issue (superscalar)
- How much ILP is common?
  - Greatly depends on the application
    - Consider memory copy
    - Unroll loop, lots of independent operations
  - Other programs, less so
- Even given unbounded ILP, superscalar has implementation limits
  - IPC (or CPI) vs clock frequency trade-off
  - Given these challenges, what is reasonable today?
    - ~4 instruction per cycle maximum

Superscalar Implementation Challenges

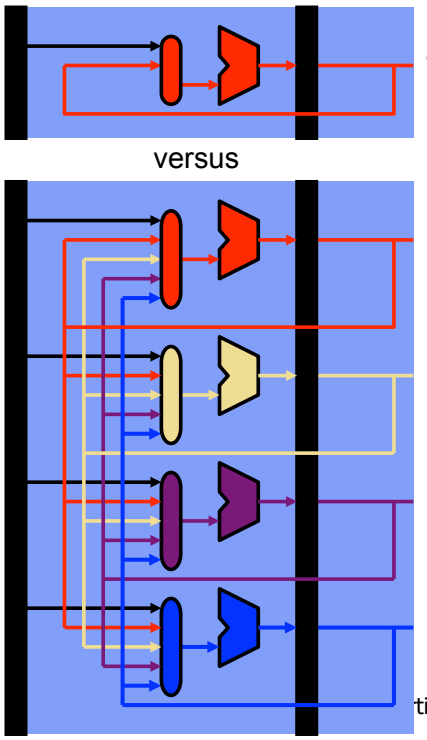
Superscalar Challenges - Front End

- **Superscalar instruction fetch**
  - Modest: fetch multiple instructions per cycle
  - Aggressive: buffer instructions and/or predict multiple branches
- **Superscalar instruction decode**
  - Replicate decoders
- **Superscalar instruction issue**
  - Determine when instructions can proceed in parallel
  - More complex stall logic - order $\mathbf{N^2}$ for *N*-wide machine
  - Not all combinations of types of instructions possible
- **Superscalar register read**
  - Port for each register read (4-wide superscalar ➔ 8 read "ports")
  - Each port needs its own set of address and data wires
    - Latency & area $\propto$ #ports^2

Superscalar Challenges - Back End

- **Superscalar instruction execution**
  - Replicate arithmetic units (but not all, say, integer divider)
  - Perhaps multiple cache ports (slower access, higher energy)
    - Only for 4-wide or larger (why? only ~35% are load/store insn)
- **Superscalar bypass paths**
  - More possible sources for data values
  - Order (N^2 * P) for *N*-wide machine with execute pipeline depth *P*
- **Superscalar instruction register writeback**
  - One write port per instruction that writes a register
  - Example, 4-wide superscalar ➔ 4 write ports
- **Fundamental challenge:**
  - Amount of ILP (instruction-level parallelism) in the program
  - Compiler must schedule code and extract parallelism

Superscalar Bypass



- **N^2 bypass network**
  - N+1 input muxes at each ALU input
  - N^2 point-to-point connections
  - Routing lengthens wires
  - Heavy capacitive load

- And this is just one bypass stage (MX)!
  - There is also WX bypassing
  - Even more for deeper pipelines

- One of the big problems of superscalar
  - Why? On the critical path of single-cycle "bypass & execute" loop

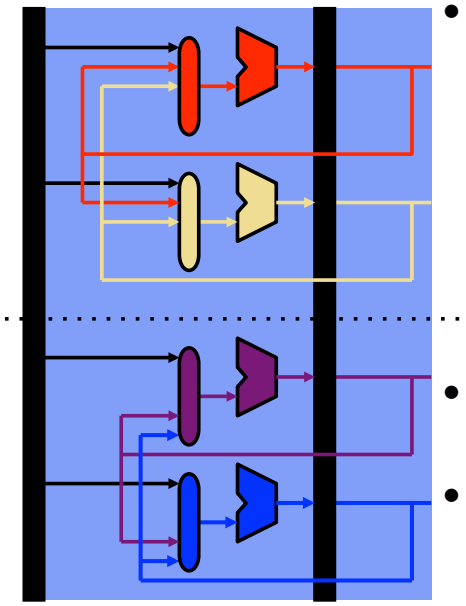
Not All N^2 Created Equal

- N^2 bypass vs. N^2 stall logic & dependence cross-check
  - Which is the bigger problem?

- N^2 bypass … by far
  - 64- bit quantities (vs. 5-bit)
  - Multiple levels (MX, WX) of bypass (vs. 1 level of stall logic)
  - Must fit in one clock period with ALU (vs. not)

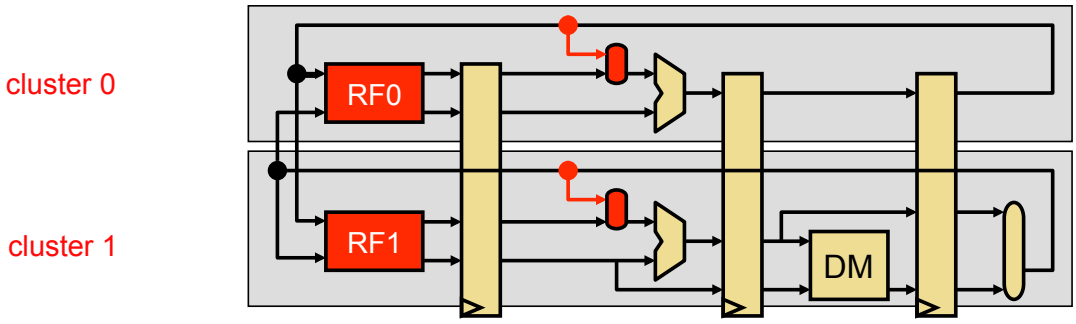
- Dependence cross-check not even 2nd biggest N^2 problem
  - Regfile is also an N^2 problem (think latency where N is #ports)
  - And also more serious than cross-check

Mitigating N^2 Bypass & Register File



- **Clustering**: mitigates N^2 bypass
  - Group ALUs into **K** clusters
  - Full bypassing within a cluster
  - Limited bypassing between clusters
    - **With 1 or 2 cycle delay**
    - Can hurt IPC, but faster clock
  - (N/K) + 1 inputs at each mux
  - $(N/K)^2$ bypass paths in each cluster
- **Steering**: key to performance
  - Steer dependent insns to same cluster
- **Cluster register file**, too
  - Replica a register file per cluster
  - All register writes update all replicas
  - Fewer read ports; only for cluster

Mitigating N^2 RegFile: Clustering++

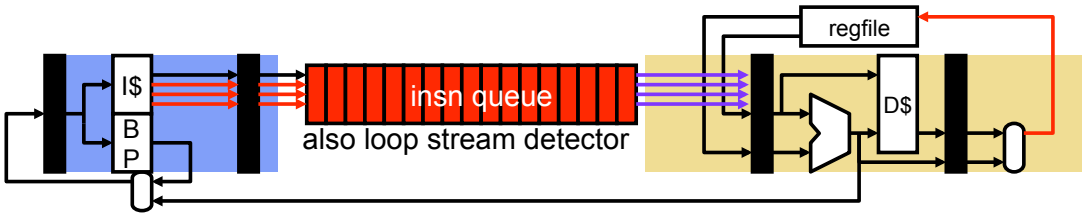


- **Clustering**: split **N**-wide execution pipeline into **K** clusters
  - With centralized register file, 2N read ports and N write ports
- **Clustered register file**: extend clustering to register file
  - Replicate the register file (one replica per cluster)
  - Register file supplies register operands to just its cluster
  - All register writes go to all register files (keep them in sync)
  - Advantage: fewer read ports per register!
    - K register files, each with 2N/K read ports and N write ports

Another Challenge: Superscalar Fetch

- What is involved in fetching multiple instructions per cycle?
- In same cache block? → no problem
  - 64-byte cache block is 16 instructions (~4 bytes per instruction)
  - Favors larger block size (independent of hit rate)
- What if next instruction is last instruction in a block?
  - Fetch only one instruction that cycle
  - Or, some processors may allow fetching from 2 consecutive blocks
- What about taken branches?
  - How many instructions can be fetched on average?
  - Average number of instructions per taken branch?
    - Assume: 20% branches, 50% taken → ~10 instructions
- Consider a 5-instruction loop with an 4-issue processor
  - Without smarter fetch, ILP is limited to 2.5 (not 4, which is bad)

Increasing Superscalar Fetch Rate



- Option #1: over-fetch and buffer
  - Add a queue between fetch and decode (18 entries in Intel Core2)
  - Compensates for cycles that fetch less than maximum instructions
  - "decouples" the "front end" (fetch) from the "back end" (execute)
- Option #2: "loop stream detector" (Core 2, Core i7)
  - Put entire loop body into a small cache
    - Core2: 18 macro-ops, up to four taken branches
    - Core i7: 28 micro-ops (avoids re-decoding macro-ops!)
  - Any branch mis-prediction requires normal re-fetch
- Other options: next-next-block prediction, "trace cache"

Multiple-Issue Implementations

- **Statically-scheduled (in-order) superscalar**
  - **What we've talked about thus far**
  - \+ Executes unmodified sequential programs
  - – Hardware must figure out what can be done in parallel
  - E.g., Pentium (2-wide), UltraSPARC (4-wide), Alpha 21164 (4-wide)
- **Very Long Instruction Word (VLIW)**
  - – **Compiler identifies independent instructions**, new ISA
  - \+ Hardware can be simple and perhaps lower power
  - E.g., TransMeta Crusoe (4-wide)
  - **Variant: Explicitly Parallel Instruction Computing (EPIC)**
    - A bit more flexible encoding & some hardware to help compiler
    - E.g., Intel Itanium (6-wide)
- **Dynamically-scheduled superscalar (next topic)**
  - **Hardware extracts more ILP by on-the-fly reordering**
  - Core 2, Core i7 (4-wide), Alpha 21264 (4-wide)

Multiple Issue Redux

- Multiple issue
  - Exploits insn level parallelism (ILP) beyond pipelining
  - Improves IPC, but perhaps at some clock & energy penalty
  - 4-6 way issue is about the peak issue width currently justifiable
    - Low-power implementations today typically 2-wide superscalar
- Problem spots
  - N^2 bypass & register file → clustering
  - Fetch + branch prediction → buffering, loop streaming, trace cache
  - N^2 dependency check → VLIW/EPIC (but unclear how key this is)
- Implementations
  - Superscalar vs. VLIW/EPIC

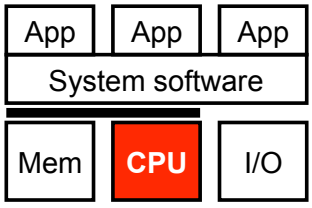
[spacer]

Computer Architecture

Unit 8: Static & Dynamic Scheduling

Slides originally developed by
Drew Hilton, Amir Roth and Milo Martin
at University of Pennsylvania

This Unit: Static & Dynamic Scheduling



- Code scheduling
  - To reduce pipeline stalls
  - To increase ILP (insn level parallelism)
- Static scheduling by the compiler
  - Approach & limitations
- Dynamic scheduling in hardware
  - Register renaming
  - Instruction selection
  - Handling memory operations

Readings

- Assigned reading
  - "Memory Dependence Prediction using Store Sets" by Chrysos & Emer
- Suggested reading
  - "The MIPS R10000 Superscalar Microprocessor" by Kenneth Yeager

Code Scheduling & Limitations

Code Scheduling

- Scheduling: act of finding independent instructions
  - "Static" done at compile time by the compiler (software)
  - "Dynamic" done at runtime by the processor (hardware)

- Why schedule code?
  - Scalar pipelines: fill in load-to-use delay slots to improve CPI
  - Superscalar: place independent instructions together
    - As above, load-to-use delay slots
    - Allow multiple-issue decode logic to let them execute at the same time

Compiler Scheduling

- Compiler can schedule (move) instructions to reduce stalls
  - **Basic pipeline scheduling**: eliminate back-to-back load-use pairs
  - Example code sequence: `a = b + c;  d = f - e;`
    - `sp` stack pointer, `sp+0` is "a", `sp+4` is "b", etc…

Before

```
ld [sp+4]➜r2
ld [sp+8]➜r3
add r2,r3➜r1 //stall
st r1➜[sp+0]
ld [sp+16]➜r5
ld [sp+20]➜r6
sub r6,r5➜r4 //stall
st r4➜[sp+12]
```

After

```
ld [sp+4]➜r2
ld [sp+8]➜r3
ld [sp+16]➜r5
add r2,r3➜r1 //no stall
ld [sp+20]➜r6
st r1➜[sp+0]
sub r6,r5➜r4 //no stall
st r4➜[sp+12]
```

Compiler Scheduling Requires

- **Large scheduling scope**
  - Independent instruction to put between load-use pairs
  - \+ Original example: large scope, two independent computations
  - – This example: small scope, one computation

Before

```
ld [sp+4]➜r2
ld [sp+8]➜r3
add r2,r3➜r1 //stall
st r1➜[sp+0]
```

After (same!)

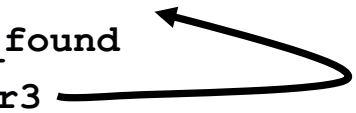
```
ld [sp+4]➜r2
ld [sp+8]➜r3
add r2,r3➜r1 //stall
st r1➜[sp+0]
```

- Compiler can create larger scheduling scopes
  - For example: loop unrolling & function inlining

Scheduling Scope Limited by Branches

r1 and r2 are inputs

```
loop:
  jz r1, not_found
  ld [r1+0]➜r3
  sub r2,r3➜r4
  jz r4, found
  ld [r1+4]➜r1
  jmp loop
```



Aside: what does this code do?

Legal to move load up past branch?

Compiler Scheduling Requires

- **Enough registers**
  - To hold additional "live" values
  - Example code contains 7 different values (including `sp`)
  - Before: max 3 values live at any time → 3 registers enough
  - After: max 4 values live → 3 registers not enough

Original

```
ld [sp+4]➜r2
ld [sp+8]➜r1
add r1,r2➜r1 //stall
st r1➜[sp+0]
ld [sp+16]➜r2
ld [sp+20]➜r1
sub r2,r1➜r1 //stall
st r1➜[sp+12]
```

Wrong!

```
ld [sp+4]➜r2
ld [sp+8]➜r1
ld [sp+16]➜r2
add r1,r2➜r1  // wrong r2
ld [sp+20]➜r1
st r1➜[sp+0]  // wrong r1
sub r2,r1➜r1
st r1➜[sp+12]
```


Compiler Scheduling Requires

- **Alias analysis**
  - Ability to tell whether load/store reference same memory locations
    - Effectively, whether load/store can be rearranged
  - Previous example: easy, loads/stores use same base register (`sp`)
  - New example: can compiler tell that `r8` != `r9`?
  - Must be **conservative**

Before

```
ld [r9+4]➜r2
ld [r9+8]➜r3
add r3,r2➜r1 //stall
st r1➜[r9+0]
ld [r8+0]➜r5
ld [r8+4]➜r6
sub r5,r6➜r4 //stall
st r4➜[r8+8]
```

Wrong(?)

```
ld [r9+4]➜r2
ld [r9+8]➜r3
ld [r8+0]➜r5 //does r8==r9?
add r3,r2➜r1
ld [r8+4]➜r6 //does r8+4==r9?
st r1➜[r9+0]
sub r5,r6➜r4
st r4➜[r8+8]
```

Compiler Scheduling Limitations

- Scheduling scope
  - Example: can't generally move memory operations past branches
- Limited number of registers (set by ISA)
- Inexact "memory aliasing" information
  - Often prevents reordering of loads above stores by compiler
- Caches misses (or any runtime event) confound scheduling
  - How can the compiler know which loads will miss vs hit?
  - Can impact the compiler's scheduling decisions

Dynamic (Hardware) Scheduling

Can Hardware Overcome These Limits?

- **Dynamically-scheduled processors**
  - Also called "out-of-order" processors
  - Hardware re-schedules insns…
  - …within a sliding window of VonNeumann insns
  - As with pipelining and superscalar, ISA unchanged
    - Same hardware/software interface, appearance of in-order

- Examples:
  - Pentium Pro/II/III (3-wide), Core 2 (4-wide), Alpha 21264 (4-wide), MIPS R10000 (4-wide), Power5 (5-wide)

Example: In-Order Limitations #1

	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [r1] ➜ r2	F	D	X	M_1	M_2	W							
add r2 + r3 ➜ r4	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor r4 ^ r5 ➜ r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] ➜ r4		F	D	**p***	**p***	**p***	X	M_1	M_2	W			

- In-order pipeline, two-cycle load-use penalty
  - 2-wide
- Why not the following:

	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [r1] ➜ r2	F	D	X	M_1	M_2	W							
add r2 + r3 ➜ **r4**	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor **r4** ^ r5 ➜ r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] ➜ **r4**		F	D	**X**	**M_1**	**M_2**	**W**						

Example: In-Order Limitations #2

	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [p1] ➜ p2	F	D	X	M_1	M_2	W							
add p2 + p3 ➜ p4	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor p4 ^ p5 ➜ p6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [p7] ➜ p8		F	D	**p***	**p***	**p***	X	M_1	M_2	W			

- In-order pipeline, two-cycle load-use penalty
  - 2-wide
- Why not the following:

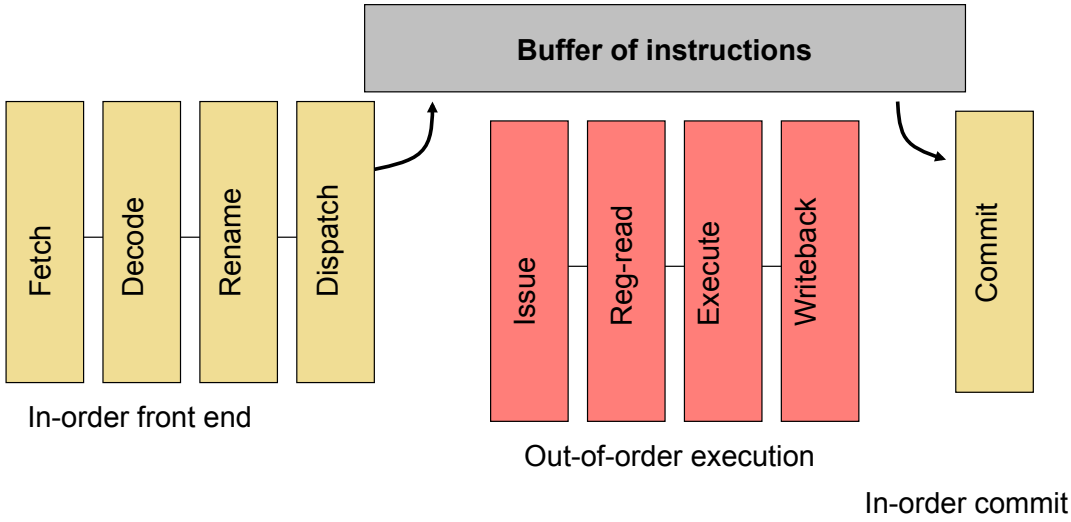
	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [p1] ➜ p2	F	D	X	M_1	M_2	W							
add p2 + p3 ➜ **p4**	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor **p4** ^ p5 ➜ p6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [p7] ➜ **p8**		F	D	**X**	**M_1**	**M_2**	**W**						

Out-of-Order to the Rescue

	0	1	2	3	4	5	6	7	8	9	10	11	12
Ld [p1] ➜ p2	F	Di	I	RR	X	M_1	M_2	W	C				
add p2 + p3 ➜ p4	F	Di				I	RR	X	W	C			
xor p4 ^ p5 ➜ p6		F	Di				I	RR	X	W	C		
ld [p7] ➜ p8		F	Di	I	RR	X	M_1	M_2	W		C		

- "Dynamic scheduling" done by the hardware
- Still 2-wide superscalar, but now out-of-order, too
  - Allows instructions to issues when dependences are ready
- Longer pipeline
  - In-order front end: Fetch, "**Dispatch**"
  - Out-of-order execution core:
    - "**Issue**", "**RegisterRead**", Execute, Memory, Writeback
  - In-order retirement: "**Commit**"

Out-of-Order Pipeline



Out-of-Order Execution

- Also call "Dynamic scheduling"
  - Done by the hardware on-the-fly during execution
- Looks at a "window" of instructions waiting to execute
  - Each cycle, picks the next ready instruction(s)
- Two steps to enable out-of-order execution:
  Step #1: Register renaming – to avoid "false" dependencies
  Step #2: Dynamically schedule – to enforce "true" dependencies
- Key to understanding out-of-order execution:
  - **Data dependencies**

Dependence types

- RAW (Read After Write) = "true dependence" (true)

  mul r0 * r1 ➜ **r2**

  ...

  add **r2** + r3 ➜ r4

- WAW (Write After Write) = "output dependence" (false)

  mul r0 * r1➜ **r2**

  ...

  add r1 + r3 ➜ **r2**

- WAR (Write After Read) = "anti-dependence" (false)

  mul r0 * **r1** ➜ r2

  ...

  add r3 + r4 ➜ **r1**

- WAW & WAR are "false", Can be **totally eliminated** by "renaming"

Step #1: Register Renaming

- To eliminate register conflicts/hazards
- "Architected" vs "Physical" registers – level of indirection
  - Names: `r1,r2,r3`
  - Locations: `p1,p2,p3,p4,p5,p6,p7`
  - Original mapping: `r1`→`p1`, `r2`→`p2`, `r3`→`p3`, `p4`–`p7` are "available"

MapTable

r1	r2	r3
p1	p2	p3
p4	p2	p3
p4	p2	p5
p4	p2	p6

FreeList

p4,p5,p6,p7
p5,p6,p7
p6,p7
p7

Original insns	Renamed insns
add r2,r3➜r1	add p2,p3➜p4
sub r2,r1➜r3	sub p2,p4➜p5
mul r2,r3➜r3	mul p2,p5➜p6
div r1,4➜r1	div p4,4➜p7

- Renaming – conceptually write each register once
  - + Removes **false** dependences
  - + Leaves **true** dependences intact!
- When to reuse a physical register? After overwriting insn done

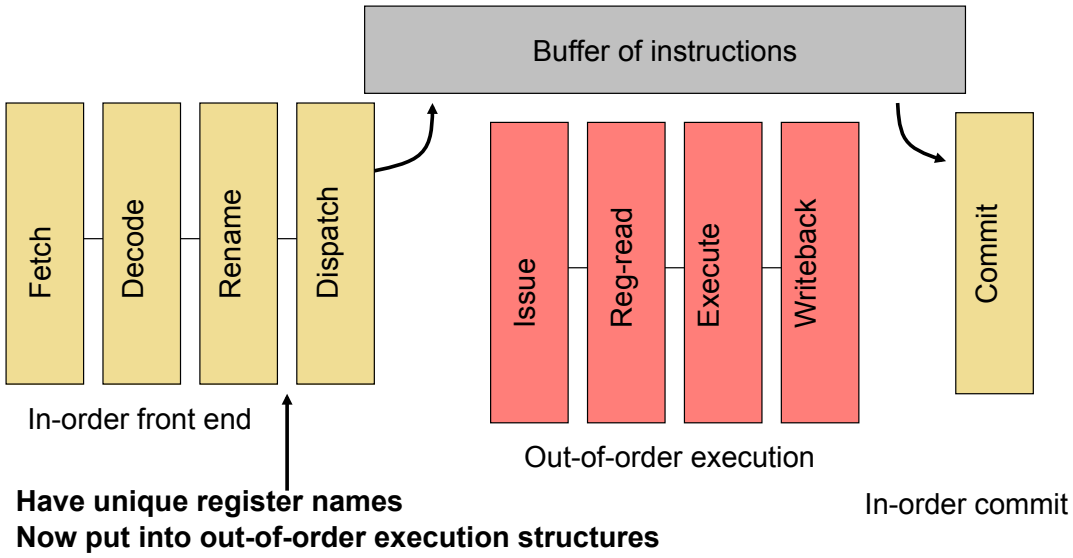
Register Renaming Algorithm

- Two key data structures:
  - maptable[architectural_reg] ➔ physical_reg
  - Free list: allocate (new) & free registers (implemented as a queue)
- Algorithm: at "decode" stage for each instruction:

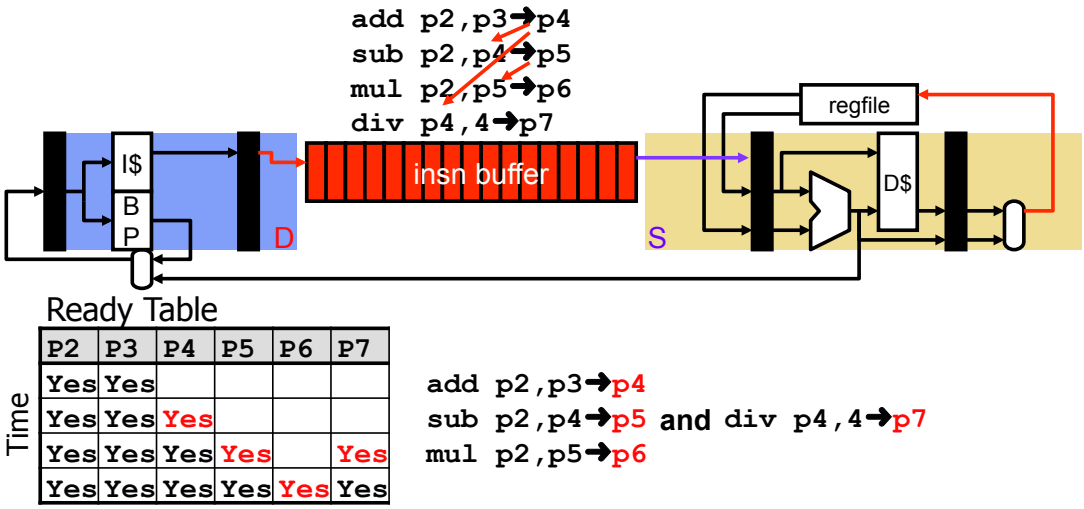
```
insn.phys_input1 = maptable[insn.arch_input1]
insn.phys_input2 = maptable[insn.arch_input2]
insn.old_phys_output = maptable[insn.arch_output]
new_reg = new_phys_reg()
maptable[insn.arch_output] = new_reg
insn.phys_output = new_reg
```

- At "commit"
  - Once all prior instructions have committed, free register `free_phys_reg(insn.old_phys_output)`

Out-of-order Pipeline



Step #2: Dynamic Scheduling



- Instructions fetch/decoded/renamed into *Instruction Buffer*
  - Also called "instruction window" or "instruction scheduler"
- Instructions (conceptually) check ready bits every cycle
  - Execute earliest "ready" instruction, set output as "ready"

Dynamic Scheduling/Issue Algorithm

- Data structures:
  - Ready table[phys_reg] ➔ yes/no (part of "issue queue")
- Algorithm at "schedule" stage (prior to read registers):

```
foreach instruction:
   if table[insn.phys_input1] == ready &&
      table[insn.phys_input2] == ready then
         insn is "ready"
select the earliest "ready" instruction
   table[insn.phys_output] = ready
```

- Multiple-cycle instructions? (such as loads)
  - For an insn with latency of N, set "ready" bit N-1 cycles in future

Register Renaming

Register Renaming Algorithm (Simplified)

- Two key data structures:
  - maptable[architectural_reg] ➔ physical_reg
  - Free list: allocate (new) & free registers (implemented as a queue)
- Algorithm: at "decode" stage for each instruction:

```
insn.phys_input1 = maptable[insn.arch_input1]
insn.phys_input2 = maptable[insn.arch_input2]

new_reg = new_phys_reg()
maptable[insn.arch_output] = new_reg
insn.phys_output = new_reg
```

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

r1	p1
r2	p2
r3	p3
r4	p4
r5	p5

Map table

p6
p7
p8
p9
p10

Free-list

Renaming example

xor **r1** ^ **r2** ➜ r3 ⟶ xor **p1** ^ **p2** ➜
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

r1	**p1**
r2	**p2**
r3	p3
r4	p4
r5	p5

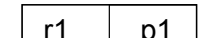
Map table

p6
p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1



xor p1 ^ p2 ➜ **p6**

r1	p1
r2	p2
r3	p3
r4	p4
r5	p5

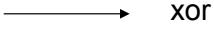
Map table

p6
p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ **r3**
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1



xor p1 ^ p2 ➜ p6

r1	p1
r2	p2
r3	**p6**
r4	p4
r5	p5

Map table

p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add **r3** + **r4** ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add **p6** + **p4** ➜

r1	p1
r2	p2
r3	**p6**
r4	**p4**
r5	p5

Map table

p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ **p7**

r1	p1
r2	p2
r3	p6
r4	p4
r5	p5

Map table

p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ **r4**
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7

r1	p1
r2	p2
r3	p6
r4	**p7**
r5	p5

Map table

p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub **r5** - **r2** ➜ r3
addi r3 + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub **p5** - **p2** ➜

r1	p1
r2	**p2**
r3	p6
r4	p7
r5	**p5**

Map table

p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1
```



```
xor  p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
```

r1	p1
r2	p2
r3	p6
r4	p7
r5	p5

Map table

p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1
```



```
xor  p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
```

r1	p1
r2	p2
r3	**p8**
r4	p7
r5	p5

Map table

p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi **r3** + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi **p8** + 1 ➜

r1	p1
r2	p2
r3	**p8**
r4	p7
r5	p5

Map table

p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ **p9**

r1	p1
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p9
p10

Free-list

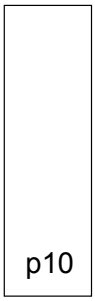
Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ **r1**

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

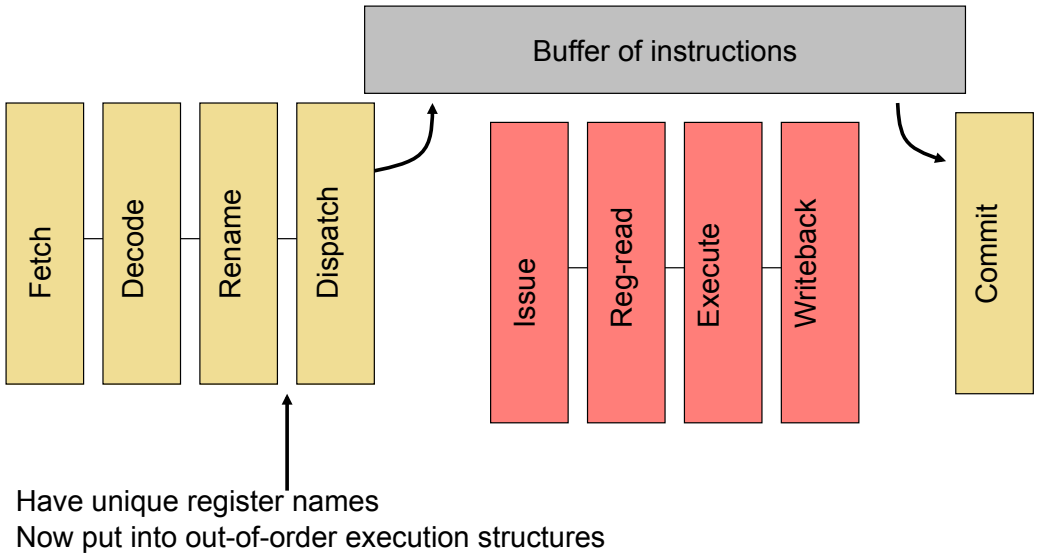
r1	**p9**
r2	p2
r3	p8
r4	p7
r5	p5

Map table



Free-list

Out-of-order Pipeline

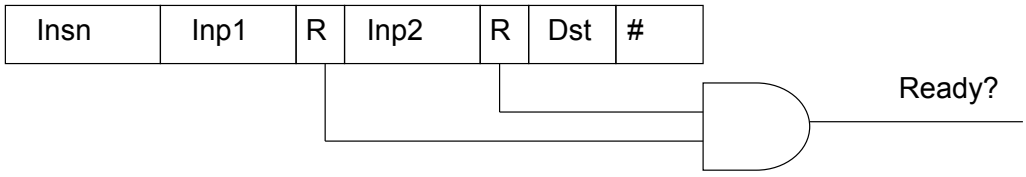


Have unique register names
Now put into out-of-order execution structures

Dynamic Scheduling Mechanisms

Dispatch

- Renamed instructions into out-of-order structures
- Re-order buffer (ROB)
  - All instruction until commit
- Issue Queue
  - Central piece of scheduling logic
  - Holds un-executed instructions
  - Tracks ready inputs
    - Physical register names + ready bit
    - "AND" the bits to tell if ready



Dispatch Steps

- Allocate Issue Queue (IQ) slot
  - Full? Stall
- Read **ready bits** of inputs
  - Table 1-bit per physical reg
- Clear **ready bit** of output in table
  - Instruction has not produced value yet
- Write data into Issue Queue (IQ) slot

Dispatch Example

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

Issue Queue

Insn	Inp1	R	Inp2	R	Dst	#

Ready bits

p1	y
p2	y
p3	y
p4	y
p5	y
p6	y
p7	y
p8	y
p9	y

Dispatch Example

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

Issue Queue

Insn	Inp1	R	Inp2	R	Dst	#
xor	p1	y	p2	y	p6	0

Ready bits

p1	y
p2	y
p3	y
p4	y
p5	y
p6	**n**
p7	y
p8	y
p9	y

Dispatch Example

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

Issue Queue

Insn	Inp1	R	Inp2	R	Dst	#
xor	p1	y	p2	y	p6	0
add	p6	n	p4	y	p7	1

Ready bits

p1	y
p2	y
p3	y
p4	y
p5	y
p6	n
p7	**n**
p8	y
p9	y

Dispatch Example

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

Issue Queue

Insn	Inp1	R	Inp2	R	Dst	#
xor	p1	y	p2	y	p6	0
add	p6	n	p4	y	p7	1
sub	p5	y	p2	y	p8	2

Ready bits

p1	y
p2	y
p3	y
p4	y
p5	y
p6	n
p7	n
p8	**n**
p9	y

Dispatch Example

xor p1 ^ p2 ➜ p6
add p6 + p4 ➜ p7
sub p5 - p2 ➜ p8
addi p8 + 1 ➜ p9

Issue Queue

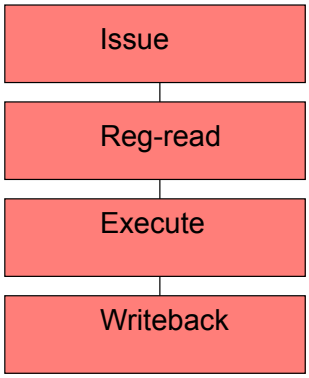
Insn	Inp1	R	Inp2	R	Dst	#
xor	p1	y	p2	y	p6	0
add	p6	n	p4	y	p7	1
sub	p5	y	p2	y	p8	2
addi	p8	n	---	y	p9	3

Ready bits

p1	y
p2	y
p3	y
p4	y
p5	y
p6	n
p7	n
p8	n
p9	**n**

Out-of-order pipeline

- Execution (out-of-order) stages
- **Select** ready instructions
  - Send for execution
- **Wakeup** dependents



Dynamic Scheduling/Issue Algorithm

- Data structures:
  - Ready table[phys_reg] ➔ yes/no (part of issue queue)

- Algorithm at "schedule" stage (prior to read registers):

```
foreach instruction:
   if table[insn.phys_input1] == ready &&
      table[insn.phys_input2] == ready then
          insn is "ready"
select the earliest "ready" instruction
   table[insn.phys_output] = ready
```

Issue = Select + Wakeup

- **Select** earliest of "ready" instructions
  - ➢ "xor" is the earliest ready instruction below
  - ➢ "xor" and "sub" are the two earliest ready instructions below
  - Note: may have resource constraints: i.e. load/store/floating point

Insn	Inp1	R	Inp2	R	Dst	#	
xor	p1	**y**	p2	**y**	p6	0	**Ready!**
add	p6	n	p4	y	p7	1	
sub	p5	**y**	p2	**y**	p8	2	**Ready!**
addi	p8	n	---	y	p9	3	

Issue = Select + Wakeup

- Wakeup dependent instructions
  - Search for destination (Dst) in inputs & set "ready" bit
    - Implemented with a special memory array circuit called a Content Addressable Memory (CAM)
  - Also update ready-bit table for future instructions

Insn	Inp1	R	Inp2	R	Dst	#
xor	p1	y	p2	y	**p6**	0
add	**p6**	**y**	p4	y	p7	1
sub	p5	y	p2	y	**p8**	2
addi	**p8**	**y**	---	y	p9	3

  - For multi-cycle operations (loads, floating point)
    - Wakeup deferred a few cycles
    - Include checks to avoid structural hazards

Ready bits	
p1	y
p2	y
p3	y
p4	y
p5	y
p6	**y**
p7	n
p8	**y**
p9	n

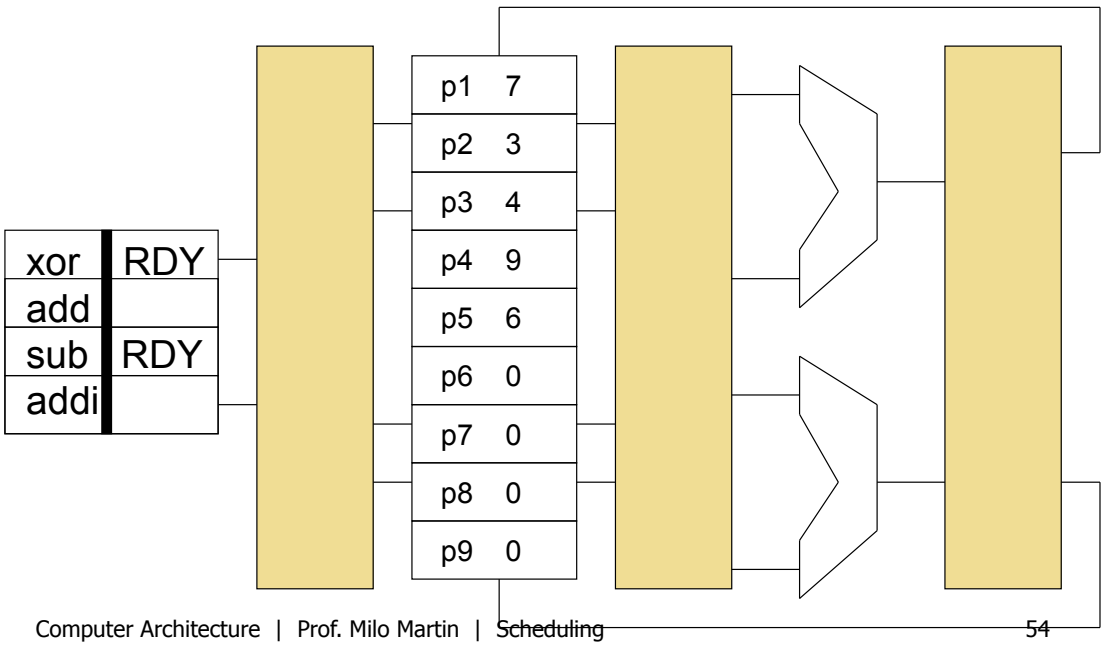
Issue

- **Select/Wakeup** one cycle
- Dependent instructions execute on back-to-back cycles
  - Next cycle: add/addi are ready:

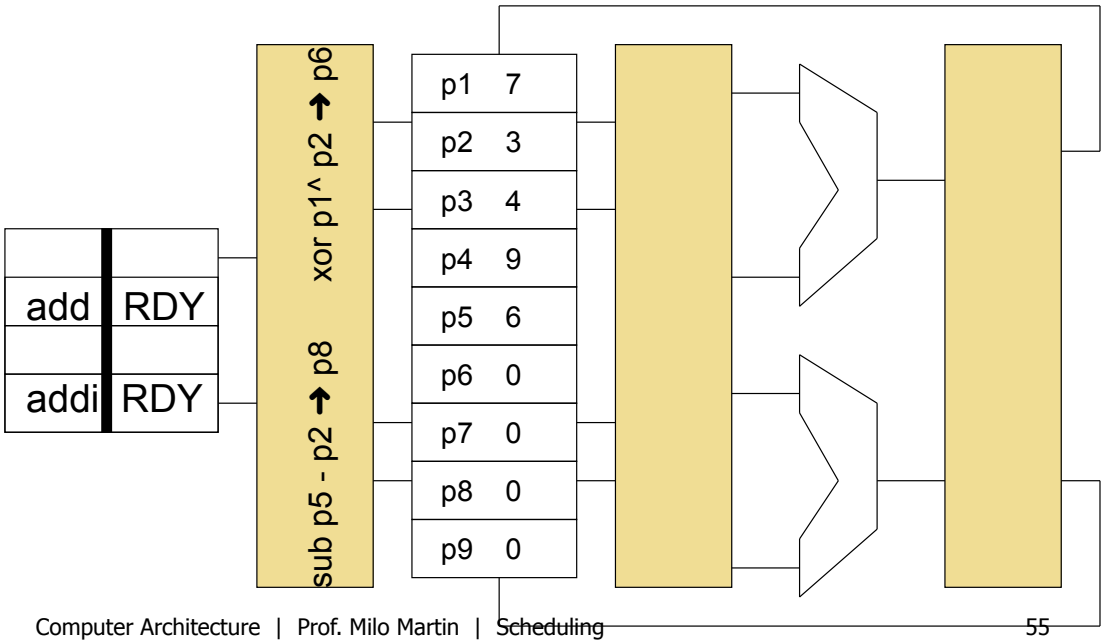
Insn	Inp1	R	Inp2	R	Dst	#
add	p6	**y**	p4	y	p7	1
addi	p8	**y**	---	y	p9	3

- Issued instructions are removed from issue queue
  - Free up space for subsequent instructions

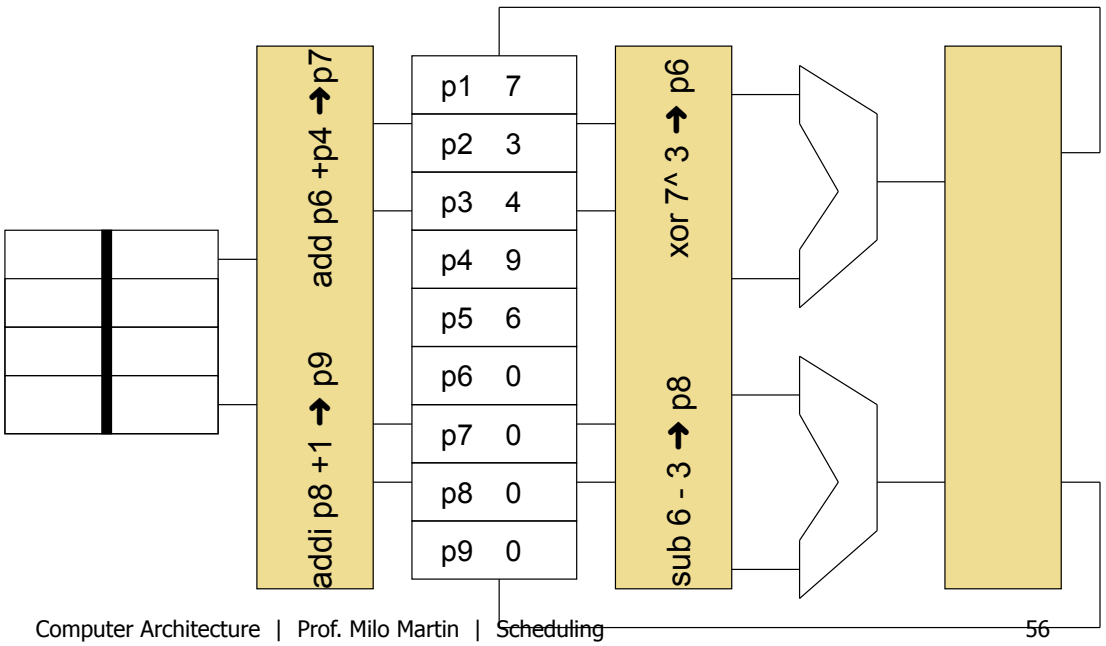
OOO execution (2-wide)



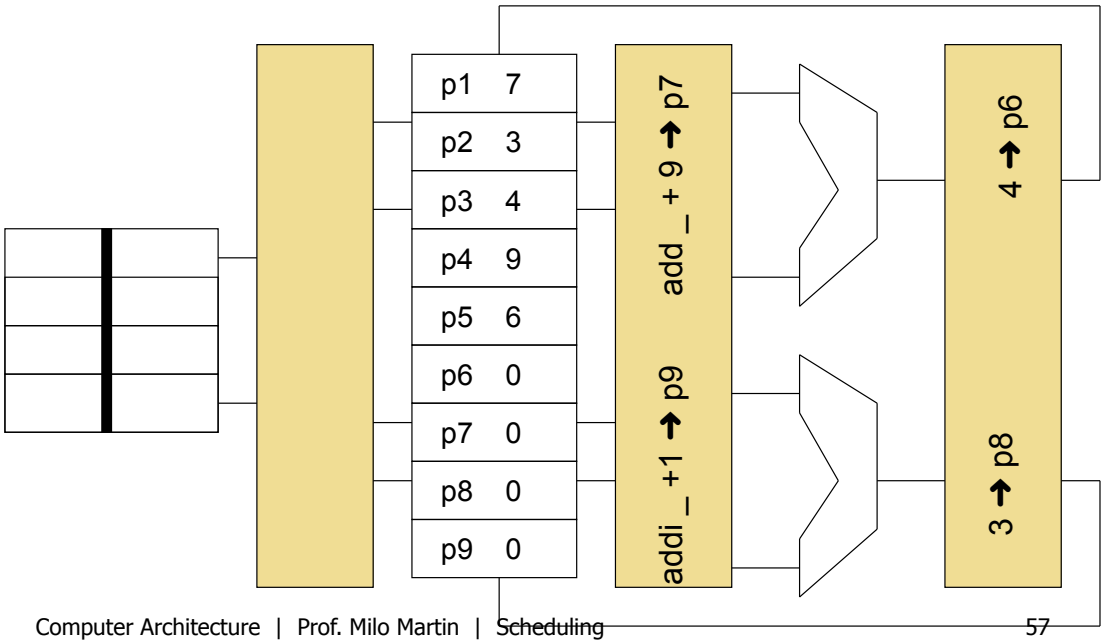
OOO execution (2-wide)



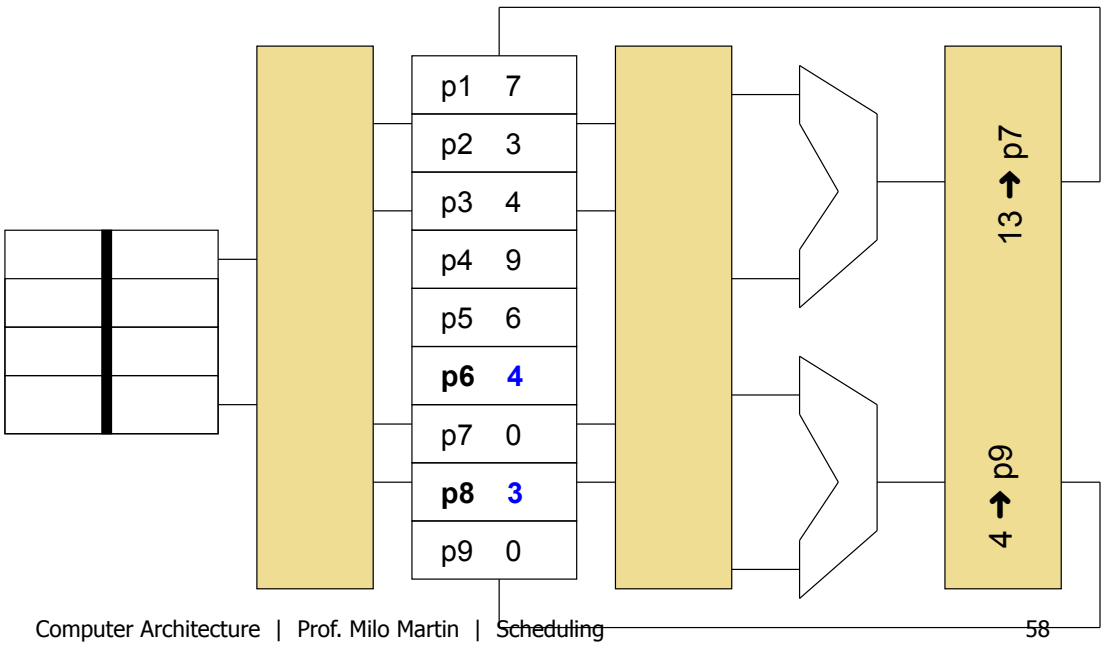
OOO execution (2-wide)



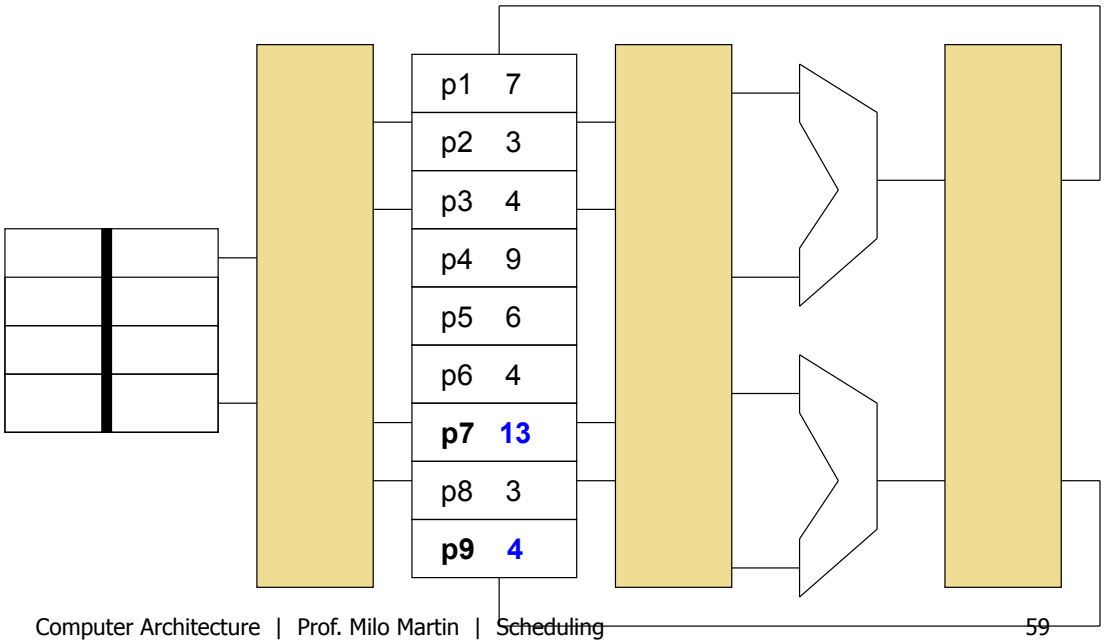
OOO execution (2-wide)



OOO execution (2-wide)

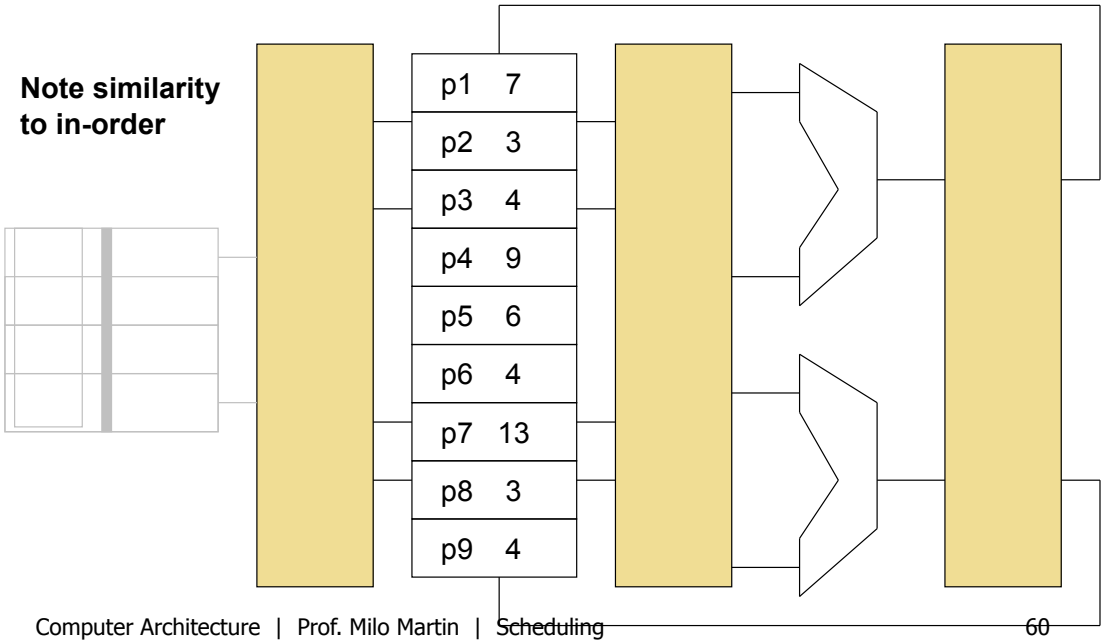


OOO execution (2-wide)



OOO execution (2-wide)

Note similarity to in-order



When Does Register Read Occur?

- Current approach: after select, right before execute
  - **Not during in-order part of pipeline, in out-of-order part**
  - Read **physical** register (renamed)
  - Or get value via bypassing (based on physical register name)
  - This is Pentium 4, MIPS R10k, Alpha 21264, IBM Power4, Intel's "Sandy Bridge" (2011)
  - Physical register file may be large
    - Multi-cycle read
- Older approach:
  - Read as part of "issue" stage, keep values in Issue Queue
    - At commit, write them back to "architectural register file"
  - Pentium Pro, Core 2, Core i7
  - Simpler, but may be less energy efficient (more data movement)

Renaming Revisited

Re-order Buffer (ROB)

- ROB entry holds all info for recover/commit
  - **All instructions** & in order
  - Architectural register names, physical register names, insn type
  - Not removed until very last thing ("commit")

- Operation
  - Dispatch: insert at tail (if full, stall)
  - Commit: remove from head (if not yet done, stall)

- Purpose: tracking for in-order commit
  - Maintain appearance of in-order execution
  - Done to support:
    - **Misprediction recovery**
    - **Freeing of physical registers**

Renaming revisited

- Track (or "log") the "overwritten register" in ROB
  - Freed this register at commit
  - Also used to restore the map table on "recovery"
    - Branch mis-prediction recovery

Register Renaming Algorithm (Full)

- Two key data structures:
  - maptable[architectural_reg] ➔ physical_reg
  - Free list: allocate (new) & free registers (implemented as a queue)
- Algorithm: at "decode" stage for each instruction:

```
insn.phys_input1 = maptable[insn.arch_input1]
insn.phys_input2 = maptable[insn.arch_input2]
insn.old_phys_output = maptable[insn.arch_output]
new_reg = new_phys_reg()
maptable[insn.arch_output] = new_reg
insn.phys_output = new_reg
```

- **At "commit"**
  - **Once all prior instructions have committed, free register**

```
free_phys_reg(insn. old_phys_output)
```


Recovery

- Completely remove wrong path instructions
  - Flush from IQ
  - Remove from ROB
  - Restore map table to before misprediction
  - Free destination registers
- How to restore map table?
  - Option #1: log-based reverse renaming to recover each instruction
    - Tracks the old mapping to allow it to be reversed
    - Done sequentially for each instruction (slow)
    - See next slides
  - Option #2: checkpoint-based recovery
    - Checkpoint state of maptable and free list each cycle
    - Faster recovery, but requires more state
  - Option #3: hybrid (checkpoint for branches, unwind for others)

Renaming example

xor r1 ^ r2 ➜ r3
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

r1	p1
r2	p2
r3	p3
r4	p4
r5	p5

Map table

p6
p7
p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3 ⟶ xor p1 ^ p2 ➜ [p3]
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1

r1	p1
r2	p2
r3	**p3**
r4	p4
r5	p5

Map table

p6
p7
p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3          ⟶   xor  p1 ^ p2 ➜ p6          [ p3 ]
add r3 + r4 ➜ r4
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1
```

r1	p1
r2	p2
r3	**p6**
r4	p4
r5	p5

Map table

p7
p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3          ⟶   xor  p1 ^ p2 ➜ p6          [ p3 ]
add r3 + r4 ➜ r4              add p6 + p4 ➜              [ p4 ]
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1
```

r1	p1
r2	p2
r3	p6
r4	**p4**
r5	p5

Map table

p7
p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3          xor  p1 ^ p2 ➜ p6          [ p3 ]
add r3 + r4 ➜ r4   ———→   add p6 + p4 ➜ p7          [ p4 ]
sub r5 - r2 ➜ r3
addi r3 + 1 ➜ r1
```

r1	p1
r2	p2
r3	p6
r4	**p7**
r5	p5

Map table

p8
p9
p10

Free-list

Renaming example

```
xor r1 ^ r2 ➜ r3          xor  p1 ^ p2 ➜ p6          [ p3 ]
add r3 + r4 ➜ r4          add p6 + p4 ➜ p7           [ p4 ]
sub r5 - r2 ➜ r3   ———→   sub p5 - p2 ➜              [ p6 ]
addi r3 + 1 ➜ r1
```

r1	p1
r2	p2
r3	**p6**
r4	p7
r5	p5

Map table

p8
p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3		xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4		add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	⟶	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1			

r1	p1
r2	p2
r3	**p8**
r4	p7
r5	p5

Map table

p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3		xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4		add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3		sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	⟶	addi p8 + 1 ➜	[p1]

r1	**p1**
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p9
p10

Free-list

Renaming example

xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	**p9**
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10

Free-list

Recovery Example

Now, let's use this info. to recover from a branch misprediction

bnz r1 loop	**bnz p1, loop**	[ ]
xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10

Free-list

Recovery Example

bnz r1 loop	bnz p1, loop	[ ]
xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	**p1**
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p9
p10

Free-list

Recovery Example

bnz r1 loop	bnz p1, loop	[ ]
xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]

r1	p1
r2	p2
r3	**p6**
r4	p7
r5	p5

Map table

p8
p9
p10

Free-list

Recovery Example

bnz r1 loop	bnz p1, loop	[ ]
xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]

r1	p1
r2	p2
r3	p6
r4	**p4**
r5	p5

Map table

p7
p8
p9
p10

Free-list

Recovery Example

bnz r1 loop	bnz p1, loop	[ ]
xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]

r1	p1
r2	p2
r3	**p3**
r4	p4
r5	p5

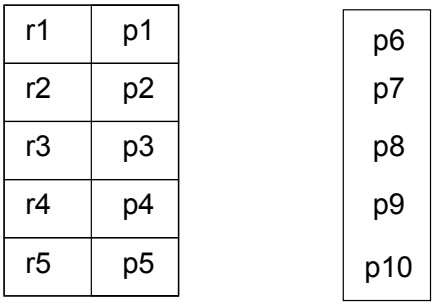
Map table

p6
p7
p8
p9
p10

Free-list

Recovery Example

bnz r1 loop	bnz p1, loop	[ ]



r1	p1
r2	p2
r3	p3
r4	p4
r5	p5

Map table

p6
p7
p8
p9
p10

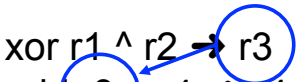
Free-list

Commit

xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

- Commit: instruction becomes **architected state**
  - In-order, only when instructions are finished
  - Free overwritten register (why?)

Freeing over-written register



xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

- P3 was r3 **before** xor
- P6 is r3 **after** xor
  - Anything before (in program order) xor should read p3
  - Anything after (in program order) xor should p6 (until next r3 writing instruction
- At commit of xor, no instructions before it are in the pipeline

Commit Example

xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10

Free-list

Commit Example

xor r1 ^ r2 ➜ r3	xor p1 ^ p2 ➜ p6	[p3]
add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10
p3

Free-list

Commit Example

add r3 + r4 ➜ r4	add p6 + p4 ➜ p7	[p4]
sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10
p3
p4

Free-list

Commit Example

sub r5 - r2 ➜ r3	sub p5 - p2 ➜ p8	[p6]
addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10
p3
p4
p6

Free-list

Commit Example

addi r3 + 1 ➜ r1	addi p8 + 1 ➜ p9	[p1]

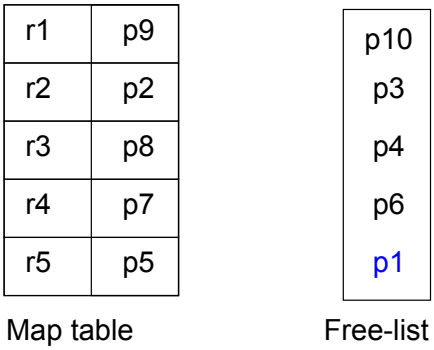
r1	p9
r2	p2
r3	p8
r4	p7
r5	p5

Map table

p10
p3
p4
p6
p1

Free-list

Commit Example



Dynamic Scheduling Example

Dynamic Scheduling Example

- The following slides are a detailed but concrete example
- Yet, it contains enough detail to be overwhelming
  - Try not to worry about the details
- Focus on the big picture take-away:

Hardware can reorder instructions
to extract instruction-level parallelism

Recall: Motivating Example

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [p1] ➜ p2	F	Di	I	RR	X	M_1	M_2	W	C				
add p2 + p3 ➜ p4	F	Di				I	RR	X	W	C			
xor p4 ^ p5 ➜ p6		F	Di				I	RR	X	W	C		
ld [p7] ➜ p8		F	Di	I	RR	X	M_1	M_2	W		C		

- How would this execution occur cycle-by-cycle?
- Execution latencies assumed in this example:
  - Loads have two-cycle load-to-use penalty
    - Three cycle total execution latency
  - All other instructions have single-cycle execution latency
- "Issue queue": hold all waiting (un-executed) instructions
  - Holds ready/not-ready status
  - Faster than looking up in ready table each cycle

Out-of-Order Pipeline – Cycle 0

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F												
add r2 + r3 ➜ r4	F												
xor r4 ^ r5 ➜ r6													
ld [r7] ➜ r4													

Map Table

r1	p8
r2	p7
r3	p6
r4	p5
r5	p4
r6	p3
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	---
p10	---
p11	---
p12	---

Reorder Buffer

Insn	To Free	Done?
ld		no
add		no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#

Out-of-Order Pipeline – Cycle 1a

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di											
add r2 + r3 ➜ r4	F												
xor r4 ^ r5 ➜ r6													
ld [r7] ➜ r4													

Map Table

r1	p8
r2	p9
r3	p6
r4	p5
r5	p4
r6	p3
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	---
p11	---
p12	---

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add		no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
ld	p8	yes	---	yes	p9	0

Out-of-Order Pipeline – Cycle 1b

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di											
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6													
ld [r7] ➜ r4													

Map Table

r1	p8
r2	p9
r3	p6
r4	p10
r5	p4
r6	p3
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	---
p12	---

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
ld	p8	yes	---	yes	p9	0
add	p9	no	p6	yes	p10	1

Out-of-Order Pipeline – Cycle 1c

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di											
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F											
ld [r7] ➜ r4		F											

Map Table

r1	p8
r2	p9
r3	p6
r4	p10
r5	p4
r6	p3
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	---
p12	---

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor		no
ld		no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
ld	p8	yes	---	yes	p9	0
add	p9	no	p6	yes	p10	1

Out-of-Order Pipeline – Cycle 2a

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I										
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F											
ld [r7] ➜ r4		F											

Map Table

r1	p8
r2	p9
r3	p6
r4	p10
r5	p4
r6	p3
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	---
p12	---

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor		no
ld		no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
ld	p8	yes	---	yes	p9	0
add	p9	no	p6	yes	p10	1

Out-of-Order Pipeline – Cycle 2b

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I										
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F											

Map Table

r1	p8
r2	p9
r3	p6
r4	p10
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	no
p12	---

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld		no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~	~~---~~	~~yes~~	~~p9~~	~~0~~
add	p9	no	p6	yes	p10	1
xor	p10	no	p4	yes	p11	2

Out-of-Order Pipeline – Cycle 2c

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I										
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F	Di										

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	no
p12	no

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
add	p9	no	p6	yes	p10	1
xor	p10	no	p4	yes	p11	2
ld	p2	yes	---	yes	p12	3

Out-of-Order Pipeline – Cycle 3

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR									
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F	Di	I									

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	no
p10	no
p11	no
p12	no

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
add	p9	no	p6	yes	p10	1
xor	p10	no	p4	yes	p11	2
ld	p2	yes	---	yes	p12	3

Out-of-Order Pipeline – Cycle 4

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X								
add r2 + r3 ➜ r4	F	Di											
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F	Di	I	RR								

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	yes
p10	no
p11	no
p12	no

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
add	p9	yes	p6	yes	p10	1
xor	p10	no	p4	yes	p11	2
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 5a

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1							
add r2 + r3 ➜ r4	F	Di				I							
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F	Di	I	RR	X							

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	yes
p10	yes
p11	no
p12	no

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
add	p9	yes	p6	yes	p10	1
xor	p10	yes	p4	yes	p11	2
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 5b

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1							
add r2 + r3 ➜ r4	F	Di				I							
xor r4 ^ r5 ➜ r6		F	Di										
ld [r7] ➜ r4		F	Di	I	RR	X							

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	yes
p10	yes
p11	no
p12	yes

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
xor	p10	yes	p4	yes	p11	2
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 6

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2						
add r2 + r3 ➜ r4	F	Di				I	RR						
xor r4 ^ r5 ➜ r6		F	Di				I						
ld [r7] ➜ r4		F	Di	I	RR	X	M_1						

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
ld	p7	no
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
xor	p10	yes	p4	yes	p11	2
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 7

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W					
add r2 + r3 ➜ r4	F	Di				I	RR	X					
xor r4 ^ r5 ➜ r6		F	Di				I	RR					
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2					

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	yes
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
ld	p7	yes
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 8a

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X					
xor r4 ^ r5 ➜ r6		F	Di				I	RR					
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2					

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	---
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
add	p5	no
xor	p3	no
ld	p10	no

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 8b

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X	W				
xor r4 ^ r5 ➜ r6		F	Di				I	RR	X				
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2	W				

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	yes
p6	yes
p7	---
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
add	p5	yes
xor	p3	no
ld	p10	yes

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 9a

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X	W	C			
xor r4 ^ r5 ➜ r6		F	Di				I	RR	X				
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2	W				

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	---
p6	yes
p7	---
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
~~add~~	~~p5~~	~~yes~~
xor	p3	no
ld	p10	yes

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 9b

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X	W	C			
xor r4 ^ r5 ➜ r6		F	Di				I	RR	X	W			
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2	W				

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	yes
p4	yes
p5	---
p6	yes
p7	---
p8	yes
p9	yes
p10	yes
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
~~add~~	~~p5~~	~~yes~~
xor	p3	yes
ld	p10	yes

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Cycle 10

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X	W	C			
xor r4 ^ r5 ➜ r6		F	Di				I	RR	X	W	C		
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2	W		C		

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	---
p4	yes
p5	---
p6	yes
p7	---
p8	yes
p9	yes
p10	---
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
~~add~~	~~p5~~	~~yes~~
~~xor~~	~~p3~~	~~yes~~
~~ld~~	~~p10~~	~~yes~~

Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

Out-of-Order Pipeline – Done!

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] ➜ r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 ➜ r4	F	Di				I	RR	X	W	C			
xor r4 ^ r5 ➜ r6		F	Di				I	RR	X	W	C		
ld [r7] ➜ r4		F	Di	I	RR	X	M_1	M_2	W		C		

Map Table

r1	p8
r2	p9
r3	p6
r4	p12
r5	p4
r6	p11
r7	p2
r8	p1

Ready Table

p1	yes
p2	yes
p3	---
p4	yes
p5	---
p6	yes
p7	---
p8	yes
p9	yes
p10	---
p11	yes
p12	yes

Reorder Buffer

Insn	To Free	Done?
~~ld~~	~~p7~~	~~yes~~
~~add~~	~~p5~~	~~yes~~
~~xor~~	~~p3~~	~~yes~~
~~ld~~	~~p10~~	~~yes~~

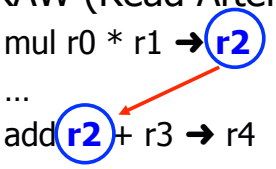
Issue Queue

Insn	Src1	R?	Src2	R?	Dest	#
~~ld~~	~~p8~~	~~yes~~		~~yes~~	~~p9~~	~~0~~
~~add~~	~~p9~~	~~yes~~	~~p6~~	~~yes~~	~~p10~~	~~1~~
~~xor~~	~~p10~~	~~yes~~	~~p4~~	~~yes~~	~~p11~~	~~2~~
~~ld~~	~~p2~~	~~yes~~		~~yes~~	~~p12~~	~~3~~

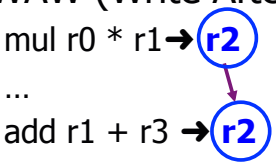
Handling Memory Operations

Recall: Types of Dependencies

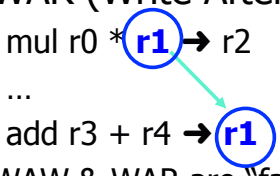
- RAW (Read After Write) = "true dependence"



- WAW (Write After Write) = "output dependence"



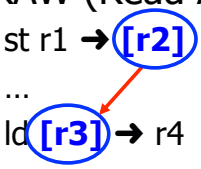
- WAR (Write After Read) = "anti-dependence"



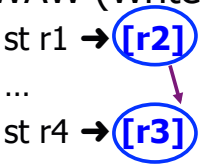
- WAW & WAR are "false", Can be **totally eliminated** by "renaming"

Also Have Dependencies via Memory

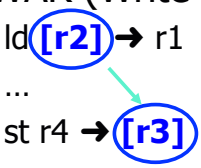
- **If value in "r2" and "r3" is the same…**
- RAW (Read After Write) – True dependency



- WAW (Write After Write)



- WAR (Write After Read)



WAR/WAW are "false dependencies"
- But can't rename memory in same way as registers
  - **Why? Address are not known at rename**
- Need to use other tricks

Let's Start with Just Stores

- Stores: Write data cache, not registers
  - Can we rename memory?
  - Recover in the cache?
- ➢ No (at least not easily)
  - Cache writes unrecoverable
- Solution: write stores into cache only when certain
  - When are we certain? At "commit"

Handling Stores

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	X	W	C		
st p5 ➜ [p3+4]		F	Di				I	RR	X	M	W	C	
st p4 ➜ [p6+8]		F	Di	I?									

- Can "st p4 ➜ [p6+8]" issue and begin execution?
  - Its registers inputs are ready…
  - Why or why not?

Problem #1: Out-of-Order Stores

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	X	W	C		
st p5 ➜ [p3+4]		F	Di				I	RR	X	**M**	W	C	
st p4 ➜ [p6+8]		F	Di	**I?**	**RR**	**X**	**M**	**W**				C	

- Can "st p4 ➜ [p6+8]" write the cache in cycle 6?
  - "st p5 ➜ [p3+4]" has not yet executed
- What if "p3+4 == p6+8"
  - The two stores write the same address! WAW dependency!
  - Not known until their "X" stages (cycle 5 & 8)
- Unappealing solution: all stores execute in-order
- We can do better…

Problem #2: Speculative Stores

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	**X**	W	C		
st p5 ➜ [p3+4]		F	Di				I	RR	X	M	W	C	
st p4 ➜ [p6+8]		F	Di	**I?**	**RR**	**X**	**M**	**W**				**C**	

- Can "st p4 ➜ [p6+8]" write the cache in cycle 6?
  - Store is still "speculative" at this point
- What if "jump-not-zero" is mis-predicted?
  - Not known until its "X" stage (cycle 8)
- How does it "undo" the store once it hits the cache?
  - Answer: it can't; stores write the cache only at **commit**
  - Guaranteed to be non-speculative at that point

Store Queue (SQ)

- Solves two problems
  - Allows for recovery of speculative stores
  - Allows out-of-order stores
- Store Queue (SQ)
  - **At dispatch, each store is given a slot in the Store Queue**
  - First-in-first-out (FIFO) queue
  - Each entry contains: "address", "value", and "#" (program order)
- Operation:
  - Dispatch (in-order): allocate entry in SQ (stall if full)
  - Execute (out-of-order): write store value into store queue
  - Commit (in-order): read value from SQ and write into data cache
  - Branch recovery: remove entries from the store queue
- Address the above two problems, plus more…

Memory Forwarding

	0	1	2	3	4	5	6	7	8	9	10	11	12
fdiv p1 / p2 ➜ p9	F	Di	I	RR	X_1	X_2	X_3	X_4	X_5	X_6	W	C	
st p4 ➜ [p5+4]	F	Di	I	RR	X	W						C	
st p3 ➜ [p6+8]		F	Di	I	RR	X	W						C
ld [p7] ➜ p8		F	Di	I?	RR	X	M_1	M_2	W				C

- Can "ld [p7] ➜ p8" issue and begin execution?
  - Why or why not?

Memory Forwarding

	0	1	2	3	4	5	6	7	8	9	10	11	12
fdiv p1 / p2 ➜ p9	F	Di	I	RR	X_1	X_2	X_3	X_4	X_5	X_6	W	C	
st p4 ➜ [p5+4]	F	Di	I	RR	X	SQ						C	
st p3 ➜ [p6+8]		F	Di	I	RR	X	SQ						C
ld [p7] ➜ p8		F	Di	I?	RR	X	M_1	M_2	W				C

- Can "ld [p7] ➜ p8" issue and begin execution?
  - Why or why not?
- If the load reads from either of the store's addresses…
  - Load must get correct value, but it isn't written to the cache until commit…

Memory Forwarding

	0	1	2	3	4	5	6	7	8	9	10	11	12
fdiv p1 / p2 ➜ p9	F	Di	I	RR	X_1	X_2	X_3	X_4	X_5	X_6	W	C	
st p4 ➜ [p5+4]	F	Di	I	RR	X	SQ						C	
st p3 ➜ [p6+8]		F	Di	I	RR	X	SQ						C
ld [p7] ➜ p8		F	Di	I?	RR	X	M_1	M_2	W				C

- Can "ld [p7] ➜ p8" issue and begin execution?
  - Why or why not?
- If the load reads from either of the store's addresses…
  - Load must get correct value, but it isn't written to the cache until commit…
- Solution: "memory forwarding"
  - Loads also searches the Store Queue (in parallel with cache access)
  - Conceptually like register bypassing, but different implementation
    - Why? Addresses unknown until execute

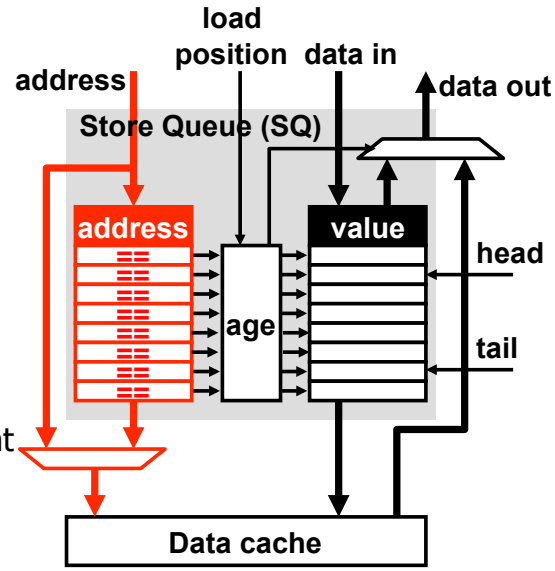
Problem #3: WAR Hazards

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	X	W	C		
ld [p3+4] ➜ p5		F	Di				I	RR	X	M_1	M_2	W	C
st p4 ➜ [p6+8]		F	Di	I	RR	X	SQ						C

- What if "p3+4 == p6 + 8"?
  - Then load and store access same memory location
- Need to make sure that load doesn't read store's result
  - Need to get values based on "program order" not "execution order"
- Bad solution: require all stores/loads to execute in-order
- Good solution: Track order, loads search SQ
  - Read from store to same address that is "earlier in program order"
  - Another reason the SQ is a FIFO queue

Memory Forwarding via Store Queue

- Store Queue (SQ)
  - Holds all in-flight stores
  - CAM: searchable by address
  - "Age" to determine which to forward from
- Store rename/dispatch
  - Allocate entry in SQ
- Store execution
  - Update SQ (Address + Data)
- Load execution
  - Search SQ to find: most recent store prior to the load (program order)
    - Match? Read SQ
    - No Match? Read cache



Store Queue (SQ)

- On load execution, select the store that is:
  - To same address as load
  - Prior to the load (before the load in program order)
- Of these, select the "youngest" store
  - The store to the address that most recently preceded the load

When Can Loads Execute?

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	**X**	W	C		
st p5 ➜ [p3+4]		F	Di				I	RR	X	SQ	C		
ld [p6+8] ➜ p7		F	Di	**I?**	**RR**	**X**	**M_1**	**M_2**	**W**			C	

- Can "ld [p6+8] ➜ p7" issue in cycle 3
  - Why or why not?

When Can Loads Execute?

	0	1	2	3	4	5	6	7	8	9	10	11	12
mul p1 * p2 ➜ p3	F	Di	I	RR	X_1	X_2	X_3	X_4	W	C			
jump-not-zero p3	F	Di					I	RR	**X**	W	C		
st p5 ➜ [p3+4]		F	Di				I	RR	X	SQ	C		
ld [p6+8] ➜ p7		F	Di	**I?**	**RR**	**X**	**M_1**	**M_2**	**W**			C	

- Aliasing! Does p3+4 == p6+8?
  - If no, load should get value from memory
    - **Can it start to execute?**
  - If yes, load should get value from store
    - By reading the store queue?
    - **But the value isn't put into the store queue until cycle 9**
- **Key challenge**: don't know addresses until execution!
  - One solution: require all loads to wait for all earlier (prior) stores

Compiler Scheduling Requires

- **Alias analysis**
  - Ability to tell whether load/store reference same memory locations
    - Effectively, whether load/store can be rearranged
  - Example code: easy, all loads/stores use same base register (`sp`)
  - New example: can compiler tell that `r8` != `r9`?
  - Must be **conservative**

Before

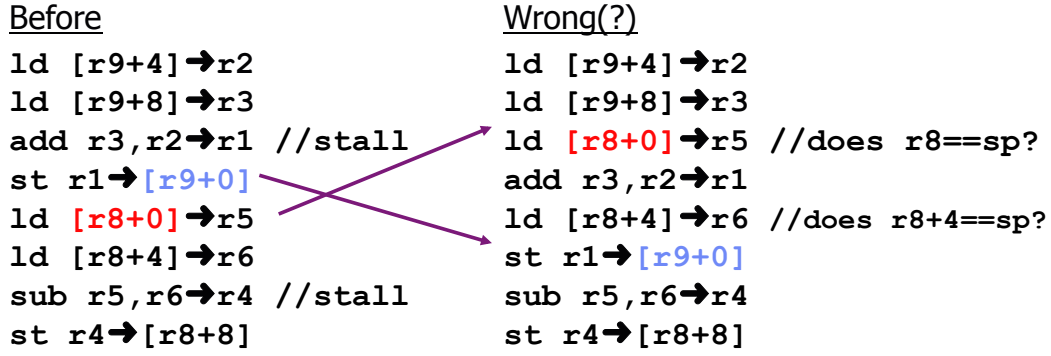
```
ld [r9+4]➜r2
ld [r9+8]➜r3
add r3,r2➜r1 //stall
st r1➜[r9+0]
ld [r8+0]➜r5
ld [r8+4]➜r6
sub r5,r6➜r4 //stall
st r4➜[r8+8]
```

Wrong(?)

```
ld [r9+4]➜r2
ld [r9+8]➜r3
ld [r8+0]➜r5 //does r8==r9?
add r3,r2➜r1
ld [r8+4]➜r6 //does r8+4==r9?
st r1➜[r9+0]
sub r5,r6➜r4
st r4➜[r8+8]
```

Dynamically Scheduling Memory Ops

- Compilers must schedule memory ops conservatively
- Options for hardware:
  - Don't execute any load until all prior stores execute (conservative)
  - Execute loads as soon as possible, detect violations (optimistic)
    - When a store executes, it checks if any later loads executed too early (to same address). If so, flush pipeline
  - Learn violations over time, selectively reorder (predictive)



Conservative Load Scheduling

- Conservative load scheduling:
  - All earlier stores have executed
    - Some architectures: split store address / store data
      - Only requires knowing addresses (not the store values)
  - Advantage: always safe
  - Disadvantage: performance (limits out-of-orderness)

Conservative Load Scheduling

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ld [p1] ➜ p4	F	Di	I	Rr	X	M_1	M_2	W	C							
ld [p2] ➜ p5	F	Di	I	Rr	X	M_1	M_2	W	C							
add p4, p5 ➜ p6		F	Di			I	Rr	X	W	C						
st p6 ➜ [p3]		F	Di				I	Rr	X	SQ	C					
ld [p1+4] ➜ p7			F	Di				I	Rr	X	M_1	M_2	W	C		
ld [p2+4] ➜ p8			F	Di				I	Rr	X	M_1	M_2	W	C		
add p7, p8 ➜ p9				F	Di						I	Rr	X	W	C	
st p9 ➜ [p3+4]				F	Di							I	Rr	X	SQ	C

Conservative load scheduling: can't issue ld [p1+4] until cycle 7!
Might as well be an in-order machine on this example
Can we do better? How?

Optimistic Load Scheduling

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ld [p1] ➜ p4	F	Di	I	Rr	X	M_1	M_2	W	C							
ld [p2] ➜ p5	F	Di	I	Rr	X	M_1	M_2	W	C							
add p4, p5 ➜ p6		F	Di			I	Rr	X	W	C						
st p6 ➜ [p3]		F	Di				I	Rr	X	SQ	C					
ld [p1+4] ➜ p7			F	Di	I	Rr	X	M_1	M_2	W	C					
ld [p2+4] ➜ p8			F	Di	I	Rr	X	M_1	M_2	W		C				
add p7, p8 ➜ p9				F	Di			I	Rr	X	W	C				
st p9 ➜ [p3+4]				F	Di				I	Rr	X	SQ	C			

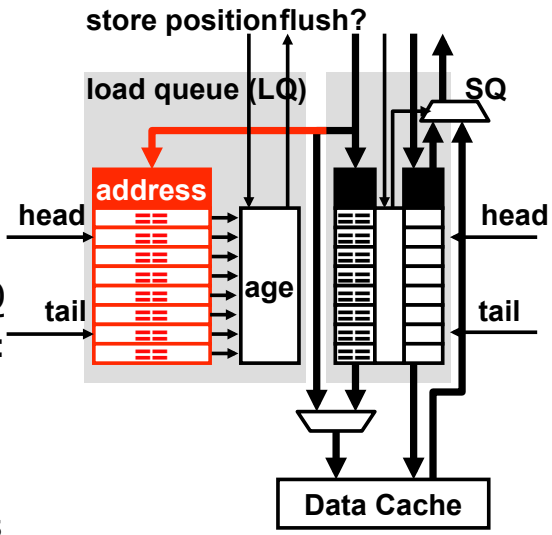
Optimistic load scheduling: can actually benefit from out-of-order!
But how do we know when out speculation (optimism) fails?

Load Speculation

- Speculation requires two things…..
  - 1. Detection of mis-speculations
    - How can we do this?
  - 2. Recovery from mis-speculations
    - Squash from offending load
    - Saw how to squash from branches: same method

Load Queue

- Detects load ordering violations
- Load execution: Write LQ
  - Write address into LQ
  - Record which in-flight store it forwarded from (if any)
- Store execution: Search LQ
  - For a store S, foreach load L:
    - Does S.addr = L.addr?
    - Is S before L in program order?
    - Which store did L gets its value from?



Store Queue + Load Queue

- Store Queue: handles forwarding
  - Entry per store (allocated @ dispatch, deallocated @ commit)
  - Written by stores (@ execute)
  - Searched by loads (@ execute)
  - Read from to write data cache (@ commit)
- Load Queue: detects ordering violations
  - Entry per load (allocated @ dispatch, deallocated @ commit)
  - Written by loads (@ execute)
  - Searched by stores (@ execute)
- Both together
  - Allows aggressive load scheduling
  - Stores don't constrain load execution

Optimistic Load Scheduling Problem

- Allows loads to issue before earlier stores
  - Increases out-of-orderness
  - \+ Good: When no conflict, increases performance
  - \- Bad: Conflict => squash => worse performance than waiting
- Can we have our cake AND eat it too?

Predictive Load Scheduling

- Predict which loads must wait for stores

- Fool me once, shame on you-- fool me twice?
  - Loads default to aggressive
  - Keep table of load PCs that have been caused squashes
    - Schedule these conservatively
  + Simple predictor
  - Makes "bad" loads wait for all stores before it is not so great

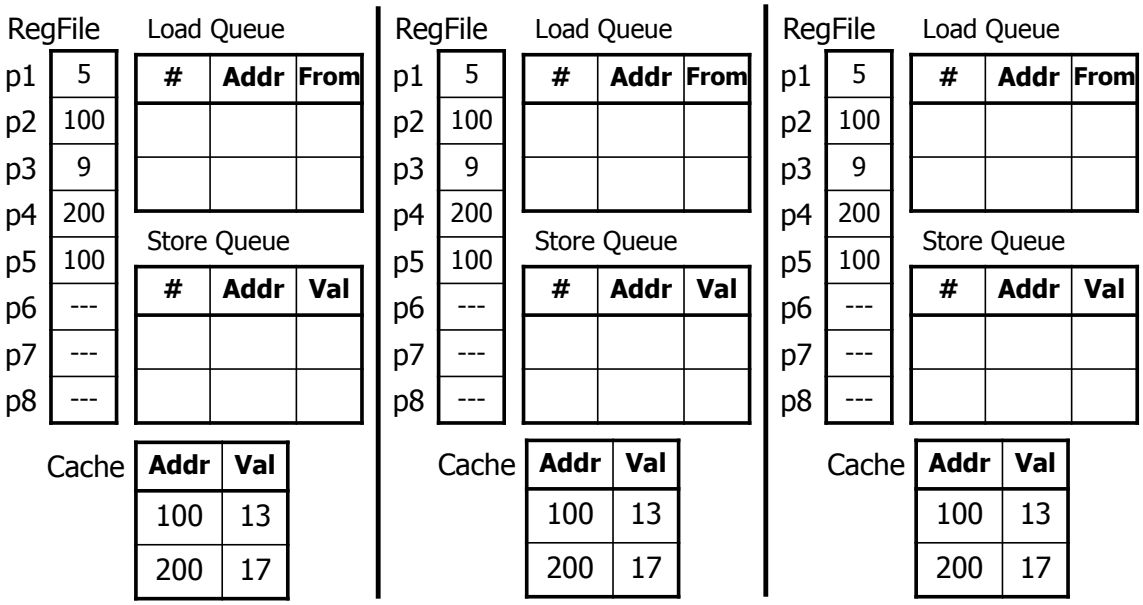
- More complex predictors used in practice
  - Predict which stores loads should wait for
  - "Store Sets" paper for next time

Load/Store Queue Examples

Initial State

(Stores to different addresses)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

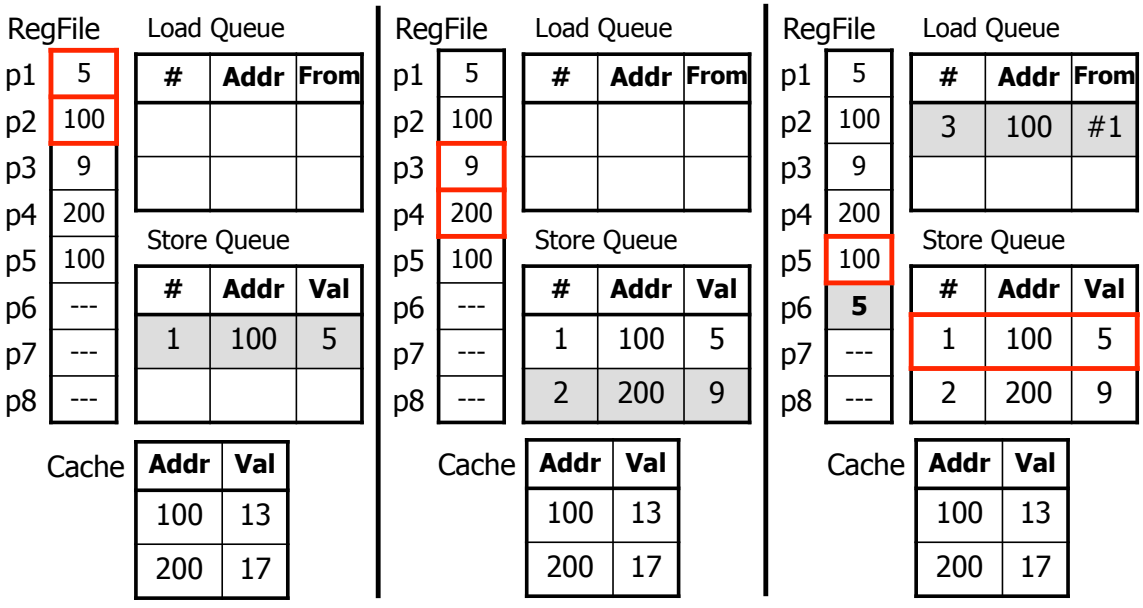


Good Interleaving

(Shows importance of address check)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

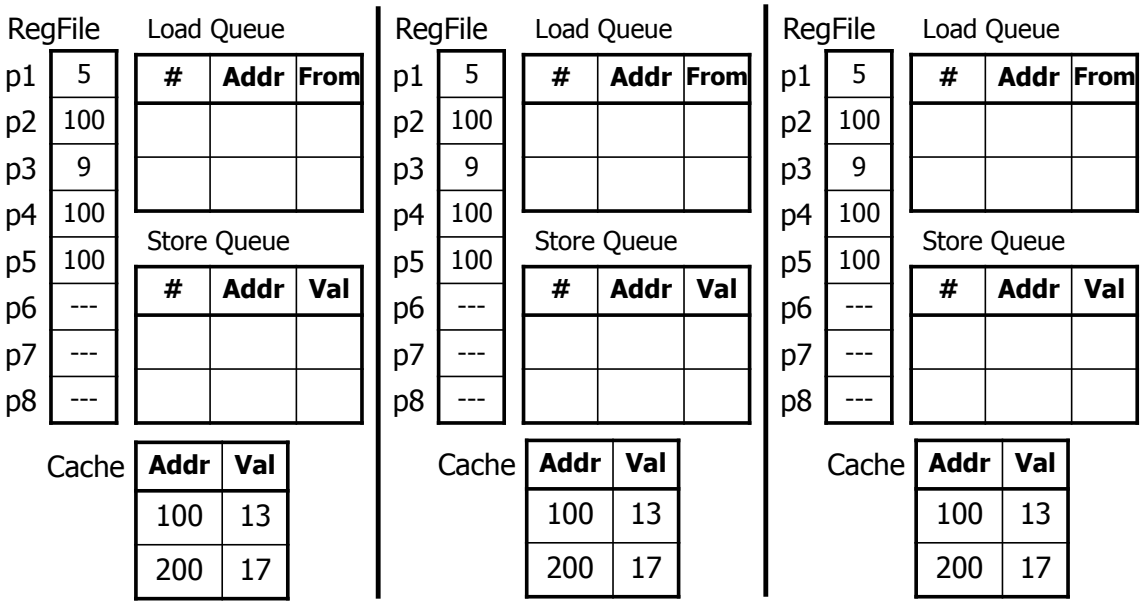
1. St p1 ➜ [p2] 2. St p3 ➜ [p4] 3. Ld [p5] ➜ p6



Different Initial State

(All to same address)

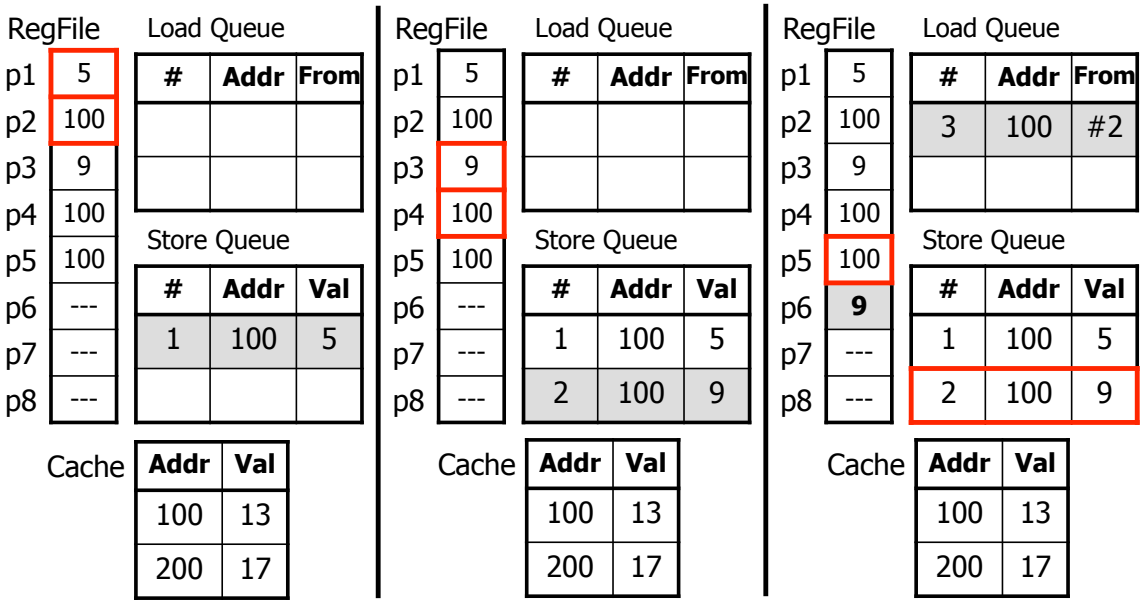
1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6



Good Interleaving #1

(Program Order)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6



Good Interleaving #2

(Stores reordered, so okay)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

2. St p3 ➜ [p4]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	---
p7	---
p8	---

Load Queue

#	Addr	From

Store Queue

#	Addr	Val
2	100	9

Cache

Addr	Val
100	13
200	17

1. St p1 ➜ [p2]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	---
p7	---
p8	---

Load Queue

#	Addr	From

Store Queue

#	Addr	Val
1	100	5
2	100	9

Cache

Addr	Val
100	13
200	17

3. Ld [p5] ➜ p6

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	**9**
p7	---
p8	---

Load Queue

#	Addr	From
3	100	#2

Store Queue

#	Addr	Val
1	100	5
2	100	9

Cache

Addr	Val
100	13
200	17

Bad Interleaving #1

(Load reads the cache, but should not)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

3. Ld [p5] ➜ p6

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	13
p7	---
p8	---

Load Queue

#	Addr	From
3	100	--

Store Queue

#	Addr	Val

Cache

Addr	Val
100	13
200	17

2. St p3 ➜ [p4]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	13
p7	---
p8	---

Load Queue

#	Addr	From
3	100	--

Store Queue

#	Addr	Val
2	100	9

Cache

Addr	Val
100	13
200	17

Bad Interleaving #2

(Load gets value from wrong store)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

1. St p1 ➜ [p2]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	---
p7	---
p8	---

Load Queue

#	Addr	From
3	100	--

Store Queue

#	Addr	Val
1	100	5

Cache

Addr	Val
100	13
200	17

3. Ld [p5] ➜ p6

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	**5**
p7	---
p8	---

Load Queue

#	Addr	From
3	100	#1

Store Queue

#	Addr	Val
1	100	5

Cache

Addr	Val
100	13
200	17

2. St p3 ➜ [p4]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	5
p7	---
p8	---

Load Queue

#	Addr	From
3	100	#1

Store Queue

#	Addr	Val
1	100	5
2	100	9

Cache

Addr	Val
100	13
200	17

Good Interleaving #3

(Using "From" field to prevent false squash)

1. St p1 ➜ [p2]
2. St p3 ➜ [p4]
3. Ld [p5] ➜ p6

2. St p3 ➜ [p4]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	---
p7	---
p8	---

Load Queue

#	Addr	From

Store Queue

#	Addr	Val
2	100	9

Cache

Addr	Val
100	13
200	17

3. Ld [p5] ➜ p6

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	**9**
p7	---
p8	---

Load Queue

#	Addr	From
3	100	#2

Store Queue

#	Addr	Val
2	100	9

Cache

Addr	Val
100	13
200	17

1. St p1 ➜ [p2]

RegFile

p1	5
p2	100
p3	9
p4	100
p5	100
p6	9
p7	---
p8	---

Load Queue

#	Addr	From
3	100	#2

Store Queue

#	Addr	Val
1	100	5
2	100	9

Cache

Addr	Val
100	13
200	17

Out-of-Order: Benefits & Challenges

Dynamic Scheduling Operation

- Dynamic scheduling
  - Totally in the hardware (not visible to software)
  - Also called "out-of-order execution" (OoO)
- Fetch many instructions into instruction window
  - Use branch prediction to speculate past (multiple) branches
  - Flush pipeline on branch misprediction
- Rename registers to avoid false dependencies
- Execute instructions as soon as possible
  - Register dependencies are known
  - Handling memory dependencies more tricky
- "Commit" instructions in order
  - Anything strange happens before commit, just flush the pipeline
- How much out-of-order? Core i7 "Haswell":
  - 192-entry reorder buffer, 168 integer registers, 60-entry scheduler

Haswell Buffer Sizes

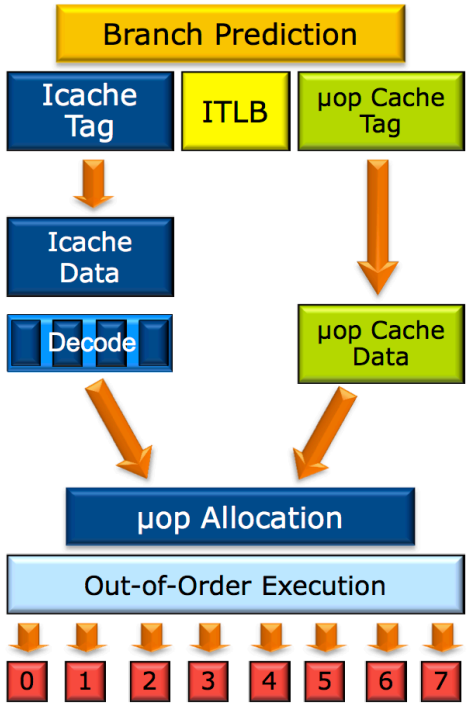
Extract more parallelism in every generation

	Nehalem	Sandy Bridge	Haswell
Out-of-order Window	128	168	192
In-flight Loads	48	64	72
In-flight Stores	32	36	42
Scheduler Entries	36	54	60
Integer Register File	N/A	160	168
FP Register File	N/A	144	168
Allocation Queue	28/thread	28/thread	56

IDF2012 INTEL DEVELOPER FORUM

11 Intel® Microarchitecture (Haswell); Intel® Microarchitecture (Nehalem); Intel® Microarchitecture (Sandy Bridge)

Haswell Core at a Glance



Next generation branch prediction
- Improves performance *and* saves wasted work

Improved front-end
- Initiate TLB and cache misses speculatively
- Handle cache misses in parallel to hide latency
- Leverages improved branch prediction

Deeper buffers
- Extract more instruction parallelism
- More resources when running a single thread

More execution units, shorter latencies
- Power down when not in use

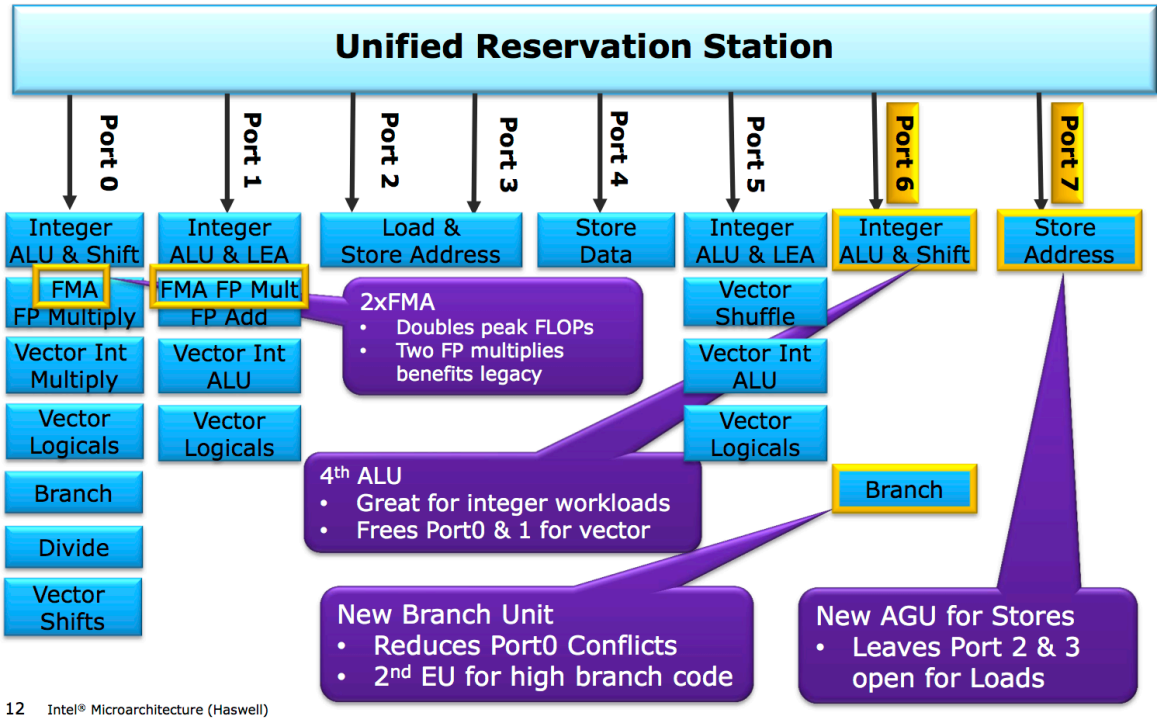
More load/store bandwidth
- Better prefetching, better cache line split latency & throughput, double L2 bandwidth
- New modes save power without losing performance

No pipeline growth
- Same branch misprediction latency
- Same L1/L2 cache latency

IDF2012 INTEL DEVELOPER FORUM

10 Intel® Microarchitecture (Haswell)

Haswell Execution Unit Overview



Core Cache Size/Latency/Bandwidth

Metric	Nehalem	Sandy Bridge	Haswell
L1 Instruction Cache	32K, 4-way	32K, 8-way	32K, 8-way
L1 Data Cache	32K, 8-way	32K, 8-way	32K, 8-way
Fastest Load-to-use	4 cycles	4 cycles	4 cycles
Load bandwidth	16 Bytes/cycle	32 Bytes/cycle (banked)	**64 Bytes/cycle**
Store bandwidth	16 Bytes/cycle	16 Bytes/cycle	**32 Bytes/cycle**
L2 Unified Cache	256K, 8-way	256K, 8-way	256K, 8-way
Fastest load-to-use	10 cycles	11 cycles	11 cycles
Bandwidth to L1	32 Bytes/cycle	32 Bytes/cycle	**64 Bytes/cycle**
L1 Instruction TLB	4K: 128, 4-way 2M/4M: 7/thread	4K: 128, 4-way 2M/4M: 8/thread	4K: 128, 4-way 2M/4M: 8/thread
L1 Data TLB	4K: 64, 4-way 2M/4M: 32, 4-way 1G: fractured	4K: 64, 4-way 2M/4M: 32, 4-way 1G: 4, 4-way	4K: 64, 4-way 2M/4M: 32, 4-way 1G: 4, 4-way
L2 Unified TLB	4K: 512, 4-way	4K: 512, 4-way	**4K+2M shared: 1024, 8-way**

All caches use 64-byte lines

Out of Order: Benefits

- Allows speculative re-ordering
  - Loads / stores
  - Branch prediction to look past branches
- Done by hardware
  - Compiler may want different schedule for different hw configs
  - Hardware has only its own configuration to deal with
- Schedule can change due to cache misses
  - Different schedule optimal from on cache hit
- **Memory-level parallelism**
  - Executes "around" cache misses to find independent instructions
  - Finds and initiates independent misses, reducing memory latency
    - Especially good at hiding L2 hits (~11 cycles in Core i7)

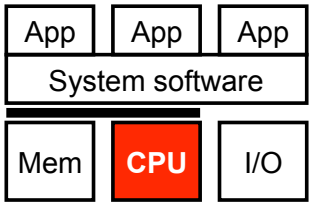
Challenges for Out-of-Order Cores

- Design complexity
  - More complicated than in-order? Certainly!
  - But, we have managed to overcome the design complexity
- Clock frequency
  - Can we build a "high ILP" machine at high clock frequency?
  - Yep, with some additional pipe stages, clever design
- Limits to (efficiently) scaling the window and ILP
  - Large physical register file
  - Fast register renaming/wakeup/select/load queue/store queue
    - Active areas of micro-architectural research
  - Branch & memory depend. prediction (limits effective window size)
    - 95% branch mis-prediction: 1 in 20 branches, or 1 in 100 insn.
  - Plus all the issues of build "wide" in-order superscalar
- Power efficiency
  - Today, even mobile phone chips are out-of-order cores

Redux: Hdw vs. Software Scheduling

- Static scheduling
  - Performed by compiler, limited in several ways
- Dynamic scheduling
  - Performed by the hardware, overcomes limitations
- Static limitation ➜ dynamic mitigation
  - Number of registers in the ISA ➜ register renaming
  - Scheduling scope ➜ branch prediction & speculation
  - Inexact memory aliasing information ➜ speculative memory ops
  - Unknown latencies of cache misses ➜ execute when ready
- Which to do? **Compiler does what it can, hardware the rest**
  - Why? dynamic scheduling needed to sustain more than 2-way issue
  - **Helps with hiding memory latency** (execute around misses)
  - Intel Core i7 is four-wide execute w/ large scheduling window
  - Even mobile phones have dynamic scheduled cores (ARM A9)

Summary: Scheduling



- Code scheduling
  - To reduce pipeline stalls
  - To increase ILP (insn level parallelism)
- Static scheduling by the compiler
  - Approach & limitations
- Dynamic scheduling in hardware
  - Register renaming
  - Instruction selection
  - Handling memory operations
- Up next: multicore

Computer Architecture
Mini-Course

March 2013

Prof. Milo Martin

Day 3 of 3

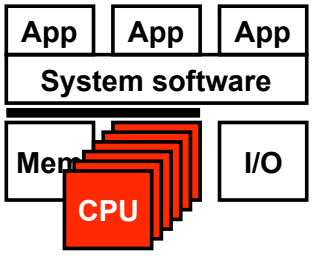
[spacer]

Computer Architecture

Unit 9: Multicore

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

This Unit: Shared Memory Multiprocessors



- Thread-level parallelism (TLP)
- Shared memory model
  - Multiplexed uniprocessor
  - Hardware multihreading
  - Multiprocessing
- Cache coherence
  - Valid/Invalid, MSI, MESI
- Parallel programming
- Synchronization
  - Lock implementation
  - Locking gotchas
  - Transactional memory
- Memory consistency models

Readings

- "Assigned" reading
  - "Why On-Chip Cache Coherence is Here to Stay" by Milo Martin, Mark Hill, and Daniel Sorin, Communications of the ACM (CACM), July 2012.

- Suggested reading
  - "A Primer on Memory Consistency and Cache Coherence" (Synthesis Lectures on Computer Architecture) by Daniel Sorin, Mark Hill, and David Wood, November 2011
  - "Speculative Lock Elision: Enabling Highly Concurrent Multithreaded Execution" by Rajwar & Goodman, MICRO 2001

Beyond Implicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy():
  for (i = 0; i < SIZE; i++)
    z[i] = a*x[i] + y[i];
```

- Lots of instruction-level parallelism (ILP)
  - Great!
  - But how much can we really exploit? 4 wide? 8 wide?
    - Limits to (efficient) super-scalar execution

- But, if SIZE is 10,000, the loop has 10,000-way parallelism!
  - How do we exploit it?

Explicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy():
  for (i = 0; i < SIZE; i++)
    z[i] = a*x[i] + y[i];
```

- Break it up into N "chunks" on N cores!
  - Done by the programmer (or maybe a really smart compiler)

```
void daxpy(int chunk_id):
  chuck_size = SIZE / N
  my_start = chuck_id * chuck_size
  my_end = my_start + chuck_size
  for (i = my_start; i < my_end; i++)
    z[i] = a*x[i] + y[i]
```

SIZE = 400, N=4

Chunk ID	Start	End
0	0	99
1	100	199
2	200	299
3	300	399

- Assumes
  - Local variables are "private" and x, y, and z are "shared"
  - Assumes SIZE is a multiple of N (that is, SIZE % N == 0)

Explicit Parallelism

- Consider "daxpy":

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy(int chunk_id):
  chuck_size = SIZE / N
  my_start = chuck_id * chuck_size
  my_end = my_start + chuck_size
  for (i = my_start; i < my_end; i++)
    z[i] = a*x[i] + y[i]
```

- Main code then looks like:

```
parallel_daxpy():
  for (tid = 0; tid < CORES; tid++) {
    spawn_task(daxpy, tid);
  }
  wait_for_tasks(CORES);
```

Explicit (Loop-Level) Parallelism

- Another way: "OpenMP" annotations to inform the compiler

```
double a, x[SIZE], y[SIZE], z[SIZE];
void daxpy() {
  #pragma omp parallel for
  for (i = 0; i < SIZE; i++) {
    z[i] = a*x[i] + y[i];
  }
```

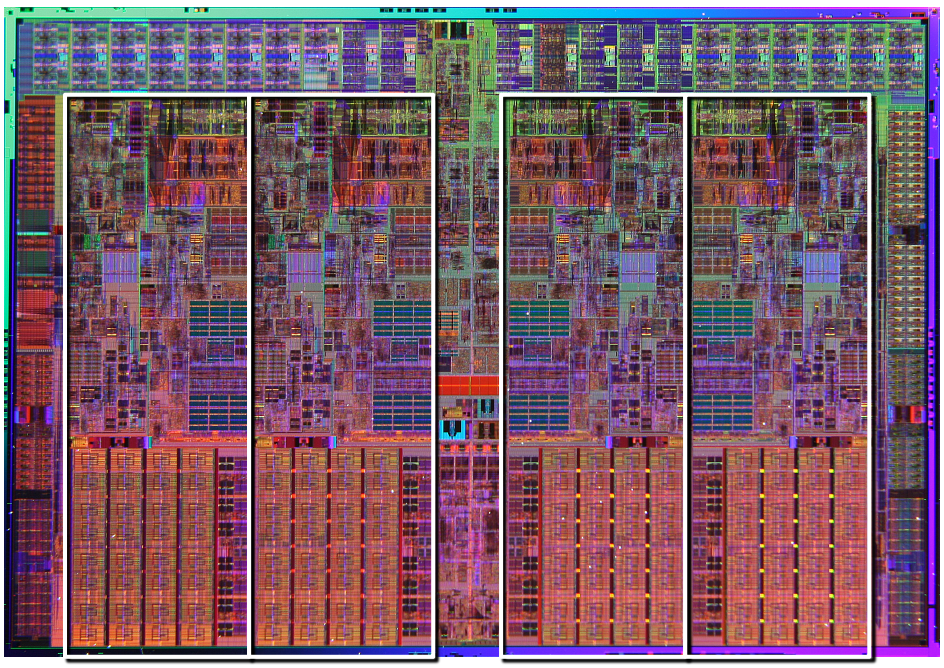
- But only works if loop is actually parallel
  - If not parallel, incorrect behavior may result in unpredictable ways

Multicore & Multiprocessor Hardware

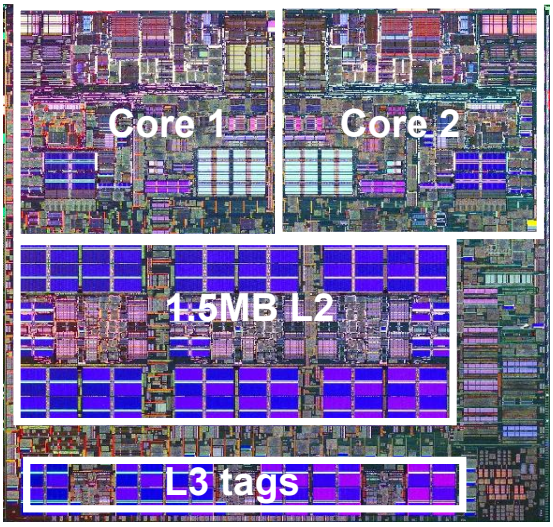
Multiplying Performance

- A single core can only be so fast
  - Limited clock frequency
  - Limited instruction-level parallelism
- What if we need even more computing power?
  - Use multiple cores! But how?
- Old-school (2000s): Ultra Enterprise 25k
  - 72 dual-core UltraSPARC IV+ processors
  - Up to 1TB of memory
  - Niche: large database servers
  - $$$, weights more than 1 ton
- Today: multicore is everywhere
  - Dual-core ARM phones

Intel Quad-Core "Core i7"



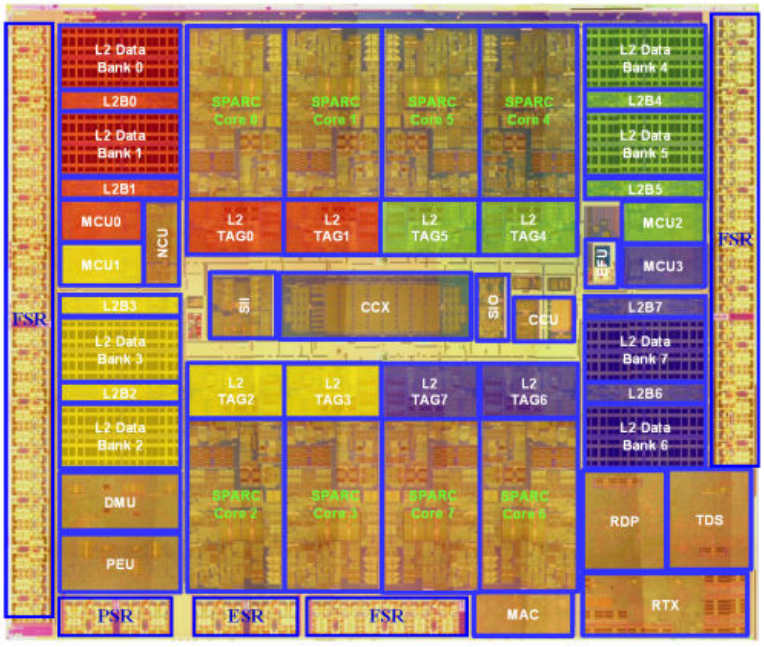
Multicore: Mainstream Multiprocessors



Why multicore? What else would you do with 1 billion transistors?

- **Multicore chips**
- **IBM Power5**
  - Two 2+GHz PowerPC cores
  - Shared 1.5 MB L2, L3 tags
- AMD Quad Phenom
  - Four 2+ GHz cores
  - Per-core 512KB L2 cache
  - Shared 2MB L3 cache
- Intel Core i7 Quad
  - Four cores, private L2s
  - Shared 8 MB L3
- Sun Niagara
  - 8 cores, each 4-way threaded
  - Shared 2MB L2
  - For servers, not desktop

Sun Niagara II



Application Domains for Multiprocessors

- **Scientific computing/supercomputing**
  - Examples: weather simulation, aerodynamics, protein folding
  - Large grids, integrating changes over time
  - Each processor computes for a part of the grid
- **Server workloads**
  - Example: airline reservation database
  - Many concurrent updates, searches, lookups, queries
  - Processors handle different requests
- **Media workloads**
  - Processors compress/decompress different parts of image/frames
- **Desktop workloads…**
- **Gaming workloads…**

But software must be written to expose parallelism

Recall: Multicore & Energy

- Explicit parallelism (multicore) is highly energy efficient
- Recall: dynamic voltage and frequency scaling
  - Performance vs power is NOT linear
  - Example: Intel's Xscale
    - 1 GHz → 200 MHz reduces energy used by 30x
- Consider the impact of parallel execution
  - What if we used 5 Xscales at 200Mhz?
  - Similar performance as a 1Ghz Xscale, but **1/6th the energy**
    - 5 cores * 1/30th = $1/6^{th}$
- And, amortizes background "uncore" energy among cores
- Assumes parallel speedup (a difficult task)
  - Subject to Ahmdal's law

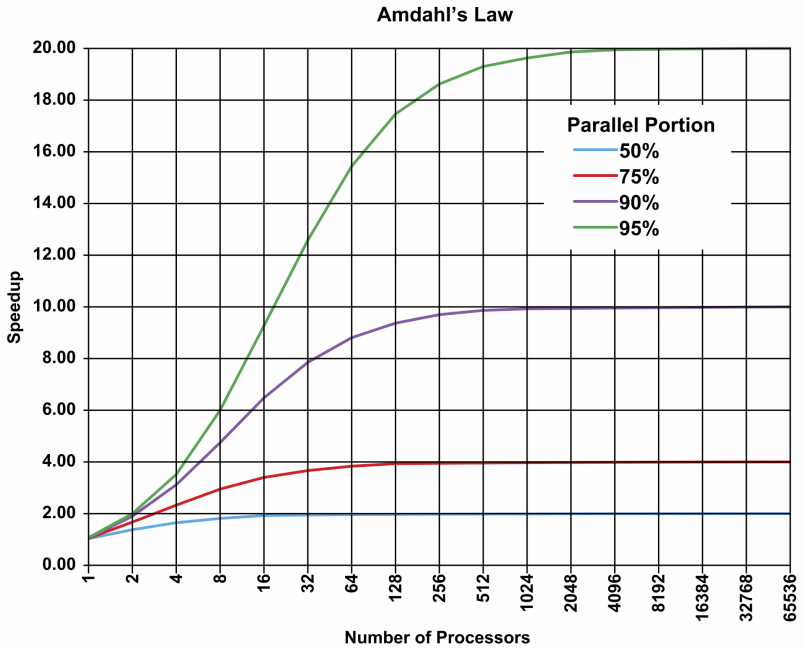
Amdahl's Law

- Restatement of the law of diminishing returns
  - Total speedup limited by non-accelerated piece
  - Analogy: drive to work & park car, walk to building

- Consider a task with a "parallel" and "serial" portion
  - What is the speedup with N cores?
  - Speedup(n, p, s) = (s+p) / (s + (p/n))
    - p is "parallel percentage", s is "serial percentage"
  - What about infinite cores?
    - Speedup(p, s) = (s+p) / s = 1 / s

- Example: can optimize 50% of program A
  - Even "magic" optimization that makes this 50% disappear…
  - …only yields a 2X speedup

Amdahl's Law Graph



Source: Wikipedia

"Threading" & The Shared Memory Execution Model

First, Uniprocessor Concurrency

- **Software "thread":** Independent flows of execution
  - "Per-thread" state
    - Context state: PC, registers
    - Stack (per-thread local variables)
  - "Shared" state: globals, heap, etc.
  - Threads generally share the same memory space
    - "Process" like a thread, but different memory space
  - Java has thread support built in, C/C++ using a thread library
- Generally, system software (the O.S.) manages threads
  - "Thread scheduling", "context switching"
  - In single-core system, all threads share the one processor
    - Hardware timer interrupt occasionally triggers O.S.
    - Quickly swapping threads gives illusion of concurrent execution
  - Much more in an operating systems course

Multithreaded Programming Model

- Programmer explicitly creates multiple threads
- All loads & stores to a single **shared memory** space
  - Each thread has its own stack frame for local variables
  - All memory shared, accessible by all threads
- A "thread switch" can occur at any time
  - Pre-emptive multithreading by OS
- Common uses:
  - Handling user interaction (GUI programming)
  - Handling I/O latency (send network message, wait for response)
  - **Expressing parallel work via Thread-Level Parallelism (TLP)**
    - This is our focus!

Shared Memory Model: Interleaving

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y	store 1 → x
load x	load y

- What value pairs can be read by the two loads?

Shared Memory Model: Interleaving

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

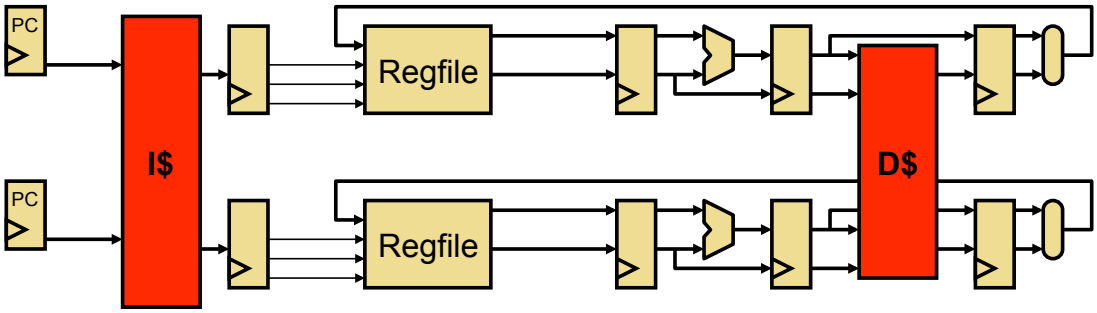
store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0, y=0)?

Shared Memory Implementations

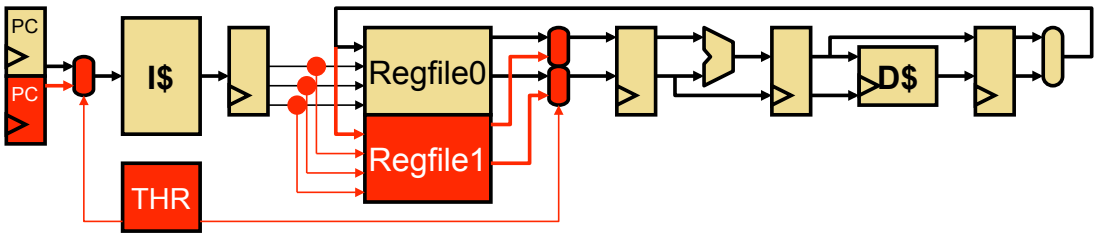
- **Multiplexed uniprocessor**
  - Runtime system and/or OS occasionally pre-empt & swap threads
  - Interleaved, but no parallelism
- **Multiprocessing**
  - Multiply execution resources, higher peak performance
  - Same interleaved shared-memory model
  - Foreshadowing: allow private caches, further disentangle cores
- **Hardware multithreading**
  - Tolerate pipeline latencies, higher efficiency
  - Same interleaved shared-memory model
- **All support the shared memory programming model**

Simplest Multiprocessor



- Replicate entire processor pipeline!
  - Instead of replicating just register file & PC
  - Exception: share the caches (we'll address this bottleneck soon)
- Multiple threads execute
  - Shared memory programming model
  - Operations (loads and stores) are interleaved "at random"
  - Loads returns the value written by most recent store to location

Hardware Multithreading



- **Hardware Multithreading (MT)**
  - Multiple threads dynamically share a single pipeline
  - Replicate only per-thread structures: program counter & registers
  - Hardware interleaves instructions
  - \+ **Multithreading improves utilization and throughput**
    - Single programs utilize <50% of pipeline (branch, cache miss)
  - **Multithreading does not improve single-thread performance**
    - Individual threads run as fast or even slower
  - **Coarse-grain MT**: switch on cache misses Why?
  - **Simultaneous MT**: no explicit switching, fine-grain interleaving

Four Shared Memory Issues

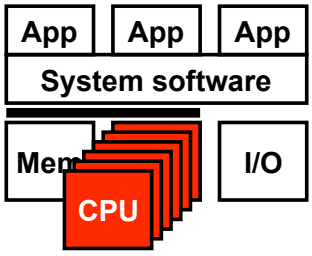
1. **Cache coherence**
   - If cores have private (non-shared) caches
   - How to make writes to one cache "show up" in others?

2. **Parallel programming**
   - How does the programmer express the parallelism?

3. **Synchronization**
   - How to regulate access to shared data?
   - How to implement "locks"?

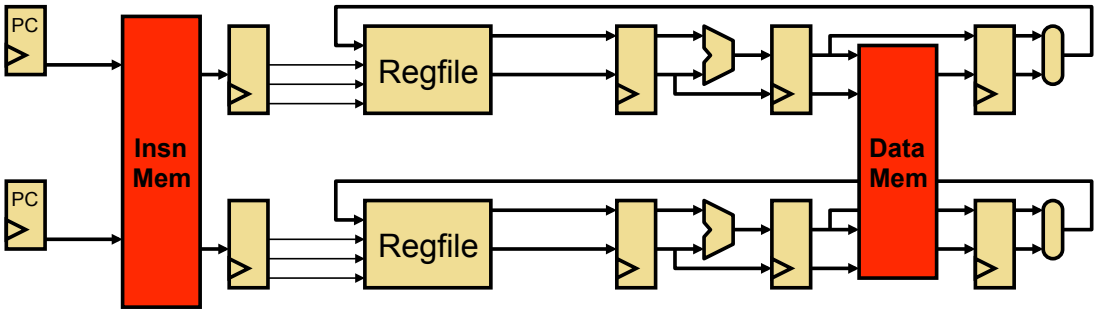
4. **Memory consistency models**
   - How to keep programmer sane while letting hardware optimize?
   - How to reconcile shared memory with compiler optimizations, store buffers, and out-of-order execution?

Roadmap Checkpoint



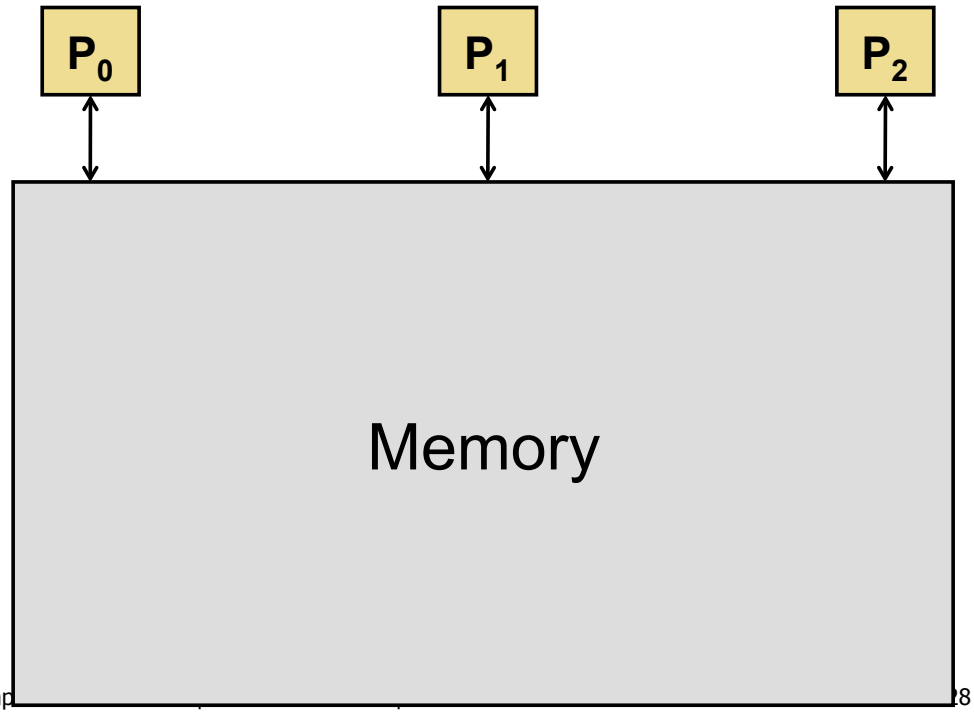
- ~~Thread-level parallelism (TLP)~~
- ~~Shared memory model~~
  - ~~Multiplexed uniprocessor~~
  - ~~Hardware multihreading~~
  - ~~Multiprocessing~~
- **Cache coherence**
  - **Valid/Invalid, MSI, MESI**
- Parallel programming
- Synchronization
  - Lock implementation
  - Locking gotchas
  - Transactional memory
- Memory consistency models

Recall: Simplest Multiprocessor

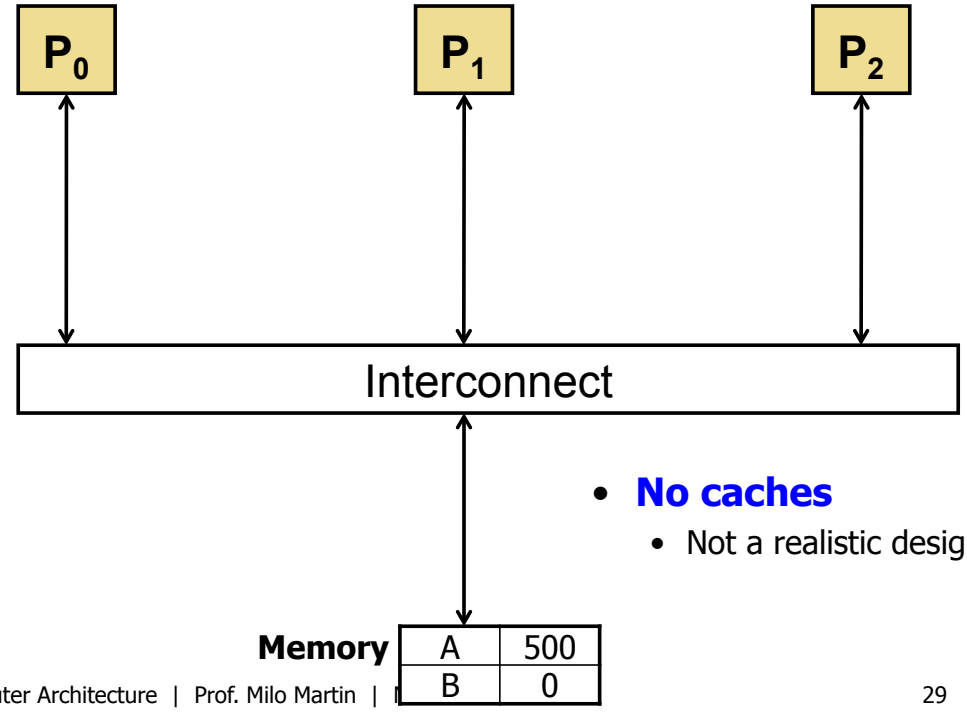


- What if we don't want to share the L1 caches?
  - Bandwidth and latency issue
- Solution: use per-processor ("private") caches
  - Coordinate them with a ***Cache Coherence Protocol***
- Must still provide shared-memory invariant:
  - **"Loads read the value written by the most recent store"**

No-Cache (Conceptual) Implementation

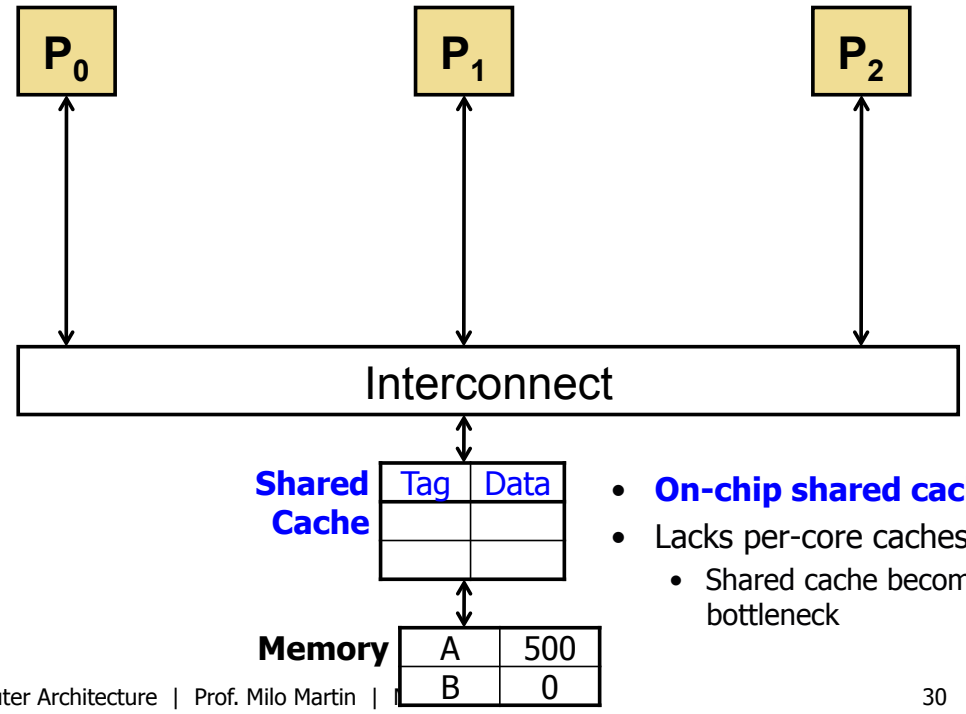


No-Cache (Conceptual) Implementation



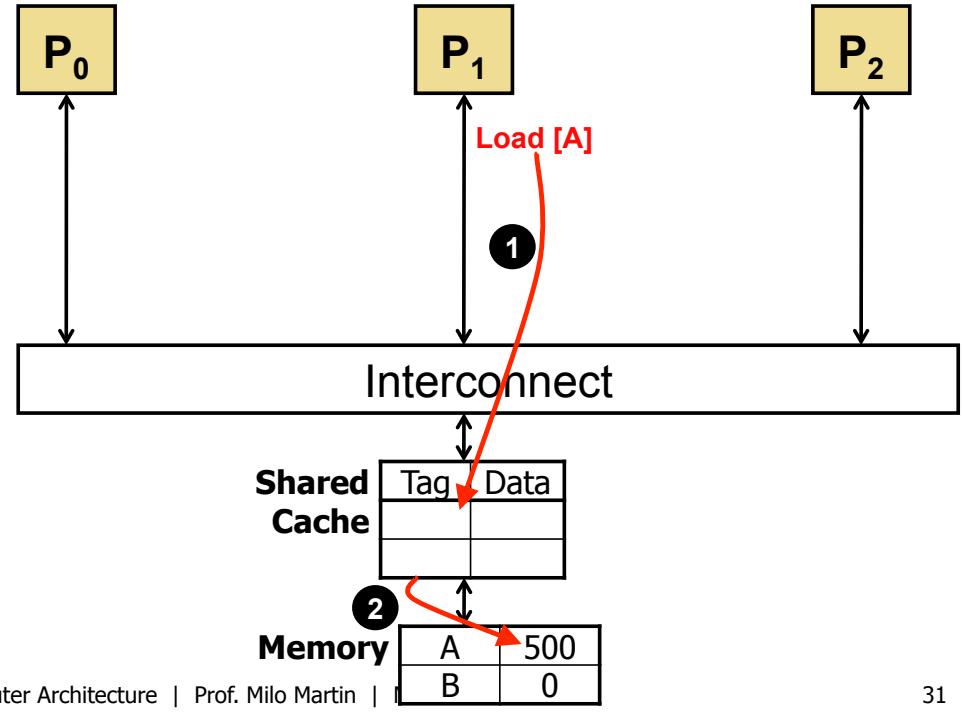
- **No caches**
  - Not a realistic design

Shared Cache Implementation

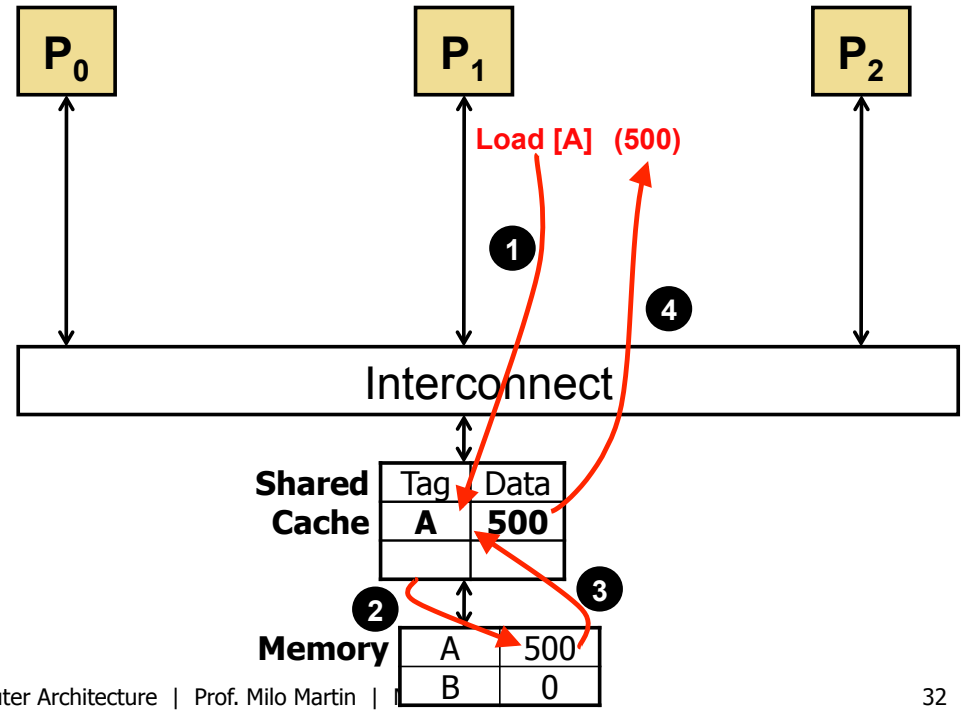


- **On-chip shared cache**
- Lacks per-core caches
  - Shared cache becomes bottleneck

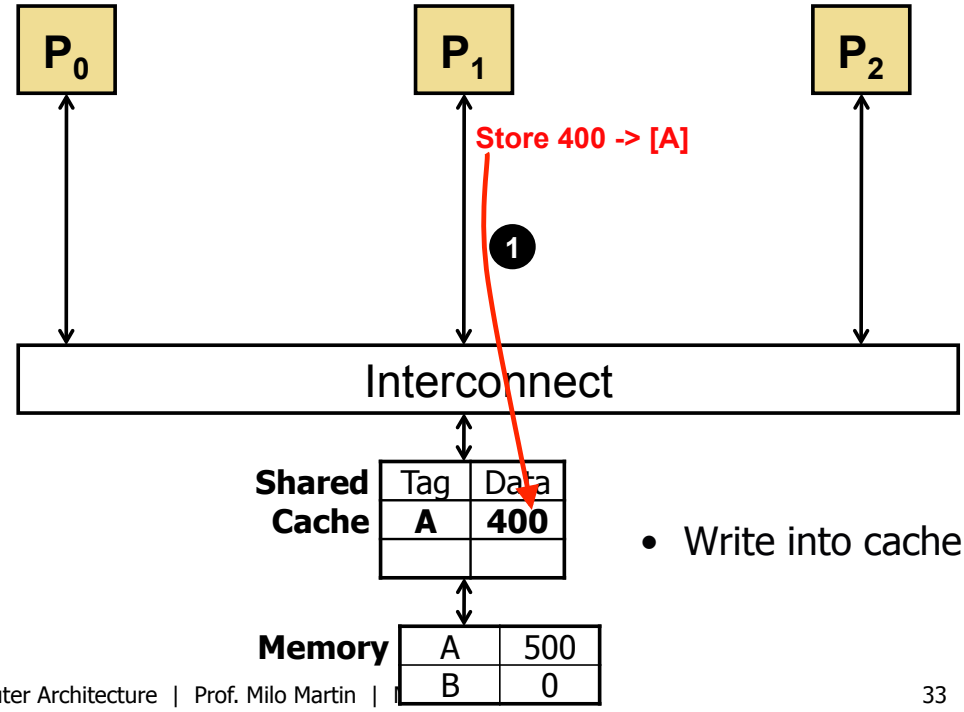
Shared Cache Implementation



Shared Cache Implementation

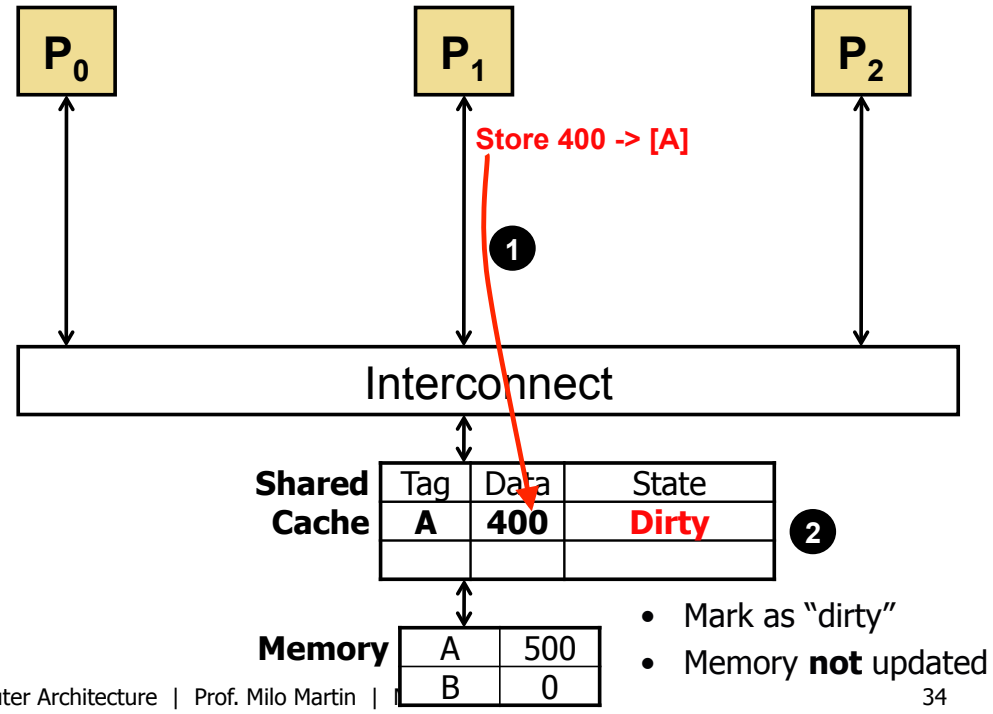


Shared Cache Implementation



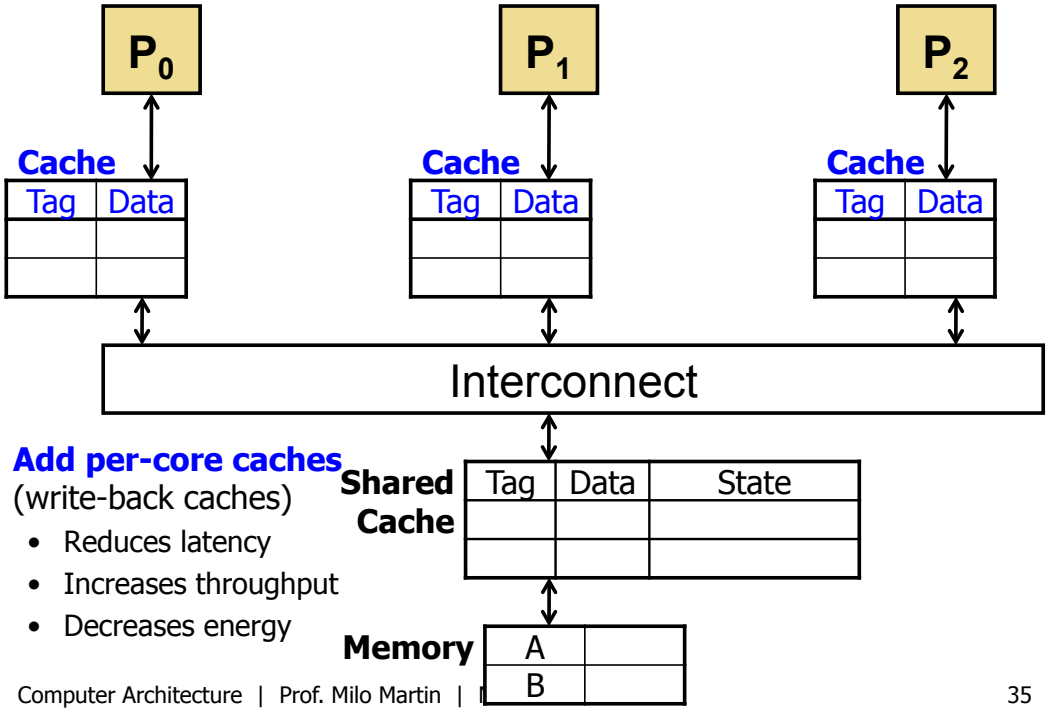
- Write into cache

Shared Cache Implementation



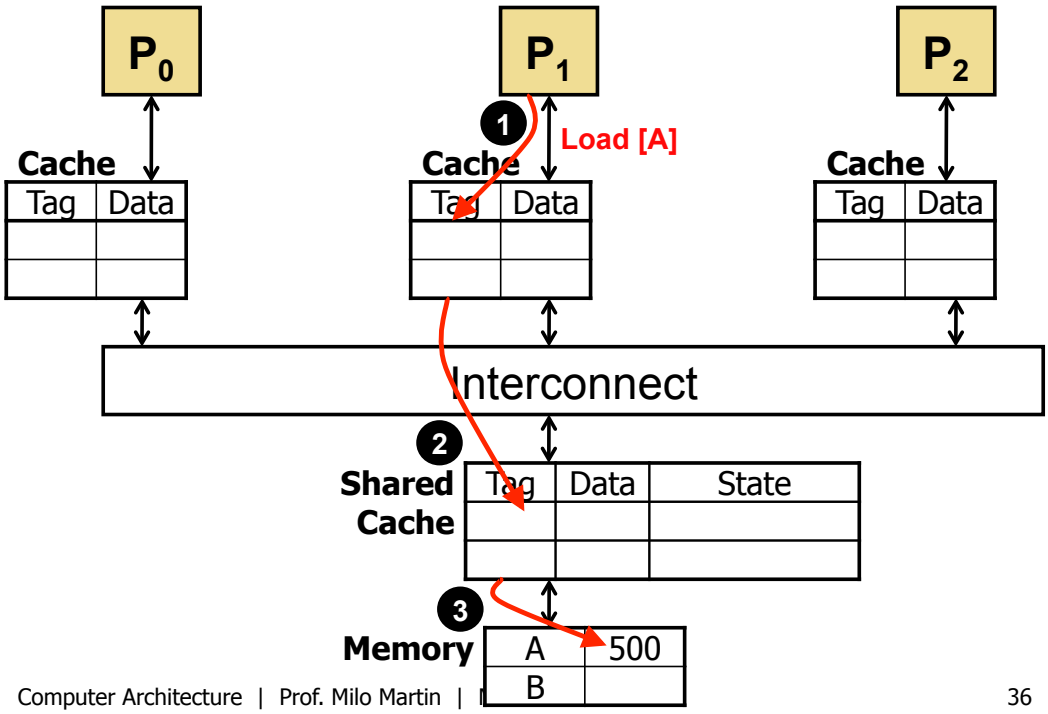
- Mark as "dirty"
- Memory **not** updated

Adding Private Caches

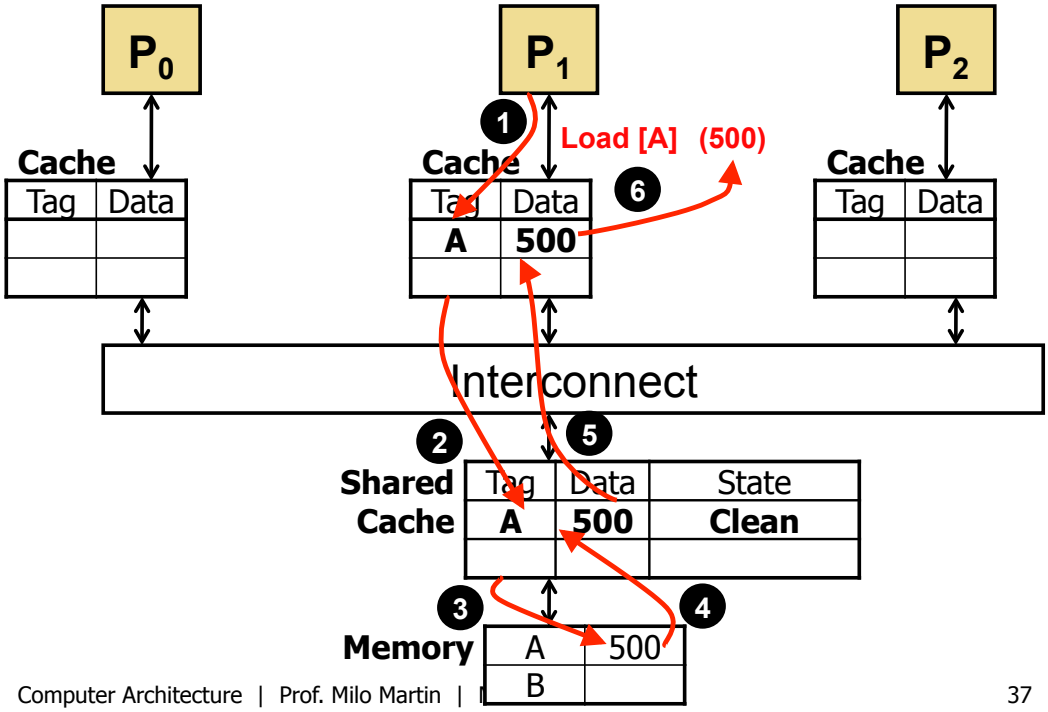


- **Add per-core caches** (write-back caches)
  - Reduces latency
  - Increases throughput
  - Decreases energy

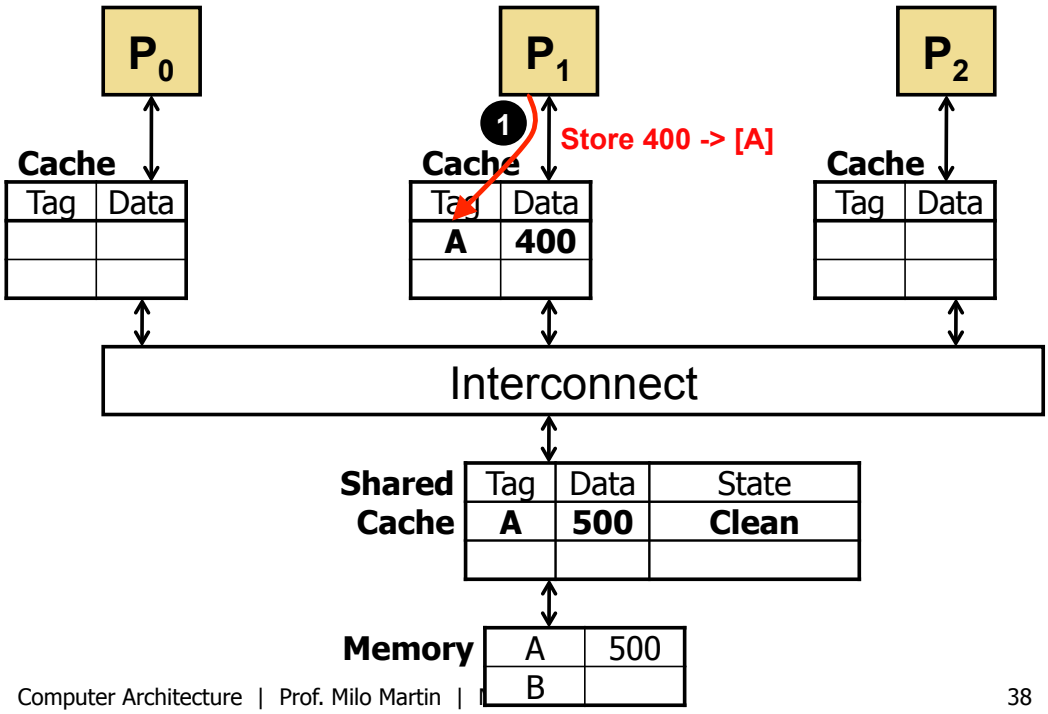
Adding Private Caches



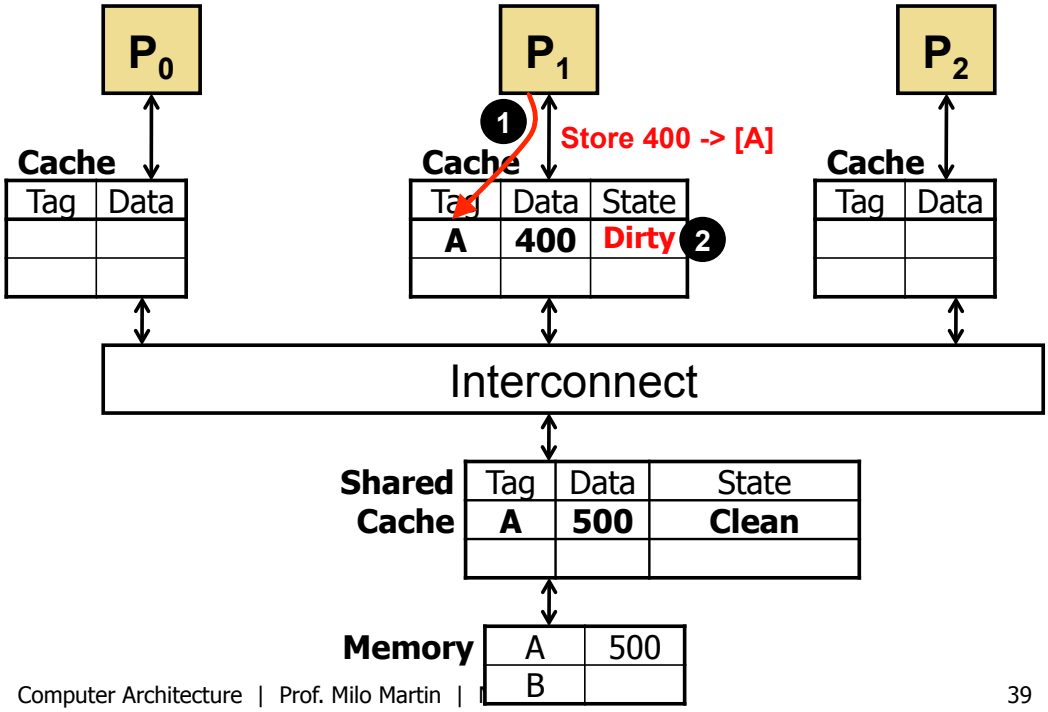
Adding Private Caches



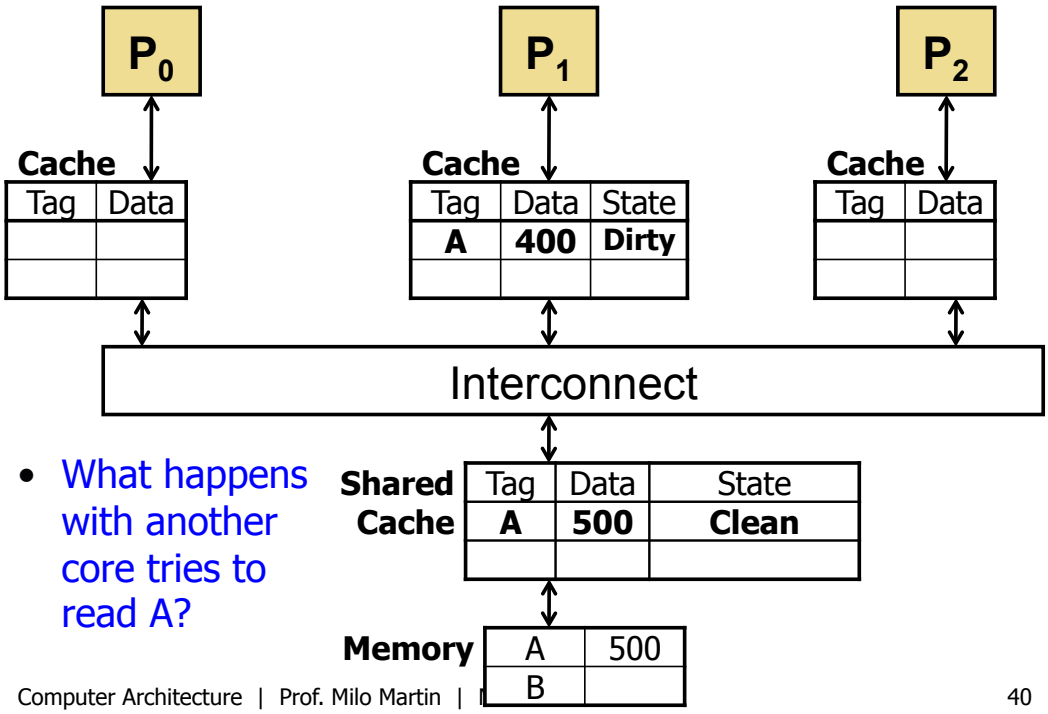
Adding Private Caches



Adding Private Caches

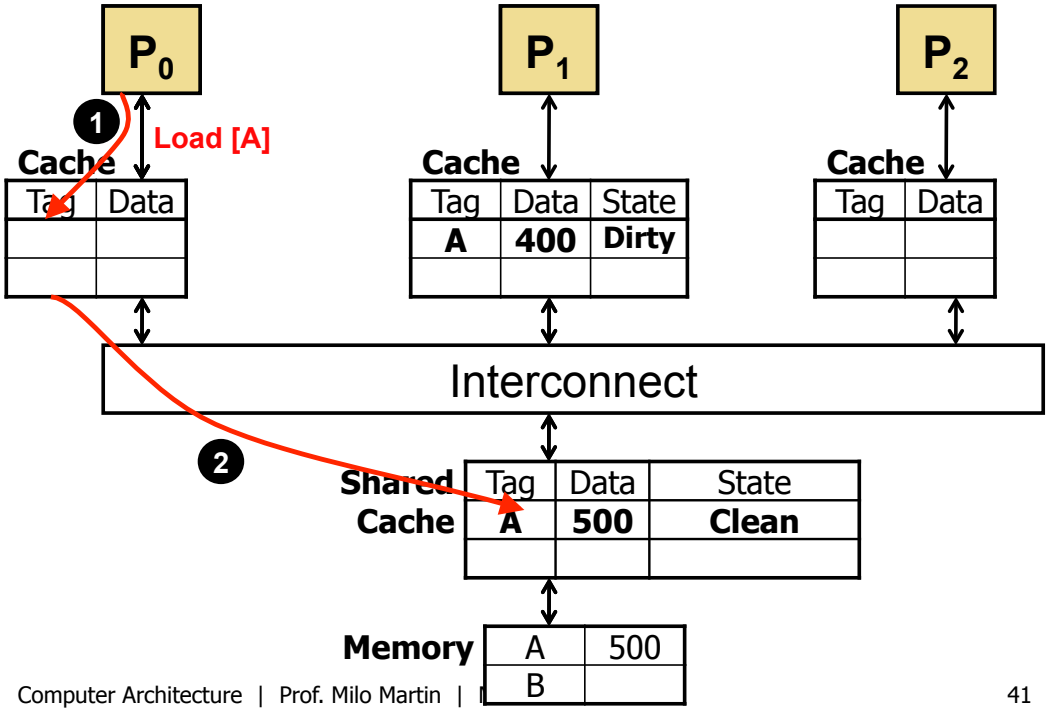


Private Cache Problem: Incoherence

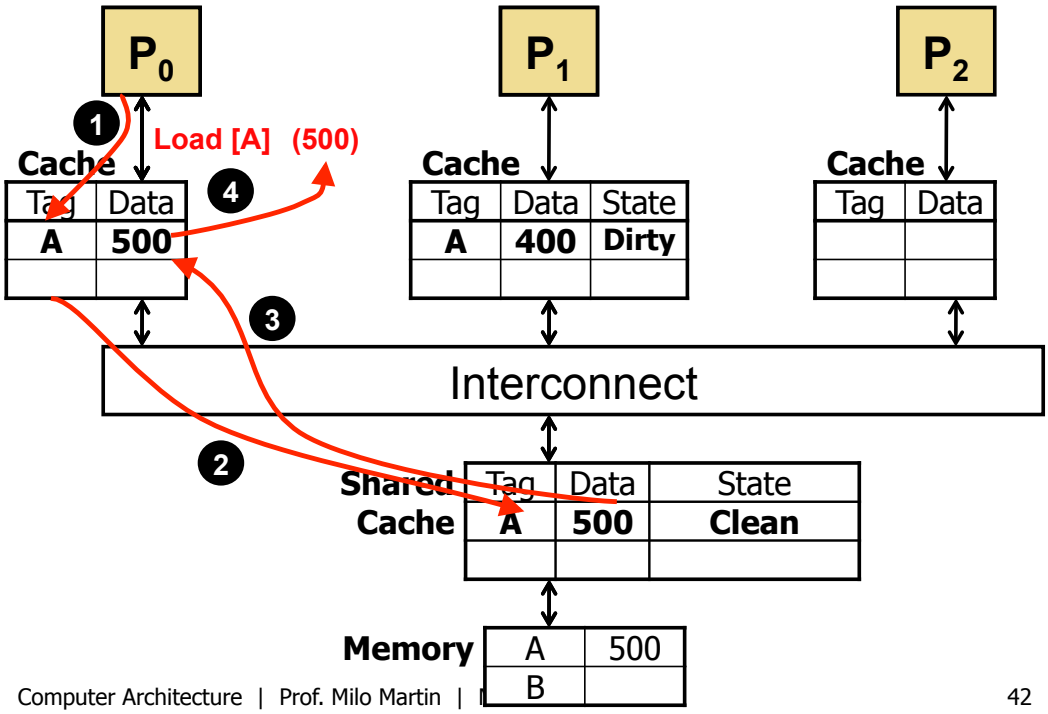


- What happens with another core tries to read A?

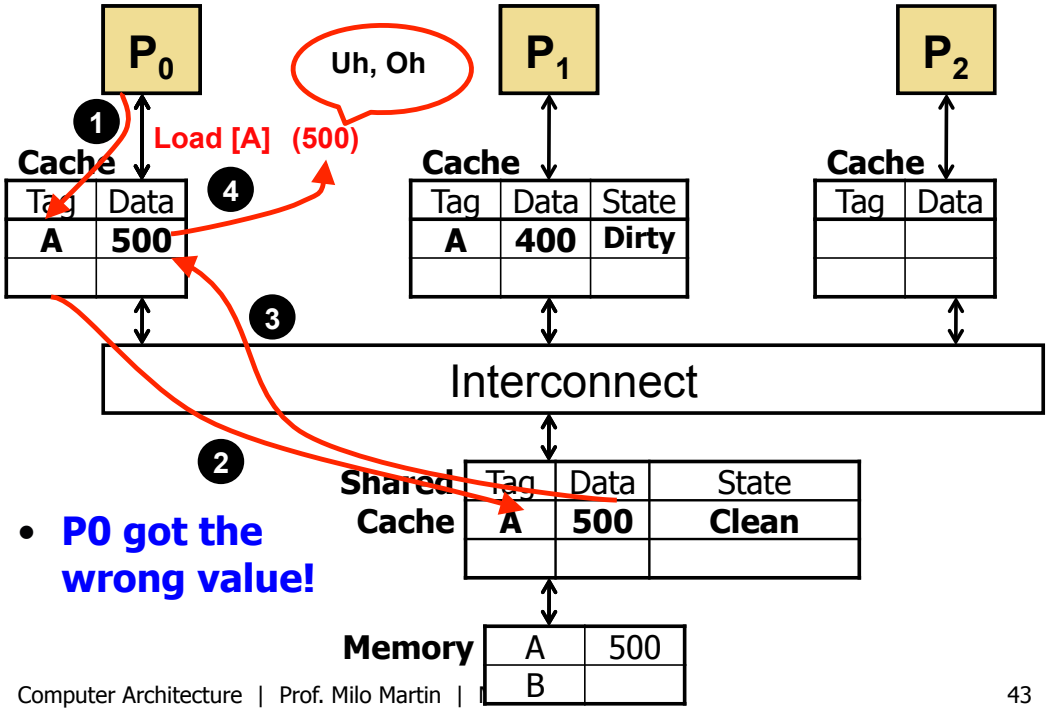
Private Cache Problem: Incoherence



Private Cache Problem: Incoherence

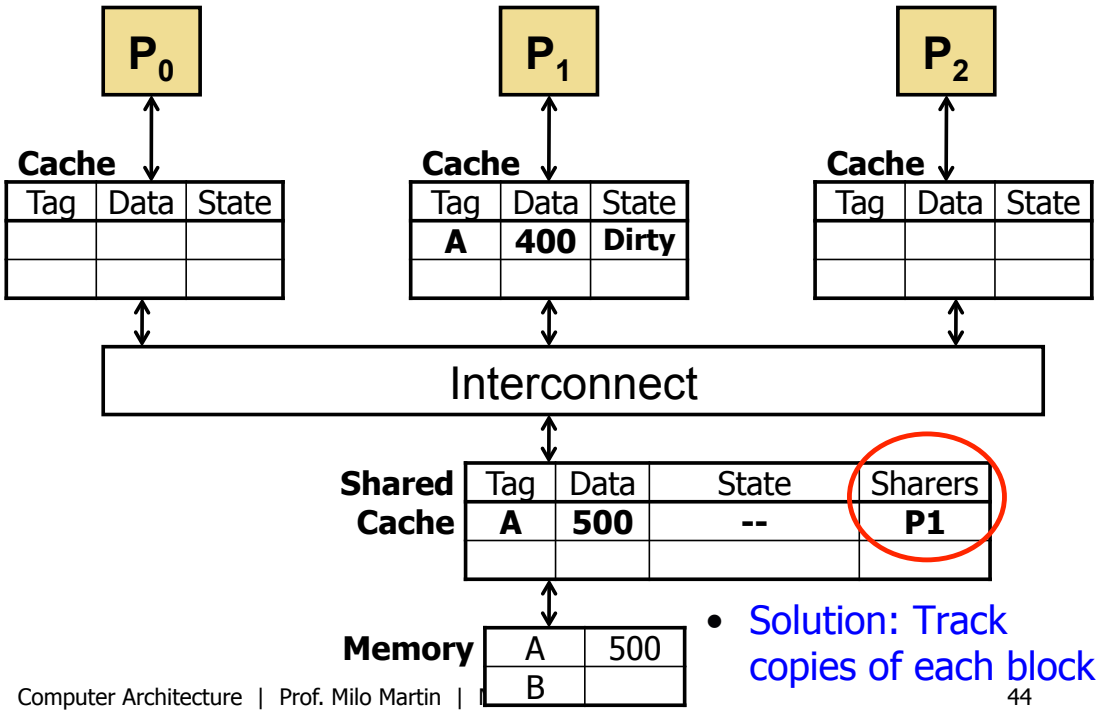


Private Cache Problem: Incoherence



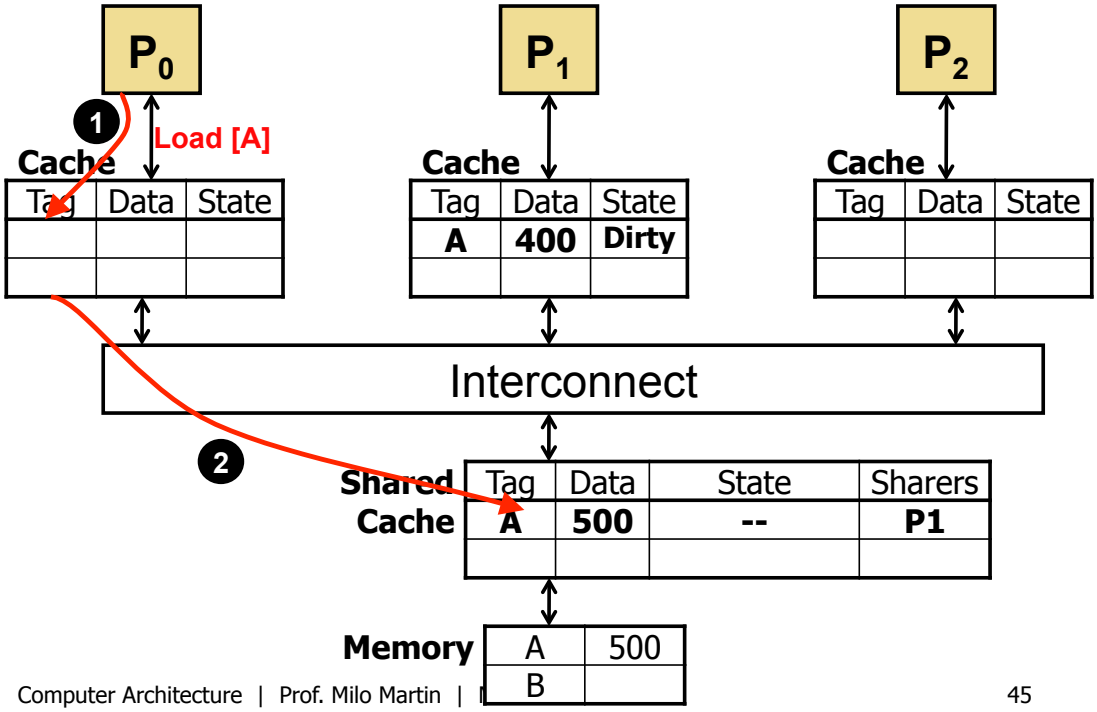
- **P0 got the wrong value!**

Rewind: Fix Problem by Tracking Sharers

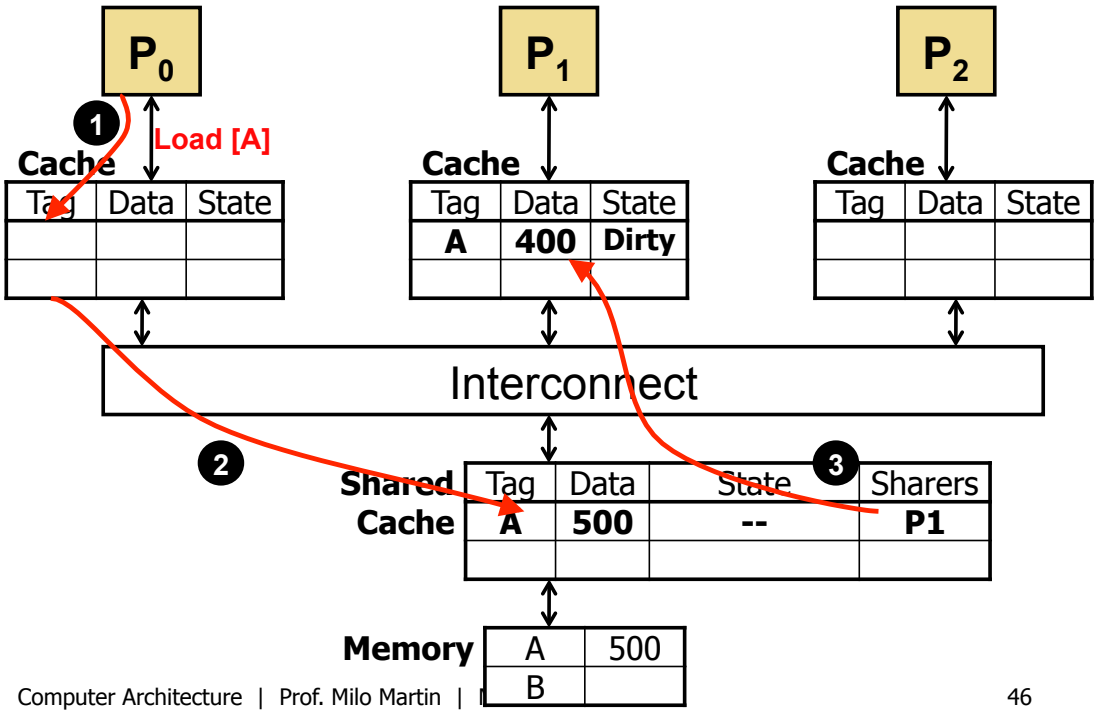


- Solution: Track copies of each block

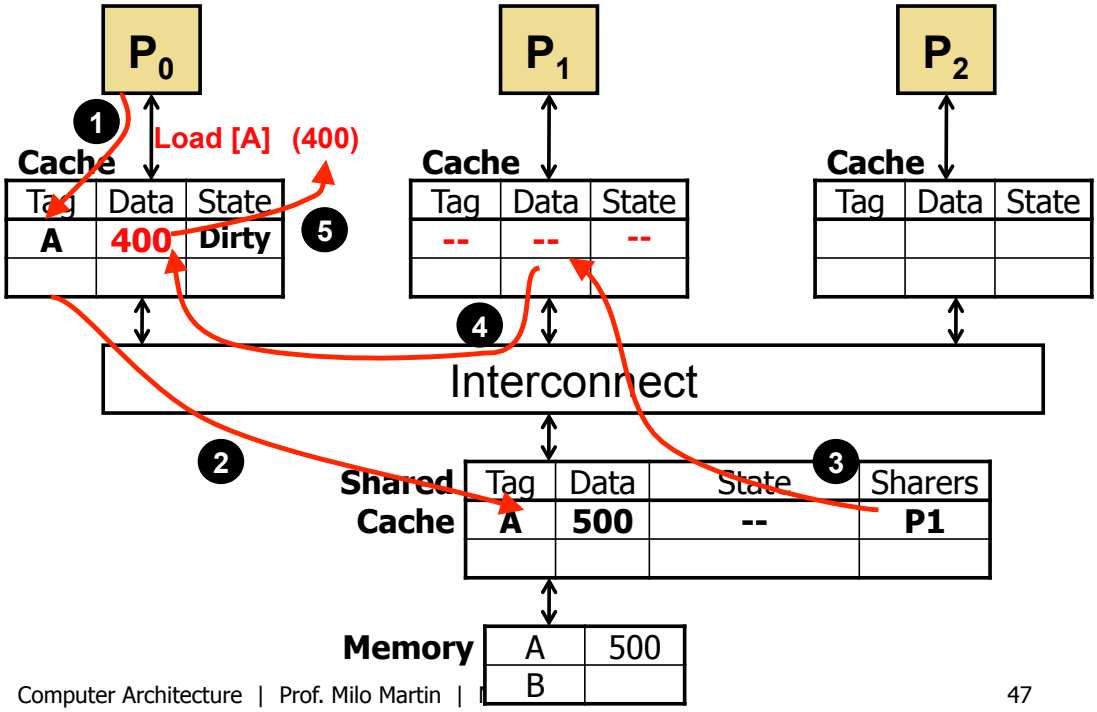
Use Tracking Information to "Invalidate"



Use Tracking Information to "Invalidate"



Use Tracking Information to "Invalidate"



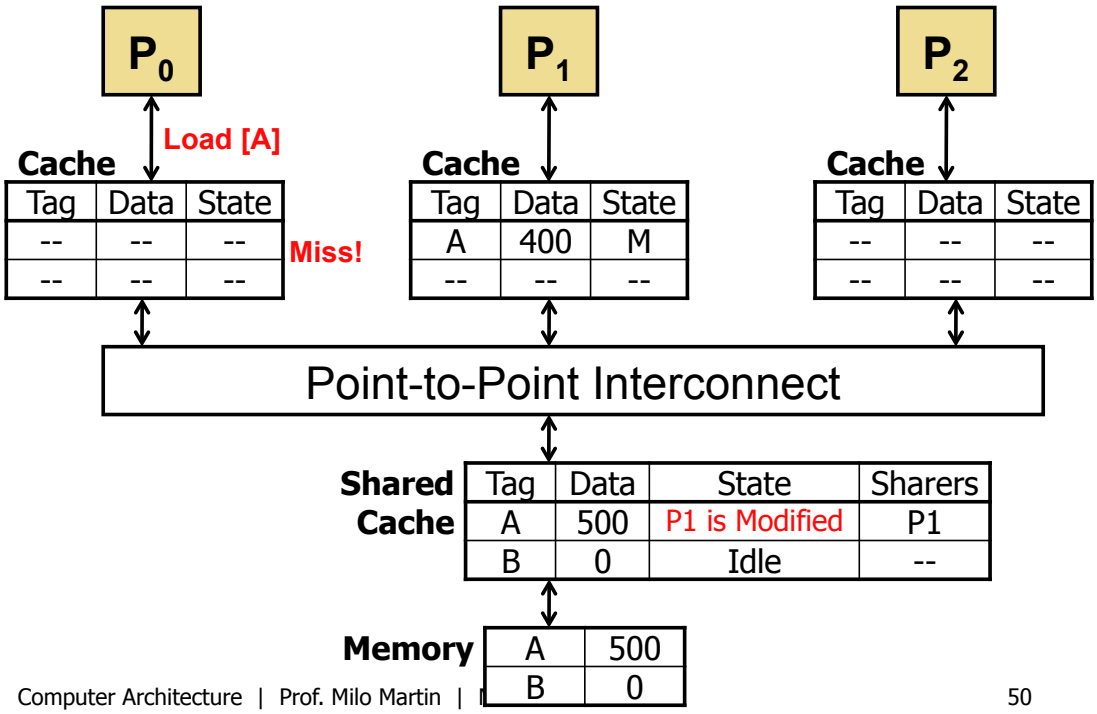
"Valid/Invalid" Cache Coherence

- To enforce the shared memory invariant…
  - "Loads read the value written by the most recent store"
- Enforce the invariant…
  - **"At most one valid copy of the block"**
  - Simplest form is a **two-state "valid/invalid" protocol**
  - If a core wants a copy, must find and "invalidate" it
- On a cache miss, how is the valid copy found?
  - Option #1 "**Snooping**": broadcast to all, whoever has it responds
  - Option #2: "**Directory**": tracker sharers at known location
- **Problem**: multiple copies can't exist, even if read-only
  - Consider mostly-read data structures, instructions, etc.

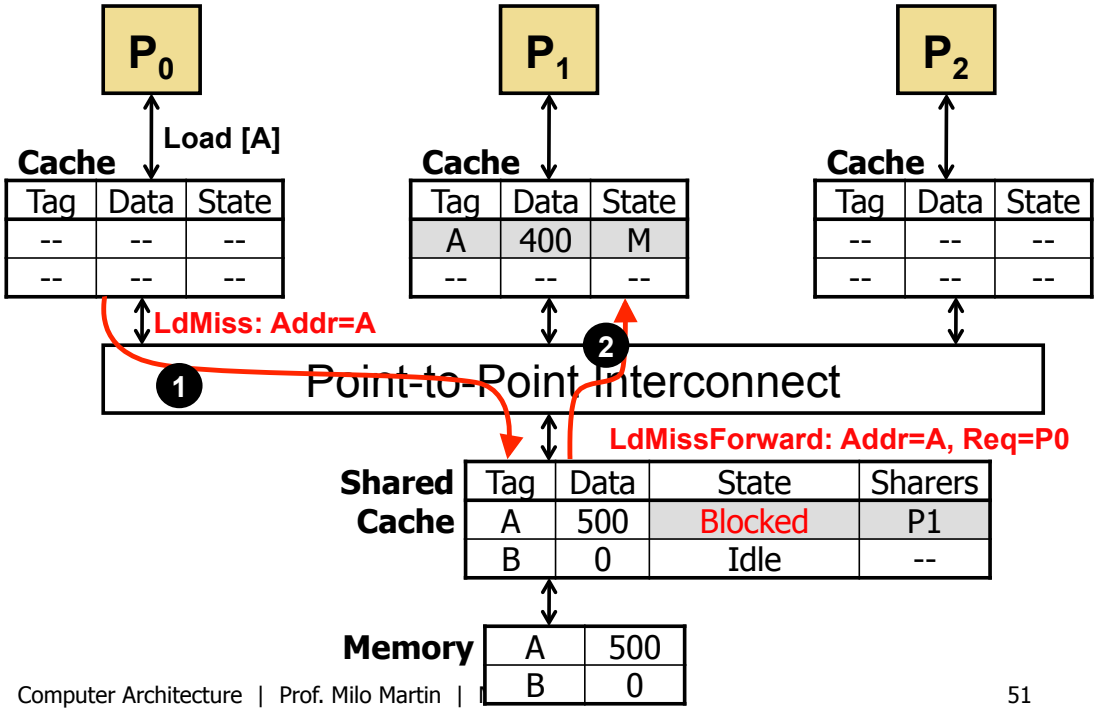
MSI Cache Coherence Protocol

- Solution: enforce the invariant…
  - **Multiple read-only copies —OR—**
  - **Single read/write copy**
- Track these MSI permissions (states) in per-core caches
  - **Modified (M): read/write permission**
  - **Shared (S): read-only permission**
  - **Invalid (I): no permission**
- Also track a **"Sharer" bit vector** in shared cache
  - One bit per core; tracks all shared copies of a block
  - Then, *invalidate all readers* when a write occurs
- Allows for many readers…
  - …while still enforcing shared memory invariant ("Loads read the value written by the most recent store")

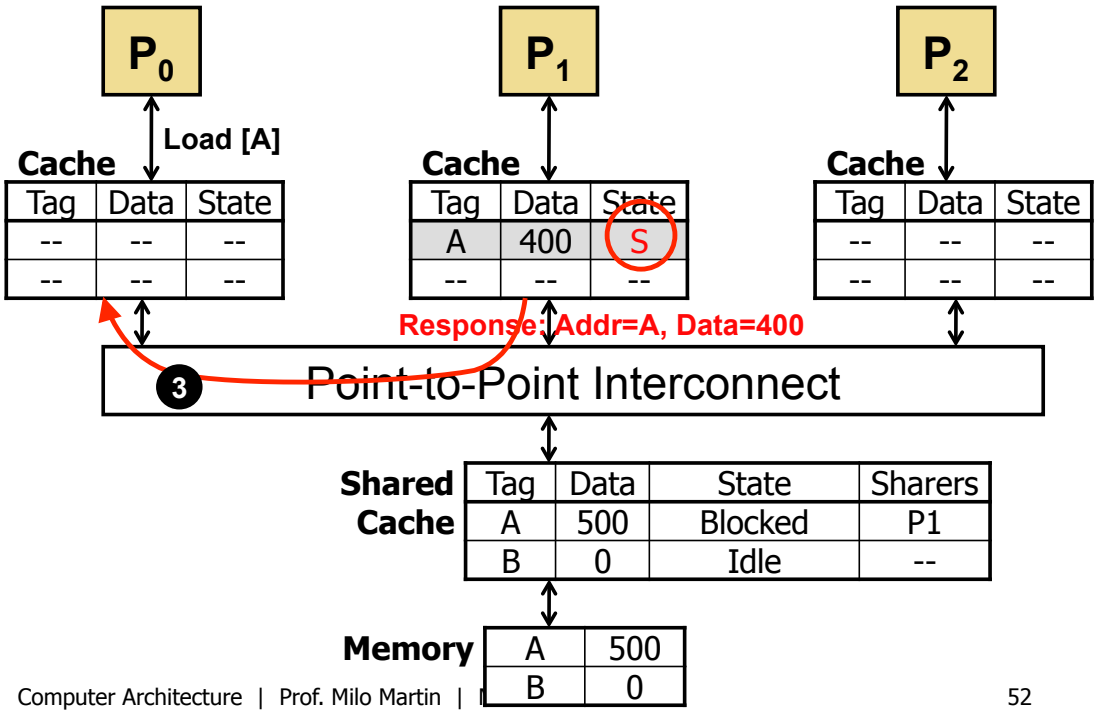
MSI Coherence Example: Step #1



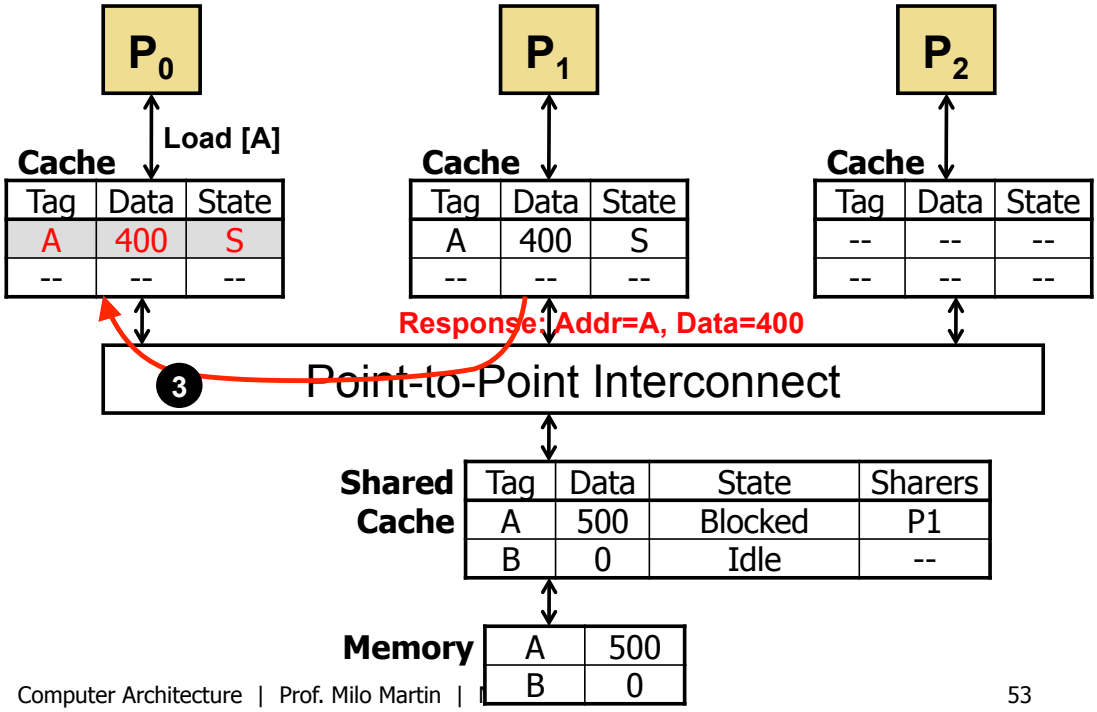
MSI Coherence Example: Step #2



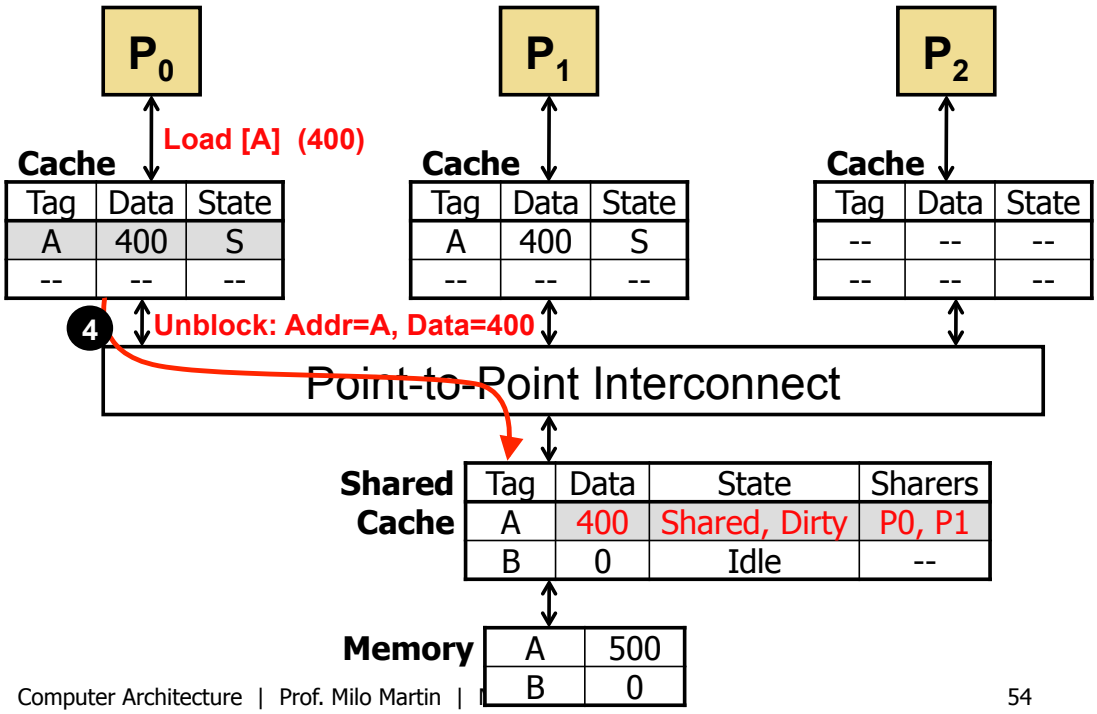
MSI Coherence Example: Step #3



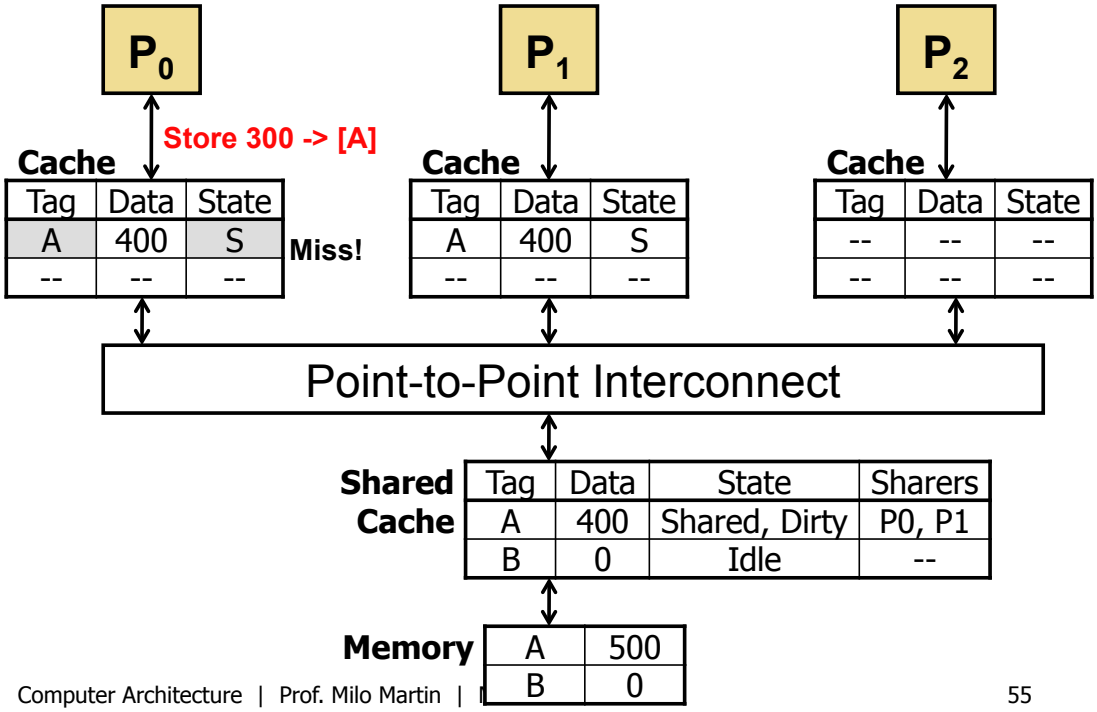
MSI Coherence Example: Step #4



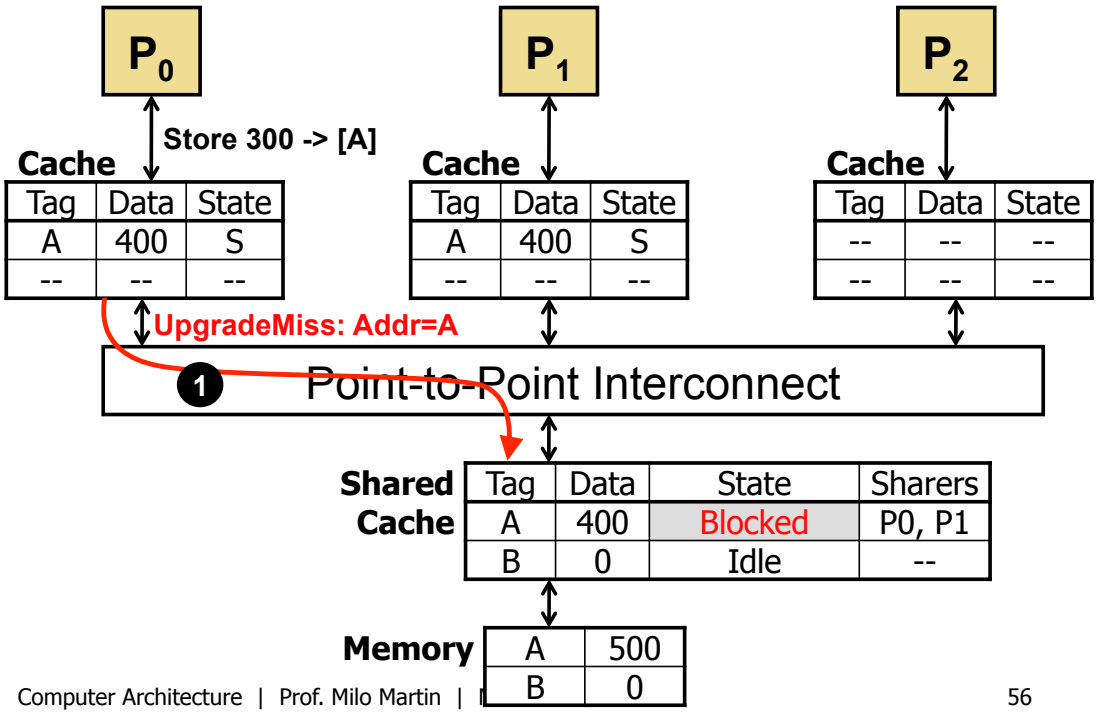
MSI Coherence Example: Step #5



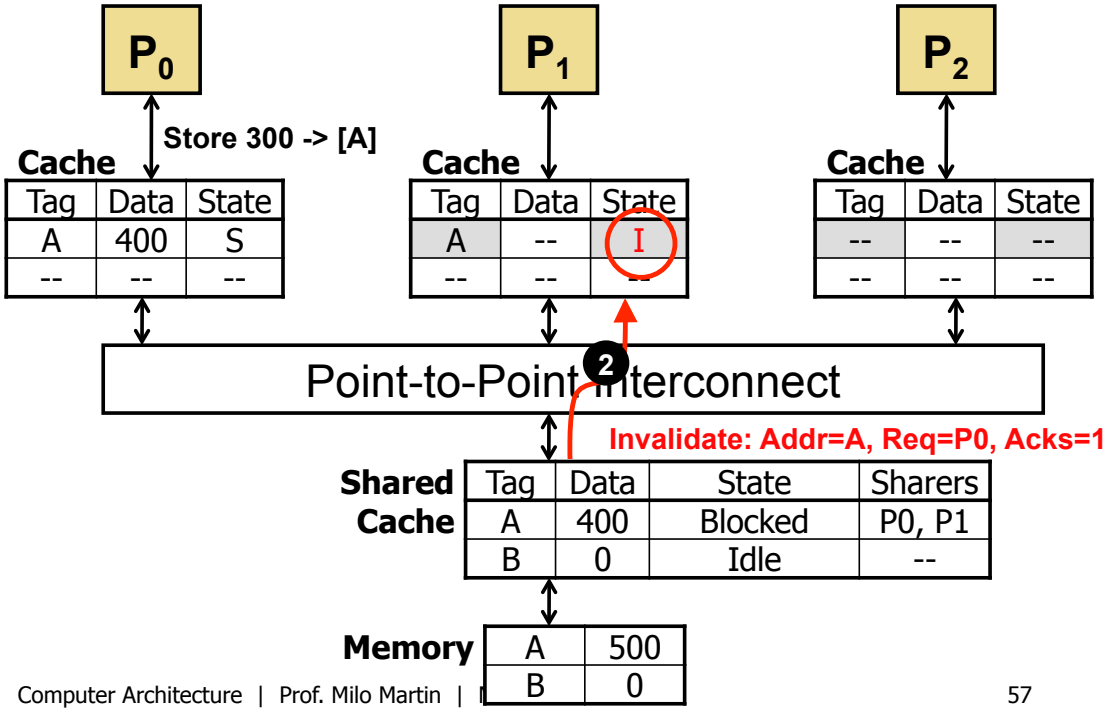
MSI Coherence Example: Step #6



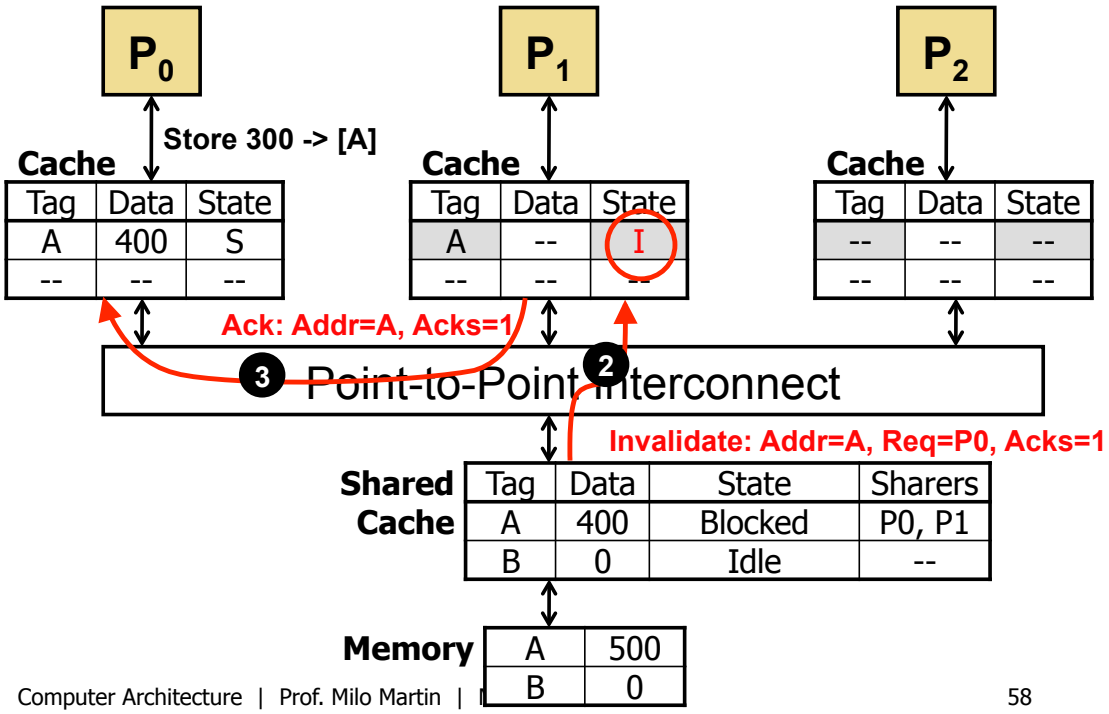
MSI Coherence Example: Step #7



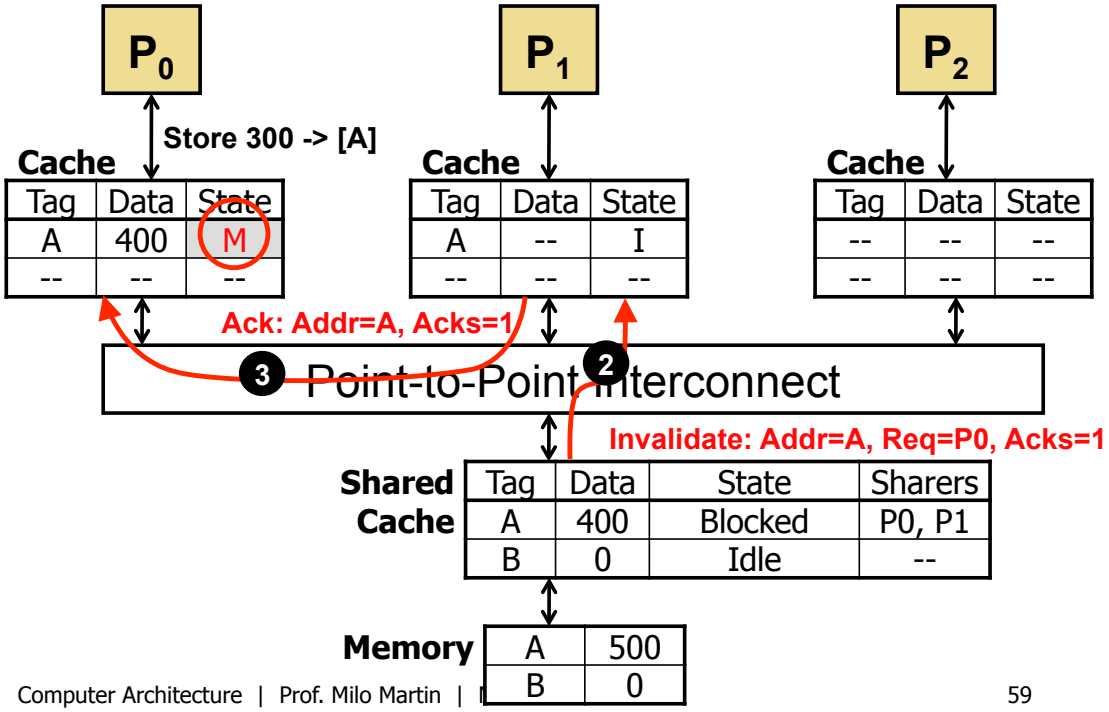
MSI Coherence Example: Step #8



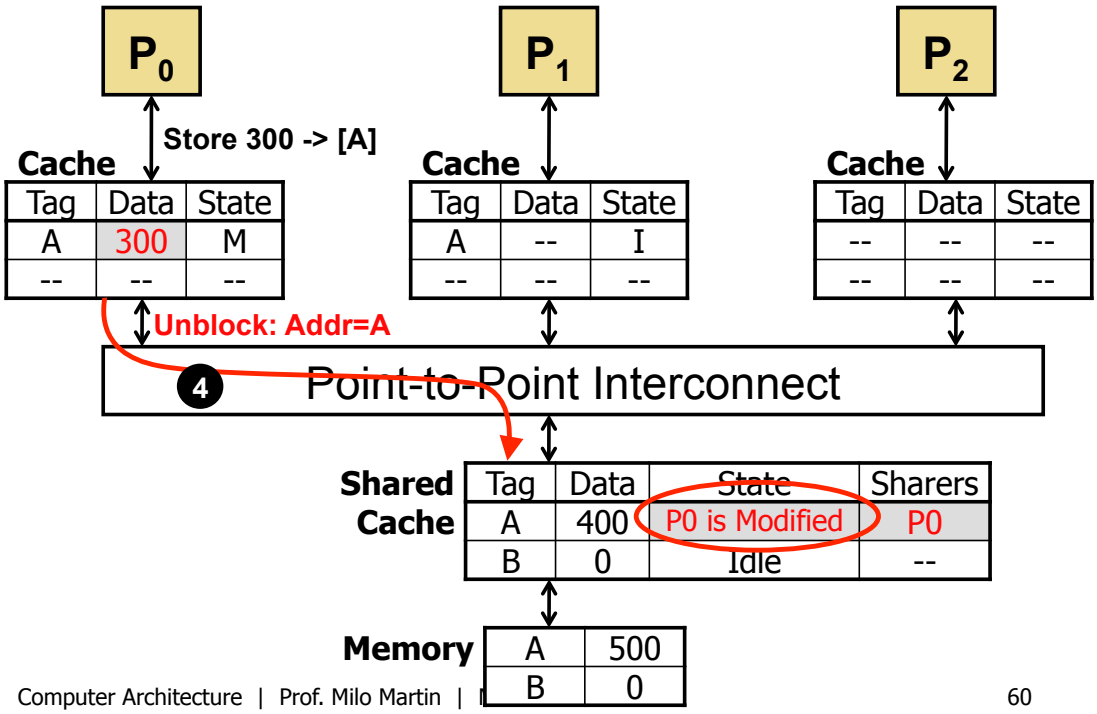
MSI Coherence Example: Step #9



MSI Coherence Example: Step #10



MSI Coherence Example: Step #11



MESI Cache Coherence

- Ok, we have read-only and read/write with MSI

- But consider load & then store of a block by same core
  - Under coherence as described, **this would be two misses: "Load miss" plus an "upgrade miss"…**
  - … even if the block isn't shared!
  - Consider programs with 99% (or 100%) private data
    - Potentially doubling number of misses (bad)

- Solution:
  - Most modern protocols also include **E (exclusive)** state
  - Interpretation: "I have the only cached copy, and it's a **clean** copy"
    - Has read/write permissions
    - Just like "Modified" but "clean" instead of "dirty".

MESI Operation

- Goals:
  - Avoid "upgrade" misses for non-shared blocks
  - While not increasing eviction (aka writeback or replacement) traffic
- Two cases on a load miss to a block…
  - **Case #1**: … with no current sharers
    (that is, no sharers in the set of sharers)
    - Grant requester "Exclusive" copy with read/write permission
  - **Case #2**: … with other sharers
    - As before, grant just a "Shared" copy with read-only permission
- A store to a block in "Exclusive" changes it to "Modified"
  - Instantaneously & silently (no latency or traffic)
- On block eviction (aka writeback or replacement)…
  - If "Modified", block is dirty, must be written back to next level
  - If "Exclusive", writing back the data is not necessary
    (but notification may or may not be, depending on the system)

Cache Coherence and Cache Misses

- With the "Exclusive" state…
  - **Coherence has no overhead on misses to non-shared blocks**
  - Just request/response like a normal cache miss
- But, coherence introduces two new kinds of cache misses
  - **Upgrade miss:** stores to read-only blocks
    - Delay to acquire write permission to read-only block
  - **Coherence miss**
    - Miss to a block evicted by another processor's requests
- Making the cache larger…
  - Doesn't reduce these types of misses
  - So, as cache grows large, these sorts of misses dominate
- **False sharing**
  - Two or more processors sharing parts of the same block
  - But *not* the same bytes within that block (no actual sharing)
  - Creates pathological "ping-pong" behavior
  - Careful data placement may help, but is difficult

Cache Coherence Protocols

- Two general types
  - Update-based cache coherence
    - Write through update to all caches
    - Too much traffic; used in the past, not common today
  - **Invalidation-based cache coherence** (examples shown)
- Of invalidation-based cache coherence, two types:
  - Snooping/broadcast-based cache coherence
    - No explicit state, but too much traffic; not common today
  - **Directory-based cache coherence** (examples shown)
    - Track sharers of blocks
- For directory-based cache coherence, two options:
  - Enforce "inclusion"; if in per-core cache, must be in last-level cache
    - **Encoding sharers in cache tags** (examples shown & Core i7)
  - No inclusion? "directory cache" parallel to last-level cache (AMD)

Scaling Cache Coherence

- **Scalable interconnect**
  - Build switched interconnect to communicate among cores
- **Scalable directory lookup bandwidth**
  - Address interleave (or "bank") the last-level cache
  - Low-order bits of block address select which cache bank to access
  - Coherence controller per bank
- **Scalable traffic**
  - Amortized analysis shows traffic overhead independent of core #
  - Each invalidation can be tied back to some earlier request
- **Scalable storage**
  - Bit vector requires n-bits for n cores, scales up to maybe 32 cores
  - Inexact & "coarse" encodings trade more traffic for less storage
- Hierarchical design can help all of the above, too
- See: "Why On-Chip Cache Coherence is Here to Stay", CACM, 2012

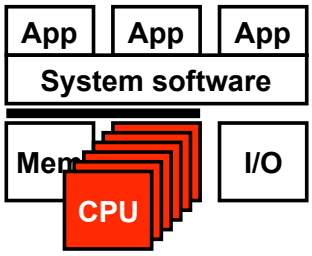
Coherence Recap & Alternatives

- Keeps caches "coherent"
  - Load returns the most recent stored value by any processor
  - And thus keeps caches transparent to software

- Alternatives to cache coherence
  - #1: no caching of shared data (slow)
  - #2: requiring software to explicitly "flush" data (hard to use)
    - Using some new instructions
  - #3: message passing (programming without shared memory)
    - Used in clusters of machines for high-performance computing

- However, directory-based coherence protocol scales well
  - Perhaps to 1000s of cores

Roadmap Checkpoint



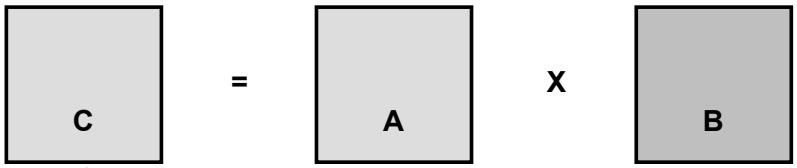
- ~~Thread-level parallelism (TLP)~~
- ~~Shared memory model~~
  - ~~Multiplexed uniprocessor~~
  - ~~Hardware multihreading~~
  - ~~Multiprocessing~~
- ~~Cache coherence~~
  - ~~Valid/Invalid, MSI, MESI~~
- **Parallel programming**
- **Synchronization**
  - **Lock implementation**
  - **Locking gotchas**
  - **Transactional memory**
- Memory consistency models

Parallel Programming

Parallel Programming

- One use of multiprocessors: **multiprogramming**
  - Running multiple programs with no interaction between them
  - Works great for a few cores, but what next?
- Or, programmers must **explicitly** express parallelism
  - "Coarse" parallelism beyond what the hardware can extract **implicitly**
  - Even the compiler can't extract it in most cases
- How? Several options:
  1. Call libraries that perform well-known computations in parallel
     - Example: a matrix multiply routine, etc.
  2. Add code annotations ("this loop is parallel"), OpenMP
  3. Parallel "for" loops, task-based parallelism, …
  4. Explicitly spawn "tasks", runtime/OS schedules them on the cores
- Parallel programming: key challenge in multicore revolution

Example #1: Parallelizing Matrix Multiply




```
for (I = 0; I < SIZE; I++)
   for (J = 0; J < SIZE; J++)
      for (K = 0; K < SIZE; K++)
         C[I][J] += A[I][K] * B[K][J];
```

- How to parallelize matrix multiply?
  - Replace outer "for" loop with "**parallel_for**" or OpenMP annotation
  - Supported by many parallel programming environments
- Implementation: give each of N processors loop iterations

```
int start = (SIZE/N) * my_id();   // my_id() from library
for (I = start; I < start + SIZE/N; I++)
  for (J = 0; J < SIZE; J++)
     for (K = 0; K < SIZE; K++)
        C[I][J] += A[I][K] * B[K][J];
```

- Each processor runs copy of loop above
  - No explicit synchronization required (implicit at end of loop)

Example #2: Bank Accounts

- Consider

```
struct acct_t { int balance; … };
struct acct_t accounts[MAX_ACCT];     // current balances

struct trans_t { int id; int amount; };
struct trans_t transactions[MAX_TRANS];  // debit amounts

for (i = 0; i < MAX_TRANS; i++) {
  debit(transactions[i].id, transactions[i].amount);
}

void debit(int id, int amount) {
  if (accounts[id].balance >= amount) {
    accounts[id].balance -= amount;
  }
}
```

- Can we do these "debit" operations in parallel?
  - Does the order matter?

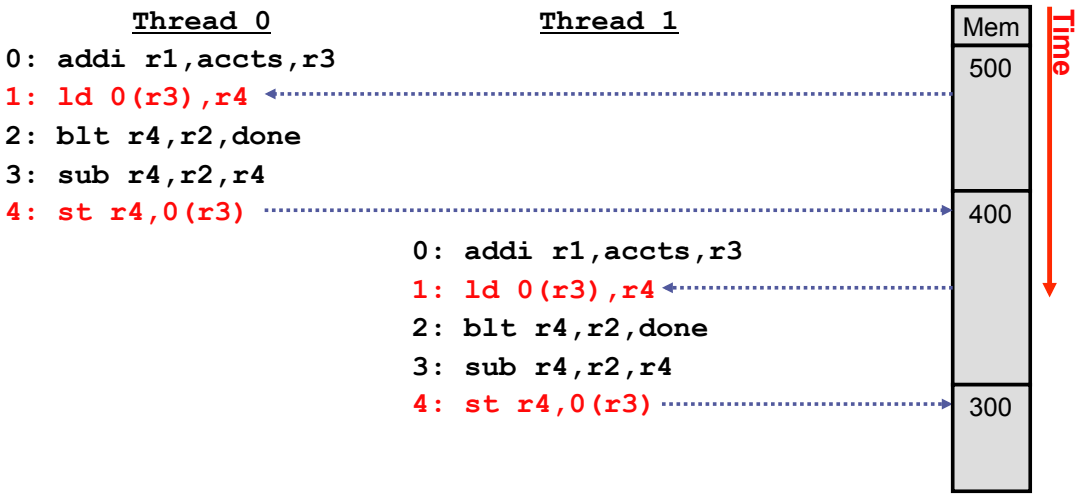
Example #2: Bank Accounts

```
struct acct_t { int bal; … };
shared struct acct_t accts[MAX_ACCT];
void debit(int id, int amt) {
  if (accts[id].bal >= amt)
  {
     accts[id].bal -= amt;
  }
}
```

```
0: addi r1,accts,r3
1: ld 0(r3),r4
2: blt r4,r2,done
3: sub r4,r2,r4
4: st r4,0(r3)
```

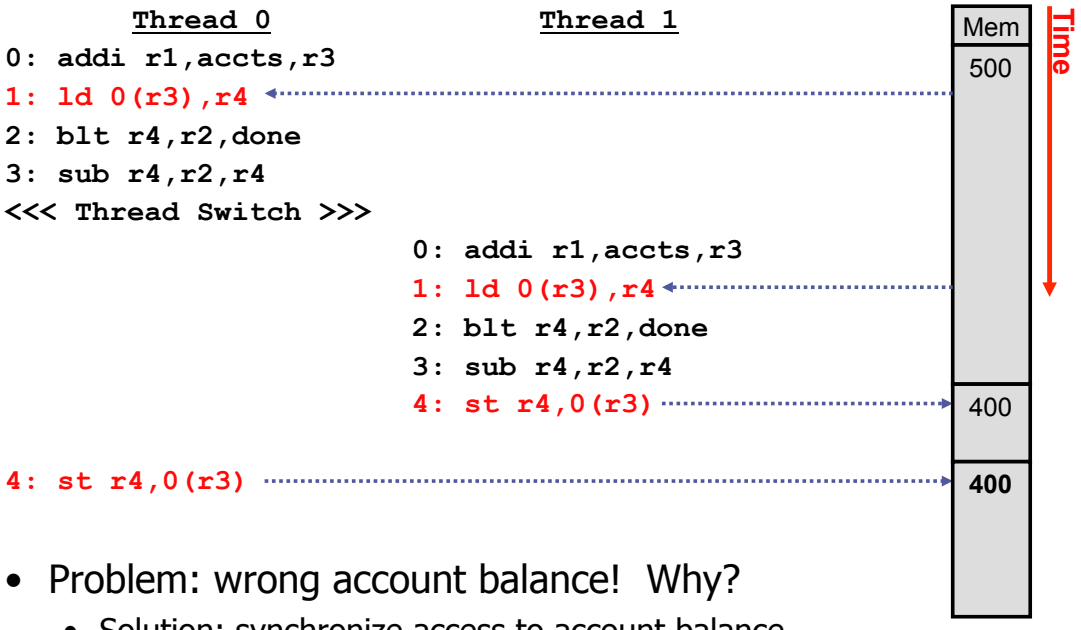
- Example of **Thread-level parallelism (TLP)**
  - Collection of asynchronous tasks: not started and stopped together
  - Data shared "loosely" (sometimes yes, mostly no), dynamically
- Example: database/web server (each query is a thread)
  - `accts` is global and thus **shared**, can't register allocate
  - `id` and `amt` are private variables, register allocated to `r1`, `r2`
- Running example

An Example Execution



- Two $100 withdrawals from account #241 at two ATMs
  - Each transaction executed on different processor
  - Track `accts[241].bal` (address is in `r3`)

A **Problem** Execution



- Problem: wrong account balance! Why?
  - Solution: synchronize access to account balance

Synchronization

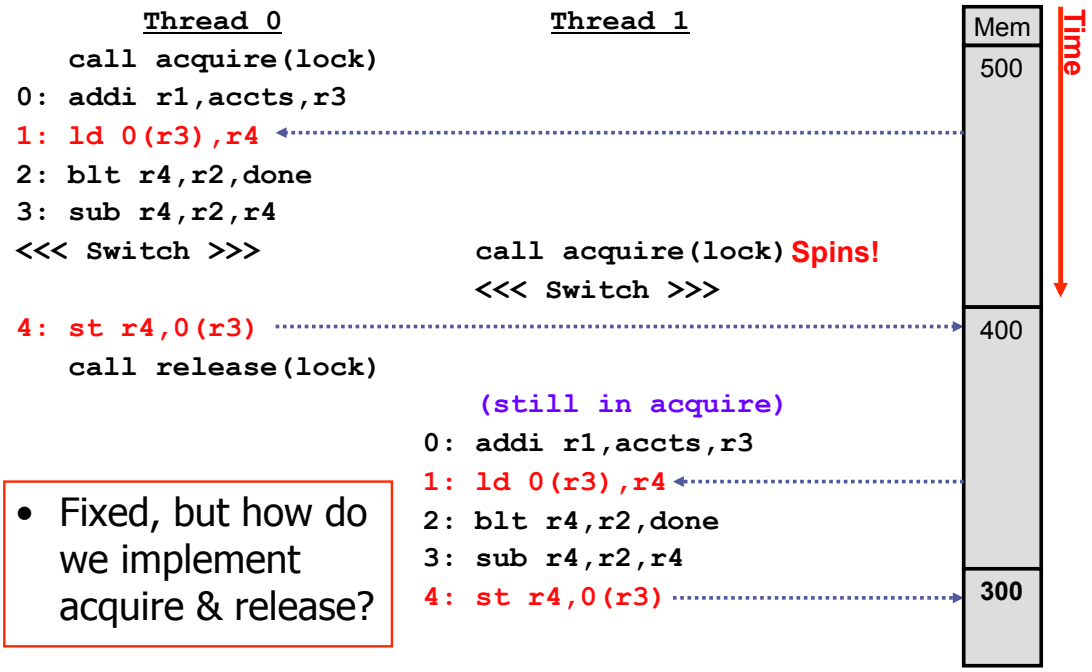
Synchronization:

- **Synchronization**: a key issue for shared memory
- Regulate access to shared data (mutual exclusion)
- Low-level primitive: **lock** (higher-level: "semaphore" or "mutex")
  - Operations: `acquire(lock)` and `release(lock)`
  - Region between `acquire` and `release` is a **critical section**
  - Must interleave `acquire` and `release`
  - Interfering `acquire` will block
- Another option: **Barrier synchronization**
  - Blocks until all threads reach barrier, used at end of "parallel_for"

```
struct acct_t { int bal; … };
shared struct acct_t accts[MAX_ACCT];
shared int lock;
void debit(int id, int amt):
  acquire(lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(lock);
```

critical section

A Synchronized Execution



- Fixed, but how do we implement acquire & release?

Strawman Lock (Incorrect)

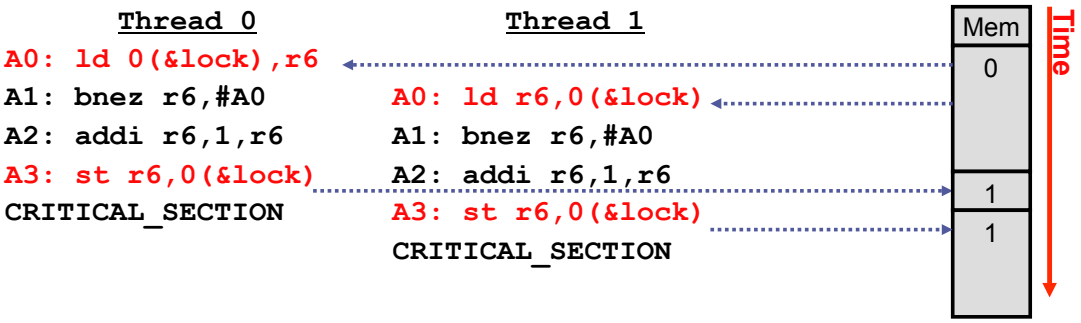
- **Spin lock**: software lock implementation
  - `acquire(lock): while (lock != 0) {} lock = 1;`
    - "Spin" while lock is 1, wait for it to turn 0

```
A0:  ld 0(&lock),r6
A1:  bnez r6,A0
A2:  addi r6,1,r6
A3:  st r6,0(&lock)
```

  - `release(lock): lock = 0;`

```
R0:  st r0,0(&lock)      // r0 holds 0
```

Incorrect Lock Implementation



- Spin lock makes intuitive sense, but doesn't actually work
  - Loads/stores of two `acquire` sequences can be interleaved
  - Lock `acquire` sequence also not atomic
  - **Same problem as before!**
- Note, `release` is trivially atomic

Correct Spin Lock: Use Atomic Swap

- ISA provides an atomic lock acquisition instruction
  - Example: **atomic swap**

    `swap r1,0(&lock)`
    - Atomically executes:

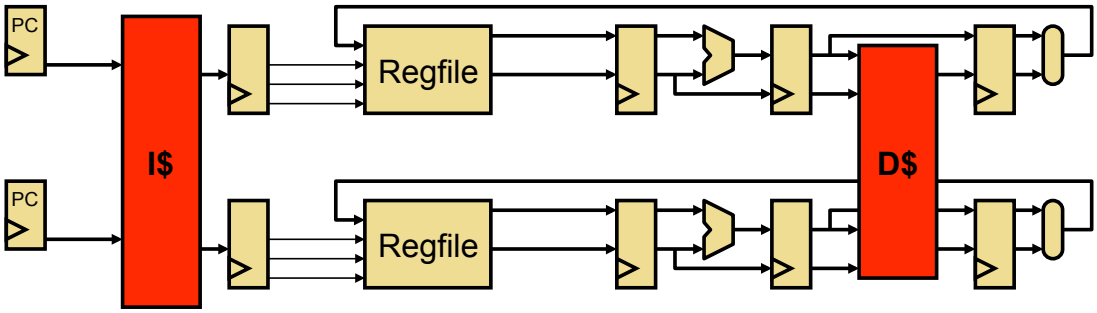
```
mov r1->r2
ld r1,0(&lock)
st r2,0(&lock)
```

- New acquire sequence

```
(value of r1 is 1)
A0: swap r1,0(&lock)
A1: bnez r1,A0
```

  - If lock was initially busy (1), doesn't change it, **keep looping**
  - If lock was initially free (0), acquires it (sets it to 1), break loop
- Insures lock held by **at most one thread**
  - Other variants: **exchange**, **compare-and-swap**, **test-and-set (t&s)**, or **fetch-and-add**

Atomic Update/Swap Implementation



- How is atomic swap implemented?
  - Need to ensure no intervening memory operations
  - Requires blocking access by other threads temporarily (yuck)

- How to pipeline it?
  - Both a load and a store (yuck)
  - Not very RISC-like

RISC Test-And-Set

- **`swap`**: a load and store in one insn is not very "RISC"
  - Broken up into micro-ops, but then how is it made atomic?
- "Load-link" / "store-conditional" pairs
  - Atomic load/store pair

```
label:
  load-link r1,0(&lock)
  // potentially other insns
  store-conditional r2,0(&lock)
  branch-not-zero label   // check for failure
```

  - On `load-link`, processor remembers address…
    - …And looks for writes by other processors
    - If write is detected, next `store-conditional` will fail
      - Sets failure condition
- Used by ARM, PowerPC, MIPS, Itanium

Lock Correctness

```
      Thread 0              Thread 1
A0: swap r1,0(&lock)
A1: bnez r1,#A0       A0: swap r1,0(&lock)
CRITICAL_SECTION      A1: bnez r1,#A0
                      A0: swap r1,0(&lock)
                      A1: bnez r1,#A0
```

+ Lock actually works…
  - Thread 1 keeps spinning

- Sometimes called a "test-and-set lock"
  - Named after the common "test-and-set" atomic instruction

"Test-and-Set" Lock Performance

```
Thread 0              Thread 1
A0: swap r1,0(&lock)
A1: bnez r1,#A0       A0: swap r1,0(&lock)
A0: swap r1,0(&lock)  A1: bnez r1,#A0
A1: bnez r1,#A0       A0: swap r1,0(&lock)
                      A1: bnez r1,#A0
```

– …but performs poorly
  - Consider 3 processors rather than 2
  - Processor 2 (not shown) has the lock and is in the critical section
  - But what are processors 0 and 1 doing in the meantime?
    - Loops of `swap`, each of which includes a `st`
    – Repeated stores by multiple processors costly
    – Generating a ton of useless interconnect traffic

Test-and-Test-and-Set Locks

- Solution: **test-and-test-and-set locks**
  - New acquire sequence

```
A0: ld r1,0(&lock)
A1: bnez r1,A0
A2: addi r1,1,r1
A3: swap r1,0(&lock)
A4: bnez r1,A0
```

  - Within each loop iteration, before doing a **`swap`**
    - Spin doing a simple test (**`ld`**) to see if lock value has changed
    - Only do a **`swap`** (**`st`**) if lock is actually free
  - Processors can spin on a busy lock locally (in their own cache)
    + Less unnecessary interconnect traffic
  - Note: test-and-test-and-set is not a new instruction!
    - Just different software

Queue Locks

- Test-and-test-and-set locks can still perform poorly
  - If lock is contended for by many processors
  - Lock release by one processor, creates "free-for-all" by others
  – Interconnect gets swamped with **`swap`** requests
- **Software queue lock**
  - Each waiting processor spins on a different location (a queue)
  - When lock is released by one processor...
    - Only the next processors sees its location go "unlocked"
    - Others continue spinning locally, unaware lock was released
  - Effectively, passes lock from one processor to the next, in order
  + Greatly reduced network traffic (no mad rush for the lock)
  + Fairness (lock acquired in FIFO order)
  – Higher overhead in case of no contention (more instructions)
  – Poor performance if one thread is descheduled by O.S.

Programming With Locks Is Tricky

- Multicore processors are the way of the foreseeable future
  - thread-level parallelism anointed as parallelism model of choice
  - Just one problem…
- Writing lock-based multi-threaded programs is tricky!
- More precisely:
  - Writing programs that are correct is "easy" (not really)
  - Writing programs that are highly parallel is "easy" (not really)
  - – **Writing programs that are both correct and parallel is difficult**
    - And that's the whole point, unfortunately
  - Selecting the "right" kind of lock for performance
    - Spin lock, queue lock, ticket lock, read/writer lock, etc.
  - **Locking granularity issues**

Coarse-Grain Locks: Correct but Slow

- **Coarse-grain locks**: e.g., one lock for entire database
  - \+ Easy to make correct: no chance for unintended interference
  - – Limits parallelism: no two critical sections can proceed in parallel

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
shared Lock_t lock;
void debit(int id, int amt) {
  acquire(lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(lock);
}
```

Fine-Grain Locks: Parallel But Difficult

- **Fine-grain locks**: e.g., multiple locks, one per record
  - + Fast: critical sections (to different records) can proceed in parallel
  - – Difficult to make correct: easy to make mistakes
    - This particular example is easy
      - Requires only one lock per critical section

```
struct acct_t { int bal, Lock_t lock; …  };
shared struct acct_t  accts[MAX_ACCT];

void debit(int id, int amt) {
  acquire(accts[id].lock);
  if (accts[id].bal >= amt) {
     accts[id].bal -= amt;
  }
  release(accts[id].lock);
}
```

- What about critical sections that require two locks?

Multiple Locks

- **Multiple locks**: e.g., acct-to-acct transfer
  - Must acquire both `id_from`, `id_to` locks
  - Running example with accts 241 and 37
  - Simultaneous transfers 241 → 37 and 37 → 241
  - Contrived… but even contrived examples must work correctly too

```
struct acct_t { int bal, Lock_t lock; …};
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  acquire(accts[id_from].lock);
  acquire(accts[id_to].lock);
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  release(accts[id_to].lock);
  release(accts[id_from].lock);
}
```

Multiple Locks And Deadlock

Thread 0

```
id_from = 241;
id_to = 37;

acquire(accts[241].lock);
// wait to acquire lock 37
// waiting…
// still waiting…
```

Thread 1

```
id_from = 37;
id_to = 241;

acquire(accts[37].lock);
// wait to acquire lock 241
// waiting…
// …
```

Multiple Locks And Deadlock

```
Thread 0                          Thread 1

id_from = 241;                    id_from = 37;
id_to = 37;                       id_to = 241;

acquire(accts[241].lock);         acquire(accts[37].lock);
// wait to acquire lock 37        // wait to acquire lock 241
// waiting…                       // waiting…
// still waiting…                 // …
```

- **Deadlock**: circular wait for shared resources
  - Thread 0 has lock 241 waits for lock 37
  - Thread 1 has lock 37 waits for lock 241
  - Obviously this is a problem
  - The solution is …

Correct Multiple Lock Program

- **Always acquire multiple locks in same order**
  - Just another thing to keep in mind when programming

```
struct acct_t { int bal, Lock_t lock; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  int id_first = min(id_from, id_to);
  int id_second = max(id_from, id_to);

  acquire(accts[id_first].lock);
  acquire(accts[id_second].lock);
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  release(accts[id_second].lock);
  release(accts[id_first].lock);
}
```

Correct Multiple Lock Execution

```
Thread 0                          Thread 1

id_from = 241;                    id_from = 37;
id_to = 37;                       id_to = 241;
id_first = min(241,37)=37;        id_first = min(37,241)=37;
id_second = max(37,241)=241;      id_second = max(37,241)=241;

acquire(accts[37].lock);          // wait to acquire lock 37
acquire(accts[241].lock);         // waiting…
// do stuff                       // …
release(accts[241].lock);         // …
release(accts[37].lock);          // …
                                  acquire(accts[37].lock);
```

- Great, are we done? No

More Lock Madness

- What if…
  - Some actions (e.g., deposits, transfers) require 1 or 2 locks…
  - …and others (e.g., prepare statements) require all of them?
  - Can these proceed in parallel?
- What if…
  - There are locks for global variables (e.g., operation id counter)?
  - When should operations grab this lock?
- What if… what if… what if…

- **So lock-based programming is difficult…**
- **…wait, it gets worse**

And To Make It Worse…

- **Acquiring locks is expensive…**
  - By definition requires a slow atomic instructions
    - Specifically, acquiring write permissions to the lock
  - Ordering constraints (see soon) make it even slower
- **…and 99% of the time un-necessary**
  - Most concurrent actions don't actually share data
  - – You paying to acquire the lock(s) for no reason
- Fixing these problem is an area of active research
  - One proposed solution "Transactional Memory"
  - Programmer uses construct: "atomic { … code … }"
    - Hardware, compiler & runtime executes the code "atomically"
    - Uses speculation, rolls back on conflicting accesses

Research: Transactional Memory (TM)

- **Transactional Memory (TM) goals:**
  - \+ Programming simplicity of coarse-grain locks
  - \+ Higher concurrency (parallelism) of fine-grain locks
    - Critical sections only serialized if data is actually shared
  - \+ Lower overhead than lock acquisition
  - Hot academic & industrial research topic (or was a few years ago)
  - No fewer than nine research projects:
    - Brown, Stanford, MIT, Wisconsin, Texas, Rochester, Sun/Oracle, Intel
    - Penn, too
- Most recently:
  - Intel announced TM support in "Haswell" core! (shipping in 2013)

Transactional Memory: The Big Idea

- Big idea I: **no locks, just shared data**

- Big idea II: **optimistic (speculative) concurrency**
  - Execute critical section speculatively, abort on conflicts
  - "Better to beg for forgiveness than to ask for permission"

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  begin_transaction();
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  end_transaction();
}
```


Transactional Memory: Read/Write Sets

- **Read set**: set of shared addresses critical section reads
  - Example: `accts[37].bal`, `accts[241].bal`
- **Write set**: set of shared addresses critical section writes
  - Example: `accts[37].bal`, `accts[241].bal`

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  begin_transaction();
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  end_transaction();
}
```

Transactional Memory: Begin

- **begin_transaction**
  - Take a local register checkpoint
  - Begin locally tracking read set (remember addresses you read)
    - See if anyone else is trying to write it
  - Locally buffer all of your writes (invisible to other processors)
  - \+ **Local actions only: no lock acquire**

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  begin_transaction();
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  end_transaction();
}
```


Transactional Memory: End

- **end_transaction**
  - Check read set: is all data you read still valid (i.e., no writes to any)
  - Yes? Commit transactions: commit writes
  - No? Abort transaction: restore checkpoint

```
struct acct_t { int bal; … };
shared struct acct_t  accts[MAX_ACCT];
void transfer(int id_from, int id_to, int amt) {
  begin_transaction();
  if (accts[id_from].bal >= amt) {
     accts[id_from].bal -= amt;
     accts[id_to].bal += amt;
  }
  end_transaction();
}
```

Transactional Memory Implementation

- How are read-set/write-set implemented?
  - Track locations accessed using bits in the cache
- Read-set: additional "transactional read" bit per block
  - Set on reads between begin_transaction and end_transaction
  - Any other write to block with set bit ➔ triggers abort
  - Flash cleared on transaction abort or commit
- Write-set: additional "transactional write" bit per block
  - Set on writes between begin_transaction and end_transaction
  - Before first write, if dirty, initiate writeback ("clean" the block)
  - Flash cleared on transaction commit
  - To abort transaction: invalidate all blocks with bit set

Transactional Execution

Thread 0

```
id_from = 241;
id_to = 37;

begin_transaction();
if(accts[241].bal > 100) {
   …
   // write accts[241].bal
   // abort
```

Thread 1

```
id_from = 37;
id_to = 241;

begin_transaction();
if(accts[37].bal > 100) {
   accts[37].bal -= amt;
   acts[241].bal += amt;
}
end_transaction();
// no writes to accts[241].bal
// no writes to accts[37].bal
// commit
```

Transactional Execution II (More Likely)

```
Thread 0                              Thread 1

id_from = 241;                        id_from = 450;
id_to = 37;                           id_to = 118;

begin_transaction();                  begin_transaction();
if(accts[241].bal > 100) {            if(accts[450].bal > 100) {
   accts[241].bal -= amt;                accts[450].bal -= amt;
   acts[37].bal += amt;                  acts[118].bal += amt;
}                                     }
end_transaction();                    end_transaction();
// no write to accts[240].bal         // no write to accts[450].bal
// no write to accts[37].bal          // no write to accts[118].bal
// commit                             // commit
```

- Critical sections execute in parallel

So, Let's Just Do Transactions?

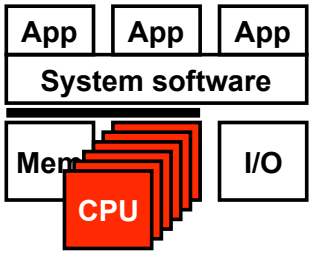
- What if…
  - Read-set or write-set bigger than cache?
  - Transaction gets swapped out in the middle?
  - Transaction wants to do I/O or SYSCALL (not-abortable)?
- How do we transactify existing lock based programs?
  - Replace `acquire` with `begin_trans` does not always work
- Several different kinds of transaction semantics
  - Are transactions atomic relative to code outside of transactions?
- Do we want transactions in hardware or in software?
  - What we just saw is **hardware transactional memory (HTM)**
- That's what these research groups are looking at
  - Best-effort hardware TM: Azul systems, Sun's Rock processor

Speculative Lock Elision (SLE)

```
Processor 0
acquire(accts[37].lock); // don't actually set lock to 1
// begin tracking read/write sets
// CRITICAL_SECTION
// check read set
//  no conflicts? Commit, don't actually set lock to 0
//  conflicts? Abort, retry by acquiring lock
release(accts[37].lock);
```

- Alternatively, keep the locks, but…
- … speculatively transactify lock-based programs in hardware
  - **Speculative Lock Elision (SLE)** [Rajwar+, MICRO'01]
    - Captures most of the advantages of transactional memory…
  - + No need to rewrite programs
  - + Can always fall back on lock-based execution (overflow, I/O, etc.)
- Intel's "Haswell" supports both SLE & best-effort TM

Roadmap Checkpoint



- ~~Thread-level parallelism (TLP)~~
- ~~Shared memory model~~
  - ~~Multiplexed uniprocessor~~
  - ~~Hardware multihreading~~
  - ~~Multiprocessing~~
- ~~Cache coherence~~
  - ~~Valid/Invalid, MSI, MESI~~
- ~~Parallel programming~~
- ~~Synchronization~~
  - ~~Lock implementation~~
  - ~~Locking gotchas~~
  - ~~Transactional memory~~
- **Memory consistency models**

Shared Memory Example #1

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

Shared Memory Example #1: "Answer"

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0, y=0)? Nope… or can it?

Shared Memory Example #2

- **Initially: all variables zero** ("flag" is 0, "a" is 0)

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- What value can be read by "load a"?

Shared Memory Example #2: "Answer"

- **Initially: all variables zero** ("flag" is 0, "a" is 0)

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- What value can be read by "load a"?
  - "load a" can see the value "1"
- Can "load a" read the value zero?
  - Are you sure?

What is Going On?

- Reordering of memory operations to different addresses!

- **In the compiler**
  - Compiler is generally allowed to re-order memory operations to different addresses
  - Many other compiler optimizations also cause problems

- **In the hardware**
  1. To tolerate write latency
     - Cores don't wait for writes to complete (via store buffers)
     - And why should they? No reason to wait on non-threaded code
  2. To simplify out-of-order execution

Memory Consistency

- **Memory coherence**
  - Creates globally uniform (consistent) view…
  - Of **a single memory location** (in other words: cache blocks)
  - Not enough
    - Cache blocks A and B can be individually consistent…
    - But inconsistent with respect to each other

- **Memory consistency**
  - Creates globally uniform (consistent) view…
  - Of **all memory locations relative to each other**

- Who cares? Programmers
  - Globally inconsistent memory creates mystifying behavior

Why? To Hide Store Miss Latency

- Why? Why Allow Such Odd Behavior?
  - Reason #1: hiding store miss latency
- Recall (back from caching unit)
  - Hiding store miss latency
  - How? Store buffer
- Said it would complicate multiprocessors
  - Yes. It does.
  - By allowing reordering of store and load (to different addresses)
- Example:

thread 1	thread 2
store 1 → y load x	store 1 → x load y

  - Both stores miss cache, are put in store buffer
  - Loads hit, receive value before store completes, sees "old" value

Shared Memory Example #1: Answer

- **Initially: all variables zero** (that is, x is 0, y is 0)

thread 1	thread 2
store 1 → y load x	store 1 → x load y

- What value pairs can be read by the two loads?

store 1 → y load x store 1 → x load y (x=0, y=1)	store 1 → y store 1 → x load x load y (x=1, y=1)	store 1 → y store 1 → x load y load x (x=1, y=1)
store 1 → x load y store 1 → y load x (x=1, y=0)	store 1 → x store 1 → y load y load x (x=1, y=1)	store 1 → x store 1 → y load x load y (x=1, y=1)

- What about (x=0,y=0)? Yes! (for x86, SPARC, ARM, PowerPC)

Why? Simplify Out-of-Order Execution

- Why? Why Allow Such Odd Behavior?
  - Reason #2: simplifying out-of-order execution
- One key benefit of out-of-order execution:
  - Out-of-order execution of loads to (same or different) addresses

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

  - **Uh, oh.**
- Two options for hardware designers:
  - Option #1: **allow** this sort of "odd" reordering ("not my problem")
  - Option #2: hardware **detects & recovers** from such reorderings
    - Scan load queue (LQ) when cache block is invalidated
- And store buffers on some systems reorder stores by same thread to different addresses (as in thread 1 above)

Shared Memory Example #2: Answer

- **Initially: all variables zero** (flag is 0, a is 0)

thread 1	thread 2
`store 1 → a` `store 1 → flag`	`loop: if (flag == 0) goto loop` `load a`

- What value can be read by "load a"?
  - "load a" can see the value "1"
- Can "load a" read the value zero? (same as last slide)
  - **Yes! (for ARM, PowerPC, Itanium, and Alpha)**
  - **No! (for Intel/AMD x86, Sun SPARC, IBM 370)**
    - Assuming the compiler didn't reorder anything…

Restoring Order (Hardware)

- Sometimes we need ordering (mostly we don't)
  - Prime example: ordering between "lock" and data
- How? insert **Fences (memory barriers)**
  - Special instructions, part of ISA
- Example
  - Ensure that loads/stores don't cross synchronization operations

```
lock acquire
fence
"critical section"
fence
lock release
```

- How do fences work?
  - They stall execution until write buffers are empty
  - Makes lock acquisition and release slow(er)
- **Use synchronization library, don't write your own**

Restoring Order (Software)

- These slides have focused mostly on **hardware** reordering
  - But the compiler also reorders instructions (reason #3)
- How do we tell the **compiler** to not reorder things?
  - Depends on the language…
- In Java:
  - The built-in "synchronized" constructs informs the compiler to limit its optimization scope (prevent reorderings across synchronization)
  - Or, programmer uses "volatile" keyword to explicitly mark variables
  - Java compiler inserts the hardware-level ordering instructions
- In C/C++:
  - More murky, as pre-2011 language doesn't define synchronization
  - Lots of hacks: "inline assembly", volatile, atomic keyword (new!)
  - Programmer may need to explicitly insert hardware-level fences
- **Use synchronization library, don't write your own**

Recap: Four Shared Memory Issues

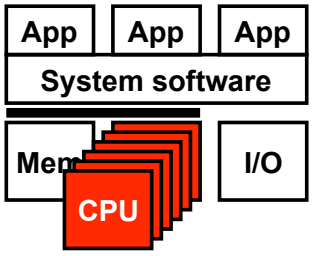
1. **Cache coherence**
   - If cores have private (non-shared) caches
   - How to make writes to one cache "show up" in others?

2. **Parallel programming**
   - How does the programmer express the parallelism?

3. **Synchronization**
   - How to regulate access to shared data?
   - How to implement "locks"?

4. **Memory consistency models**
   - How to keep programmer sane while letting hardware optimize?
   - How to reconcile shared memory with compiler optimizations, store buffers, and out-of-order execution?

Summary



- Thread-level parallelism (TLP)
- Shared memory model
  - Multiplexed uniprocessor
  - Hardware multihreading
  - Multiprocessing
- Cache coherence
  - Valid/Invalid, MSI, MESI
- Parallel programming
- Synchronization
  - Lock implementation
  - Locking gotchas
  - Transactional memory
- Memory consistency models

[spacer]

Computer Architecture

Unit 10: Data-Level Parallelism: Vectors & GPUs

Slides developed by Milo Martin & Amir Roth at the University of Pennsylvania
with sources that included University of Wisconsin slides
by Mark Hill, Guri Sohi, Jim Smith, and David Wood

How to Compute This Fast?

- Performing the **same** operations on **many** data items
  - Example: SAXPY

```
for (I = 0; I < 1024; I++) {
  Z[I] = A*X[I] + Y[I];
}
```

```
L1: ldf [X+r1]->f1  // I is in r1
    mulf f0,f1->f2  // A is in f0
    ldf [Y+r1]->f3
    addf f2,f3->f4
    stf f4->[Z+r1}
    addi r1,4->r1
    blti r1,4096,L1
```

- Instruction-level parallelism (ILP) - fine grained
  - Loop unrolling with static scheduling –or– dynamic scheduling
  - Wide-issue superscalar (non-)scaling limits benefits
- Thread-level parallelism (TLP) - coarse grained
  - Multicore
- Can we do some "medium grained" parallelism?

Data-Level Parallelism

- **Data-level parallelism (DLP)**
  - Single operation repeated on multiple data elements
    - SIMD (**S**ingle-**I**nstruction, **M**ultiple-**D**ata)
  - Less general than ILP: parallel insns are all same operation
  - Exploit with **vectors**
- Old idea: Cray-1 supercomputer from late 1970s
  - Eight 64-entry x 64-bit floating point "vector registers"
    - 4096 bits (0.5KB) in each register! 4KB for vector register file
  - Special vector instructions to perform vector operations
    - Load vector, store vector (wide memory operation)
    - Vector+Vector or Vector+Scalar
      - addition, subtraction, multiply, etc.
    - In Cray-1, each instruction specifies 64 operations!
  - ALUs were expensive, so one operation per cycle (not parallel)

Example Vector ISA Extensions (SIMD)

- Extend ISA with floating point (FP) vector storage …
  - **Vector register**: fixed-size array of 32- or 64- bit FP elements
  - **Vector length**: For example: 4, 8, 16, 64, …
- … and example operations for vector length of 4
  - Load vector: `ldf.v [X+r1]->v1`

    `ldf [X+r1+0]->v1`$_0$

    `ldf [X+r1+1]->v1`$_1$

    `ldf [X+r1+2]->v1`$_2$

    `ldf [X+r1+3]->v1`$_3$
  - Add two vectors: `addf.vv v1,v2->v3`

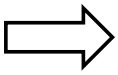
    `addf v1`$_i$`,v2`$_i$`->v3`$_i$ `(where i is 0,1,2,3)`
  - Add vector to scalar: `addf.vs v1,f2,v3`

    `addf v1`$_i$`,f2->v3`$_i$ `(where i is 0,1,2,3)`
- Today's vectors: short (128 or 256 bits), but fully parallel

Example Use of Vectors – 4-wide

```
ldf [X+r1]->f1
mulf f0,f1->f2
ldf [Y+r1]->f3
addf f2,f3->f4
stf f4->[Z+r1]
addi r1,4->r1
blti r1,4096,L1
```

7x1024 instructions



```
ldf.v [X+r1]->v1
mulf.vs v1,f0->v2
ldf.v [Y+r1]->v3
addf.vv v2,v3->v4
stf.v v4,[Z+r1]
addi r1,16->r1
blti r1,4096,L1
```

7x256 instructions
(4x fewer instructions)

- Operations
  - Load vector: `ldf.v [X+r1]->v1`
  - Multiply vector to scalar: `mulf.vs v1,f2->v3`
  - Add two vectors: `addf.vv v1,v2->v3`
  - Store vector: `stf.v v1->[X+r1]`
- Performance?
  - Best case: 4x speedup
  - But, vector instructions don't always have single-cycle throughput
    - Execution width (implementation) vs vector width (ISA)

Vector Datapath & Implementatoin

- Vector insn. are just like normal insn… only "wider"
  - Single instruction fetch (no extra N^2 checks)
  - Wide register read & write (not multiple ports)
  - Wide execute: replicate floating point unit (same as superscalar)
  - Wide bypass (avoid N^2 bypass problem)
  - Wide cache read & write (single cache tag check)
- Execution width (implementation) vs vector width (ISA)
  - Example: Pentium 4 and "Core 1" executes vector ops at half width
  - "Core 2" executes them at full width
- Because they are just instructions…
  - …superscalar execution of vector instructions
  - Multiple n-wide vector instructions per cycle

Intel's SSE2/SSE3/SSE4/AVX…

- **Intel SSE2 (Streaming SIMD Extensions 2)** - 2001
  - 16 128bit floating point registers (`xmm0-xmm15`)
  - Each can be treated as 2x64b FP or 4x32b FP ("packed FP")
    - Or 2x64b or 4x32b or 8x16b or 16x8b ints ("packed integer")
    - Or 1x64b or 1x32b FP (just normal scalar floating point)
  - Original SSE: only 8 registers, no packed integer support
- Other vector extensions
  - AMD 3DNow!: 64b (2x32b)
  - PowerPC AltiVEC/VMX: 128b (2x64b or 4x32b)
- Looking forward for x86
  - Intel's "Sandy Bridge" brings 256-bit vectors to x86
  - Intel's "Xeon Phi" multicore will bring 512-bit vectors to x86

Other Vector Instructions

- These target specific domains: e.g., image processing, crypto
  - Vector reduction (sum all elements of a vector)
  - Geometry processing: 4x4 translation/rotation matrices
  - Saturating (non-overflowing) subword add/sub: image processing
  - Byte asymmetric operations: blending and composition in graphics
  - Byte shuffle/permute: crypto
  - Population (bit) count: crypto
  - Max/min/argmax/argmin: video codec
  - Absolute differences: video codec
  - Multiply-accumulate: digital-signal processing
  - Special instructions for AES encryption
- More advanced (but in Intel's Xeon Phi)
  - Scatter/gather loads: indirect store (or load) from a vector of pointers
  - Vector mask: predication (conditional execution) of specific elements

Using Vectors in Your Code

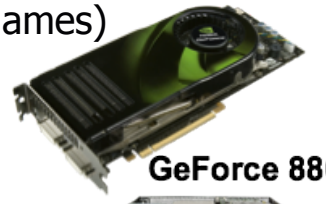
- Write in assembly
  - Ugh
- Use "intrinsic" functions and data types
  - For example: _mm_mul_ps() and "__m128" datatype
- Use vector data types
  - typedef double v2df __attribute__ ((vector_size (16)));
- Use a library someone else wrote
  - Let them do the hard work
  - Matrix and linear algebra packages
- Let the compiler do it (automatic vectorization, with feedback)
  - GCC's "-ftree-vectorize" option, -ftree-vectorizer-verbose=**n**
  - Limited impact for C/C++ code (old, hard problem)

Recap: Vectors for Exploiting DLP

- Vectors are an efficient way of capturing parallelism
  - Data-level parallelism
  - Avoid the N^2 problems of superscalar
  - Avoid the difficult fetch problem of superscalar
  - Area efficient, power efficient
- The catch?
  - Need code that is "vector-izable"
  - Need to modify program (unlike dynamic-scheduled superscalar)
  - Requires some help from the programmer
- Looking forward: Intel "Xeon Phi" (aka Larrabee) vectors
  - More flexible (vector "masks", scatter, gather) and wider
  - Should be easier to exploit, more bang for the buck

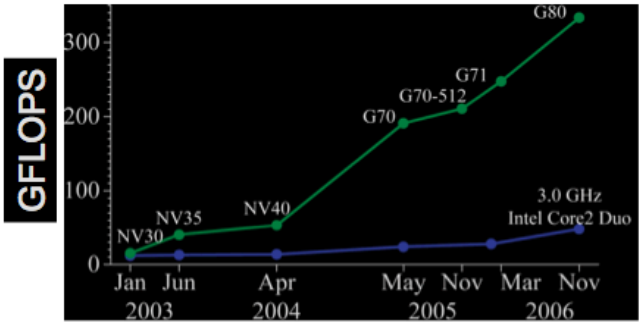
Graphics Processing Units (GPU)

- Killer app for parallelism: graphics (3D games)
- A quiet revolution and potential build-up
  - Calculation: 367 GFLOPS vs. 32 GFLOPS
  - Memory Bandwidth: 86.4 GB/s vs. 8.4 GB/s
  - Until recently, programmed through graphics API



GeForce 8800

Tesla S870



G80 = GeForce 8800 GTX
G71 = GeForce 7900 GTX
G70 = GeForce 7800 GTX
NV40 = GeForce 6800 Ultra
NV35 = GeForce FX 5950 Ultra
NV30 = GeForce FX 5800

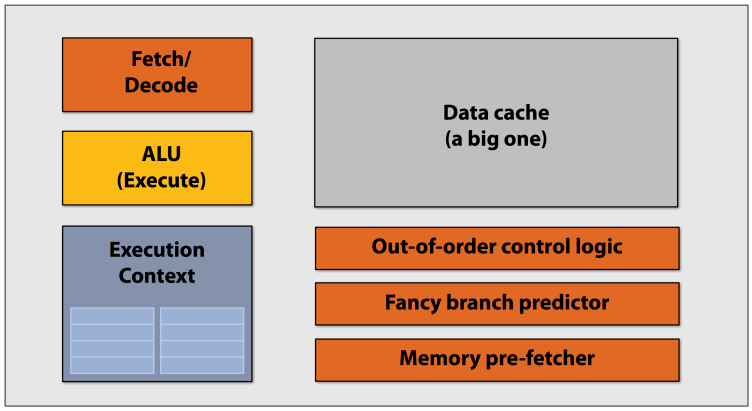
- GPU in every desktop, laptop, mobile device – massive volume and potential impact

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2009 ECE 498AL, University of Illinois, Urbana-Champaign

GPUs and SIMD/Vector Data Parallelism

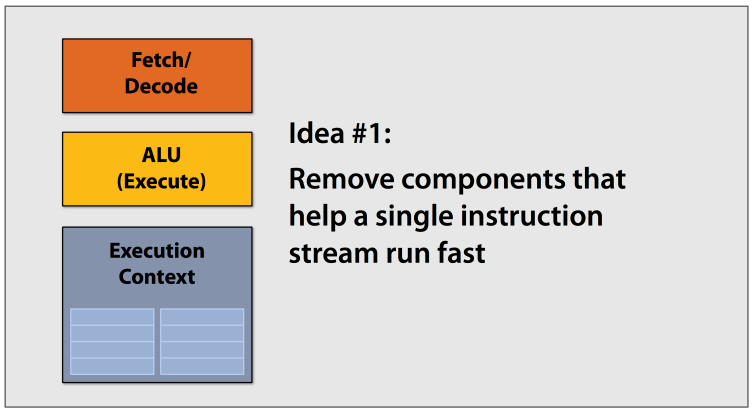
- How do GPUs have such high peak FLOPS & FLOPS/Joule?
  - Exploit massive data parallelism – focus on total throughput
  - Remove hardware structures that accelerate single threads
  - Specialized for graphs: e.g., data-types & dedicated texture units
- "SIMT" execution model
  - Single instruction multiple threads
  - Similar to both "vectors" and "SIMD"
  - A key difference: better support for conditional control flow
- Program it with CUDA or OpenCL
  - Extensions to C
  - Perform a "shader task" (a snippet of scalar computation) over many elements
  - Internally, GPU uses scatter/gather and vector mask operations

"CPU-style" cores



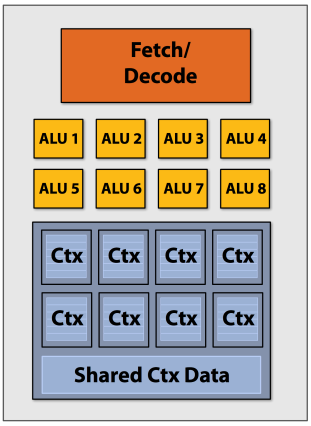
Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

Slimming down



Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

Add ALUs



Idea #2:
Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD processing

Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

Stalls!

Stalls occur when a core cannot run the next instruction because of a dependency on a previous operation.

Texture access latency = 100's to 1000's of cycles

We've removed the fancy caches and logic that helps avoid stalls.

Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

But we have **LOTS** of independent fragments.

Idea #3:

Interleave processing of many fragments on a single core to avoid stalls caused by high latency operations.

Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

My chip!

16 cores

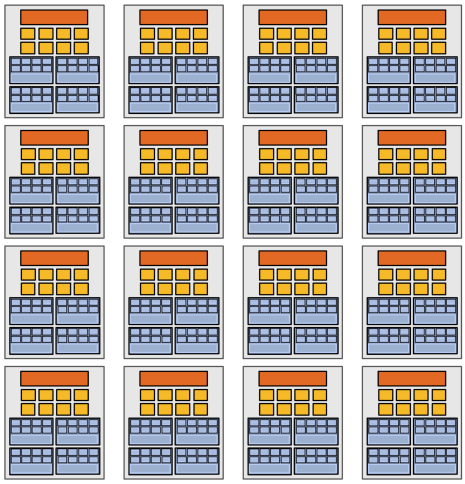
8 mul-add ALUs per core
(128 total)

16 simultaneous
instruction streams

64 concurrent (but interleaved)
instruction streams

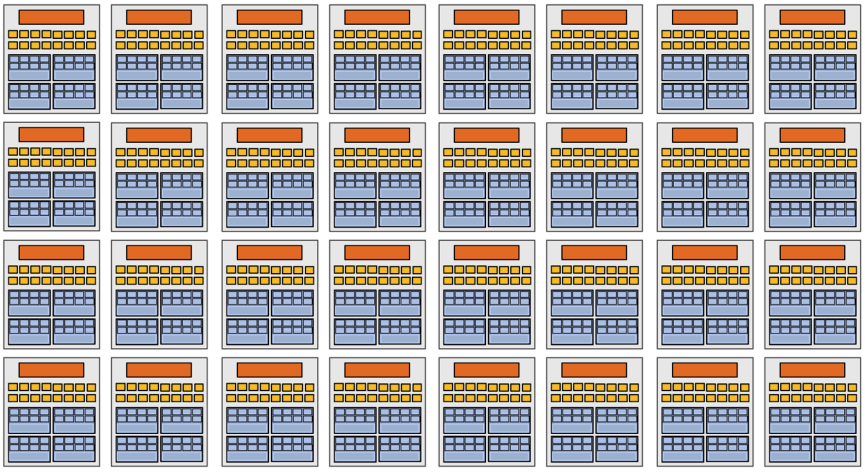
512 concurrent fragments

= 256 GFLOPs (@ 1GHz)



Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

My "enthusiast" chip!



32 cores, 16 ALUs per core (512 total) = 1 TFLOP (@ 1 GHz)

Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

Summary: three key ideas

1. Use many "slimmed down cores" to run in parallel

2. Pack cores full of ALUs (by sharing instruction stream across groups of fragments)
   - Option 1: Explicit SIMD vector instructions
   - Option 2: Implicit sharing managed by hardware

3. Avoid latency stalls by interleaving execution of many groups of fragments

Slide by Kayvon Fatahalian - http://bps10.idav.ucdavis.edu/talks/03-fatahalian_gpuArchTeraflop_BPS_SIGGRAPH2010.pdf

Data Parallelism Summary

- Data Level Parallelism
  - "medium-grained" parallelism between ILP and TLP
  - Still one flow of execution (unlike TLP)
  - Compiler/programmer must explicitly expresses it (unlike ILP)
- Hardware support: new "wide" instructions (SIMD)
  - Wide registers, perform multiple operations in parallel
- Trends
  - Wider: 64-bit (MMX, 1996), 128-bit (SSE2, 2000), 256-bit (AVX, 2011), 512-bit (Xeon Phi, 2012?)
  - More advanced and specialized instructions
- GPUs
  - Embrace data parallelism via "SIMT" execution model
  - Becoming more programmable all the time
- Today's chips exploit parallelism at all levels: ILP, DLP, TLP

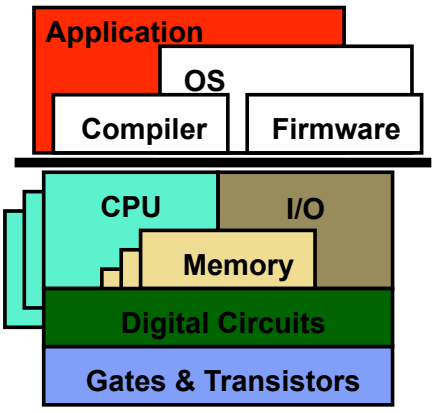
[spacer]

Computer Architecture

Unit 11: Putting it All Together:
Anatomy of the XBox 360 Game Console

Slides originally developed by Milo Martin &
Amir Roth at University of Pennsylvania

This Unit: Putting It All Together



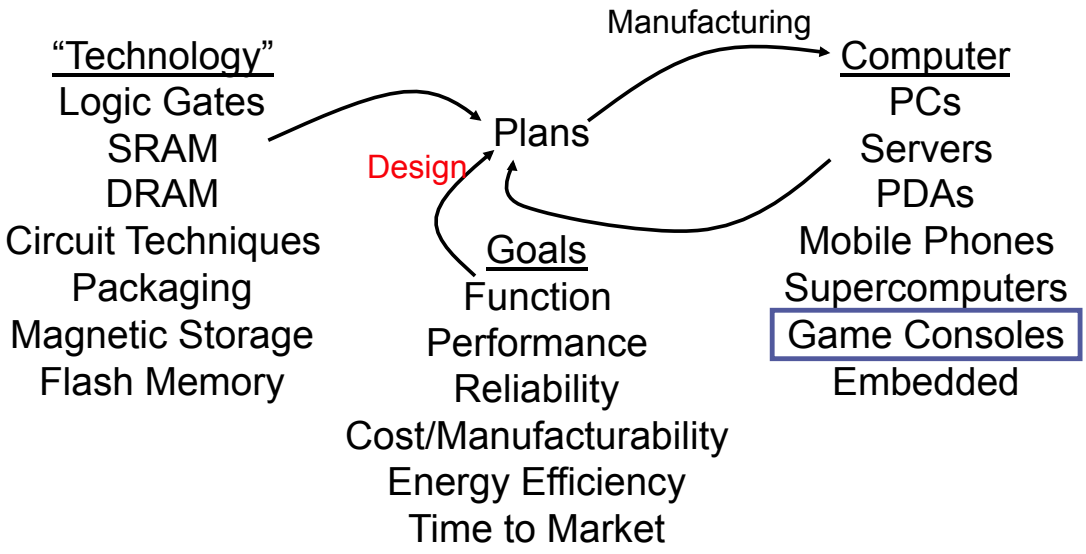
- Anatomy of a game console
  - Microsoft XBox 360
- Focus mostly on CPU chip
- Briefly talk about system
  - Graphics processing unit (GPU)
  - I/O and other devices

Sources

- **Application-customized CPU design: The Microsoft Xbox 360 CPU story**, Brown, IBM, Dec 2005
  - http://www-128.ibm.com/developerworks/power/library/pa-fpfxbox/
- **XBox 360 System Architecture**, Andrews & Baker, IEEE Micro, March/April 2006
- Microprocessor Report
  - **IBM Speeds XBox 360 to Market**, Krewell, Oct 31, 2005
  - **Powering Next-Gen Game Consoles**, Krewell, July 18, 2005

What is Computer Architecture?

The role of a *computer* architect:



Microsoft XBox Game Console History

- XBox
  - First game console by Microsoft, released in 2001, $299
  - Glorified PC
    - 733 Mhz x86 Intel CPU, 64MB DRAM, NVIDIA GPU (graphics)
    - Ran modified version of Windows OS
  - ~25 million sold
- XBox 360
  - Second generation, released in 2005, $299-$399
  - All-new custom hardware
    - 3.2 Ghz PowerPC IBM processor (custom design for XBox 360)
    - ATI graphics chip (custom design for XBox 360)
  - 45 million sold, as of Sept 2010 [Source: Wikipedia]
    - 70 million sold as of Sept 2012 [Source: Wikipedia]

Microsoft Turns to IBM for XBox 360

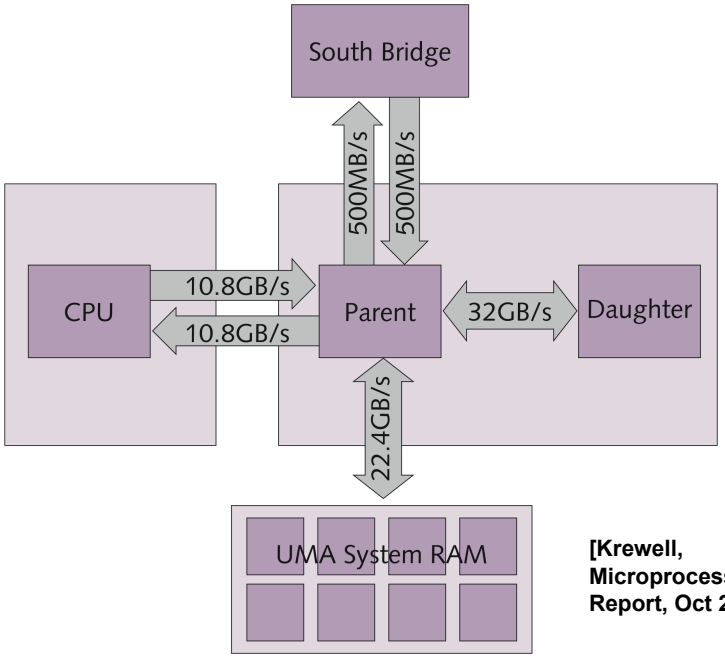
- Microsoft is mostly a software company
  - Turned to IBM & ATI for XBox 360 design
  - Sony & Nintendo also turned to IBM (for PS3 & Wii, respectively)

- Design principles of XBox 360 [Andrews & Baker, 2006]
  - Value for 5-7 years
    - ➔ big performance increase over last generation
  - Support anti-aliased high-definition video (720*1280*4 @ 30+ fps)
    - ➔ extremely high pixel fill rate (goal: 100+ million pixels/s)
  - Flexible to suit dynamic range of games
    - ➔ balance hardware, homogenous resources
  - Programmability (easy to program)
    - ➔ listened to software developers

More on Games Workload

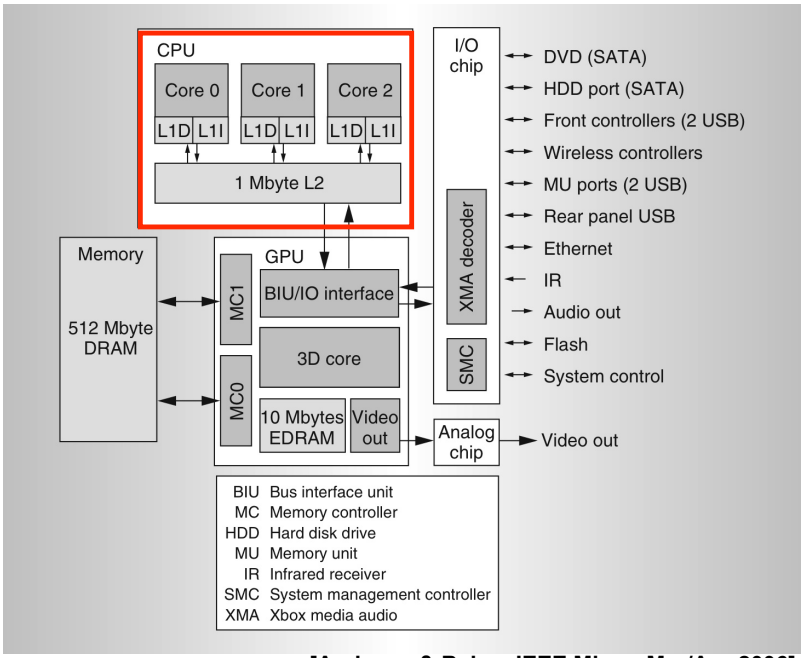
- Graphics, graphics, graphics
  - Special highly-parallel graphics processing unit (GPU)
  - Much like on PCs today
- But general-purpose, too
  - "The high-level game code is generally a database management problem, with plenty of object-oriented code and pointer manipulation. Such a workload needs a large L2 and high integer performance." [Andrews & Baker, 2006]
- Wanted only a modest number of modest, fast cores
  - Not one big core
  - Not dozens of small cores (leave that to the GPU)
  - Quote from Seymour Cray

XBox 360 System from 30,000 Feet



[Krewell, Microprocessor Report, Oct 21, 2005]

XBox 360 System



[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

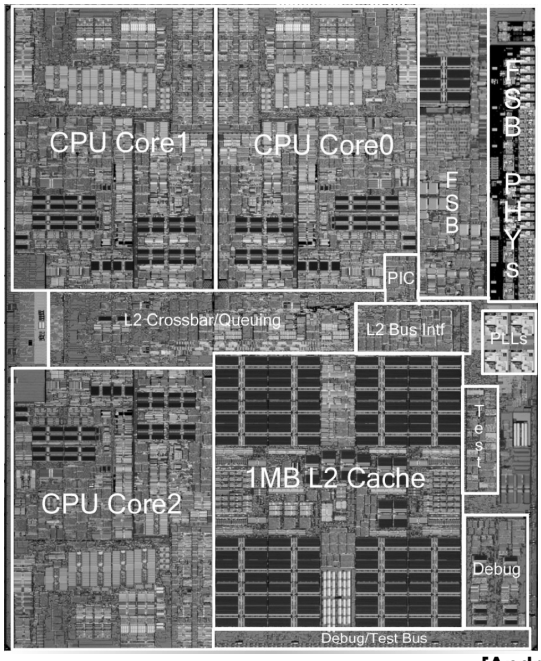
XBox 360 "Xenon" Processor

- ISA: 64-bit PowerPC chip
  - RISC ISA
    - Like MIPS, but with condition codes
  - Fixed-length 32-bit instructions
  - 32 64-bit general purpose registers (GPRs)
- ISA Extended with VMX-128 operations
  - **128 registers, 128-bits each**
  - Packed "vector" operations
  - Example: four 32-bit floating point numbers
    - One instruction: VR1 * VR2 ➔ VR3
    - Four single-precision operations
    - Also supports conversion to Microsoft DirectX data formats
  - Similar to Altivec (and Intel's MMX, SSE, SSE2, etc.)
  - Works great for 3D graphics kernels and compression

XBox 360 "Xenon" Processor

- Peak performance: ~75 gigaflops
  - Gigaflop = 1 billion floating points operations per second
- Pipelined superscalar processor
  - 3.2 Ghz operation
  - Superscalar: two-way issue
  - VMX-128 instructions (four single-precision operations at a time)
  - Hardware multithreading: two threads per processor
  - Three processor cores per chip
- Result:
  - 3.2 * 2 * 4 * 3 = ~77 gigaflops

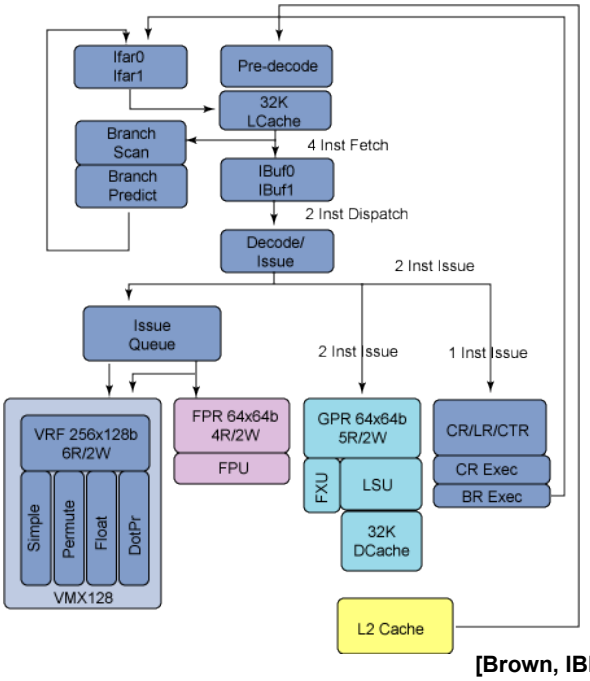
XBox 360 "Xenon" Chip (IBM)



- 165 million transistors
  - IBM's 90nm process
- Three cores
  - 3.2 Ghz
  - Two-way superscalar
  - Two-way multithreaded
- Shared 1MB cache

[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

"Xenon" Processor Pipeline



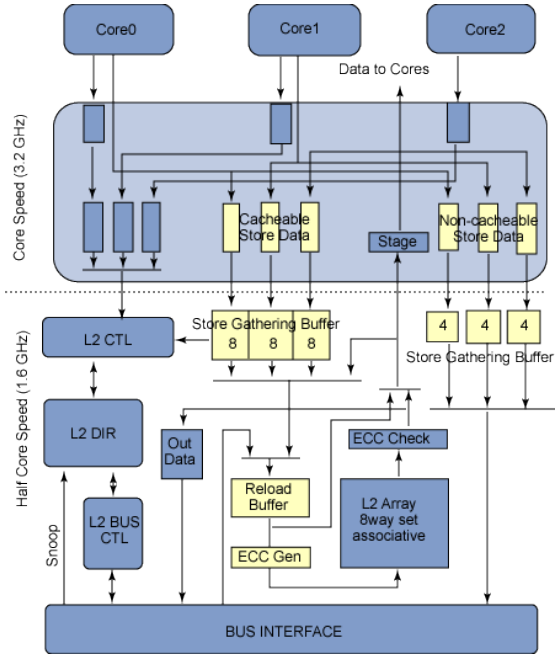
- Four-instruction fetch
- Two-instruction "dispatch"
- Five functional units
- "VMX128" execution "decoupled" from other units
  - 14-cycle VMX dot-product
- Branch predictor:
  - "4K" G-share predictor
  - Unclear if 4KB or 4K 2-bit counters
  - Per thread

[Brown, IBM, Dec 2005]

XBox 360 Memory Hiearchy

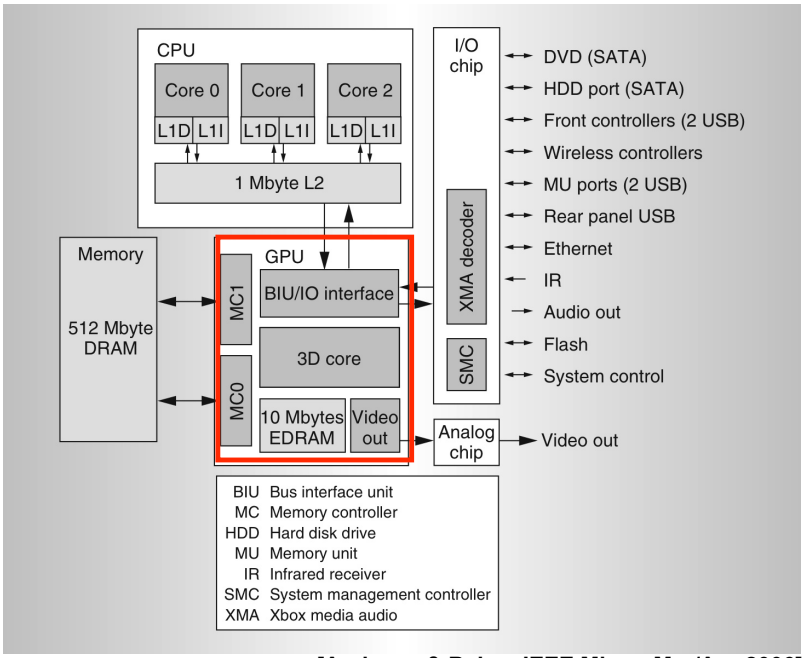
- 128B cache blocks throughout
- 32KB 2-way set-associative instruction cache (per core)
- 32KB 4-way set-associative data cache (per core)
  - Write-through, lots of store buffering
  - Parity
- 1MB 8-way set-associative second-level cache (per chip)
  - Special "skip L2" prefetch instruction
  - MESI cache coherence
  - Error Correcting Codes (ECC)
- 512MB GDDR3 DRAM, dual memory controllers
  - Total of 22.4 GB/s of memory bandwidth
- Direct path to GPU

Xenon Multicore Interconnect



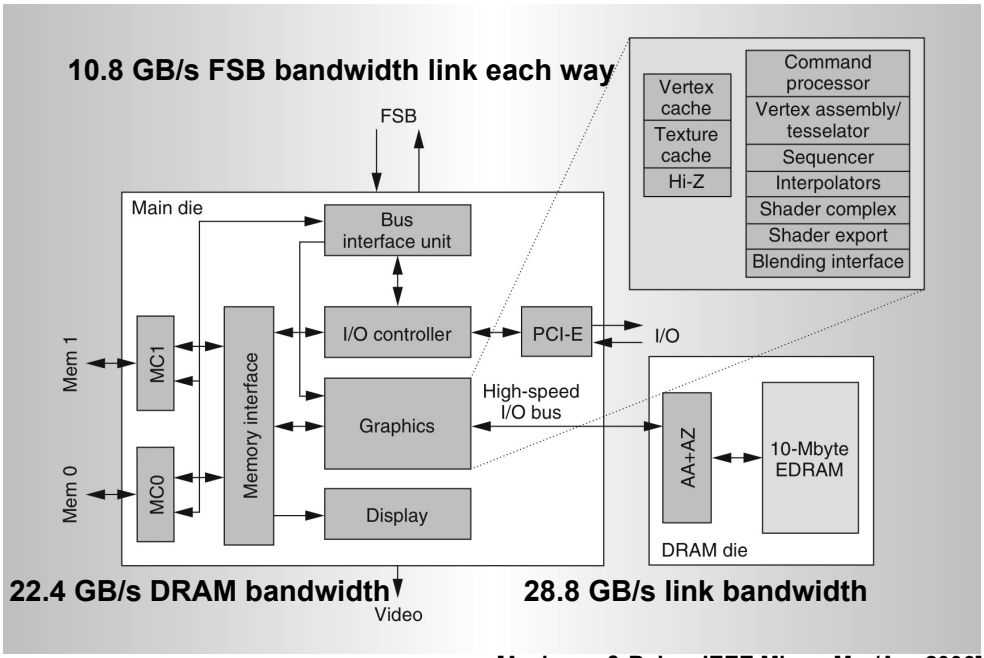
 [Brown, IBM, Dec 2005]

XBox 360 System



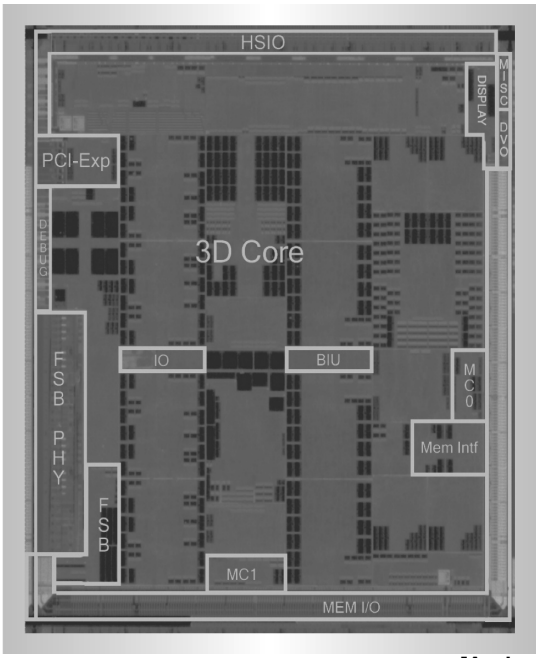
[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

XBox Graphics Subsystem



[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

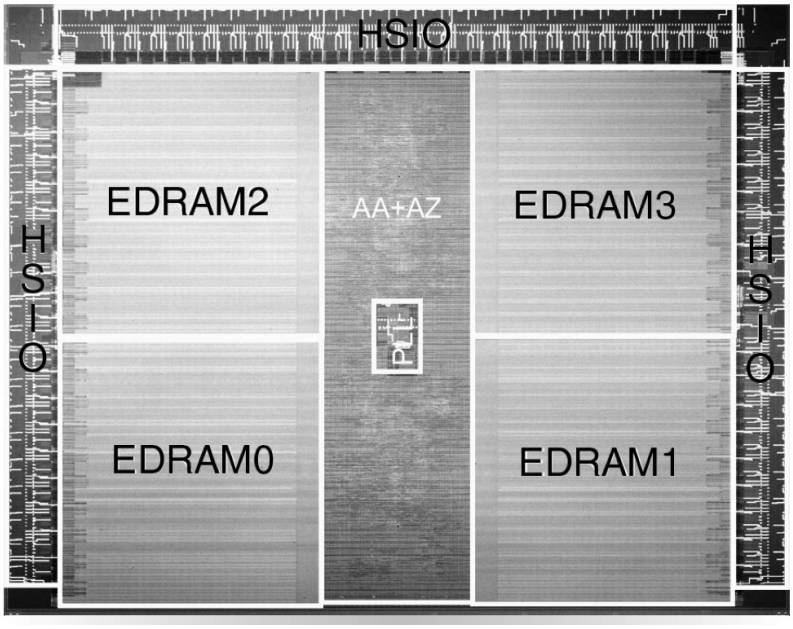
Graphics "Parent" Die (ATI)



- 232 million transistors
- 500 Mhz
- 48 unified shader ALUs
  - Mini-cores for graphics

[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

GPU "daughter" die (NEC)



- 100 million transistors
- 10MB eDRAM
  - "Embedded"
- NEC Electronics
- Anti-aliasing
  - Render at 4x resolution, then sample
- Z-buffering
  - Track the "depth" of pixels
- 256GB/s internal bandwidth

[Andrews & Baker, IEEE Micro, Mar/Apr 2006]

Putting It All Together

- Unit 1: Introduction
- Unit 2: ISAs
- Unit 3: Technology
- Unit 4: Caches
- Unit 5: Virtual Memory
- Unit 6: Pipelining & Branch Prediction
- Unit 7: Superscalar
- Unit 8: Scheduling
- Unit 9: Multicore
- Unit 10: Vectors & GPUs